

nat-os

An Operating System Written From Scratch for the ESP32

The Complete Engineering Narrative

USED MEDIAS LLC · EMBEDDED SYSTEMS DIVISION

nat-os — An Operating System Written From Scratch for the ESP32

The Complete Engineering Narrative

Compiled from the source tree, the commit history, and engineering reports UM-NATOS-001 through UM-NATOS-028.

The kernel and this book are released under the MIT Licence. The two binaries in `vendor/` are unmodified ESP-IDF build artefacts, copyright Espressif Systems, redistributed under the Apache Licence 2.0; the MIT licence does not apply to them.

Developed and verified on the ESP32-2432S028R, the board commonly sold as the “Cheap Yellow Display”. Every measurement in this book was taken on that hardware, and in most cases on one physical unit. Where a result depends on that, the text says so.

Claims verified on hardware are marked as such; claims taken from documentation or from reasoning are marked separately. Chapter 30 is the consolidated inventory of what this system does *not* establish.

ESP32 is a trademark of Espressif Systems.

Contents

FRONT MATTER

Preface	1
Conventions, and a Note on Evidence	4

PART I — FOUNDATIONS

1 The Machine, and the Problem It Cannot Solve	8
2 call0: The Calling Convention That Made a Scheduler Tractable	15
3 Boot: From Silicon to <code>_start</code>	24
4 The Memory Map, and an Overlap That Was Not There	31
5 The Build Pipeline	40
6 Milestone 0: Proving Ownership of the Machine	47

PART II — THE KERNEL

7 Interrupts and the Tick	53
8 Tasks, Frames, and the Defect That Cost Twelve Builds	63
9 Scheduling: Priority, Sleep, Ageing, and Two Clocks That Lied	76
10 Memory: A Heap With One Invariant, and Arenas	89
11 Locking, and Why Contention Cost Is a Count and Not a Duration	100
12 Failure Handling: Three Mechanisms That Had Never Fired	111

PART III — THE VIRTUAL MACHINE

13 Why a Bytecode VM Is the Only Isolation Available	125
14 The Instruction Set and the Interpreter	131
15 The Producer: <code>vasm.py</code> and the Programs	144
16 Applications: Lifecycle, Three Levels of Scheduling, Messaging	156
17 Syscalls and the Confinement Model	167

PART IV — DRIVERS

18 The Display: From 387 ms to 43 ms, and a Stall Still Open	179
19 Touch: Four Attempts, and an Axis Inverted for Three Months	194
20 Persistence: A Read Defect That Looked Like the Wrong Thing	208
21 microSD, and a Pad Table That Is Not in Pin Order	218
22 The Interrupt Matrix, the ADC, and I ² C	224
23 Audio, and Three Ways to Be Silent	242

PART V — THE DEVICE

24 The Launcher, and Four Defects It Found by Existing 248

25 The Renderer 259

26 The Note Pad and the Shell on the Panel 268

27 WiFi: Mixed ABIs, a Vendor Blob, and Frames Off the Air 281

PART VI — METHOD

28 The Instruments That Lied 297

29 The Standing Rules 305

30 What Is Not Established 313

31 Where It Could Go Next 321

APPENDICES

A File Inventory 326

B Bytecode ISA Reference 332

C Shell Command Reference 338

D Address and Register Map 343

E Report Index and Cross-Reference 349

F Every Measured Number 354

G Timeline from the Commit History 363

Preface

Why this book exists

There are two kinds of documentation an operating system produces. The first describes what the system does: the API, the data structures, the state machines. The second describes what the system's authors *believed* at each point, what measurement said instead, and what changed as a result.

nat-os has an unusual amount of the second kind. Twenty-eight engineering reports exist under `docs/`, and they follow two rules that are worth stating up front because they shape everything in this book:

Measured is separated from assumed. On a from-scratch kernel the difference between "this is true" and "this should be true" is the difference between a working boot and a silent reset, so claims verified on hardware are marked as such and claims taken from documentation are marked separately.

Every report ends with what it does *not* establish. Those sections are the most useful part. They are where the known gaps live, and several of them correctly predicted the next defect.

That second rule turns out to be load-bearing. UM-NATOS-011 §4 predicted, in writing, that the next thing to break would be a flash write colliding with the data cache. It broke exactly that way, on the first version of the driver, and the report that records it (UM-NATOS-018 §3) says so. UM-NATOS-010 §8 recorded that `arena_contains()` existed and nothing called it; the isolation guarantee arrived two milestones later and the report that delivered it cites the gap. UM-NATOS-009 §9 recorded that nothing had audited the other consequences of `PS.EXCM`; nothing has, and it is still listed as open in Chapter 30.

The reports are the primary record. This book is not a replacement for them and does not try to be. What it adds is *continuity*: a defect found in the touch driver in Chapter 19 is the same defect that decided the launcher's icon selection in Chapter 24, and the reports say so in cross-references while this book says so in a narrative. A reader picking up UM-NATOS-017 cold learns that the X axis was inverted; a reader of this book learns why nothing noticed for three months, which is the transferable part.

The shape of the thing being described

nat-os runs on the ESP32-2432S028R — the board sold everywhere as the "Cheap Yellow Display". A dual-core Xtensa LX6 at 240 MHz nominal (the bootloader leaves it at 80 MHz and nothing in the kernel raises it), 520 KB of internal SRAM of which about 176 KB is usable as data, 4 MB of flash executed in place through a cache, an

ILI9341 240×320 panel, an XPT2046 resistive touch controller, a microSD slot, a light sensor, a speaker connector, and an RGB LED.

No PSRAM. No MMU paging. That second absence is the single fact from which most of this system's architecture follows, and Chapter 1 is about it.

The kernel starts at what UM-NATOS-001 calls layer L2 — scheduler, memory, synchronisation — and owns everything above it. L1, the second-stage bootloader, is borrowed: 17,536 bytes that configure the flash controller, program the flash MMU, enable the cache and jump to an entry point. The interface between L1 and nat-os is the image header and nothing else, which is why replacing L1 later is a contained change rather than a rewrite.

How to read this

If you are new to the project, read Chapters 1 through 6 in order. That gives you the hardware, the calling convention, how the thing boots, where everything lives, how it is built, and what "milestone 0" proved. About 60 pages, and after them the rest of the book can be read in any order.

If you are picking up implementation work, read Chapter 30 first. It is the consolidated inventory of everything the system does *not* establish, drawn from the "what this does not establish" section of every report. It is where the next defect probably lives.

If you are debugging something, Chapter 28 is the catalogue of every instrument in this kernel that has been caught reporting confidently and wrongly, and Chapter 29 is the set of standing rules those failures produced. Both are short and both have saved time more than once.

If you want the reference material, the appendices carry the ISA, the shell commands, the register map, a file inventory, and a table of every number this project has measured on hardware.

A note on the failures

This book records the defects honestly, including the embarrassing ones. A touch axis that was inverted for three months behind a calibration that could only ever return one answer. A tick deadline that raced into the future on every yield. An idle task that silently failed to be created and whose absence was printed on screen and read without being registered. A self-test that could not fail by construction and was believed anyway. A capture harness that rebooted the board it was measuring and then reported twelve seconds of a system nobody was touching. Three separate peripherals whose registers read back byte-for-byte correct while the hardware sat completely dead.

None of that is included for colour. The failures are more instructive than the successes because the successes are mostly the ordinary application of published technique, and the failures are where this project learned something it could not have read. Where a defect produced a rule, the rule is stated at the point of the defect and again in Chapter 29.

One theme recurs often enough to name here rather than let it accumulate: **the kernel's own instruments were wrong more often than the hardware was.** Chapter 28 counts seven separate cases in a single session. The pattern is always the same — a signal that the measurement was invalid, sitting next to a number that looked reasonable, and the number believed.

Acknowledgement of scope

This is one board, one panel, one flash chip, one microSD card, one finger, and one person's judgement about whether an icon is legible at 24 pixels. Where a result depends on that, this book says so. The scheduler, the heap, the arena model, the bytecode VM and the application model assume an ESP32 and nothing more; the board-specific parts are the pin maps and the panel and touch controllers, and they are confined to their own files.

Conventions, and a Note on Evidence

Typographic conventions

Source is quoted verbatim from the tree, with its comments intact. The comments in `nat-os` carry a substantial part of the design rationale — in several files the comment explaining why a line exists is longer than the file's code — so stripping them for brevity would remove most of what makes the quotation worth reading.

Where a quotation is abridged, the elision is marked:

```
/* ... */
```

Register addresses are given in the form the source uses. Hex constants are lower case in code and upper case in prose, matching the reports.

Report citations use the reports' own numbering: **UM-NATOS-017 §7.1** is section 7.1 of `docs/UM-NATOS-017-touch.md`. The renaming from `cyd-os` to `nat-os` did not change document or section numbers, so `UM-CYDOS-014 §5.2` in an old commit message is `UM-NATOS-014 §5.2` here.

The evidence grading used throughout

The reports draw a hard line between three kinds of claim, and this book keeps it. Every quantitative statement in this book falls into one of these:

Grade	Meaning	Example
Measured	Observed on the target board, with the raw output recorded	"43 ms full-screen fill via SPI2 + DMA"
Derived	Computed from measured quantities, with the arithmetic shown	"~109 cycles per bytecode instruction" — from a quarter-share assumption, so a ceiling
Transcribed	Taken from documentation or a vendor header, not independently confirmed	The SRAM region boundaries in Chapter 4

The distinction matters more than it usually does. `UM-NATOS-004 §2` transcribes the ESP32's SRAM boundaries from the Technical Reference Manual and marks them as unverified, while marking the addresses the kernel actually uses as confirmed on hardware. `UM-NATOS-024 §3` is the story of a constant transcribed *from memory* rather than from the vendor header — `SENS_SARI_EN_PAD_FORCE` written as bit 27 instead of bit 31 — which produced a driver whose every register read back exactly as written and which was converting nothing.

Where this book states a number, it says which grade it is if there is any ambiguity.

What "verified on hardware" means here, and what it does not

UM-NATOS-019 is entirely about the gap between a mechanism *existing* and a mechanism *working*. Three separate safety mechanisms — stack guards, the panic handler, and the hang detector — had each been confirmed to exist and none had been observed to fire. The rule that came out of it:

A startup artefact is not evidence the thing it introduces works. The shell was signed off because its banner printed; the banner proves a task was created and the TRANSMIT path works, and says nothing about receive. The receive path was one byte behind for the shell's entire existence.

So in this book, "verified on hardware" means one of:

- A counter was read back with a value that could only be produced by the mechanism working (`escapes=0` across 136 fills; `corrupt=0` across 3,418 ticks).
- The mechanism was **triggered deliberately** and its output captured (`fault`, `smash` and `hang` exist in the shell for exactly this).
- A quantity was measured against an independent clock (the TSF timer's 1 MHz rate, confirmed against CCOUNT over half a second, two clocks with no connection to each other).

It does not mean "the screen looked right", except where the book says so explicitly — and Chapter 18 records three occasions where a number improved, the change was reported, and the panel had not been looked at.

The counter idiom

nat-os instruments nearly everything, and the instrumentation follows a pattern worth naming because it appears in every chapter.

A claim is turned into a **pair** of counters where possible: one that must be non-zero and one that must be zero. Alone, either is uninformative.

```
touch g/w = 81/109211      last = ...->191,174
```

81 touches delivered to an application and 109,211 withheld for landing outside its viewport. Delivery alone would not show confinement; withholding alone would be indistinguishable from a broken syscall. Both together are the claim (UM-NATOS-017 §8.2).

```
vp calls=136  escapes=0  maxy=250/320
```

136 rectangles reached the panel; none escaped its viewport; the lowest row any application painted is 250, which is exactly slot 2's viewport bottom edge (`168 + 2×28 + 26`). "Not one row below it, and nowhere near the panel's 320."

The same idea applies to policies that discard work. `display_try_lock()` lets a drawing primitive be skipped when the panel is busy, and the skip is counted:

```
if (!display_try_lock()) {
    g_draw_skipped++;
    return;
}
```

with the reasoning recorded beside it in `vm.c`:

```
/* Drawing primitives dropped because the panel was busy. Counted, not hidden: a
 * best-effort policy that silently discards work is indistinguishable from a
 * broken one, and the ratio of skipped to drawn is the only thing that says
 * whether the policy is reasonable or is starving applications of the screen. */
static uint32_t g_draw_skipped;
```

Measured at 45 of 1,325 primitives, 3.4%.

Units and clocks

Three clocks appear in this book and they measure different things. Confusing them has produced at least four wrong conclusions in this project's history, so:

Clock	What it counts	Where it lies
<code>xt_ccount()</code>	CPU cycles, free-running	Wall clock. Keeps running while the reading task is descheduled. A full-screen fill measures 43 ms single-threaded and 249–362 ms from a task — the same bytes to the same panel.
<code>task_cpu_cycles()</code>	Cycles the calling task has actually run	Only advances at context switches, so it cannot bound a spin <i>inside</i> one slice.
<code>timer_ticks()</code>	Scheduler ticks, 10 ms each	Counted ISR entries rather than elapsed time until UM-NATOS-028 §4. Measured at 217 ticks per real second where 100 is correct.

The CPU runs at 80 MHz. This is **derived**, not specified: UM-NATOS-008 §5.2 counted 25 ticks of 2,400,000 cycles within a ~10 s capture window and inferred a clock near 80 MHz. The bootloader leaves the core at its default and nothing in the kernel raises it, so this is a measurement of the current configuration rather than a specification figure, and it should be re-derived if boot configuration ever changes.

The tick is 10 ms (`TICK_INTERVAL_CYCLES` = 800,000 cycles at 80 MHz).

A word on the tone of the source comments

The comments quoted throughout are unusual for kernel source in that they frequently argue with themselves, record retracted conclusions, and name what a piece of code *cannot* do. That is deliberate and it is the same discipline as the reports. From `touch.c`:

```
* ---- why the original calibration passed -----
*
* This axis has been backwards since the touch driver was written, and the
* calibration in UM-NATOS-017 §7 did not catch it – but NOT because it ignored
* direction. It tested direction explicitly, and concluded the opposite:
*
*     "the horizontal drag ended at rx=3527 against a maximum of 3536,
*       so raw X increases left to right"
*
* The flaw is which sample it trusted.
```

A comment that records the wrong answer alongside the right one is a comment that stops the wrong answer being rediscovered. Several of them in this tree have already done that job.

1. The Machine, and the Problem It Cannot Solve

Sources: docs/UM-NATOS-001-architecture.md, docs/UM-NATOS-004-memory-map.md Code: kernel/arena.h, kernel/vm.h, kernel/task.h

1.1 The target

Every architectural decision in nat-os traces back to a property of one specific piece of silicon, so it is worth being precise about what that silicon is before anything else.

Property	Value	Consequence for the design
SoC	ESP32-D0WD, dual-core Xtensa LX6 @ 240 MHz	Real SMP is possible; deferred indefinitely — APP_CPU is still never started
Internal SRAM	520 KB (SRAM0 192 KB / SRAM1 128 KB / SRAM2 200 KB)	Ample for a kernel; tight once a graphics stack is added
PSRAM	None	No external RAM to fall back on
Flash	4 MB, executed in place via cache	Code can exceed IRAM, but requires cache setup
MMU	Flash-cache address translation only	No per-process virtual address spaces
Display	ILI9341 240×320 with resistive touch	The framebuffer question, §1.4

The chip identifies itself, and the identification was checked rather than assumed. UM-NATOS-010 §7 records it: **ESP32-D0WD-V3 rev 3.1**, features `WiFi`, `BT`, `Dual Core`, `240MHz`, `VRef calibration in efuse`, `Coding Scheme None`. No embedded PSRAM, and none on the module. That check exists because "add external RAM" is the obvious escape hatch from the memory constraint in §1.4, and it is closed without a hardware change.

The board is the ESP32-2432S028R, sold as the "Cheap Yellow Display". Every measurement anywhere in this project was taken on it, and mostly on one physical unit. Where that matters — the touch calibration, the SPI clock ceiling, the speaker's frequency response — the book says so.

1.2 Five layers, and where the project starts writing

UM-NATOS-001 divides the system into five layers. The scope decision is not "what does the OS contain" but *where the project starts writing its own code*.

Layer	Contents	nat-os
L0	Mask-ROM bootloader	Silicon. Cannot be replaced.
L1	Second-stage bootloader: clock config, flash cache/MMU init, image loading	Borrowed. Replaceable later.
L2	Scheduler, context switching, synchronisation, memory allocation	Written from scratch
L3	Driver model, virtual filesystem, inter-process communication	Written from scratch
L4	Application format, loader, virtual machine, shell	Written from scratch

The argument for starting at L2

Writing L1 means implementing clock trees, flash cache configuration, and SPI timing calibration against the technical reference manual. That is silicon bring-up, not operating system design. It is a legitimate project and a different one, and it delays the first scheduler by weeks.

Borrowing L1 costs one binary dependency — 17,536 bytes — and yields a running CPU with flash mapped and a well-defined handoff. The borrowed component is isolated behind a documented interface (the image header, Chapter 3), so replacing it later is a contained change rather than a rewrite.

The cost of that decision is not zero and it is recorded. UM-NATOS-002 §6 lists three places where the kernel currently relies on L1 or the ROM having done something:

1. **UART0 is already configured.** The kernel writes bytes into the TX FIFO without setting baud rate, pin mux, or line discipline. This works because the ROM configured UART0 at 115200 for its own output. A replacement L1 that did not do this would produce a silent kernel.
2. **Flash cache is configured but unused** at M0. When the kernel started mapping `.rodata` from flash (Chapter 4), this became load-bearing.
3. **CPU clock.** The kernel neither reads nor sets it.

Item 2 came due. Chapter 4 covers the DROM region that depends on it; Chapter 20 covers the flash driver that had to reconcile with the cache it shares a bus with.

1.3 The problem: this hardware cannot isolate anything

The ESP32 cannot provide memory protection between applications. Its MMU translates flash addresses for execute-in-place caching; it does not implement per-process page tables. There is one address space, and any code can write any address.

This is a hardware property, not an implementation gap. It cannot be engineered around at the kernel level. Its direct consequence is that **a native-code application can corrupt the kernel and every other application, and no kernel design prevents this.**

That sentence is the hinge of the whole project. Two responses were considered.

Option A — native applications via an ELF loader

Applications compiled separately, stored on SD, loaded into heap at runtime, relocated, and executed natively.

- Fast — applications run at full CPU speed.
- Precedent exists on ESP32.
- **No isolation whatsoever.** One bad pointer takes down the system.
- Relocation and symbol binding are intricate and a rich source of subtle bugs.

Option B — a bytecode virtual machine (selected)

Applications compiled to a bytecode the kernel interprets. Every memory access executes as a VM instruction, so the interpreter can bounds-check it.

- **Isolation becomes achievable in software** — the VM refuses out-of-range access rather than performing it. This recovers, in the interpreter, the guarantee the silicon will not give.
- The instruction set is ours, so it can be kept small and auditable.
- **Preemption becomes trivial.** The dispatch loop can check a quantum counter between instructions; an application can always be interrupted at a known-safe boundary, with no native context switch required for application tasks.
- Slower than native — the interpretation overhead was, at the time of the decision, real and unmeasured. It has since been derived at roughly 109 cycles per bytecode instruction (Chapter 14).
- Costs flash for the interpreter, and requires a compiler or assembler.

Decision: Option B. The report states the argument in one sentence:

An operating system whose applications can silently corrupt each other is firmware with extra steps, and on hardware without an MMU the VM is the only mechanism that can deliver the property.

The preemption consequence is worth stating separately because it simplifies the kernel enormously: native preemptive context switching is needed only for kernel and driver tasks, not for applications. There are currently nine native tasks and the number is expected to stay small.

The strong form of the guarantee

It is easy to describe the VM as "refusing" illegal accesses, and that undersells it. UM-NATOS-013 §5.2 makes the sharper claim after running a program written for no other purpose than to escape:

The deeper point is what the rogue could not attempt. A VM address is an offset into its own arena, so there is no value it could load that names another application's memory.

Reaching a neighbour is not refused — it is **unrepresentable**. The bounds check exists for the weaker case of a program walking off its own end, which is precisely what was observed.

That distinction — *refused* versus *unrepresentable* — recurs for every resource an application touches, and by the end of Chapter 17 it covers three:

Resource	Confined by	Outside its allocation
Memory	Arena bounds check	Unrepresentable — an address outside the arena cannot be formed
Pixels	Viewport clipping	Clipped before it exists
Input	Viewport test on delivery	Reported as no touch

The design intent is visible in the header that declares arenas:

```
/* nat-os - VM application arenas.
 *
 * An arena is a contiguous block of DRAM with a recorded base and length. It is
 * the unit of memory an application owns, and its bounds are what the bytecode
 * interpreter will check every load and store against (UM-NATOS-001 §4.2).
 *
 * This is the ONLY isolation mechanism this kernel will have. The ESP32 has no
 * MMU paging, so there is no hardware that can be asked to enforce these
 * bounds – the interpreter must do it in software on every access, and it must
 * not be compiled out for speed. Native tasks are not confined by arenas and
 * never will be; they are trusted code.
 */
```

The last sentence is the boundary of the claim, and it is stated everywhere it applies. From `task.h`:

```
* There is no memory protection between them. The ESP32 has no MMU paging, so
* a native task can corrupt any other. Stack guards catch the most common case
* (overflow) but nothing catches a wild pointer.
```

nat-os isolates *applications*. It does not isolate *drivers*, and it never will on this silicon.

1.4 The second constraint: 176 KB, and a display that wants 150 of it

The other number that shapes the architecture is DRAM.

Of the 520 KB of SRAM, the kernel's linker script claims about 176 KB as data memory (Chapter 4 explains why it starts where it does). After kernel data, `.rodata`, task stacks and a 4 KB boot-stack reservation, UM-NATOS-010 §7 measured **166,432 bytes allocatable** at M3 — later 167,680 B once `.rodata` moved to flash, then 158,048 B once `TASK_MAX` rose from 4 to 8 and added 8 KB of statically allocated stacks.

Against that, a full-screen 16-bit framebuffer for a 240×320 panel is:

$$240 \times 320 \times 2 = 153,600 \text{ bytes}$$

Consumer	Bytes	Share of heap
Full 240×320 16-bit framebuffer	153,600	92.3%
Remaining for all arenas	12,832	7.7%

A full-screen framebuffer and any meaningful set of concurrent VM applications cannot coexist in internal DRAM. With one committed, 12.8 KB is left — a single small arena, with no room for a second application.

Four escapes were considered and three closed:

1. **No full framebuffer.** Drive the ILI9341 from a line or tile buffer and push updates incrementally. A 240-pixel line buffer is 480 B.
2. **Reduced colour depth.** 8bpp halves it to 76,800 B — still 46% of the heap.
3. **Partial/dirty-region updates.**
4. **External PSRAM. Confirmed absent.** Closed without a hardware change.

And flash, the obvious fifth option, is closed for reasons of mechanism rather than capacity (UM-NATOS-010 §7.1):

- Flash is memory-mapped **read-only** at runtime. It cannot be the target of a store instruction.
- Writing requires erasing a 4 KB sector and then programming pages, costing tens of milliseconds — roughly three orders of magnitude more than a frame's budget.
- Endurance is ~100,000 erase cycles per sector. A 153,600 B framebuffer spans about 38 sectors; at 30 fps each is erased 30 times per second, exhausting rated endurance in **under an hour**.
- Flash writes require the cache to be disabled, stalling any code executing from flash.

Flash as framebuffer is therefore not slow-but-workable. It destroys the part.

The premise, challenged

Option 1 was selected, and then UM-NATOS-010 §7.2 went further and questioned whether the framebuffer was needed at all:

The ILI9341 contains its own frame memory — a GRAM of 240×320 at 18bpp, roughly 172,800 bytes — and drives the panel from it autonomously. Pixels written to the controller stay there; the host is not refreshing a surface continuously.

The ESP32 therefore never requires a full local copy. It requires a *transfer* buffer: 480 B for one 240-pixel line at 16bpp. The 153,600 B in §7 was never a hardware requirement — it is the cost of keeping a redundant second copy of something the display already stores.

This strengthened option 1 "from *the least bad choice* to *the correct one*", and the display driver was written with no framebuffer anywhere in the system.

The story does not end there, and the ending is instructive enough to preview. A framebuffer for the 3D view *was* eventually added, measured as worthless, defaulted off, and then — after two unrelated defects were fixed — re-measured as clearly worth having and defaulted on. Chapter 18 §18.7 is that reversal, and what made it findable was that the framebuffer was a *runtime switch* rather than a compile-time decision:

Keeping the switch means the claim can be rechecked whenever the display path changes underneath it, rather than being inherited as folklore.

1.5 Explicit non-goals

Stated in UM-NATOS-001 §6 and still true:

- **POSIX compatibility.** Nothing here targets a standard API.
- **Binary compatibility with ESP-IDF or Arduino.** Existing sketches will not run. Chapter 2 explains why this is not merely unimplemented but structurally impossible for anything compiled the usual way.
- **Reuse of the sibling BibleOS/The Word Device codebase.** That is LVGL-on-Arduino firmware and is not a migration target.
- **Memory protection between native tasks.** Impossible on this silicon; protection applies to VM applications only.

1.6 The component inventory, then and now

UM-NATOS-001 §5 listed what existed at the time it was written. The comparison with the present tree is the shape of the whole book:

Component	Layer	At M0	Now
Entry stub (<code>start.S</code>)	L2	Complete	Unchanged, 44 lines
UART console (<code>uart.c</code>)	L3	Output only	Output, polled receive, an output tee
Kernel entry (<code>kmain.c</code>)	L2	Placeholder with self-checks	1,820 lines: nine tasks, six self-tests, the boot report
Link map (<code>linker.ld</code>)	L2	RAM-only	IRAM + DRAM + DROM, <code>.dram.rodata</code> by object file
Build pipeline (<code>build.ps1</code>)	—	Complete	Plus bytecode assembly, windowed-ABI objects, vendor archives
Timer / interrupt controller	L2	Not started	<code>timer.c</code> , <code>intr.c</code> , a routable matrix
Scheduler and context switch	L2	Not started	<code>task.c</code> , 689 lines, three priority levels with ageing

Component	Layer	At M0	Now
Heap allocator	L2	Not started	heap.c with a ten-case structural check
Bytecode VM	L4	Not started	vm.c, 35 opcodes, 12 syscalls
Display, touch, SD drivers	L3	Not started	All three, plus ADC, I ² C, audio, flash, WiFi

1.7 The open architectural questions, revisited

UM-NATOS-001 §7 listed four. Their fates are worth recording because two were answered by measurement and two by not needing an answer.

1. SMP. "Whether the scheduler becomes SMP-aware or core 1 is dedicated to drivers is undecided and should be settled before M2 fixes the scheduler's shape." It was never settled and the scheduler's shape was fixed anyway. APP_CPU has never been started. The absence has cost exactly once, and expensively: the GPIO interrupt-enable field selects *which CPU* receives a pin, and the first choice of bit delivered PENIRQ edges "perfectly to a core that was halted" (Chapter 22).

2. Flash execute-in-place. Resolved for *data* in UM-NATOS-011 (Chapter 4). Never attempted for *code*: the kernel is 116 KB of text against 128 KB of IRAM and still fits. The 132r literal-pool constraint in §4.3 is why an IROM segment would be more than a linker-script change.

3. Application toolchain. Answered by `tools/vasm.py` (Chapter 15) — an assembler, not a compiler. It runs on the host, which keeps the on-device side a pure interpreter with no parsing and no attack surface from untrusted text.

4. Persistence. Answered partially. There is a flash-backed record and a microSD *reader*, and there is no filesystem. Writing FAT remains "a substantial subproject" and remains unwritten. Chapter 21 stops at reading a FAT16 signature at LBA 240.

Next: the calling convention, which is the one decision that had to be made before any code could be written and cannot be revisited cheaply afterward.

2. call0: The Calling Convention That Made a Scheduler Tractable

Sources: docs/UM-NATOS-003-abi.md Code: kernel/start.S, kernel/vectors.S, kernel/window.S, build.ps1

2.1 The decision, and why it is load-bearing

The Xtensa LX6 supports two calling conventions: the **windowed** ABI and the **call0** ABI. nat-os is built entirely with call0.

This is the single change that makes a from-scratch scheduler tractable on this architecture, and it cannot be revisited cheaply once drivers and a VM exist, because ABIs cannot be mixed within a link. It had to be decided before the first line of kernel code was written, and it was — and it was *tested* rather than assumed available (§2.4).

2.2 What the windowed ABI does

The Xtensa architecture optionally implements a **register window**: 64 physical address registers of which 16 are visible, with the visible window rotating on call and return. `ENTRY` rotates the window and allocates a stack frame; `RETW` reverses it.

When the window wraps, the hardware raises a **window overflow exception**, and the handler must spill registers to the stack. Returning past a spilled frame raises **window underflow**, and the handler must reload them.

For an operating system, three consequences follow:

- A context switch cannot simply save 16 registers. Live windows belonging to the outgoing task exist in the physical register file and must be spilled first, or restored state will be silently wrong.
- The spill mechanism must work during the switch itself, which constrains what the switch code may touch.
- Failures do not manifest at the switch. They manifest several returns later, in a function that did nothing wrong, with a corrupted frame.

UM-NATOS-003 §2 names this plainly:

This is the single hardest part of writing an Xtensa kernel and the point at which such projects most often stall.

2.3 What call0 does instead

The call0 ABI does not use windows. It behaves as a conventional RISC calling convention:

Register	Role under call0
a0	Return address
a1	Stack pointer
a2-a7	Arguments / return values
a12-a15	Callee-saved

No ENTRY, no RETW, no window exceptions. **A context switch becomes a conventional register save and restore.**

The entry stub says so in its own header comment:

```
/* nat-os - entry stub.
 *
 * The first instruction of the OS. Runs with whatever state the bootloader
 * left behind, so it establishes its own stack before touching C.
 *
 * Built with -mabi=call0: no ENTRY/RETW, no register windows, no spill
 * exceptions. a1 is the stack pointer and a0 is the return address, exactly
 * as on a conventional architecture. That choice is what makes the context
 * switch later a plain register save rather than window-spill surgery.
 */
```

2.4 Verification: the ABI was tested, not assumed

The availability of call0 for this target was tested against the installed toolchain (xtensa-esp32-elf-gcc, crosstool-NG esp-14.2.0_20241119) **before any project code was written.**

Test input:

```
int add(int a, int b) { return a + b; }
int chain(int x)      { return add(x, add(x, x)); }
```

Compiled at -O2 under each ABI. The relevant emitted instructions:

-mabi=windowed (default)	-mabi=call0
entry sp, 32	(none)
retw.n	ret.n

The windowed build allocates a window frame on entry and returns with RETW. The call0 build emits neither — a plain ret.n, with no window management at all.

Result: call0 is supported by this toolchain for this target. Confirmed again in the linked kernel; the disassembly of _start shows the .bss clear loop using bgeu / s32i.n / addi.n with no entry or retw present.

That loop, as written:

```
/* Zero .bss. The bootloader only copies segments that have file content,
 * so uninitialised data arrives as whatever was in RAM at reset. */
movi    a2, _bss_start
movi    a3, _bss_end
movi    a4, 0
.Lbss_loop:
    bgeu    a2, a3, .Lbss_done
    s32i    a4, a2, 0
    addi    a2, a2, 4
    j       .Lbss_loop
.Lbss_done:

/* Into C. kmain must not return. */
call0    kmain
```

`call0 kmain` — the instruction that gives the ABI its name — rather than `call4 / call8 / call12`, which rotate the window.

2.5 The costs, all three of them

2.5.1 ROM routines are unusable

The ESP32 ROM exports hundreds of functions — memory routines, SPI helpers, cryptography — all compiled for the **windowed** ABI. Calling them from `call0` code corrupts state.

The report assessed the impact as low, because a from-scratch OS writes its own drivers by definition, and named the concrete instance:

```
kernel/uart.c writes UART registers directly instead of calling the ROM's output
routines, specifically for this reason.
```

The file confirms it:

```
/* nat-os - raw UART0 output.
 *
 * Register-level, no ROM calls (ROM routines are windowed-ABI and this kernel
 * is call0). ...
 */
```

The report also noted that "an assembly wrapper that switches conventions is possible, but none is currently needed". That sentence stayed true for two days and then stopped being true, decisively — see §2.7.

2.5.2 All code must use one ABI

Every object in the link must agree. Any third-party library compiled as windowed cannot be linked in. This closes off casual reuse of ESP-IDF components.

It also produced an unexpected benefit three weeks later. UM-NATOS-024 §2, on the ADC:

ADC2 is unusable on this part whenever WiFi is running, which is the standard reason to avoid it. **That reason does not apply here.** The `-mabi=call0` build cannot link Espressif's precompiled radio libraries at all (UM-NATOS-003 §5.1), so WiFi will never run and ADC2 is permanently free.

That reasoning was correct when written and was overtaken by §2.7, which is a pleasing failure mode for a constraint.

2.5.3 Slightly larger code

Without windows, callee-saved registers are spilled to the stack conventionally, so code is typically marginally larger. At 1,124 bytes of text this was "not currently measurable and not expected to matter". At the present 116,692 bytes against 131,072 of IRAM it still has not mattered, though the margin is now thinner than it was.

2.6 The payoff, made concrete

With `call0`, a task switch is:

1. Push `a0`, `a12` – `a15` and any live caller-saved registers onto the outgoing task's stack.
2. Store the resulting stack pointer in the outgoing task's control block.
3. Load the incoming task's stack pointer.
4. Pop the registers.
5. `ret` — which returns into the incoming task.

No spilling, no window state, no exception interaction. UM-NATOS-003 §6 predicted this would make M2 "a manageable milestone rather than the project's wall".

That prediction was half right, and the way it was wrong is Chapter 8. M2 did consume twelve build cycles and three wrong hypotheses — but not one of them had anything to do with register windows. The defect was `PS.EXCM` disabling the zero-overhead loop, which is an entirely different mechanism and one that would have bitten a windowed kernel identically.

The interrupt entry sequence in `vectors.S` is where the saving is visible. It saves twenty-one words and nothing else:

```
_handler_level3:
    addi    a1, a1, -96
    s32i    a0, a1, 0
    s32i    a2, a1, 4
    /* ... a3 through a15 ... */
    rsr.sar a2
    s32i    a2, a1, 60
    rsr.epc3 a2
    s32i    a2, a1, 64
    rsr.eps3 a2
    s32i    a2, a1, 68
```

UM-NATOS-008 §3.2 states the accounting directly:

This is the entire context-save cost of the call0 decision. Under the windowed ABI the same handler would additionally have to spill live register windows before touching the register file.

2.7 The reversal: making windowed code run anyway

Two days after the report declared ROM routines unusable and vendor libraries unlinkable, both became possible — without changing the kernel's ABI at all.

The mechanism is that the window vectors had never been implemented. `vectors.S` recorded the intent:

Because the kernel is built `-mabi=call0` there are no register windows, so the window overflow/underflow vectors are never taken by ITS OWN code.

and then recorded the correction, which is the interesting part:

```
* They were described here as "left as traps", which was the intent and not
* what the linker script did: nothing was placed before _vecbase + 0x1C0, so
* slots 0x000..0x180 were zero-filled and a window exception would have
* executed zeros. PS.WOE is set by the ROM, so the mechanism was armed with
* nothing behind it.
*
* They are now real handlers — see kernel/window.S — which is what lets
* WINDOWED code run here at all.
```

`window.S` implements the six window vectors. Its header makes the safety argument for adding them:

```
* ---- why this is safe to add -----
*
* These vectors are currently EMPTY. ...
*
* PS.WOE is already SET — the ROM leaves it that way and task.c preserves it
* when fabricating task frames — so the mechanism is armed and only the
* handlers are missing.
*
* Filling empty vectors is therefore purely additive. Code that never triggers
* a window exception cannot tell the difference, which is the whole reason this
* can go in without risking anything that already works.
```

That `task.c` detail is not incidental. Task creation inherits PS from the running kernel rather than fabricating one:

```
/* Resume at the entry point, with interrupts admitted. PS is taken
 * from the running kernel with INTLEVEL forced to 0, so the task
 * inherits the same execution mode rather than a guessed one — the
 * ROM leaves WOE and CALLINC set, and fabricating a different PS would
 * put the task in a subtly different state from its creator. */
frame[TASK_FRAME_IDX_EPC3] = (uint32_t)entry;
frame[TASK_FRAME_IDX_EPS3] = xt_get_ps() & ~0xFu;
```

A decision made in Chapter 8 for reasons of caution turned out, in Chapter 27, to be the reason windowed code could run inside a task at all.

The handlers themselves

Six vectors, taken verbatim rather than derived, and the reason is stated:

```
* Taken verbatim from Espressif's xtensa_vectors.S rather than derived. A wrong
* register number here does not fault – it silently reloads a caller's frame
* with the wrong values, which is the least debuggable failure this project
* could construct. This is the third time today that fetching a definition
* beat recalling one (UM-NATOS-024 §4).
```

The `CALL4` pair is short enough to show whole:

```
.section .vectors.window.of4, "ax"
.align 4
.global _WindowOverflow4
_WindowOverflow4:
s32e    a0, a5, -16          /* save a0 to call[j+1]'s stack frame */
s32e    a1, a5, -12
s32e    a2, a5, -8
s32e    a3, a5, -4
rftwo                                /* rotate back to call[i] */

.section .vectors.window.uf4, "ax"
.align 4
.global _WindowUnderflow4
_WindowUnderflow4:
l32e    a0, a5, -16
l32e    a1, a5, -12
l32e    a2, a5, -8
l32e    a3, a5, -4
rftwu
```

`CALL8`'s overflow handler is where a subtle ABI requirement lives, and it cost this project a full debugging session in Chapter 27:

```
_WindowOverflow8:
s32e    a0, a9, -16
l32e    a0, a1, -12          /* a0 <- call[j-1]'s sp */
s32e    a1, a9, -12
s32e    a2, a9, -8
s32e    a3, a9, -4
s32e    a4, a0, -32
s32e    a5, a0, -28
s32e    a6, a0, -24
s32e    a7, a0, -20
rftwo
```

That second instruction — `l32e a0, a1, -12` — fetches the *caller's* stack pointer from a slot every windowed frame is required to carry. `ENTRY` does not write it; the caller's prologue does. `Call0` code has no reason to know that, and when `nat-os` first called into the `PHY` blob on a private stack, nothing had written it. The handler loaded

a fresh `.bss` zero and spilled to address `0 - 32`. Chapter 27 §27.5 is the full story, including why the reported fault address was never where the store was.

The verification: a checksum over handler invocations

`window.h` records how the handlers were proved rather than merely observed not to crash:

```
/* Calls a windowed-ABI function from call0 code and returns what it returned.
 *
 * win_probe(n) recurses n deep and returns n. With CALL8 the 64 physical
 * registers are exhausted after 8 frames, so any depth above that MUST take
 * window overflow exceptions on the way down and underflows on the way back.
 * The return value is therefore a checksum over every handler invocation: a
 * handler that reloads the wrong register returns the wrong number rather than
 * merely failing to crash. */
uint32_t win_call_probe(uint32_t depth);
```

Then three escalating proofs, each with a distinct thing it establishes:

```
/* Calls vendor_probe() in vendor/windowed/, which is compiled -mabi=windowed by
 * the same compiler that builds Espressif's libraries. Standing proof that this
 * kernel can call vendor-ABI code, not just hand-written windowed assembly. */
uint32_t win_call_vendor(uint32_t depth, uint32_t seed);

/* Calls a three-argument WINDOWED function at an arbitrary address.
 *
 * Espressif's ROM holds hundreds of routines at fixed addresses, all windowed,
 * needing no linking and no environment. This makes every one of them callable.
 * crc32_le lives at 0x4005CFEC and is a pure function, which is why it is the
 * first vendor code this kernel runs. */
uint32_t rom_call3(uint32_t fn, uint32_t a, uint32_t b, uint32_t c);

/* Calls a windowed function that calls BACK into call0 code on every
 * iteration -- proof the reverse bridge round-trips. */
uint32_t win_call_bridge(uint32_t fn_add, uint32_t depth);
```

`crc32_le` was chosen as the first ROM call because it is a *pure function with a known answer*. A CRC-32 that comes back matching the textbook value cannot have been produced by a broken window handler.

The commit history reads as a ladder:

```
window: implement the register-window vectors; windowed-ABI code now runs
window: link and call a real -mabi=windowed object from the call0 kernel
window: run Espressif ROM code -- crc32_le returns the textbook CRC-32
```

Three commits, three independent levels of the same claim.

2.8 Mixed ABIs in one image

The build compiles two sets of objects with different ABIs and links them together. From `build.ps1`:

```
# Objects built for the WINDOWED ABI, linked alongside the call0 kernel.
#
# Everything in kernel/ is -mabi=call0 and always will be. This directory holds
# code that is not: the ABI every precompiled Espressif library uses. Mixing the
# two in one image is a link-time question, not a compile-time one, and the
# register-window handlers in kernel/window.S are what make the resulting calls
# actually work at run time.
$winsrc = Get-ChildItem "$root\vendor\windowed\*.c" -ErrorAction SilentlyContinue
if ($winsrc) {
    Write-Host "== compiling windowed ABI ==" -ForegroundColor Cyan
    $wflags = @("-mabi=windowed", "-mlongcalls", "-ffreestanding", "-fno-builtin",
        "-fno-stack-protector", "-Os", "-Wall", "-Wextra", "-std=c11")
    foreach ($src in $winsrc) {
        $obj = Join-Path $build ($src.BaseName + ".o")
        Write-Host (" {0} [windowed]" -f $src.Name)
        & $gcc @wflags -c $src.FullName -o $obj
        if ($LASTEXITCODE -ne 0) { throw "compile failed: $($src.Name)" }
        $objs += $obj
    }
}
```

The two halves meet through explicit bridges in `window.S`. The WiFi OSI table is the largest consumer:

```
libpp (windowed) → wifi_osi.c entry (windowed, does nothing)
                  → w2c_callN (window.S)
                  → wifi_osi_impl.c (call0: heap, scheduler, tick)
```

declared as:

```
/* Windowed -> call0 bridges, for the OSI table's bodies. See window.S. */
uint32_t w2c_call0f(uint32_t fn);
uint32_t w2c_call1(uint32_t fn, uint32_t a);
uint32_t w2c_call2(uint32_t fn, uint32_t a, uint32_t b);
uint32_t w2c_call3(uint32_t fn, uint32_t a, uint32_t b, uint32_t c);
```

The design note in `wifi_osi_impl.c` explains why the split is where it is:

```
* call0, deliberately. The OSI table itself must be windowed because libpp
* calls it, but the table's only job is to forward: the work happens here,
* where the kernel's heap, scheduler and tick can be used directly. Each
* windowed entry crosses back through w2c_callN() in window.S.
*
* That split is the design. Writing these bodies in the windowed file would
* mean either duplicating the kernel's primitives or calling into them across
* a boundary that does not permit it – the fault that put IllegalInstruction
* at 0x4008a810 the first time it was tried.
```

2.9 The build flags

Set in `build.ps1` for compile, and `-mabi=call0` must appear on the **link** line as well, because the driver uses it to select compatible internal libraries:

```
$cflags = @(
    "-mabi=call0", "-mtext-section-literals", "-mlongcalls",
```

```

"-ffreestanding", "-fno-builtin", "-fno-stack-protector",
"-fno-tree-loop-distribute-patterns",
"-Os", "-Wall", "-Wextra", "-std=c11",
"-I", "$root\kernel"
)

```

Flag	Purpose
-mabi=call0	Select the non-windowed ABI. The load-bearing flag.
-mtext-section-literals	Place literal pools inside .text so they land in IRAM. Without this they land in a section the linker script does not map, and 132r loads fault
-mlongcalls	Permit calls beyond the short-displacement range

Chapter 5 covers the rest.

2.10 Residual risk

UM-NATOS-003 §8 listed two, and both have now been paid.

Exception and interrupt handlers. "Their entry sequence must save state explicitly and correctly; call0 simplifies but does not eliminate this work." Correct. Chapter 8's defect was in exactly that sequence, though not in the part that saves registers.

Toolchain internals. "libgcc routines pulled in for operations such as 64-bit division must also be call0. The current build links -nostdlib and uses none, but that will change and should be re-checked at that point."

It changed. Two ways:

1. kstring.c had to be written because GCC synthesises calls to memcpy and memset from ordinary C — struct assignments, large local initialisers — even under -fno-builtin (Chapter 14 §14.9).
2. The WiFi link needed __divsf3 from libgcc, and needed Espressif's ROM symbol table without letting it displace the kernel's own memcpy. The comment in build.ps1 records what went wrong on the first attempt:

```

# The first attempt linked Espressif's newlib and libgcc ROM scripts too,
# which define memcpy and sprintf by BARE ASSIGNMENT rather than PROVIDE.
# That silently redirected the kernel's own call0 memcpy to a windowed ROM
# routine and panicked the board on boot.

```

The fix was to rename those references inside the vendor archive with objcopy, answer them in vendor/windowed/, and link only esp32.rom.ld — "whose every entry is PROVIDE, verified".

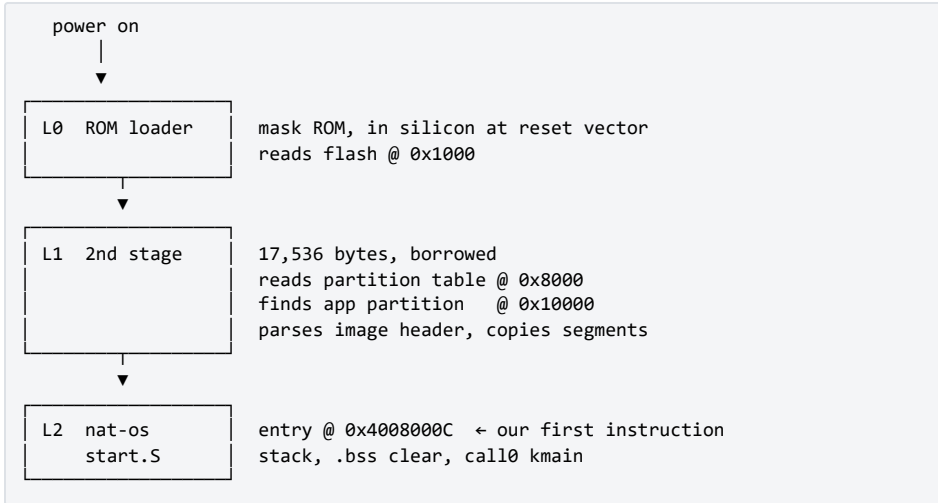
| Nothing the kernel defines can be displaced, because nothing strong is defined at all.

Next: how a .bin file on flash becomes the first instruction of _start.

3. Boot: From Silicon to `_start`

Sources: docs/UM-NATOS-002-boot-chain.md, docs/UM-NATOS-006-m0-verification.md
Code: kernel/start.S, kernel/linker.ld, vendor/README.md, build.ps1

3.1 Four stages



Stage L0 — mask ROM

Executes from ROM on reset. Determines boot mode from strapping pins (GPIO0 high = normal flash boot), configures the SPI flash interface, and loads the second-stage bootloader from flash offset 0x1000. It also brings up UART0 at 115200 baud for its own diagnostic output — a side effect nat-os currently depends on.

Not modifiable.

Stage L1 — second-stage bootloader

Borrowed. Its responsibilities, as they matter to nat-os:

1. Read the partition table at 0x8000.
2. Locate the application partition (offset 0x10000).
3. Read the image header at that offset.
4. Copy each declared segment to its declared load address.
5. Verify the image checksum and SHA-256 hash.
6. Jump to the declared entry point.

Plus one thing not in that list that became load-bearing later: it recognises load addresses in the flash-mapped ranges, programs the flash MMU to point at the image data in place, and enables the cache *before* jumping. Chapter 4 §4.4 depends entirely on this.

The interface between L1 and nat-os is entirely the image header. Nothing else is shared — no symbols, no runtime, no calling convention. This is why the bootloader is replaceable without touching kernel code.

`vendor/README.md` records what the binaries are and how to reproduce them:

```
| file | size | sha256 (first 32) |
|---|---|---|
| `bootloader.bin` | 17536 B | `3d234a7471f67b013686dabd4dee7c1f` |
| `partitions.bin` | 3072 B | `6a88d59601a83a16a19a08114b59d338` |
```

They are unmodified ESP-IDF build artefacts, Apache 2.0, redistributed under that licence rather than under nat-os's MIT. Reproducing them needs a stock `esp32dev` PlatformIO project and nothing else:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
```

They were originally read from an unrelated project's build directory, which UM-NATOS-007 §8.4 flagged as a coupling worth removing — "deleting or rebuilding that project breaks this one's flash step". They now live in `vendor/`, and `-Vendor <path>` overrides.

3.2 The image format

`esptool elf2image` converts the linked ELF into the format L1 expects. The header declares, for each segment, a length and a load address, plus one entry point for the whole image.

The M0 build:

```
Image version: 1
Entry point: 0x4008000C
2 segments
  Segment 1: len 0x00188 load 0x3FFB0000 file_offs 0x018 [BYTE_ACCESSIBLE, DRAM]
  Segment 2: len 0x002E0 load 0x40080000 file_offs 0x1A8 [IRAM]
Checksum: 0x8F (valid)
Validation hash: 8e3a272a...5c72f3 (valid)
```

Both load addresses correspond directly to MEMORY regions in `kernel/linker.ld`. After `.rodata` moved to flash (Chapter 4), the same build produces three:

```
Segment 1: len 0x004e0 load 0x3f400020 file_offs 0x00000018 [DROM]
Segment 2: len 0x00004 load 0x3ffb0000 file_offs 0x00000050 [BYTE_ACCESSIBLE, DRAM]
```

```
Segment 3: len 0x01510 load 0x40080000 file_offs 0x0000050c [IRAM]
```

There is no magic in this format. It is a list of "copy N bytes to address A", followed by "jump to address E". Any producer that emits a conforming header will boot.

Why the entry point is `+0x0C`

The entry point is `_start + 0x0C` rather than `+0x00` because `-mtext-section-literals` places the literal pool at the head of the section, and the first 12 bytes are literals. Xtensa loads large constants through a literal pool referenced by `132r`.

This produces a disassembly artefact worth remembering when reading a fault address:

`objdump` cannot distinguish literals from instructions, so disassembling from `0x40080000` shows plausible but meaningless instructions before the real code.

3.3 The entry stub, in full

`kernel/start.S` is 44 lines and does three things.

```
.section .text.start, "ax"
.align 4
.global _start
.type _start, @function

_start:
/* Stack first – nothing below this line may call C. */
movi    a1, _stack_top
/* Xtensa wants the stack 16-byte aligned. */
movi    a2, ~15
and     a1, a1, a2

/* Zero .bss. The bootloader only copies segments that have file content,
 * so uninitialised data arrives as whatever was in RAM at reset. */
movi    a2, _bss_start
movi    a3, _bss_end
movi    a4, 0
.Lbss_loop:
bgeu    a2, a3, .Lbss_done
s32i    a4, a2, 0
addi    a2, a2, 4
j       .Lbss_loop
.Lbss_done:

/* Into C. kmain must not return. */
call0   kmain

/* If it does return anyway, park rather than execute whatever follows. */
.Lhang:
j       .Lhang

.size _start, . - _start
```

Three observations.

The stack comes first and the comment says why. "Nothing below this line may call C" is an executable constraint, not a style note: the `.bss` clear is written in assembly rather than C precisely because C requires a stack.

.bss is zeroed here because nobody else will. It is declared `NOLOAD` in the linker script, so it costs no image bytes and arrives as whatever was in RAM at reset. M0's third assertion tests exactly this.

The final `j` .Lhang is defensive. `kmain` is documented as never returning, and if it does anyway, parking is better than executing whatever the linker happened to place next.

`_start` is placed first in `.text` deliberately:

```
.text : ALIGN(4)
{
    /* Entry stub first so the image entry point is predictable. */
    KEEP(*(.text.start))
}
```

3.4 The measured boot trace

Captured 2026-08-14 from the target board on COM5 at 115200 baud, with the serial port opened *before* reset so no output was missed:

```
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1184
load:0x40078000,len:13232
load:0x40080400,len:3028
entry 0x400805e4

=====
nat-os milestone 0 - kernel alive
=====
```

Reading it

Line	Meaning
ets Jul 29 2019	ROM build date — identifies the silicon revision's ROM
rst:0x1 (POWERON_RESET)	Clean cold boot. Diagnostically important: a watchdog or panic reset shows a different code, and is the first thing to check when a kernel change stops booting
boot:0x13 (SPI_FAST_FLASH_BOOT)	Normal flash boot; GPIO0 was high
mode:DIO, clock div:2	Flash interface configuration
load:0x... ×3	The bootloader loading itself , not nat-os

Line	Meaning
entry 0x400805e4	Entry into the <i>bootloader</i> , not our kernel

That last pair is the trap:

The three `load:` lines and the `entry` line are frequently misread as the kernel loading. They are not. They are L1 placing its own segments before it has even read our partition. Our own load is silent — L1 does not log it — and the first evidence of `nat-os` is the banner.

Misreading them cost this project a false alarm; Chapter 4 §4.5 is that story.

What the banner alone proves

The banner appearing at all proves the header was well-formed, the checksum matched, both segments were copied, and the CPU reached `0x4008000C`. That is four independent things, and it is also the ceiling of what a banner can prove — a point Chapter 12 makes at length after the shell was signed off on the same evidence and turned out to have a broken receive path.

3.5 Flash layout

Offset	Contents	Size	Origin
0x1000	Second-stage bootloader	17,536 B	Borrowed
0x8000	Partition table	3,072 B	Borrowed
0x10000	nat-os kernel image	1,216 B at M0, 37,248 B now	Ours
0x200000	Persistent record sector	4,096 B	Ours (Chapter 20)
0x201000	Message store sector	4,096 B	Ours (Chapter 26)

Everything from `0x10000` onward is available to the project. The two borrowed regions total under 21 KB of 4 MB.

The 2 MB offset for data is not arbitrary. `flash.c` refuses any erase or write below `FLASH_DATA_ADDR`, and UM-NATOS-018 §2.1 gives the reason:

This is not defence against a subtle bug. It is defence against the specific failure where a wrong constant erases the bootloader and the board stops enumerating over USB — which turns a five-minute fix into a recovery job. Every failure in this driver stays recoverable over serial.

3.6 Capturing boot output, and the harness that lied

Attaching a monitor *after* reset loses the banner, because the kernel prints within milliseconds of the jump. The capture harness opens the port first and then pulses the auto-reset circuit:

- `RTS` → `EN` (reset), asserted then released

- `DTR` → `GPIO0` (boot mode), held high for normal flash boot

Sequence: open port → `DTR=False`, `RTS=True` → 150 ms → `RTS=False` → read.

There is a trap here that this project fell into twice, and the second time was by someone who had just read the write-up of the first time. From `README.md`:

Note that opening a serial port on Windows asserts `DTR`, which resets the board; `deassert dtr` and `rts before open()`. UM-NATOS-017 §5 is the story of learning that twice.

The symptom was three consecutive runs, across different builds and different user actions, reporting **exactly 150 samples**.

A counter that lands on an identical value every time is not accumulating. It is measuring a fixed window.

Every capture described as "no reset" was rebooting the board, waiting for it to boot, and then reading roughly twelve seconds of a system nobody was touching. Setting `dtr` and `rts` before `open()` fixed it; the same read then reported 647,406 samples and no boot banner.

Two published findings were retracted as a result, and the fix that mattered was not the code change but the *verification*:

What is new is the verification — uptime is now sampled twice across a connection and confirmed to increase, rather than the absence of a reset being assumed. Knowing the failure mode was not enough; a check was needed.

Chapter 28 catalogues this alongside its six siblings.

3.7 Failure modes, and the first diagnostic step

Symptom	Most likely cause
No output at all	Image checksum invalid, or entry point outside a loaded segment
ROM trace then silence	Kernel reached but faulted before UART output — suspect stack or <code>.bss</code>
Boot loop with <code>rst:0x3</code> / <code>rst:0xc</code> / <code>rst:0x10</code>	Watchdog — kernel hung before feeding or disabling it
Boot loop with <code>rst:0x7</code>	TIMG0 watchdog — the hang detector fired (Chapter 12)
Garbled output	Baud mismatch

In every case the **reset reason on the first line** is the highest-value single datum, which is why the capture procedure resets the board explicitly rather than attaching to an already-running system.

`rst:0x10` (`RTCWDT_RTC_RESET`) deserves special mention because it was misread for three milestones. UM-NATOS-006 §6 originally reasoned that surviving an 8-second

capture window implied no watchdog was armed. That inference was wrong and is corrected in place in the report:

CORRECTED 2026-08-14. The inference was wrong. The RTC watchdog **is** armed by the second-stage bootloader, which expects the application to take ownership of it. M0 survived its capture window by luck of timing, not because nothing was running.

Chapter 12 covers what the kernel does about it now.

3.8 Reproduction

```
cd C:\src\nat-os
.\build.ps1 -Flash -Port COM5
# then capture with the port opened before reset
```

One caution, learned expensively and recorded as a standing rule (UM-NATOS-018 §5):

`build.ps1 builds`; `build.ps1 -Flash` builds *and* flashes.

Two hypotheses were once recorded as tested against a board that had never been reflashed, because the invocation used named a script that does not exist and PowerShell's error went to a filtered stream. Every hypothesis returned bit-identical output, which was read as "none of these are the cause" when it meant "no experiment has run yet". Chapter 20 §20.5 is the full account. The mitigation is narrow and dull:

the flash step now always shows its `Hash of data verified.` line, and that line is checked before any capture is interpreted.

Next: where the segments the bootloader copies actually land, and why one of them appeared to collide with the bootloader itself.

4. The Memory Map, and an Overlap That Was Not There

Sources: docs/UM-NATOS-004-memory-map.md, docs/UM-NATOS-011-flash-cache.md Code: kernel/linker.ld

4.1 The regions

The ESP32 has 520 KB of internal SRAM in three blocks, some visible at more than one address depending on whether it is accessed as instruction or data memory.

Block	Size	Data (DRAM) view	Instruction (IRAM) view
SRAM0	192 KB	—	0x40070000–0x400A0000
SRAM1	128 KB	0x3FFE0000–0x40000000	0x400A0000–0x400C0000
SRAM2	200 KB	0x3FFAE000–0x3FFE0000	—

External flash is additionally mapped through the cache:

Purpose	Address base
Flash instruction (execute in place)	0x400D0000
Flash read-only data	0x3F400000

These are **transcribed**, not verified, and the report says so explicitly:

These boundaries are transcribed from the ESP32 Technical Reference Manual's system address mapping and should be checked against it before being relied on for anything beyond the current layout. The addresses nat-os actually uses are confirmed working on hardware; the surrounding region boundaries are not independently verified.

That distinction is the whole discipline of Chapter 0b applied to an address table, and it matters: three separate defects in this project came from a constant recalled rather than fetched (Chapters 22 and 23).

Two properties that constrain everything

IRAM requires aligned 32-bit access. Byte or unaligned halfword loads from IRAM fault. Read-only data — notably string literals, which string handling touches unaligned constantly — must therefore live in DRAM or flash, not IRAM, even though it is logically part of the program image.

Part of SRAM0 serves as instruction cache. When flash execute-in-place is enabled, a portion of SRAM0 is consumed and unavailable as IRAM. nat-os does not

execute from flash, so this does not currently apply, and it is listed as unmeasured in §4.7.

4.2 The layout, as declared

```
MEMORY
{
  /* Internal SRAM0 – instruction RAM. */
  iram (rwx) : ORIGIN = 0x40080000, LENGTH = 0x20000 /* 128 KB */

  /* Internal SRAM2 – data RAM, below the ROM's reserved region. */
  dram (rw)  : ORIGIN = 0x3FFB0000, LENGTH = 0x2C200 /* ~176 KB */

  drom (r)   : ORIGIN = 0x3F400020, LENGTH = 0x400000 - 0x20
}
```

Section	Region	Rationale
.vectors	IRAM, first	VECBASE must be 1024-byte aligned
.text (code + literals)	IRAM	Executable
.dram.rodata	DRAM	Must survive a disabled flash cache — §4.5
.flash.rodata	DROM	Everything else read-only
.data	DRAM	Writable initialised data
.bss	DRAM	Zeroed by start.S; NOLOAD, so it costs no image bytes
Heap	DRAM	Between .bss and the boot stack reservation
Stack	DRAM, top-down from _stack_top	ORIGIN + LENGTH = 0x3FFDC200

Why DRAM starts at 0x3FFB0000 and not 0x3FFAE000

SRAM2 begins 8 KB lower. The origin is set higher because the upper region of DRAM (above roughly 0x3FFE0000) is used by the ROM for its own stack and data during boot; starting low and ending below that region avoids being overwritten while the bootloader is still running. The chosen length places the top at 0x3FFDC200, comfortably clear.

Confirmed on hardware at M0: .bss spanning 0x3FFB0188–0x3FFB018C, stack top 0x3FFDC200, executing code observed at 0x40080088.

Stack top 0x3FFDC200 — equals `ORIGIN(dram) + LENGTH(dram) = 0x3FFB0000 + 0x2C200`, confirming the linker script's arithmetic reached the running image.

That is a small thing and it is a real check. A linker script can be arithmetically correct and still not be the script that was used.

4.3 The vectors section

The CPU dispatches exceptions and interrupts to fixed offsets from `VECBASE`, so slots are placed by *absolute position* in the linker script, not by declaration order:

```
.vectors : ALIGN(1024)
{
    _vecbase = .;

    . = _vecbase + 0x000;
    KEEP*(.vectors.window.of4))
    . = _vecbase + 0x040;
    KEEP*(.vectors.window.uf4))
    . = _vecbase + 0x080;
    KEEP*(.vectors.window.of8))
    . = _vecbase + 0x0C0;
    KEEP*(.vectors.window.uf8))
    . = _vecbase + 0x100;
    KEEP*(.vectors.window.of12))
    . = _vecbase + 0x140;
    KEEP*(.vectors.window.uf12))

    . = _vecbase + 0x1C0;
    KEEP*(.vectors.level3))

    . = _vecbase + 0x300;
    KEEP*(.vectors.kernel))

    . = _vecbase + 0x340;
    KEEP*(.vectors.user))

    . = _vecbase + 0x3C0;
    KEEP*(.vectors.double))

    . = _vecbase + 0x400;
} > iram
```

The architectural offsets, from the script's own comment:

* 0x180 level 2	0x1C0 level 3	0x200 level 4
* 0x240 level 5	0x280 debug	0x2C0 NMI
* 0x300 kernel exc	0x340 user exc	0x3C0 double exc

Only the slots the kernel uses are populated; the rest are left as gap. A stray dispatch into a gap is a bug, and the resulting fault is caught by the panic vectors — which is the design intent stated in the script.

The window slots at `0x000`–`0x180` were empty until Chapter 2 §2.7, and the linker script now records the correction:

```
* These were previously left EMPTY – the section jumped straight to 0x1C0,
* so a window exception executed zero-filled space. Nothing in this kernel
* takes them (it is -mabi=call0 throughout), but PS.WOE is set by the ROM,
* so the mechanism was armed with no handlers behind it.
```

Verified before flashing, not after

A misplaced vector produces a silent reset with no output, which is close to undiagnosable without a debugger. UM-NATOS-008 §4.1 records the pre-flight check performed on the linked ELF using `nm`:

```
_vecbase = 0x40080000    1024-aligned: True

symbol          address      offset  expected
_vector_level3   0x400801C0    0x01C0   OK
_vector_kernel   0x40080300    0x0300   OK
_vector_user      0x40080340    0x0340   OK
_vector_double    0x400803C0    0x03C0   OK

_handler_level3 = 0x40080924    (1892 bytes from vector; j range ~128KB)
```

Each slot is 64 bytes and contains a single `j` to a handler in `.text`. A jump rather than an address load, because `j` has ± 128 KB range — sufficient for the whole kernel — and avoids needing a literal pool inside a fixed-offset slot.

4.4 Flash-mapped read-only data

`.rodata` originally lived in DRAM because IRAM cannot serve unaligned reads. That constraint is specific to IRAM. Flash mapped through the *data* cache has no such restriction, so the reasoning that forced the original placement simply does not carry over.

Moving it required **zero lines of C**. The change is entirely in the link map, because the borrowed bootloader already does the hard part: it recognises load addresses in the flash-mapped ranges, programs the flash MMU to point at the image data in place, enables the cache, and only then jumps.

The `0x20` that matters

```
drom (r) : ORIGIN = 0x3F400020, LENGTH = 0x400000 - 0x20
```

The linker script explains the offset at length, because getting it wrong produces an image that links, flashes, and then reads garbage:

- * The `0x20` offset is not arbitrary. The MMU maps in 64 KB pages, and the
- * mapped page begins at the start of the image — which is occupied by the
- * 24-byte image header plus an 8-byte segment header. Starting the region
- * 32 bytes in makes the virtual address agree with the flash offset, which is
- * the congruence `esptool` and the bootloader both assume.

Confirmed in the generated image: `file_offs` reports each segment's *header*, so the DROM payload begins at `0x20`. Flashed at `0x10000`, the data sits at `0x10020` and maps to `0x3f400020`. The congruence holds.

Literal pools deliberately stay in IRAM

```
* Literal pools are deliberately NOT moved. __mtext-section-literals keeps
* them beside the code in IRAM because __l32r addresses backwards within
* 256 KB of the PC; a literal in flash would be out of range and the link
* would fail – or worse, not fail.
```

"or worse, not fail" is the operative phrase, and UM-NATOS-011 §2.2 expands:

This would most likely fail at link time — but "most likely" is not "certainly", and a literal silently resolving to the wrong address is a substantially worse outcome than a failed link.

The measurement

Check	Before	After
<code>.rodata</code> location	DRAM 0x3ffb0000	flash, mapped 0x3f400020
<code>.bss</code> start	0x3ffb04f0	0x3ffb0010
Heap usable	166,432 B	167,680 B
Image size	6,720 B	6,736 B

The heap grew by 1,248 bytes — **exactly** the size of the section that moved. An inexact match would have indicated something else shifting at the same time.

Image size grew by 16 bytes: one additional segment header plus alignment. The `.rodata` bytes were always in the image; only their destination changed.

The change is also close to self-verifying:

every string the kernel prints is `.rodata`. If the mapping were wrong, the banner would be garbage or the board would fault before producing output.

1,248 bytes is 0.7% of the heap and was never the point. The point was what it unblocks: fonts, sprites and UI bitmaps at zero DRAM cost, and — eventually — application bytecode executing in place so an arena holds only an application's *data*. The first of those arrived immediately; the 5×8 font is 475 bytes of `.rodata` and costs zero DRAM, which is why the flash-cache work was scheduled before the display driver rather than after.

4.5 The section that must not be in flash

Mapping `.rodata` into flash introduced a hazard, and UM-NATOS-011 §4 predicted it in writing before anything triggered it:

Cache-off windows become hazardous. Any future code that writes flash must disable the cache while doing so. During that window, *any* access to `.rodata` faults — including a string in an interrupt handler, and including the panic handler's own messages. `nat-os` has no equivalent [of `IRAM_ATTR`] yet and does not need one until it writes flash, but the

panic path is the case to fix first: a fault handler that cannot print because the cache is off is worse than no fault handler.

The prediction came due in Chapter 20, and the answer is the `.dram.roddata` section:

```
/* Read-only data that must survive a disabled flash cache.
 *
 * UM-NATOS-011 §4 records the hazard: writing flash requires the cache to be
 * turned off, and while it is off EVERY access to flash-mapped .roddata
 * faults. That includes the panic handler's own message strings – a fault
 * during a flash write would reach a panic handler that then faults trying to
 * describe itself, which is the worst failure mode available to a kernel with
 * no debugger.
 *
 * Selecting by object file rather than by attributing individual strings:
 * string literals are awkward to place by hand, and a rule that catches an
 * object's roddata wholesale cannot be defeated by someone adding a new
 * message later and forgetting the annotation. */
.dram.roddata : ALIGN(4)
{
    *panic.o(.roddata .roddata.*)
    *uart.o(.roddata .roddata.*)
    *watchdog.o(.roddata .roddata.*)

    /* flash.o and store.o run with the flash chip busy: any flash-mapped read
     * here would hit a chip that cannot answer. */
    *flash.o(.roddata .roddata.*)
    *store.o(.roddata .roddata.*)
    . = ALIGN(4);
} > dram
```

The by-object-file selection is the interesting part, and UM-NATOS-018 §3.1 argues it explicitly:

The linker rule selects **by object file** rather than by annotation precisely so that adding a string to one of these files cannot quietly break it. An annotation can be forgotten on a new string; a file-scoped rule cannot.

Note also what nat-os does *not* do. ESP-IDF disables the cache around flash writes. nat-os never disables it, and instead makes the dangerous reads impossible:

- all kernel code executes from IRAM, so instruction fetch never touches flash
- the five files above have their `.roddata` in DRAM
- interrupts are masked for the whole operation, so no handler can run

A cache *hit* is harmless — it never reaches the chip. Only a miss would, and with no flash-mapped address referenced there is nothing to miss on.

The residual risk is stated rather than hidden: this argument depends on every function reachable from the driver keeping its read-only data out of flash, and nothing instruments it. It is listed in Chapter 30.

4.6 The heap, placed by the linker

```
/* Kernel stack grows down from the top of DRAM. */
_stack_top = ORIGIN(dram) + LENGTH(dram);

/* Boot stack reservation. ... 4 KB is far more than the boot path
 * uses and costs 2% of DRAM. */
_boot_stack_size = 4K;

/* Heap: everything between the end of .bss and the boot stack reservation.
 * Placed by the linker rather than sized by hand so it absorbs changes in
 * .bss automatically – a fixed heap base silently shrinks when a static
 * array grows, which is the sort of thing that shows up as a mysterious
 * allocation failure much later. */
_heap_start = ALIGN(_bss_end, 8);
_heap_end   = _stack_top - _boot_stack_size;
```

Two decisions, both argued in UM-NATOS-010 §2.4.

The heap is placed, not sized. A hand-chosen base would silently shrink the heap whenever a static array grew, and surface much later as an allocation failure with no obvious connection to the change that caused it.

The boot stack reservation is permanent even though the boot context is abandoned at handoff, "because the heap must not be adjacent to a stack that grew further than expected before that point".

4.7 The overlap that was not there

This section is about a risk that was raised as "high severity, blocks M1", was wrong, and produced a rule.

The observation

The measured boot trace shows the bootloader loading its own segments:

```
load:0x3fff0030,len:1184
load:0x40078000,len:13232
load:0x40080400,len:3028
```

The third lands at **0x40080400** — inside the region nat-os declares as its IRAM (0x40080000, length 0x20000). At M0 the kernel's `.text` occupied 0x40080000 – 0x400802E0, ending **288 bytes short** of it. M1 would cross it.

Why it is harmless

The ESP-IDF bootloader links itself into **two** IRAM segments:

```
iram_loader_seg : org = 0x40078000, len = 0x8000 /* 32KB, APP CPU cache */
iram_seg        : org = 0x40080400, len = 0xfc00
```

Segment	Receives
iram_loader_seg @ 0x40078000	.iram1.*, the loader support code — the code that copies the application
iram_seg @ 0x40080400	.entry.text, .init — early startup only

The bootloader's own commentary states the intent:

"The main purpose is to make sure the bootloader can load into main memory without overwriting itself."

"IRAM POOL1, used for APP CPU cache. Bootloader runs from here during the final stage of loading the app because APP CPU is still held in reset."

By the time application segments are copied, execution has moved to iram_loader_seg. The region at 0x40080400 holds initialisation code that has already completed and is *expected* to be overwritten.

Documentation was not treated as sufficient

A kernel was built with ~24 KB forced into .text so its IRAM segment spanned well past the disputed boundary, with sentinel words at both ends:

Image size	26,048 bytes (vs 1,216 baseline)
IRAM span	0x4008025C ... 0x40086258 — crosses 0x40080400 by ~24 KB
First sentinel	intact
Last sentinel	intact
Boot	normal; all M0 assertions still pass; heartbeat advancing

The entire span was copied intact and the kernel ran normally.

The corrected guidance, and the rule

- The IRAM origin of 0x40080000 is **correct** and should not be changed.
- Moving it to 0x40088000 — the previously recommended "cheap mitigation" — would have discarded 32 KB of IRAM to solve a problem that does not exist.
- **M1 is not blocked by this.**

The report is candid about how the wrong conclusion was reached:

The risk was raised from the boot trace alone: an address in the log fell inside a region the linker script claimed, and the conclusion was drawn from geometry without consulting the bootloader's design. The lesson for later milestones is that an apparent conflict in an address map is a prompt to read the other party's link script, not to move our own.

This is the same shape as three later defects: a constant recalled instead of fetched (Chapter 22 §22.4), a register field remembered instead of read (Chapter 22 §22.3), and a window handler that could have been derived and was copied instead (Chapter

2 §2.7). In every case, going to the source of truth was cheaper than reasoning around it.

4.8 Unverified assumptions

Carried forward from UM-NATOS-004 §6, and still open:

Assumption	Status
DRAM above 0x3FFE0000 is ROM-reserved	Believed; not independently confirmed
IRAM window 0x40080000+0x20000 is fully available	Confirmed by §4.7's experiment
SRAM region boundaries in §4.1	Transcribed, not verified against silicon
Cache consumption of SRAM0 when XIP is enabled	Not measured — relevant from the first flash-resident code build

One more, added later and worth its own line because it was a *constraint that turned out not to exist*. UM-NATOS-028 §3 records that the WiFi work had been shaped for weeks by a belief that referencing libphy would cost 48 KB of IRAM:

Referencing those four functions cost **2,459 bytes** (116,692 → 119,151 text against 131,072 of IRAM), not the 48 KB this project has warned about since `MAC-NEXT.md`. That figure came from a `--whole-archive` measurement, which pulls every object; a real link pulls only what is referenced. A constraint that had been shaping decisions for weeks simply did not exist.

Next: how the source becomes the image whose segments this chapter placed.

5. The Build Pipeline

Sources: docs/UM-NATOS-005-build-pipeline.md Code: build.ps1, tools/vasm.py, kernel/linker.ld, vendor/README.md

5.1 PlatformIO as a toolchain, not a build system

PlatformIO is a **build orchestrator and package manager**. It downloads toolchains, resolves libraries, invokes the compiler and linker, and calls `esptool`. Nothing it produces is required on the target.

nat-os uses it as a **source of tools**:

Tool	Path
xtensa-esp32-elf-gcc (14.2.0)	~/.platformio/packages/toolchain-xtensa-esp32/bin/
xtensa-esp32-elf-objcopy, -objdump, -size	same
esptool.py (4.5.1)	~/.platformio/packages/tool-esptoolpy/
Python with pyserial	~/.platformio/penv/Scripts/python.exe

It is **not** used to build, for one reason:

PlatformIO builds are organised around a `framework` (`arduino` or `espidf`), and each framework links a substantial runtime — FreeRTOS, a heap, a C library, and a startup path that ends by calling user code. nat-os replaces all of that. The framework is therefore the obstacle rather than the foundation, and `-nostdlib -nostartfiles` says precisely that.

A framework-less PlatformIO configuration is possible but works against the tool's assumptions. The report states the preference plainly:

For kernel development, every flag should be visible rather than inferred, which a 90-line script achieves and a `platformio.ini` does not.

The discovery is defensive — a missing tool is a named failure rather than a confusing one:

```
function Find-Tool($pattern) {
    $hit = Get-ChildItem "$env:USERPROFILE\.platformio\packages\$pattern" -
ErrorAction SilentlyContinue |
        Select-Object -First 1
    if (-not $hit) { throw "could not find $pattern" }
    return $hit.FullName
}
```

5.2 The stages



Five stages, of which two did not exist when UM-NATOS-005 was written: bytecode assembly (Chapter 15) and the windowed-ABI compile (Chapter 2 §2.8).

5.3 Compiler flags, and why each one

```
$cflags = @(
    "-mabi=call0", "-mtext-section-literals", "-mlongcalls",
    "-ffreestanding", "-fno-builtin", "-fno-stack-protector",
    "-fno-tree-loop-distribute-patterns",
    "-Os", "-Wall", "-Wextra", "-std=c11",
    "-I", "$root/kernel"
)
```

Flag	Purpose
-mabi=call0	Select the non-windowed ABI. The load-bearing flag — Chapter 2
-mtext-section-literals	Place literal pools inside .text. Without this they land in a separate section the linker script does not map into IRAM, and 132r loads fault
-mlongcalls	Permit calls beyond the short-displacement range; required once code spans more than a small region
-ffreestanding	No hosted C environment — no main convention, no library assumptions
-fno-builtin	Prevent GCC replacing loops with calls to memcpy/memset, which do not exist in this link
-fno-stack-protector	The stack guard requires runtime support that does not exist
-fno-tree-loop-distribute-patterns	§5.4
-Os	Optimise for size; IRAM is the scarce resource
-Wall -Wextra	In a kernel, an unused-variable warning is often a real bug

Flag	Purpose
-std=c11	Explicit language version

5.4 The flag that prevents infinite recursion

-fno-tree-loop-distribute-patterns earns its own section because the failure it prevents is silent, infinite, and would surface arbitrarily far from its cause.

The chain, from UM-NATOS-012 §7:

1. The link failed on an undefined reference to `memcpy` from a function whose source never mentions it. GCC synthesises calls to `memcpy`/`memset` from ordinary C — byte-copy loops, struct assignments, large local initialisers — and does so **even under -fno-builtin**, which only governs the treatment of those names when written explicitly.
2. Under `-nostdlib` nothing supplies them. So `kernel/kstring.c` was written to provide `memcpy`, `memset`, `memmove` and `memcmp`.
3. And then:

GCC will recognise the copy loop *inside* `memcpy` as a `memcpy` and rewrite it into a call to itself. That bug is silent, infinitely recursive, and would surface as a stack overflow in whatever unrelated code first copied a struct.

The build comment records it in one sentence:

```
# Stops GCC rewriting a hand-written copy loop into a call to memcpy –
# which, inside memcpy itself, is silent infinite recursion. See kstring.c.
"-fno-tree-loop-distribute-patterns",
```

5.5 Linker flags

```
$ldflags = @(
    "-mabi=call0", "-nostdlib", "-nostartfiles",
    "-Wl,--gc-sections",
    "-Wl,-Map,$build\ntos.map",
    "-T", "$root\kernel\linker.ld"
)
& $gcc @ldflags -o $elf @objs @phylibs
```

Flag	Purpose
-nostdlib	Link no standard library
-nostartfiles	Link no C runtime startup — <code>_start</code> is ours
-T kernel/linker.ld	Our memory map (Chapter 4)
-Wl,--gc-sections	Discard unreferenced sections
-Wl,-Map,build/ntos.map	The authoritative record of what landed where
-mabi=call0	Required on the link line too

Note the quoting comment above them:

```
# Quote every -Wl,... argument: PowerShell otherwise reads the comma as an
# array separator and the parser dies before gcc is ever invoked.
```

That is one of the two bring-up defects UM-NATOS-005 §9 records, and neither involved kernel source:

Defect	Symptom	Resolution
Unquoted -Wl, arguments	PowerShell parse error before GCC ran; "Missing argument in parameter list"	Quote every -Wl,... string
Wrong Python interpreter	ModuleNotFoundError: No module named 'serial' during offline elf2image	Use PlatformIO's penv interpreter: esptool imports serial unconditionally, even for operations that touch no serial port

The second is handled with a fallback:

```
# PlatformIO's own interpreter – it already has pyserial, which esptool needs
# even for offline elf2image (its loader module imports serial unconditionally).
$python = "$env:USERPROFILE\.platformio\penv\Scripts\python.exe"
if (-not (Test-Path $python)) { $python = "python" }
```

5.6 Bytecode assembly

Every `tools/*.vasm` file is assembled to a C header in `kernel/generated/` before compilation:

```
# Bytecode is assembled on the host: the VM on the device is a pure interpreter
# and carries no assembler. Generated headers are build products, not sources.
$gen = Join-Path $root "kernel\generated"
New-Item -ItemType Directory -Force -Path $gen | Out-Null
$vasm = Get-ChildItem "$root\tools\*.vasm" -ErrorAction SilentlyContinue
if ($vasm) {
    Write-Host "== assembling bytecode ==" -ForegroundColor Cyan
    foreach ($src in $vasm) {
        $hdr = Join-Path $gen ($src.BaseName + ".h")
        & $python "$root\tools\vasm.py" $src.FullName -o $hdr --name ("vm_" +
$src.BaseName)
        if ($LASTEXITCODE -ne 0) { throw "vasm failed: $($src.Name)" }
    }
}
```

Twelve programs currently assemble this way. Chapter 15 covers the assembler.

5.7 The vendor archive link order

The most intricate part of the build, and the one with the longest comment. Two static archives that call each other, plus a ROM symbol script:

```
$phylibs = @(
    "$root\vendor\phy\libpp_natos.a",
    "$root\vendor\phy\libphy_natos.a",
    "$root\vendor\phy\libpp_natos.a",
    "-T", "$sdk\ld\esp32.rom.ld"
)
```

Three points, each recorded in the script:

Not `--whole-archive`.

```
# Linked normally, NOT --whole-archive: the linker pulls in only the objects
# actually referenced, so a build that calls one small function costs a few KB
# rather than libphy's full 56 KB.
```

That distinction dissolved a constraint that had shaped the WiFi work for weeks (Chapter 4 §4.8).

`libpp` before `libphy`, and again after.

```
# libpp_natos.a goes BEFORE libphy: it is the caller, and a static archive
# only satisfies references the linker has already seen to its left.
# Listing it after would leave ic_mac_init and friends unresolved even
# though they are sitting in the archive.
#
# Repeated at the end too, because the two archives call each other and a
# single pass in either order leaves something behind. Cheaper to reason
# about than --start-group, and equivalent for two libraries.
```

Only `esp32.rom.ld`, and only the patched archives. The reason is in Chapter 2 §2.10: Espressif's `newlib` and `libgcc` ROM scripts define `memcpy` and `sprintf` by *bare assignment* rather than `PROVIDE`, which silently redirected the kernel's own `call0 memcpy` to a windowed ROM routine and panicked the board on boot.

5.8 Packaging and flashing

```
& $python $esptool --chip esp32 elf2image --flash_mode dio --flash_freq 40m --
flash_size 4MB -o $bin $elf
```

Flash write targets three regions, and checks the borrowed binaries are present first — a named failure rather than a confusing one:

```
if ($Flash) {
    foreach ($f in @("bootloader.bin", "partitions.bin")) {
        if (-not (Test-Path (Join-Path $borrowed $f))) { throw "missing $f in
$borrowed - see vendor/README.md" }
    }
    Write-Host "== flashing $Port ==" -ForegroundColor Cyan
    & $python $esptool --chip esp32 --port $Port --baud 460800 write_flash -z `
        0x1000 (Join-Path $borrowed "bootloader.bin") `
        0x8000 (Join-Path $borrowed "partitions.bin") `
        0x10000 $bin
}
```



```
if ($LASTEXITCODE -ne 0) { throw "flash failed" }  
}
```

5.9 Usage

```
.\build.ps1                                # build  
.\build.ps1 -Flash -Port COM5              # build and flash  
.\build.ps1 -Flash -Monitor -Port COM5     # build, flash, attach monitor  
.\build.ps1 -Vendor <path>                 # use your own bootloader artefacts
```

The distinction between the first two lines is not cosmetic. Chapter 3 §3.8 and Chapter 20 §20.5 record what happens when a debugging session assumes the first one flashed: two hypotheses tested against a board that had never been reflashed, producing bit-identical output that was read as "none of these are the cause" when it meant "no experiment has run yet".

The mitigation is procedural, and it is worth restating here since this is the build chapter:

the flash step now always shows its Hash of data verified. line, and that line is checked before any capture is interpreted.

5.10 Known gaps

From UM-NATOS-005 §10, all still open:

- **No dependency tracking.** Every build recompiles every file. That was "trivial at three files"; it is now 40 translation units and the full build is noticeably slower, though still under a minute.
- **No incremental or parallel build.**
- **No test target.** There is no host-side unit test path; everything is verified on hardware. This is the largest gap in the list and the one Chapter 31 argues hardest for closing — several defects in this book (the ring-buffer arithmetic in Chapter 26, the offset-domain bounds checks in Chapter 14) are pure functions that a host test could exercise exhaustively.
- **Hard-coded paths.** `build.ps1` contains absolute paths to the toolchain root. The borrowed-binary path was fixed by vendoring; the toolchain path was not.

One gap was closed. "Hard-coded paths to the borrowed binaries" became `vendor/` plus a `-Vendor` override, and the assembler learned not to bake an author's home directory into a generated header:

```
if args.output.EndsWith(".h"):  
    # Repo-relative, never absolute. An absolute path bakes the author's  
    # home directory into a generated header that is then committed, which  
    # is noise in a diff and a small privacy leak in a public repository.  
    src_rel = os.path.relpath(os.path.abspath(args.source),
```

```
os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
```

Next: what the first working image actually proved, and the four things it deliberately did not.

6. Milestone 0: Proving Ownership of the Machine

Sources: docs/UM-NATOS-006-m0-verification.md, docs/UM-NATOS-007-roadmap.md Code:
kernel/start.S, kernel/kmain.c, kernel/linker.ld

6.1 Why M0 asserts rather than prints "hello"

A conventional first program prints a greeting. That proves the CPU reached the kernel, and nothing else. It does not distinguish a working image from one where `.data` was never copied, `.bss` holds reset garbage, or code is executing from an unintended region.

Those three failures are the characteristic failure modes of a hand-written linker script, and each produces symptoms that surface much later, in unrelated code, as inexplicable corruption. M0 therefore checks them explicitly at boot and reports each result, so a broken link map is caught at the point of change rather than during scheduler debugging.

And the reason this matters more here than it usually would:

This matters more than usual here because **no JTAG probe is yet available**. Self-reporting is the only diagnostic channel.

The probe was ordered at M0 and never arrived. Every defect in this book was found without one.

6.2 Configuration

Item	Value
Date	2026-08-14
Target	ESP32 CYD board, fresh unit, COM5
Image	build/natos.bin, 1,216 bytes
Bootloader	Borrowed, 17,536 bytes @ 0x1000
Partition table	Borrowed, 3,072 bytes @ 0x8000
Toolchain	xtensa-esp32-elf-gcc 14.2.0, -mabi=call0
Capture	Port opened before reset; auto-reset pulsed; 8 s window

Flash write verified by `esptool`: all three regions reported "Hash of data verified."

6.3 The captured output

```
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1184
load:0x40078000,len:13232
load:0x40080400,len:3028
entry 0x400805e4

=====
nat-os milestone 0 – kernel alive
=====
.data loaded : ok (0xc0deface)
.bss cleared : ok
bss span      : 0x3ffb0188 .. 0x3ffb018c (4 bytes)
stack top     : 0x3ffdc200
code at       : 0x40080088 (IRAM ok)

no scheduler yet – next: timer interrupt + context switch
heartbeat: 0 1 2 3 4 5 6
```

694 bytes received over the 8-second window.

6.4 The six assertions

#	Assertion	Method	Result
1	Image header is well-formed and the bootloader accepts it	Banner appears at all; esptool reported checksum and SHA-256 valid	PASS
2	.data segment was copied to DRAM	Canary initialised to 0xC0DEFACE in .data, read back at runtime	PASS — read back 0xc0deface
3	.bss was zeroed by start.S	Canary in .bss compared against zero	PASS
4	Code executes from the declared IRAM window	Address of a function compared against 0x40080000–0x400A0000	PASS — 0x40080088
5	Stack is usable; call/return works under call0	Every self-check is a C function call that returned	PASS (implicit)
6	Kernel remains running	Heartbeat counter advanced 0→6 over the capture window	PASS

Assertion 5 is worth pausing on. It is not a separate test — it is the observation that assertions 1 through 4 *printed at all*, which required a working stack, a working call sequence, and a working return under an ABI whose availability had been verified only by inspecting compiler output (Chapter 2 §2.4). It is the cheapest test in the file and it closes the largest question.

Supporting observations

- `rst:0x1 (POWERON_RESET)` — a clean cold boot. Not a watchdog (`rst:0x3 / 0xc / 0x10`) or panic reset, so the kernel did not crash and restart. This line is checked first on every subsequent capture in this book.
- `.bss span 0x3FFB0188 - 0x3FFB018C` — 4 bytes, matching the single declared canary, in the DRAM region the linker script declared.
- **Stack top** `0x3FFDC200` — equals `ORIGIN(dram) + LENGTH(dram)`, confirming the linker script's arithmetic reached the running image.

The last two are the pattern this project uses everywhere: **a computed value compared against an independently derived one**. The linker script says `0x3FFB0000 + 0x2C200`; the running kernel prints `0x3FFDC200`. If those two ever disagree, the image being run is not the image that was linked.

6.5 What M0 does not establish

Stated explicitly in the report so later work does not over-claim, and every item came true:

No interrupt handling. No vector table is installed. Behaviour on any exception or interrupt is undefined. → Chapter 7.

No timing accuracy. The heartbeat uses a spin loop of arbitrary length against an unknown CPU clock. The counter proves liveness, not rate. → The clock was derived at 80 MHz in Chapter 7 §7.6, "which also explains the apparent slowness of the M0 spin-loop heartbeat".

No watchdog management. And here the report is corrected in place, which is the most instructive entry:

The kernel neither feeds nor disables any watchdog. That it survived 8 seconds suggests watchdogs are not armed at this point in boot, but this is inference, not measurement, and may change once interrupts are enabled.

CORRECTED 2026-08-14. The inference was wrong. The RTC watchdog **is** armed by the second-stage bootloader, which expects the application to take ownership of it. M0 survived its capture window by luck of timing, not because nothing was running. Measured directly during M2: `rst:0x10 (RTCWDT_RTC_RESET)` on every boot once the CPU was kept busy.

An inference explicitly labelled as inference, then measured, then found wrong, then corrected in place rather than quietly dropped. The labelling is what made the correction cheap.

No memory beyond the first few bytes. DRAM and IRAM were exercised only at their lowest addresses. → Directly relevant to the overlap question in Chapter 4 §4.7, which was settled by an experiment that deliberately spanned 24 KB.

Single-core only. Core 1 is untouched and in an unknown state. → Still true. It cost exactly one debugging session (Chapter 22 §22.3).

6.6 The conclusion, and the number

Milestone 0 passes. nat-os boots from flash on target hardware, its segments load where the linker script directs, the entry stub produces a working C environment under the call0 ABI, and execution is sustained.

The kernel is 1,124 bytes of text with no ESP-IDF, Arduino, FreeRTOS, or C library linked. Every instruction from `0x4008000C` onward is project code.

1,124 bytes. For comparison, the present kernel is 116,692 bytes of text and 37,248 bytes of image, and the whole of the rest of this book is the growth between those two numbers.

6.7 The roadmap that followed

UM-NATOS-007 defined five milestones. It is quoted here as written, because — unusually — the report refuses to update its plan sections after the fact:

All five are complete, each with a verification report carrying its own PASS. The plan sections below are left exactly as written at M0: they are the plan, and a plan rewritten after the fact to match what happened is not a record of anything.

ID	Deliverable	Principal risk	Verified by
M0	Kernel boots, self-checks pass	Link map errors	UM-NATOS-006
M1	Timer interrupt and tick counter	Vector installation; silent faults	UM-NATOS-008
M2	Two native tasks, preemptive switching	Context save correctness	UM-NATOS-009
M3	Heap allocator and VM memory model	Fragmentation; arena sizing	UM-NATOS-010
M4	Bytecode interpreter executing a program	Instruction set design	UM-NATOS-012
M5	Two VM applications time-sliced in bounded arenas	Isolation enforcement	UM-NATOS-013

And then the assessment of the risk column, which is the most useful sentence in the report:

Every principal risk in that column materialised except one. Vector installation did produce silent resets; context save correctness cost the LOOP-state defect that made every task switch to itself; isolation enforcement is the only one that went in cleanly first time.

Five for six is a good hit rate for a risk register, and the one that missed missed in the good direction.

The exit criteria

Each milestone carried numbered exit criteria, written before the work and measured against afterwards. They are reproduced here because the discipline of writing them first is visible in how specific they are:

M1 — tick counter advances at a measured, stable rate; interrupted code resumes correctly, verified by a checksum over registers across an interrupt; system survives ≥ 60 seconds without reset.

M2 — two tasks alternate for $\geq 10,000$ switches without corruption; register-pattern check passes on every resume; stack guard patterns intact.

M3 — allocate/free cycles leave no leak across 10,000 iterations; arena bounds are queryable by the interpreter; out-of-memory returns a failure rather than corrupting.

M4 — program computing a known result returns it correctly; out-of-bounds access is refused and reported, not performed; interpreter yields at quantum expiry; instructions-per-second measured and recorded.

M5 — two applications interleave correctly; an application attempting out-of-bounds access is terminated while the other continues unaffected; terminating an application releases its arena completely.

Every one of those is a *number or a state that can be printed*, not a judgement. Chapter 12 is largely about what happens when a criterion is not of that shape.

What came after M5, none of it planned

The roadmap's own revision 1.1 adds a table of eleven unplanned reports and then draws two conclusions:

It is longer than the plan it follows. Eleven reports of unplanned work against five of planned, which says the milestones were a plan for a kernel and what got built afterwards was a device.

The one structural item on it is missing. Every driver above is reachable only from the kernel. The VM has twelve syscalls, all hardcoded, and no device model — so an application cannot read the light sensor, scan the I2C bus or receive a keypress. Each new peripheral has meant a kernel edit plus a hand-written syscall, which was tolerable at two and is the obvious next piece of *architecture* rather than more drivers.

That gap is now seventeen reports old and Chapter 31 argues it is the single most valuable thing left to build.

6.8 Two cross-cutting items from the roadmap

Version control — outstanding at M0. The roadmap flagged it with a specific reason:

Not yet initialised. The sibling project The Word Device was lost and recovered only because an editor's local history happened to retain it. This should be resolved before M1.

It was initialised 2026-08-14, the same day, and the first commit is *"Initial commit — cyd-os through Milestone 1"*. Appendix G is the resulting timeline, and it is only possible because of that decision.

JTAG probe — ordered, not in hand.

From M1 onward, failures are increasingly silent. UART cannot report a fault in the interrupt vector itself. Sequencing M1 after the probe arrives is recommended; if not, budget substantially more time.

M1 was not sequenced after the probe, and the extra time was duly spent — twelve build cycles on one defect in M2 alone. But the absence also shaped the system in ways that turned out to be valuable: the panic handler exists because there is no debugger, the counters exist because there is no watchpoint, and Chapter 28 exists because the only instruments available were the ones this kernel built for itself.

Part I ends here. The machine is understood, the ABI is chosen and proved, the boot chain is traced, the memory map is placed and one false alarm in it is disproved, the build is reproducible, and the first image has demonstrated that every one of those things is true on the hardware rather than in principle.

Part II starts where M1 does: with a vector table, a comparator, and the first interrupt this kernel ever took.

7. Interrupts and the Tick

Sources: docs/UM-NATOS-008-m1-verification.md Code: kernel/vectors.S,
kernel/xtensa.h, kernel/timer.c, kernel/panic.c

7.1 What M1 had to establish

Milestone 1 establishes that nat-os can be interrupted and resume correctly: a vector table is installed, a periodic timer interrupt is dispatched, and code interrupted mid-execution continues with its register state intact.

The third property is the one that matters, and the report says why:

Every scheduler is built on it, and a defect in the interrupt save/restore path does not announce itself — it surfaces later as corruption in unrelated code. M1 therefore measures it rather than assuming it.

7.2 Timer source: the core comparator, not a peripheral

The ESP32 offers two families of timer: the TIMG peripherals, and the core's own CCOUNT/CCOMPARE comparators. **CCOMPARE1 was selected.**

	TIMG peripheral	CCOMPARE1 (selected)
Clock gating	Required	None — inside the core
Interrupt matrix routing	Required	None — fixed internal interrupt
Prescaler / config registers	Several	One comparator register
Ways to be wrong about something other than interrupts	Many	Few

The reasoning generalises well beyond timers, and it is the same argument that puts a bit-banged SPI in front of the display (Chapter 18) and a bit-banged I²C on the sensor bus (Chapter 22):

For the first interrupt on a kernel with no driver model, minimising unrelated setup is worth more than the peripheral's extra features.

Every element of that table was later paid for by some other driver. Clock gating cost the audio driver a full evening (Chapter 23 §23.4); matrix routing cost the interrupt chapter four distinct failures (Chapter 22 §22.3). Choosing the source with none of those for the *first* interrupt meant that when it did not work, there were few candidates.

CCOMPARE1 raises **internal interrupt 15**, which is **level 3** on this core:

```

/* CCOMPARE1 fires internal interrupt 15, which is a level-3 interrupt on this
 * core. Level 3 has a dedicated vector, so the handler is entered only for this
 * class of event and needs no EXCCAUSE decoding. */
#define XT_TIMER1_INTERRUPT    15u
#define XT_TIMER1_LEVEL       3u

```

7.3 Interrupt level 3, not 1

Level 1 interrupts on Xtensa are dispatched through the general exception vector with `EXCCAUSE = 4`, requiring the handler to distinguish interrupts from genuine exceptions. Levels 2 and above have **dedicated vectors** and are entered only for that class of event.

Level 3 was chosen: the handler needs no `EXCCAUSE` decoding, and the exception vectors stay free for the panic path.

7.4 The comparator is one-shot, and the write is the acknowledgement

CCOMPARE has no auto-reload. The handler must recompute and rewrite the next deadline, and **that write is also the interrupt acknowledgement** — there is no separate acknowledge step.

```

/* Writing CCOMPAREn also acknowledges that comparator's pending interrupt -
 * there is no separate acknowledge step for these. */
static inline void xt_set_ccompare1(uint32_t v)
{
    __asm__ volatile ("wsr.ccompare1 %0; esync" :: "a"(v));
}

```

UM-NATOS-008 §2.3 analysed the failure mode of forgetting it:

Forgetting it produces immediate re-entry rather than a missed tick, which is a loud failure and therefore an acceptable one.

That analysis is correct and it is incomplete, and the report adds a revision note admitting so:

Revision 1.1 note. This section is correct and incomplete. It reasons about the handler forgetting to rewrite the deadline, and concludes the failure would be loud. It does not consider a **second writer** to the same register, which arrived with the scheduler two milestones later and failed silently.

§7.9 is that defect.

7.5 The vector table and the handler

Slots

Four vector slots are populated at M1. Each is 64 bytes and contains a single jump:

```

.section .vectors.level3, "ax"
.align 4
.global _vector_level3
_vector_level3:
    j      _handler_level3

.section .vectors.kernel, "ax"
.align 4
.global _vector_kernel
_vector_kernel:
    j      _handler_panic

.section .vectors.user, "ax"
.align 4
.global _vector_user
_vector_user:
    j      _handler_panic

.section .vectors.double, "ax"
.align 4
.global _vector_double
_vector_double:
    j      _handler_panic

```

Three of the four go to a panic handler that did not exist in the milestone's specification. UM-NATOS-008 §2.4 explains the addition:

No JTAG probe is available yet. Without this, any unexpected fault produces a silent reset with no evidence. The handler runs on a dedicated 512-byte stack because the faulting stack may be the cause of the fault, and it halts rather than resetting so the output is not scrolled away by a boot loop.

```

_handler_panic:
    /* Use a known-good stack: the faulting one may be why we are here. */
    movi    a1, _panic_stack_top
    rsr.exccause a2
    rsr.epc1   a3
    rsr.ps     a4
    call0    kernel_panic
.Lpanic_hang:
    j      .Lpanic_hang

.size _handler_panic, . - _handler_panic

.section .bss
.align 16
_panic_stack:
    .space 512
_panic_stack_top:

```

Chapter 12 covers what `kernel_panic` grew into: three reporting channels ordered by decreasing reliability, and a screen drawn on a panel by a driver operating under three suspended assumptions.

Entry and exit

Hardware saves **nothing** on entry. It records the interrupted PC in `EPC3`, the processor state in `EPS3`, raises `PS.INTLEVEL`, and jumps. Everything else is the handler's responsibility.

```
_handler_level3:
    addi    a1, a1, -96
    s32i    a0, a1, 0
    s32i    a2, a1, 4
    s32i    a3, a1, 8
    s32i    a4, a1, 12
    s32i    a5, a1, 16
    s32i    a6, a1, 20
    s32i    a7, a1, 24
    s32i    a8, a1, 28
    s32i    a9, a1, 32
    s32i    a10, a1, 36
    s32i    a11, a1, 40
    s32i    a12, a1, 44
    s32i    a13, a1, 48
    s32i    a14, a1, 52
    s32i    a15, a1, 56
    rsr.sar a2
    s32i    a2, a1, 60
    rsr.epc3 a2
    s32i    a2, a1, 64
    rsr.eps3 a2
    s32i    a2, a1, 68
```

At M1 this saved only `a0`, `a2` – `a11` and `SAR` — 12 words, padded to 64 bytes. `a12` – `a15` were deliberately *not* saved in assembly, because they are callee-saved under `call0` and are preserved by the C handler itself.

The frame shown above is the M2 version: 96 bytes, 21 words, saving `a12` – `a15` as well because "the resumed task expects *its* values, not the interrupted task's". Chapter 8 is why. It also saves three more registers that M1 had never heard of, and Chapter 8 §8.6 is that story.

Return is `RFI 3`, which restores PC and PS from `EPC3` / `EPS3` atomically.

7.6 Synchronising instructions, baked into the accessors

Several special registers require a synchronising instruction before the write takes architectural effect:

Register	Required after write
VECBASE	ISYNC — affects instruction fetch
PS	RSYNC
INTENABLE, CCOMPARE1	ESYNC

These are built into `xtensa.h`'s accessors rather than left to call sites:

```

/* Thin inline wrappers, no abstraction: kernel code that touches these needs to
 * see exactly which register it is hitting. ...
 *
 * Omitting those produces failures that appear several instructions later, so
 * they are baked into the accessors rather than left to call sites.
 */

```

The whole of `xtensa.h` is worth reading as an example of the style: no abstraction, one function per register, and a comment on every non-obvious one.

```

static inline void xt_set_vecbase(uint32_t base)
{
    __asm__ volatile ("wsr.vecbase %0; isync" :: "a"(base));
}

/* PS.INTLEVEL masks interrupts at or below its value. Setting it to 0 admits
 * every level; the other PS bits are preserved because clobbering WOE or UM
 * here would change the execution mode out from under the kernel. */
static inline void xt_set_intlevel(uint32_t level)
{
    xt_set_ps((xt_get_ps() & ~0xFu) | (level & 0xFu));
}

```

That preservation of `WOE` in `xt_set_intlevel` is the same caution as the PS inheritance in `task_create` (Chapter 2 §2.7), and for the same reason: the ROM leaves bits set whose meaning this kernel does not use but must not disturb.

One accessor carries a warning about how its result may be used:

```

/* Which CPU interrupt lines are asserting RIGHT NOW, regardless of whether they
 * are enabled. A handler must mask this with INTENABLE before acting on it:
 * INTERRUPT reports the hardware's opinion, and a line can be pending for a
 * peripheral this kernel has not enabled and cannot service. */
static inline uint32_t xt_get_interrupt(void)

```

Chapter 22 §22.2 shows the consumer that made that warning necessary.

7.7 The timer driver

`timer.c` is 121 lines, of which roughly half are comments about two defects. The state is minimal:

```

/* Written by the ISR, read by the main context – volatile, and read/written
 * as a single 32-bit word so no lock is needed on this core. */
static volatile uint32_t g_ticks;
static volatile uint32_t g_last_delta; /* cycles between the last two ticks */
static volatile uint32_t g_late;      /* re-arms that had already elapsed */

static uint32_t g_interval;           /* cycles between ticks */
static uint32_t g_next;               /* CCOUNT value of the next tick */

```

Starting it is four lines and one deliberate ordering:

```

void timer_start(uint32_t interval_cycles)
{
    g_interval = interval_cycles;
    g_ticks = 0;
    g_late = 0;
    g_last_delta = 0;

    g_next = xtccount() + interval_cycles;
    xt_set_ccompare1(g_next);

    xt_enable_interrupt(XT_TIMER1_INTERRUPT);
    xt_set_intlevel(0);          /* admit all levels; nothing is masked now */
}

```

`xt_set_intlevel(0)` last, after the comparator is armed and the source is enabled. Reversing those would admit an interrupt before there was a deadline for it to have missed.

7.8 The M1 results

Captured 2026-08-14, 10-second window:

```

.data loaded : ok
.bss cleared : ok
stack top    : 0x3ffdc200
code at      : 0x4008046c (IRAM ok)
vecbase      : 0x40080000 (installed)
tick every   : 2400000 cycles
intenable    : 0x00008000
ps           : 0x00060720

waiting for first tick... arrived

tick 25  delta=2400030cy  late=0  regchecks=1556680  corrupt=0

```

#	Exit criterion	Result
1	Tick counter advances at a stable rate	PASS — delta = 2,400,030 cycles against 2,400,000 requested
2	Interrupted code resumes correctly	PASS — 1,556,680 checks, 0 corruptions
3	System survives without reset	PASS — full capture window, no panic, no reboot

How criterion 2 was measured

Six caller-saved registers are loaded with distinct non-zero patterns and verified immediately afterwards, in a continuous loop. Because ticks fire asynchronously, a large fraction of iterations are interrupted mid-sequence — which is exactly the case a faulty save or restore would corrupt.

Distinct patterns (`0x11111111`, `0x22222222`, ...) were used so that a stray zero, or a neighbouring register's value, is unmistakable rather than plausible.

That last clause is the same reasoning as the heap's magic words (Chapter 10 §10.3) and the stack fill pattern (Chapter 8 §8.4): a canary whose corruption produces a *plausible* value is a canary that will be believed.

Reading the numbers

Handler overhead is 30 cycles. The measured interval exceeds the requested one by a constant 30 cycles, which is the prologue cost before `CCOUNT` is sampled. Constant rather than growing, so there is no cumulative drift.

`late = 0` — no deadline was ever missed.

`intenable = 0x00008000` — bit 15 only, confirming `CCOMPARE1` is the sole enabled source. Verified rather than assumed, which mattered later when a second source was routed.

`ps = 0x00060720` — `INTLEVEL = 0`, so nothing is masked. The upper bits are `WOE / CALLINC` left by the ROM; harmless under `call0` — and, two milestones later, the reason windowed code could be made to run at all.

The CPU frequency, derived

The kernel does not know the CPU frequency, so the interval is expressed in cycles and the real rate is derived by counting ticks against the *host's* clock during a capture window of known duration.

25 ticks within ~10 s at 2,400,000 cycles per tick implies a CPU clock near **80 MHz** — not the 240 MHz maximum. The bootloader leaves the core at its default and nothing in the kernel raises it.

This is a **measurement of the current configuration**, not a specification figure, and it should be re-derived rather than assumed if boot configuration changes. It also explains the apparent slowness of the M0 spin-loop heartbeat.

80 MHz appears throughout the rest of this book — in the display's `delay_us`, in the watchdog's prescaler, in every cycles-to-milliseconds conversion — always as a derived figure. `display.c` labels it as such:

```
/* Derived in UM-NATOS-008 §5.2 from the measured tick rate. Only used for the
 * panel's reset and sleep-out delays, where being wrong by a factor of three
 * still leaves them long enough. */
#define CPU_HZ 80000000
```

7.9 The second writer, and 183 milliseconds of stopped clock

This section is UM-NATOS-008 revision 1.1, added after the tick was found stalling.

The setup

§7.4 records that `CCOMPARE` has no auto-reload, so the handler recomputes the deadline and that write doubles as the acknowledgement. It analyses one failure —

the handler forgetting to write — and correctly calls it loud.

The failure that actually occurred needs *two parties*, and M1 had only one.

Two writers, one shadow

`task_yield()` ends a slice early by pulling `CCOMPARE1` back to `ccount + 64`. It writes the hardware register directly and does not tell `timer.c`, which keeps its own `g_next` shadow of what the deadline should be.

So the handler fires 64 cycles after a yield, sees no reason to think anything unusual happened, and executes:

```
g_next += g_interval;
```

adding a **whole interval** to a deadline that was already a whole interval away. Each yield pushes the tick one interval further into the future.

Measured at the moment of failure:

```
ccount      0x04672eab
ccompare1   0x05433b92    14,630,119 cycles ahead
                                = 18 tick periods = 183 ms
```

The guard existed and faced the wrong way

The handler already had a correction:

```
if ((int32_t)(g_next - xt_ccount()) <= 0) { /* deadline in the PAST */
    g_next = xt_ccount() + g_interval;
    g_late++;
}
```

That is the direction anyone reasons about: the handler was delayed, the deadline slipped behind, catch up rather than storm. It is right, it was exercised, and `g_late` counted it.

Nothing guarded the other direction, because a deadline arriving *too early* has no obvious cause — until a second writer exists.

The fix bounds it both ways, and is in the shipped source:

```
int32_t ahead = (int32_t)(g_next - xt_ccount());
if (ahead <= 0 || ahead > (int32_t)g_interval) {
    g_next = xt_ccount() + g_interval;
    g_late++;
}
```

Why it stayed hidden for three milestones

The rate is self-correcting on average. A yield defers the tick; the next handler still advances `g_next` by one interval; nothing accumulates unless yields outpace ticks.

Shortening the display task's sleep from 8 ticks to 2 multiplied the yield rate until they did. That commit is where the symptom appears in a bisect, and it did not introduce the defect — it removed the margin that had been concealing it.

And the part that should worry anyone reading a performance number from this project:

Nothing else showed a symptom. Sleeps ran long, timeouts were loose, and frame-rate figures computed as frames-per-tick were taken against a clock that intermittently stopped. Only the M6 critical-section self-test noticed, because it is the one test that asserts something about the tick *resuming* rather than about work getting done.

Chapter 18 §18.7 is where that consequence lands: a framebuffer measured as worthless, and re-measured as clearly worthwhile once this and one other defect were fixed.

The standing rule the defect produced — and the rule it defeated

The scheduler already had a rule, from Chapter 16's freeze:

A routine adjusting a scheduler deadline must only ever move it earlier.

`task_yield()` obeys that rule exactly, and this defect happened anyway. The report is precise about why:

The rule constrains the **writer**. The defect was in the other party's **bookkeeping**: `timer.c` maintained a shadow of a register it did not exclusively own, and a shadow is only as good as its exclusivity. Moving the deadline earlier is safe for the deadline and quietly invalidates every assumption anyone else has cached about it.

A shared hardware register with a software shadow has exactly one safe shape: either one writer, or every writer maintains the shadow. Two writers and one shadow is a defect waiting for enough traffic to surface.

Recovered

```
[6a] critical : PASS  entered at 31, held at 31 across 2 periods, resumed at 62
frames/s      12.4 -> 16.0
```

The frame rate improved because `task_sleep(2)` had been waiting on ticks that were not arriving.

7.10 The third defect in the same function

There is a further chapter to `timer_isr`, from UM-NATOS-028 §4, and it is included here rather than in the WiFi chapter because it belongs to this file.

`g_ticks++` sat at the bottom of the handler *unconditionally*. But `task_yield()` makes the handler run on every yield as well as on every real deadline. So `timer_ticks()` counted **scheduling activity, not time**:

Measured at **217 ticks per real second** where 100 is correct, and far higher when a task sat in an idle-yield loop.

The present handler opens by refusing to count an early entry, and the comment is the longest in the file:

```
void timer_isr(void)
{
    uint32_t now = xt_ccount();

    /* Not every entry here is a tick.
     *
     * task_yield() ends a slice by pulling CCOMPARE1 back to ccount+64, so this
     * handler runs on every yield as well as on every real deadline. Counting
     * entries therefore counts yields, and g_ticks++ used to sit at the bottom
     * of this function unconditionally.
     *
     * That made timer_ticks() a measure of scheduling activity rather than of
     * time, and every deadline expressed in ticks came due early in proportion
     * to how much yielding the system happened to be doing. ...
     *
     * So: advance the clock only when the deadline has actually passed. An
     * early entry falls through to the re-arm below, which restores CCOMPARE1
     * to the real deadline the yield displaced -- the yield still got its
     * context switch, because _handler_level3 calls the scheduler regardless of
     * what this function decides. */
    if ((int32_t)(now - g_next) < 0) {
        xt_set_ccompare1(g_next);
        return;
    }

    g_last_delta = now - (g_next - g_interval);

    g_next += g_interval;
    /* ... the two-sided bound from §7.9 ... */
    xt_set_ccompare1(g_next); /* re-arm and acknowledge */
    g_ticks++;
}
```

The final clause of that comment is the elegant part. An early entry returns without doing bookkeeping, but the *switch still happens*, because the scheduler is called from `_handler_level3` and not from `timer_isr`. The yield gets what it asked for; the clock does not get corrupted. Two concerns that were tangled are separated by a single early return.

Three defects in 121 lines, all in the same function, all silent, all found by something else breaking. The file's own summary of the second one applies to all three:

Nothing else showed a symptom, which is the uncomfortable part.

Next: the milestone that turned one interruptible context into several, and the twelve build cycles it took.

8. Tasks, Frames, and the Defect That Cost Twelve Builds

Sources: docs/UM-NATOS-009-m2-verification.md Code: kernel/task.h, kernel/task.c, kernel/vectors.S

8.1 The claim, and how each half of it fails

M1 proved that the kernel could be interrupted and resume *itself*. M2 is the harder claim, because the interrupt now returns to a **different** context than the one it interrupted.

Claim	How it fails if untrue
A task can be created that has never run	The first switch jumps to a fabricated frame and lands nowhere
A running task can be suspended mid-instruction-stream	Registers or PC are lost; corruption appears later, elsewhere
A suspended task can be resumed exactly	Silent data corruption in the resumed task
The scheduler distributes time	One task starves the others; looks like a hang
Tasks do not corrupt each other's stacks	Overflow into a neighbour; no MMU to catch it

The third and fourth are the ones that actually broke.

8.2 One way for a task to come into existence

An earlier design adopted the boot context as task 0, capturing its stack pointer on the first interrupt, and fabricated frames for every other task.

That is two mechanisms for the same thing, and only one of them worked: tasks 1 and 2 ran, task 0 never resumed.

The design was changed so that **every task without exception is created by fabricating a frame that looks as though it had already been interrupted**. `kmain` creates the tasks, starts the tick, and is then abandoned — its stack is never reclaimed and it never runs again.

`g_current` starts at `-1`, which tells the scheduler there is no outgoing context to save on the first switch:

```
/* -1 means "no task is running yet": the boot context is about to be abandoned
 * and its stack pointer must NOT be saved, because doing so would overwrite the
 * fabricated frame of whichever task occupies that slot.
 *
 * An earlier design adopted the boot context as task 0 instead, capturing its
```

```

* stack pointer on the first interrupt. That created two ways for a task to
* come into existence – fabricated and captured – and only the fabricated path
* worked: tasks 1 and 2 ran, task 0 never resumed. Deleting the second path was
* cheaper than debugging it, and leaves one code path to be correct about. */
static int g_current = -1;

```

This deleted the failing path rather than debugging it, and left one code path to be correct about instead of two.

This principle — *delete the second mechanism rather than debug it* — recurs. It reappears in §8.3 (one switching mechanism, not two), in Chapter 26 (one command set, not two), and in Chapter 16 (one `retire()` path, not three).

8.3 The switch happens inside the interrupt, not beside it

There is no separate `switch_to()` routine. The level-3 handler saves the full context onto the interrupted task's own stack, passes the stack pointer to `task_schedule()`, and resumes on whatever pointer comes back. If that pointer belongs to a different task, the return is the context switch.

```

/* Hand the current stack pointer to the scheduler; it returns the stack
 * pointer to resume on, which may belong to a different task. From this
 * instruction onward a1 may no longer be the stack we arrived on. */
mov     a2, a1
call0   task_schedule
mov     a1, a2

```

Three instructions. Everything a scheduler does is on the other side of that `call0`.

`task_yield()` therefore does not switch directly — it pulls the comparator deadline forward so the ordinary tick fires almost immediately:

One switching mechanism, exercised constantly, rather than a second one exercised rarely.

8.4 EPC3 and EPS3 must travel in the frame

This is the trick that makes a stack swap a task switch, and it is worth stating carefully because it is the one thing about the design that is not obvious.

RFI 3 restores PC and PS from EPC3 / EPS3. These are **single registers, not per-task state**. Swapping stack pointers alone would resume the new task's *registers* at the *old* task's program counter.

From `vectors.S`:

```

* EPC3 and EPS3 hold the interrupted PC and processor state, and RFI 3 restores
* from them. They are single registers, not per-task. Swapping stacks alone
* would therefore resume the new task's registers but the OLD task's program
* counter. Saving them into each task's frame and restoring before RFI is what
* makes a switch actually switch.

```

The frame layout

96 bytes, 21 words, 16-byte aligned as Xtensa requires.

Offset	Word	Contents	Why
0	0	a0	Return address; clobbered by call0 and saved before that happens
4-56	1-14	a2-a15	Both caller- and callee-saved sets: the resumed task expects <i>its</i> values
60	15	SAR	Clobbered by any shift
64	16	EPC3	Interrupted PC
68	17	EPS3	Interrupted PS
72-80	18-20	LBEG, LEND, LCOUNT	Zero-overhead loop state — §8.6

a1 is not stored: the frame's own address *is* the saved stack pointer.

The layout is declared in `task.h` and written by hand in `vectors.S`, and the two must agree. There is a static assertion for the size and an explicit acknowledgement that field order is not enforced:

```
/* Saved-context frame, in 32-bit words. Assembly in vectors.S writes this
 * layout by hand; the two MUST agree. A static assertion in task.c checks the
 * size, but the field order is only guaranteed by keeping these in step. */
#define TASK_FRAME_WORDS 21
#define TASK_FRAME_BYTES 96          /* 21 words, padded to 16-byte alignment */

#define TASK_FRAME_IDX_SAR    15
#define TASK_FRAME_IDX_EPC3  16
#define TASK_FRAME_IDX_EPS3  17
#define TASK_FRAME_IDX_LBEG  18
#define TASK_FRAME_IDX_LEND  19
#define TASK_FRAME_IDX_LCOUNT 20
```

```
_Static_assert(TASK_FRAME_BYTES >= TASK_FRAME_WORDS * 4,
               "frame must hold every saved word");
_Static_assert((TASK_FRAME_BYTES % 16) == 0,
               "Xtensa requires a 16-byte aligned stack");
```

The handler sequence, annotated

```
addi a1, a1, -96      ; frame on the interrupted task's stack
save a0, a2..a15, SAR, EPC3, EPS3, LBEG, LEND, LCOUNT
clear PS.EXCM        ; MUST precede any C – see §8.7
call0 intr_dispatch   ; tick bookkeeping, comparator re-arm
mov a2, a1
call0 task_schedule   ; a2 in = current sp, a2 out = sp to resume
mov a1, a2            ; from here a1 may be a different task's stack
restore EPS3, EPC3, SAR, LBEG, LEND, LCOUNT, a0, a2..a15
addi a1, a1, 96
rfi 3
```

Two ordering constraints, both recorded in the source:

Loop state is saved before any C is called, because `task_schedule` contains a loop of its own and would otherwise destroy what it was meant to preserve.

LCOUNT is restored last of the three:

```
/* LOOP state. LCOUNT is written last: it is the register that actually arms
 * the hardware loop, so LBEG and LEND must already describe the right body
 * before it becomes live. */
l32i      a2, a1, 72
wsr.lbeg  a2
l32i      a2, a1, 76
wsr.lend  a2
l32i      a2, a1, 80
wsr.lcount a2
```

8.5 Task creation

```
int task_create(const char *name, task_entry_fn entry)
{
    for (int id = 0; id < TASK_MAX; id++) {
        if (g_tasks[id].state != TASK_UNUSED) {
            continue;
        }

        uint32_t *stack = g_stacks[id];
        for (int i = 0; i < TASK_STACK_WORDS; i++) {
            stack[i] = STACK_FILL;
        }
        stack[0] = STACK_GUARD;

        /* Frame sits at the top of the stack, 16-byte aligned. */
        uint32_t top = (uint32_t)&stack[TASK_STACK_WORDS];
        top &= ~15u;
        uint32_t *frame = (uint32_t *)(top - TASK_FRAME_BYTES);

        for (int i = 0; i < TASK_FRAME_WORDS; i++) {
            frame[i] = 0;
        }

        /* Resume at the entry point, with interrupts admitted. PS is taken
         * from the running kernel with INTLEVEL forced to 0, so the task
         * inherits the same execution mode rather than a guessed one – the
         * ROM leaves WOE and CALLINC set, and fabricating a different PS would
         * put the task in a subtly different state from its creator. */
        frame[TASK_FRAME_IDX_EPC3] = (uint32_t)entry;
        frame[TASK_FRAME_IDX_EPS3] = xt_get_ps() & ~0xFu;
        frame[TASK_FRAME_IDX_SAR] = 0;

        g_tasks[id].sp      = (uint32_t)frame;
        g_tasks[id].state   = TASK_READY;
        /* ... */
        return id;
    }
}
```

```

    return -1;
}

```

Two constants, both chosen so their corruption is unmistakable:

```

/* Written into the lowest stack word; if it changes, the task overflowed. */
#define STACK_GUARD 0x57ACC0DEu

/* Fill pattern, so untouched stack is distinguishable from used stack and
 * headroom can be measured rather than guessed. */
#define STACK_FILL 0xEEEEEEEEu

```

The fill pattern is what makes `task_stack_headroom()` a measurement rather than an estimate:

```

uint32_t task_stack_headroom(int id)
{
    if (id < 0 || id >= TASK_MAX || g_tasks[id].stack_base == 0) {
        return 0;
    }
    uint32_t untouched = 0;
    /* Walk up from just above the guard until the fill pattern stops. */
    for (int i = 1; i < TASK_STACK_WORDS; i++) {
        if (g_tasks[id].stack_base[i] != STACK_FILL) {
            break;
        }
        untouched++;
    }
    return untouched;
}

```

Chapter 12 §12.4 records what happened when this function was finally called for the first time — it had existed, unused, for six milestones.

That function has a failure mode worth noting, and Chapter 27 §27.8 pays for it: an *unprimed* `.bss` stack is all zeros, and a scan for a fill pattern reads that as fully consumed. The PHY stack reported 6144 of 6144 bytes used in a real panic report, which "looked exactly like a stack exhaustion that had not happened".

8.6 A hypothesis that measurement rejected, and a fix kept anyway

Before the real cause was found, the first explanation for M2's failure was stale `LCOUNT`: a task suspended mid-loop, its loop state lost across the switch, the hardware later branching back to a previous `LEND`.

The context frame was extended to save `LBEG / LEND / LCOUNT`, and the build reflashed.

The symptom did not change, and every dumped frame showed `lct=0x00000000`. No task had ever been suspended mid-loop.

And then the decision that makes this section worth reading:

The frame change was nevertheless **kept**, on its own merits: a task genuinely can be suspended mid-loop, and losing that state would corrupt it on resume. ESP-IDF's context

switch saves these three registers for the same reason. A real fix for a real hazard — just not for this defect. Recorded because a confident prediction that measurement rejected is the most reusable part of the exercise.

The `task.h` comment records the whole arc, including the observation that *fixed* the symptom without explaining it:

```
* LBEG/LEND/LCOUNT back the Xtensa zero-overhead LOOP instruction, and they are
* part of a task's context whether or not that task ever knows it. GCC emits
* LOOP for ordinary counted C loops, so any task can be suspended mid-loop with
* LCOUNT non-zero. Omitting these three words cost a full debugging session:
* the scheduler's own round robin compiled to a LOOP, stale LCOUNT survived
* into the handler, the hardware branched back to LEND, and the "no candidate
* matched" fallback overwrote a selection that had in fact matched. It presented
* as a task switching to itself forever. Adding a uart_puts() inside the loop
* made the symptom disappear, because a loop body containing a call cannot be a
* zero-overhead loop and GCC emitted a plain branch instead.
```

That last sentence is a Heisenbug in one line: *the instrumentation fixed it*. Which is precisely what §8.7 explains.

8.7 The defect: `PS.EXCM` disables the zero-overhead loop

The symptom

Tasks entered correctly and then the board went silent. Switch 4 reported `2 -> 2` — the scheduler selecting the task it was already running — and repeated forever. The task table printed microseconds later showed all three tasks `st=1 (READY)` with stable stack pointers.

The scheduler was refusing candidates that were plainly available.

Two earlier conclusions that were wrong

Both were recorded in the repository before being disproved, and corrected rather than quietly dropped.

"The timer interrupt is not firing." It was firing. `ticks=0` was sampled before the first tick had occurred, because the reporter waited 100 ticks before printing and the diagnostic loop printed faster than the tick period.

"The board goes silent, so the switch is fatal." The switch worked. The silence was the workers doing exactly what they were written to do: spin without producing output.

The lesson from the second is one of the most useful in the whole report set:

Every failure mode in this kernel — masked interrupt, bad `RFI`, dead task, working-but-quiet task — presents identically as nothing on the wire. One entry marker per task turned that silence into a sequence and settled it immediately. **That should have been the first instrument, not the fourth.**

The entry markers are still in the tree, with that reasoning attached:

```
/* Entry markers, same purpose as task 0's: they distinguish "the round robin
 * reached this task" from "the switch died on the way there". Tick 2 enters
 * worker-a, tick 3 enters worker-b, and tick 4 is the first time any task is
 * RESUMED from a frame the handler actually saved rather than one fabricated
 * by task_create – three different mechanisms that all fail as silence. */
static void task_a(void) { uart_puts(" [task 1 entered]\n"); worker(&work_a_count,
...); }
static void task_b(void) { uart_puts(" [task 2 entered]\n"); worker(&work_b_count,
...); }
```

and in the reporter:

```
/* First thing a fabricated task ever does. If this appears, the RFI landed
 * on the entry point and the task owns the CPU; if it never appears, the
 * switch itself is where control is lost. Those two failures are otherwise
 * indistinguishable, because both present as silence. */
uart_puts("\n [task 0 entered]\n");
```

The root cause

GCC compiled the round robin into an Xtensa **zero-overhead LOOP**.

On Xtensa, the zero-overhead loop-back is disabled while **PS.EXCM** is set. The **LEND** comparison never fires, so the body executes exactly once and falls straight through into whatever instruction occupies the **LEND** slot. Here that instruction is **or a12, a3, a3** — the **next = g_current** fallback.

Hardware sets **EXCM** on interrupt entry. The handler had never cleared it, so every call into C ran with hardware loops silently degraded to a single iteration.

This accounts for the entire observation set, including the parts that looked arbitrary:

Observation	Explanation
Switches 1–3 correct	All were first-iteration hits — one iteration is enough
Switch 4 wrong	First case needing a second iteration; task 3 is UNUSED
Result was always g_current	That is precisely the instruction at LEND
Instrumentation fixed it	A body containing a call cannot be a hardware loop
volatile counter fixed it	Denies GCC the constant trip count
Saving LCOUNT did nothing	LCOUNT was never the mechanism

Six observations, one cause. That is the shape of a correct diagnosis, and it is the contrast with Chapter 27 §27.6, where four theories each explained *part* of a symptom set and none explained all of it.

The scope is far wider than the scheduler

This is the sentence that makes the defect worth a chapter:

GCC emits hardware loops for ordinary counted C loops, so *every* C function reachable from an interrupt handler was affected — every future driver, and the VM interpreter. The scheduler is simply where it happened to become visible, and it became visible only because a single wrong iteration still produced a plausible-looking answer three times in a row.

The decisive experiment

The hypotheses were separated with a test hook, `task_select_probe()`, holding the selection loop in its original hardware-loop form. Calling the **same function** from two contexts in the **same build**:

```
from kmain (PS.EXCM = 0):  probe(2) = 0    correct
from the ISR (PS.EXCM = 1): probe(2) = 2    wrong
```

with the handler's own PS dumped alongside: `isr ps=0x00060733 — INTLEVEL=3`, and bit 4 `EXCM` **set**. After the fix, the same line reads `isr ps=0x00060723` with `EXCM` clear and `probe(2) = 0`.

The methodological point is stated explicitly:

This is what a controlled comparison is for. The earlier A/B varied the *code generation* and could only ever show correlation; varying the *context* while holding the code identical isolates the cause.

The probe is still in the tree, with its purpose recorded:

```
/* Test hook. Byte-for-byte the selection loop as it was WITHOUT the volatile
 * workaround, so GCC is free to emit a zero-overhead LOOP again. Called from
 * kmain before timer_start(), i.e. single-threaded with no interrupt source
 * armed and no context switching in existence.
 *
 * This separates two very different explanations for the M2 defect:
 * - wrong answers here => the loop is mis-executed on its own, and interrupts
 *   were never involved
 * - correct answers here => the loop is fine in isolation and something about
 *   interrupt context corrupts it
 */
int task_select_probe(int current)
{
    int next = current;
    for (int i = 1; i <= TASK_MAX; i++) {
        int candidate = (current + i) % TASK_MAX;
        if (g_tasks[candidate].state == TASK_READY) {
            next = candidate;
            break;
        }
    }
    return next;
}
```

The fix: five instructions

```
/* Clear PS.EXCM before calling any C.
 *
 * Hardware sets EXCM on interrupt entry, and while it is set the Xtensa
 * zero-overhead loop is DISABLED: the LEND comparison never fires, so a
 * hardware loop body executes exactly once and falls through to whatever
 * instruction sits at LEND. GCC emits these loops for ordinary counted C
 * loops, which means every C function reachable from this handler is
 * affected, not just the one that exposed it.
 *
 * That is what broke M2. ...
 *
 * Safe to clear here: PS.INTLEVEL is still 3, so interrupts stay masked,
 * and EPC3/EPS3 are already saved, so the eventual RFI is unaffected. It
 * also means a fault inside the handler reaches the kernel exception
 * vector and the panic handler, instead of the double-exception vector. */
rsr.ps    a2
movi      a3, ~0x10
and       a2, a2, a3
wsr.ps    a2
rsync
```

Note the third benefit in the safety argument: with `EXCM` cleared, a fault inside the handler reaches the *kernel* exception vector and therefore the panic handler, rather than the double-exception vector. That is a debuggability improvement obtained for free.

The `volatile` workaround was removed. The selection loop is a plain counted loop again, compiles to a hardware `LOOP`, and is correct — with the history recorded in place so nobody reintroduces the workaround:

```
/* Plain counted loop. GCC is free to emit a zero-overhead LOOP here, and
 * does; that is correct now that _handler_level3 clears PS.EXCM before
 * calling C. This loop previously needed a `volatile` counter to force
 * ordinary branches, which worked but treated the symptom. */
```

The standing rule

Standing rule for interrupt handlers — clear `PS.EXCM` before calling C. Hardware sets `EXCM` on interrupt entry, and while it is set the Xtensa zero-overhead loop-back is disabled. GCC emits these loops for ordinary counted C loops, so this silently degrades **every C function reachable from a handler** — drivers and the VM interpreter included, not just the scheduler where it was found. `_handler_level3` now clears it; any future handler must do the same.

8.8 Results

Sustained run, 10 ms tick, three tasks:

t=202	switches r/a/b=68/67/67	work a/b=1528960/1698281	guards=ok
corrupt=0			
t=403	switches r/a/b=135/134/134	work a/b=3104989/3426830	guards=ok

```

corrupt=0
t=604 switches r/a/b=202/201/201 work a/b=4681019/5155380 guards=ok
corrupt=0
t=805 switches r/a/b=269/268/268 work a/b=6257049/6883930 guards=ok
corrupt=0
t=1006 switches r/a/b=336/335/335 work a/b=7833079/8612480 guards=ok
corrupt=0
t=1207 switches r/a/b=403/402/402 work a/b=9409109/10341029 guards=ok
corrupt=0

```

Distribution. 403/402/402 across three tasks — even to within one switch, which is the expected residue of where the run was sampled.

Integrity. `corrupt=0` across ~1,200 preemptions.

Stacks. 463 of 512 words free in both workers, stable across the run: about 196 bytes used, no growth. Guards intact.

Switch cost. 96 bytes of stack and 21 register transfers per switch, twice (save and restore).

Final verification figures: **3,418 ticks, 1,140/1,139/1,139 switches, zero corruption.**

How the workers make corruption visible

The verification tasks hold four values live across a compiler barrier positioned where an interrupt can land:

```

static void worker(volatile uint32_t *count, volatile uint32_t *bad,
                  volatile uint32_t *bumps, uint32_t seed)
{
    uint32_t n      = 0;
    uint32_t magic = seed;
    uint32_t alt    = ~seed;
    uint32_t acc    = seed ^ 0x9E3779B9u;

    for (;;) {
        n++;
        acc = acc * 1664525u + 1013904223u;

        /* Barrier only — keeps the values live across a point where an
         * interrupt can land, without dictating where they live. */
        __asm__ volatile ("": "+r"(n), "+r"(magic), "+r"(alt), "+r"(acc));
        /* ... */
        if (magic != seed || alt != ~seed) {
            (*bad)++;
            magic = seed;      /* repair, so one fault is not counted forever */
            alt   = ~seed;
        }
        *count = n;
    }
}

```

The barrier is a *constraint*, not a *placement*, and that distinction is recorded because getting it wrong manufactured a bug:

Registers are deliberately *not* pinned with explicit `__asm__ ("aN")` bindings: that claims registers the compiler may already be using, and the writes then land on arbitrary memory. An earlier build did exactly this and manufactured a bug that did not exist.

The repair line matters too: without it a single fault would be re-detected forever and the counter would report a corruption rate rather than a corruption count.

The comment above the worker's declaration in `kmain.c` explains why they are still running, ten chapters later:

```
* These tasks are M2 artefacts: they prove a context switch preserves registers
* across arbitrary suspension, and they have now done so 460,800 times with
* corrupt=0. The claim is established; what remains is a REGRESSION CHECK, and
* a regression check does not need most of the machine.
* ...
* Deleting them was the alternative. Kept instead because guards=ok and
* corrupt=0 are the only continuous evidence that the scheduler still does what
* UM-NATOS-009 says it does.
```

8.9 The metrics, including the ones about the process

Quantity	Value
Image size	5,312 B (includes retained instrumentation)
Frame	96 B / 21 words
Tick interval	800,000 cycles (~10 ms)
Tasks	3 of 4 slots at M2; 9 of 12 now
Stack per task	2 KB; ~196 B peak use
Ticks observed	3,418
Switches observed	1,140 / 1,139 / 1,139
Corruption events	0
Build cycles spent on the defect	12
Wrong hypotheses recorded	3
Instructions in the fix	5

The last three rows are unusual to publish and are the most informative in the table. Twelve build-and-flash cycles, three recorded wrong hypotheses, and a five-instruction fix.

8.10 The switch tracing that is still there

Two `#define` switches remain in `task.c`, both defaulted off, both kept for a stated reason:

```
/* Switch tracing. Set to 0 for a quiet boot; raise it to watch the first N
 * switches when touching the handler or the frame layout. Retained rather than
 * deleted because it is what turned M2's silence into a sequence. */
#define TRACE_SWITCHES 0
```

```

/* A/B switch. With the in-loop probes compiled in, the selection loop is
 * correct; with them out, switch 4 returned 2 -> 2 while the task table said
 * every task was READY. Same source otherwise. Flip this to reproduce. */
#define TRACE_PROBES 0

```

When enabled, the trace prints the whole frame about to be restored, plus the task table, and marks whether the frame was fabricated or saved:

```

uart_puts(fabricated ? " (fabricated frame)\n" : " (SAVED frame)\n");

```

Ticks 1-3 restore frames fabricated by task_create; tick 4 is the first restore of a frame the handler itself saved. Dumping both means the saved frame can be read against a known-good fabricated one instead of against expectations.

The trace also honestly declares its own side effect:

```

* This runs at interrupt level 3 and blocks on the UART for the length of the
* dump, which is far longer than a tick period. Ticks are missed as a result
* and timer_late_count() will climb – that is expected and harmless here,
* because the question is what the frame CONTAINS, not when it arrives.

```

And the task table is dumped alongside the frame for a specific diagnostic reason:

```

/* The whole task table. If the round robin stops offering a task, the
 * question is whether the scheduler skipped it or whether its state field
 * stopped saying READY – and those are different bugs. */

```

That is the general form of the counter idiom from Chapter 0b: instrument the two candidate explanations separately so the reading distinguishes them.

8.11 What M2 did not establish

- **No memory protection.** A native task can corrupt any other; guards catch overflow, nothing catches a wild pointer. Isolation arrives only with the bytecode VM.
- **~~No blocking.**~~ → Chapter 9. What remains missing is any wait other than a lock or a deadline — there is no way to wait for an *event*.
- **~~No priorities.**~~ → Chapter 9.
- **~~Fixed ceiling, TASK_MAX = 4.**~~ Now 12. "The ceiling moved, it did not stop being a ceiling" — stacks are still 2 KB and statically allocated.
- **~~No idle task.**~~ Added with blocking, and it executes `WAITI` rather than spinning. Chapter 24 §24.5 records the day it was discovered *not to exist*.
- **Single core.** APP_CPU is untouched.
- **No audit of other EXCM consequences.** §8.7 establishes that hardware loops were degraded while EXCM was set. Whether anything else in the kernel silently depended on EXCM being set for the duration of the handler has not been examined:

Nothing suggests it does — the handler is short and the only C it calls is the tick and the scheduler — but the question was not asked systematically.

That is still true and is in Chapter 30.

Next: what the round robin became once things needed to sleep, block, and be prevented from starving each other.

9. Scheduling: Priority, Sleep, Ageing, and Two Clocks That Lied

Sources: docs/UM-NATOS-009-m2-verification.md §11, docs/UM-NATOS-028 §4 Code: kernel/task.c, kernel/task.h, kernel/kmain.c

9.1 Round robin, and nothing cleverer

M2 shipped a strict round robin over ready tasks, and the report defends the absence of everything else:

Selection scans forward from the current task and takes the first `READY` one. No priorities, no sleeping, no blocking — none of those have a consumer yet, and each would be an untested mechanism in the one code path that must be correct.

Four things were added afterwards, and the order matters:

each was forced by the one before it, and the fourth was forced by a failure the third made possible.

1. `TASK_BLOCKED` arrived with the mutex, which needed it.
2. An **idle task** arrived with blocking, which needed it.
3. `TASK_SLEEPING` arrived with priorities, which needed it.
4. **Ageing** arrived because priorities starved a task to silence.

9.2 Blocking, and the three things it required

`TASK_BLOCKED` is a state the scheduler skips:

```
typedef enum {
    TASK_UNUSED = 0,
    TASK_READY,
    TASK_BLOCKED,      /* waiting on something; the scheduler will not pick it */
    TASK_SLEEPING,     /* waiting for a tick deadline */
} task_state_t;
```

`task_block()` deliberately does not yield

This is the classic lost-wakeup guard and it is worth quoting the whole interface contract, because the split it demands is unusual:

```
/* ---- blocking -----
 *
 * task_block() marks the calling task not-runnable. It does NOT yield — the
 * caller is expected to hold a critical section (critical.h) while blocking so
 * that the decision to block and the record of what it is waiting for cannot be
 * split by a tick, then to leave that section and call task_yield().
```



```

*
*   uint32_t s = crit_enter();
*   wait_list |= 1u << task_current();
*   task_block();
*   crit_exit(s);
*   task_yield();
*
* Nothing here knows what a task is waiting for. That belongs to whatever built
* the wait – see mutex.c. */
void task_block(void);

```

The implementation restates the reasoning at the point where somebody would be tempted to "fix" it:

```

void task_block(void)
{
    if (g_current < 0) {
        return; /* boot context has nothing to block */
    }
    g_tasks[g_current].state = TASK_BLOCKED;
    /* Deliberately does NOT yield. The caller marks itself blocked while still
    * holding a critical section – so the decision to block and the record of
    * what it is waiting for cannot be split by a tick – then leaves the
    * critical section and yields. Yielding here instead would either fire the
    * tick with the wait-list half-updated, or be silently deferred until the
    * caller's crit_exit(), which reads as a bug at the call site. */
}

```

The failure it prevents: *the holder releases, sees an empty wait list, and the waiter sleeps forever*. Chapter 11 shows `mutex_lock()` obeying the protocol exactly.

The idle task

With blocking, a moment where every task is waiting has nothing to switch to, and the old fallback of resuming the interrupted task would run a task that had just blocked.

Idle is registered rather than being a special slot, and is kept **out of the round robin**:

```

/* Registers a task as the idle task: it is chosen only when nothing else is
* runnable, rather than taking an equal share of the round robin. Without one,
* a system where every task blocks has nothing to run. */
void task_set_idle(int id);

```

leaving it in the rotation would spend a seventh of the machine deliberately doing nothing.

It executes `WAITI 0`, which stops the core until an interrupt arrives:

```

/* Idle. Runs only when every other task is blocked. WAITI stops the core until
* an interrupt arrives, so an idle system draws less power than one spinning –
* and the tick is what wakes it. */
static void task_idle(void)
{
    for (;;) {
        __asm__ volatile ("waiti 0");
    }
}

```

```

    }
}

```

Chapter 24 §24.5 records that this task silently failed to be created for months, which cost both the power saving and — more seriously — the invariant it exists to protect.

9.3 Sleeping

A sleeping task carries a tick deadline; the scheduler wakes expired sleepers on every tick, so a sleep resolves to one tick:

```

/* Wakes any sleeping task whose deadline has passed. Called from the scheduler,
 * which runs every tick, so a sleep resolves to one tick. */
static void wake_sleepers(void)
{
    uint32_t now = timer_ticks();
    for (int i = 0; i < TASK_MAX; i++) {
        if (g_tasks[i].state == TASK_SLEEPING &&
            (int32_t)(now - g_tasks[i].wake_tick) >= 0) {
            g_tasks[i].state = TASK_READY;
        }
    }
}

```

task_wake() and the woken flag

There is a second way out of a sleep, and its design is one of the more careful things in the kernel. `task_wake()` takes **SLEEPING as well as BLOCKED**, and the comment explains why the combination is what a peripheral wait needs:

```

void task_wake(int id)
{
    if (id < 0 || id >= TASK_MAX) {
        return;
    }
    /* Deliberately takes SLEEPING as well as BLOCKED, which is the whole
     * difference from task_unblock().
     *
     * A task waiting on hardware wants two things at once: to be released the
     * instant the device speaks, and to be released anyway if it never does. A
     * plain block gives the first without the second, and a task blocked on a
     * peripheral that has failed is deaf forever. A plain sleep gives the second
     * without the first.
     *
     * Sleeping with a deadline and letting an interrupt cut it short gives both,
     * and the failure mode is the good one: if the interrupt never arrives, the
     * deadline still fires and the caller falls back to polling. That is how the
     * touch task survives a PENIRQ that does not work — it degrades to exactly
     * what it did before this existed rather than to nothing.
     *
     * Safe from an ISR. It only promotes a state to READY; wake_tick is left
     * alone, and the scheduler's sleep sweep ignores a task that is no longer
     * SLEEPING. */
    if (g_tasks[id].state == TASK_BLOCKED ||
        g_tasks[id].state == TASK_SLEEPING) {

```

```

        g_tasks[id].state = TASK_READY;
        g_tasks[id].woken = 1;
    }
}

```

"It degrades to exactly what it did before this existed rather than to nothing" is the design goal, and Chapter 22 §22.6 is the case where it was needed: PENIRQ is routed, its handler runs, and no finger has ever been observed to trigger it. The touch task polls, and the interrupt path is one line from reinstatement.

The woken flag distinguishes the two exits:

```

/* Set by task_wake(), cleared by task_sleep(). Distinguishes "an interrupt
 * cut this sleep short" from "the deadline has not arrived yet", which
 * task_sleep must tell apart to re-arm without defeating touch_irq_wait. */
volatile uint8_t woken;

```

9.4 The sleep that did not sleep

`task_sleep()` is a loop, and it became one because of a defect found during the Wi-Fi work (UM-NATOS-028 §4). The reason is subtle enough that the source carries the measurement:

```

void task_sleep(uint32_t ticks)
{
    if (g_current < 0) {
        return;                      /* no context to sleep */
    }

    /* Loops, because task_yield() does not switch – it ARMS a switch.
     *
     * It writes CCOMPARE1 to ccount + 64 and returns; the context change
     * happens when that comparator fires, roughly sixty cycles later. So the
     * caller keeps running past this function for a short window, and whatever
     * it does in that window happens DURING what it believes is a sleep.
     *
     * Measured directly: task_sleep(50) – half a second – returned to its
     * caller in 107 cycles, which is the cost of the function body and nothing
     * else. The sleep did happen; it just started after the caller had already
     * read the clock and concluded no time had passed. Two separate
     * measurements this session were wrong because of it, including the first
     * TSF check, which is what exposed it.
     *
     * Re-arming until the deadline genuinely arrives closes that window and
     * costs one extra pass in the normal case.
     *
     * The `woken` flag is what keeps this from breaking touch_irq_wait. That
     * caller wants a sleep an interrupt can cut short, so "the deadline has not
     * arrived" must not be confused with "task_wake released me deliberately" –
     * without the flag, this loop would put the task straight back to sleep and
     * turn every early wake into a full-length one. */
    uint32_t deadline = timer_ticks() + ticks;

    uint32_t crit = crit_enter();
    g_tasks[g_current].woken = 0;
}

```

```

crit_exit(crit);

for (;;) {
    crit = crit_enter();
    if (g_tasks[g_current].woken) {
        g_tasks[g_current].woken = 0;
        crit_exit(crit);
        return;                /* released early, on purpose */
    }
    if ((int32_t)(timer_ticks() - deadline) >= 0) {
        crit_exit(crit);
        return;                /* the time really has passed */
    }
    g_tasks[g_current].wake_tick = deadline;
    g_tasks[g_current].state     = TASK_SLEEPING;
    crit_exit(crit);
    task_yield();
}
}

```

Two bugs were stacked here and are worth separating.

task_yield() arms a switch; it does not perform one. The caller runs on for ~60 cycles into what it believes is a sleep. `task_sleep(50)` returned in 107 cycles.

timer_ticks() counted ISR entries, not time (Chapter 7 §7.10), so the deadline the loop was comparing against was measured in the wrong unit.

After both fixes:

`task_sleep(50)` now takes 639 ms / 64 ticks, and $64 \times 10 \text{ ms} = 640 \text{ ms}$ — the tick count and the wall clock agree, which they did not before.

And the sting:

But the wider point is the uncomfortable one, and `timer.c` had already written it down about an *earlier* bug in the same function: **nothing showed a symptom**. Sleeps ran short, timeouts ran loose, and every frame-rate figure this project has ever recorded was measured against a clock running fast by a load-dependent factor.

The touch regression this fix caused

Fixing `task_sleep` broke something that had been accidentally relying on it being broken, which is worth recording as a category of hazard.

The touch loop originally called `task_sleep(1u)` back when `task_sleep` neither slept nor yielded. So it polled **repeatedly inside whatever slice it got**, which is exactly why touch felt continuous. Replacing it with `task_yield()` looked equivalent and was not:

a yield surrenders the CPU after **every single poll**, so the task sampled once per slice instead of many times, and a quick tap fell between samples.

Fixing `task_sleep` had, with perfect irony, broken the thing that was accidentally relying on it being broken.

The final configuration is a real 10 ms sleep at HIGH priority, and `kmain.c` records the reasoning and the measurement:

```
/* HIGH, with the renderer, and polling on a real 10 ms sleep.
 * ...
 * HIGH because touch is user INPUT: at NORMAL it runs only when ageing
 * rescues it from behind a renderer that never sleeps, roughly every
 * 300 ms, and a quick tap falls between samples. A real sleep rather than
 * a yield because a yield surrenders the CPU after EVERY poll, sampling
 * once per slice instead of many times. It costs one SPI read per tick.
 *
 * Measured: ~15 samples per reporter interval before, ~150 after. */
task_set_priority(id_touch, TASK_PRIO_HIGH);
```

Ten times the sample rate for *less* CPU than the old flat-out polling, and — unlike it — "a rate that is a property of the clock rather than of whatever else happens to be running".

9.5 Priorities

Three levels, strictly ordered, round-robin among equals:

```
#define TASK_PRIO_LOW    0
#define TASK_PRIO_NORMAL 1
#define TASK_PRIO_HIGH   2
#define TASK_PRIO_LEVELS 3
```

The implementation is one loop, and the elegance is that two properties fall out of one condition:

The scan starts past the current task and takes a candidate only on *strictly* greater priority, which yields both properties at once — among equals the first one met wins, and the scan begins somewhere different each time.

Strict priority is only safe because tasks sleep

```
* Strict ordering is only safe because tasks SLEEP. A high-priority task that
* spins instead of sleeping starves everything beneath it completely – this
* kernel's previous `while (...) task_yield();` idiom would do exactly that.
* task_sleep() is therefore not a convenience here; it is what makes priorities
* usable at all.
```

That is not hypothetical. The demonstration workers were first placed at LOW and performed **exactly zero iterations**, because the NORMAL band never empties.

The failure was quiet in the worst way: `corrupt=0` continued to be reported, and means nothing when no work is being done. An instrument reading healthy because its subject had stopped is the same shape as [the EXCM defect] and as [the frozen marker].

`kmain.c` records the resulting policy in full, including the reason nothing sits at LOW:

```

/* Priorities. The renderer is the only HIGH task: it wakes, draws a frame,
 * and sleeps, so it takes what it needs and then stands aside. Strict
 * priority is only safe because of that sleep – a HIGH task that spun would
 * starve every level beneath it outright.
 *
 * Nothing sits at LOW. Strict priority starves it absolutely: the workers
 * were put there first and did exactly zero iterations, because the NORMAL
 * band never empties – the application host never sleeps. That killed the
 * M2 register-integrity evidence, since corrupt=0 means nothing when no
 * work is being done.
 *
 * The renderer's speedup came from being HIGH, not from anything being LOW,
 * so everything else shares NORMAL. ... */

```

Measured

```

renderer alone at HIGH:    2 fps -> 8.5 fps
with a busy NORMAL band:  ~3 fps
workers: 821,248 iterations each, exactly equal, corrupt=0

```

with a caveat added in a later revision that is the point of Chapter 28:

These frame rates were computed as frames per TICK, and the tick was later found to stall for up to 183 ms at a time. The comparison is still valid — both figures came from the same clock — but the absolute values are not trustworthy. The renderer measured 16.0 fps once the clock was corrected.

The workers also sleep one tick per N iterations, and the reasoning is precise about what the evidence requires:

They exist to prove the switch preserves registers across arbitrary suspension, which needs them running *continuously*, not *constantly*. Spinning flat out bought no extra evidence, cost the renderer half its frame rate, and — because it ended the mutex thrash between them — sleeping raised their own throughput almost tenfold.

9.6 Ageing, because sleeping was not enough

§9.5 argues strict priority is safe **because tasks sleep**. That argument is sound and it is not a guarantee: it depends on every high-priority task being written to yield often enough, and nothing enforces it.

It failed the first time a task was made hungrier. Shortening the display task's sleep from 8 ticks to 2 left it ready roughly 61% of the time, and the reporter task stopped being scheduled at all:

```

reporter lines in 20 s    0
crash                    none
watchdog reset           none

```

And the reason nothing caught it:

Nothing was wrong that any existing mechanism could see. **The hang detector asks whether ANY distinct switch happened, not whether every ready task got a turn,** and switches were happening constantly between the display task and its neighbours. The only reason the starvation was noticed at all is that the starved task happened to be the one that prints. A quieter victim would have produced no symptom.

The policy

```
effective = base + min(waiting / TASK_AGE_TICKS, TASK_AGE_MAX)
```

```
#define TASK_AGE_TICKS 30u
#define TASK_AGE_MAX 3u /* cap, so ageing cannot invert LOW past HIGH twice over */
```

Three properties are deliberate.

Ageing is a property of the SELECTION, not of the task.

```
/* Effective priority = base + ageing credit, computed per candidate below.
 * The base priority is never modified: ageing is a property of the
 * SELECTION, not of the task, so a task that finally runs returns to its
 * declared priority automatically rather than needing to be restored. That
 * distinction is what keeps this separate from priority inheritance, which
 * really does change a task's priority and really does have to undo it. */
```

The report puts the general form well: **a mechanism that must remember to restore something is a mechanism that can forget.**

Only READY tasks earn credit. A task waiting on a deadline or a mutex is not being treated unfairly by the scheduler; crediting it would let it barge ahead the moment it became runnable.

The bound is a latency, not a throughput guarantee. `TASK_AGE_TICKS × 2` is 600 ms at the shipped values. "A task that is aged in and then immediately blocks still makes no progress, and that is the caller's problem."

The selection loop, whole

```
int next = -1;
int best = -1;
for (int i = 1; i <= TASK_MAX; i++) {
    int candidate = (g_current + i) % TASK_MAX;
    if (candidate == g_idle_id) {
        continue;
    }
    if (g_tasks[candidate].state == TASK_READY) {
        uint32_t credit = g_tasks[candidate].waiting / TASK_AGE_TICKS;
        if (credit > TASK_AGE_MAX) {
            credit = TASK_AGE_MAX;
        }
        int eff = (int)g_tasks[candidate].prio + (int)credit;
        if (eff > best) {
            best = eff;
            next = candidate;
        }
    }
}
```

```

    }
}
/* Nothing else runnable – fall back to idle. Resuming the interrupted task
 * is NOT an acceptable answer once blocking exists: that task may be the
 * one that just blocked, and running a blocked task defeats the whole
 * mechanism. */
if (next < 0 && g_idle_id >= 0 && g_tasks[g_idle_id].state == TASK_READY) {
    next = g_idle_id;
}

if (next < 0) {
    /* No runnable task and no idle task. Before blocking existed this could
     * only happen at boot; it can now also mean every task is blocked with
     * nobody left to wake them, which is a deadlock the kernel cannot
     * resolve. Resuming the interrupted context is the least-bad answer and
     * at least keeps the console alive to say so. */
    return current_sp;
}

```

And the bookkeeping, done once the decision is final:

```

/* Every READY task that was NOT chosen waits one more tick; the chosen one
 * resets. Sleeping and blocked tasks are untouched – a task waiting on a
 * deadline or a mutex is not being treated unfairly by the scheduler, and
 * crediting it would let it barge ahead the moment it becomes runnable. */
for (int i = 0; i < TASK_MAX; i++) {
    if (g_tasks[i].state != TASK_READY) {
        continue;
    }
    if (i == next) {
        if (g_tasks[i].waiting > g_max_wait) {
            g_max_wait = g_tasks[i].waiting;
        }
        if (g_tasks[i].waiting >= TASK_AGE_TICKS) {
            g_age_rescues++; /* it only ran because ageing lifted it */
        }
        g_tasks[i].waiting = 0;
    } else {
        g_tasks[i].waiting++;
    }
}

```

Measured, including the number that is zero

Re-running the configuration that caused the starvation:

```

reporter lines in 20 s    0 -> 8
fair maxwait             35 ticks (350 ms), bounded as designed
fair rescues             114 decisions changed by ageing

```

At the shipped sleep value it reads `rescues=0, maxwait=15` — **inert in normal operation**, which is correct for a backstop. Both numbers are reported anyway, and the reason is a good general principle:

a fairness policy nobody measures is a fairness policy nobody has, and `rescues=0` is the only thing that distinguishes "never needed" from "never working".

```
/* Longest any ready task has waited without being scheduled, in ticks, and the
 * number of times ageing changed the decision. Reported because a fairness
 * policy nobody measures is a fairness policy nobody has. */
uint32_t task_max_wait(void);
uint32_t task_age_rescues(void);
```

What it did not do

It did not raise the frame rate. With ageing in place *and* the display sleep shortened, the renderer still ran at 3.0 fps — identical to before. The frame rate was bounded by lock contention, which is a different problem that ageing does not touch.

Ageing fixed a class of silent failure. It is not a performance feature and is not recorded as one.

9.7 Priority inheritance

Priorities existed for less than a day before inversion did. The renderer runs at HIGH and takes the display lock; the applications beneath it run at NORMAL and take the same lock. A NORMAL task holding it while the renderer waits can be preempted by any other NORMAL task, so the renderer waits not for the critical section but for the whole NORMAL band to drain.

```
/* Priority, and the boost used for inheritance. task_boost() only ever raises;
 * task_unboost() returns to the priority the task was created with. */
void task_boost(int id, int prio);
void task_unboost(int id);
```

```
void task_boost(int id, int prio)
{
    if (id >= 0 && id < TASK_MAX && prio > (int)g_tasks[id].prio) {
        g_tasks[id].prio = (uint8_t)prio;
    }
}

void task_unboost(int id)
{
    if (id >= 0 && id < TASK_MAX) {
        g_tasks[id].prio = g_tasks[id].base_prio;
    }
}
```

Release restores the **base** priority rather than tracking nested holds, and the limitation is recorded rather than solved:

a task holding two boosted mutexes loses the boost on the first release. Recorded rather than solved: nothing in this kernel holds two, and inventing a nesting scheme with no consumer would be guessing at requirements.

And the finding that matters more than the feature:

Inheritance bounds the *cost of waiting*. It does nothing about [the frequency] lesson, which is about the *number of waits*. A raycaster took the display lock once per column and spent 25 seconds per frame on 37 ms of work; holding it once per frame instead was a **194×** improvement. Inheritance would not have helped there at all.

Chapter 11 is that argument in full.

9.8 CPU time accounting: the third clock

`xt_ccount()` measures wall clock, which counts every preemption as though it were the work. This produced wrong numbers throughout the project until a per-task cycle count was added:

```
/* ---- CPU time accounting -----
 *
 * Cycles each task has actually RUN, as opposed to how much wall-clock passed
 * while it was trying to.
 *
 * Every timing figure in this kernel used to be a pair of xt_ccount() reads
 * around some work, which counts every tick, every other task's slice and every
 * interrupt that landed in the middle as though it were the work itself. The
 * measured cost of a full-screen fill was 43 ms before the scheduler existed
 * and 249-362 ms from a task afterwards – the same bytes to the same panel,
 * differing by eight times in nothing but bookkeeping.
 *
 * Charged at the switch, which is the only place that knows a task stopped
 * running. Time spent inside the level-3 handler is charged to whichever task
 * it interrupted; that is a small and deliberate inaccuracy, and the
 * alternative is accounting inside the ISR that measures the ISR.
 *
 * MEASURED, and the correction is not uniform. A full-screen fill from the
 * SHELL task read 249-362 ms wall-clock and 63-79 ms of work – the shell runs
 * at NORMAL and is preempted constantly, so nearly all of that was other tasks.
 * The raycaster's blit did not move at all: 55.5 ms either way, because the
 * display task runs at HIGH and is barely interrupted, so wall-clock was
 * already very close to its work.
 *
 * Which means the fix matters most exactly where the old numbers were least
 * trusted, and changes nothing where they were quietly correct. A conclusion
 * drawn from one task's timings could not have been carried to another's. */
static uint32_t g_run_cycles[TASK_MAX];
static uint32_t g_slice_start; /* ccount when the running task got the CPU */
```

The accessor needs a critical section, and the reason is a good illustration of how a two-word read can be non-atomic:

```
uint32_t task_cpu_cycles(void)
{
    uint32_t crit = crit_enter();
    uint32_t v = (g_current >= 0)
        ? g_run_cycles[g_current] + (xt_ccount() - g_slice_start)
        : xt_ccount();
    crit_exit(crit);
}
```

```

    return v;
}

```

- * The critical section is not optional: without it a tick landing between the
- * two loads returns an accumulated total from before the switch added to a
- * slice start from after it, which reads as a wildly negative interval.

And then this clock lied too. UM-NATOS-028 §8 lists it first among the instruments that lied:

The blit timer. `t_blit` uses `task_cpu_cycles()`, which only advances when the scheduler credits a slice at a context switch — it does not tick *inside* one. So the figure tracks context switches, not work. A 55.85 → 31.3 ms shift was read as a serious regression, reported to the user as a 44% *improvement* an hour earlier, and was neither. It was an artifact both times.

Three clocks, each correct for a different question, and each having been used for the wrong one at least once. The summary is in Chapter 0b's table and it is worth keeping in front of you for the rest of the book.

9.9 The final task set

Nine tasks, created in `kmain` with checked creation:

```

id_report = must_create("report", task_report);
id_a      = must_create("worker-a", task_a);
id_b      = must_create("worker-b", task_b);
id_vm     = must_create("vm-host", task_vm);
id_apps   = must_create("app-host", task_apps);
id_shell  = must_create("shell", task_shell);

/* Created last and registered as idle, so it is outside the round robin and
 * chosen only when every other task is blocked. Without it, a moment where
 * all tasks are waiting has nothing to switch to. */
id_disp   = must_create("display", task_display);
id_touch  = must_create("touch", task_touch);
id_idle   = must_create("idle", task_idle);
task_set_idle(id_idle);

```

`must_create` exists because of Chapter 24 §24.5:

```

/* Creates a task or stops the kernel. See the note at the first call site. */
static int must_create(const char *name, task_entry_fn entry)
{
    int id = task_create(name, entry);
    if (id < 0) {
        kernel_panic_msg("task table full - raise TASK_MAX", 0);
    }
    return id;
}

```

with the call-site note:

```
/* Checked, because the unchecked version cost this kernel its idle task.
 * task_create() returns -1 when the table is full, kmain made nine calls
 * against a TASK_MAX of 8, and the ninth failed silently for months: the
 * return value was assigned to id_idle, passed to task_set_idle(), which
 * bounds-checks and ignores it, and never printed. A creation that cannot
 * fail quietly is worth the four lines. */
```

Task	Priority	Role
report	NORMAL	Telemetry every 200 ticks; runs the critical-section self-test
worker-a, worker-b	NORMAL	M2 regression: register integrity across preemption
vm-host	NORMAL	The kernel's own VM, running spin
app-host	NORMAL	app_tick() — the third scheduling level
shell	NORMAL	Polls UART0
display	HIGH	Renderer / launcher / note pad / terminal
touch	HIGH	100 Hz XPT2046 poll
idle	—	WAITI 0, outside the rotation

9.10 Metrics

Quantity	Value
Priority levels	3, plus ageing credit up to 3
Ageing threshold	30 ticks per level, capped at 3
Worst-case wait for a ready task	~600 ms, bounded
Longest wait observed under stress	35 ticks (350 ms)
Ageing rescues at shipped settings	0 — inert, as intended for a backstop
Tasks	9 of 12 slots
task_sleep(50) before / after	107 cycles / 639 ms
Touch samples per reporter interval, before / after	~15 / ~150

Next: the allocator underneath all of this, and the single invariant that makes it checkable.

10. Memory: A Heap With One Invariant, and Arenas

Sources: docs/UM-NATOS-010-m3-verification.md, docs/UM-NATOS-011-flash-cache.md
Code: kernel/heap.c, kernel/heap.h, kernel/arena.c, kernel/arena.h

10.1 What M3 delivers, and what it actually settles

Milestone 3 delivers the kernel allocator and the arena model that bytecode applications run inside. Its exit criteria are narrow and measurable: no leak across 10,000 allocate/free cycles, arena bounds queryable by the interpreter, and exhaustion that fails rather than corrupts.

All three pass. But:

The more consequential outcome is not the pass — it is the number the allocator finally puts on the DRAM budget, which settles a question M5 could otherwise only have guessed at.

That number is §10.7.

10.2 The structure: address-ordered, physically linked

The heap is a doubly-linked list of blocks in address order, covering the region with no gaps. Each block carries a 16-byte header: both physical neighbours, its payload size, and a magic word that doubles as the used/free flag.

```
typedef struct block {
    struct block *prev;    /* physically previous block, NULL at heap start */
    struct block *next;    /* physically next block, NULL at heap end */
    uint32_t size;        /* payload bytes, always a multiple of HEAP_ALIGN */
    uint32_t magic;       /* BLK_FREE or BLK_USED */
} block_t;
```

	Segregated free lists	Address-ordered physical list (selected)
Allocation	O(1) typical	O(n) first fit
Coalescing	Needs a search or boundary tags	O(1) both directions
State to verify	Several lists plus size classes	One list, one invariant
Overhead	Lower	16 B per block

The choice is explicitly not about throughput:

Throughput is not the constraint. This allocator serves arena creation and a small number of kernel structures, not millions of short-lived objects. What matters is that the structure can be *checked*, and a single address-ordered list tiles the heap exactly, which

makes the invariant trivially statable: **every block's payload must end precisely where the next block's header begins.**

That one sentence is what makes `heap_check()` (§10.5) possible.

The file header restates it:

```
* Address-ordered with physical links means coalescing is O(1) in both
* directions and needs no search. The cost is 16 bytes per allocation, which is
* the right trade here: arenas are large and few, and the alternative – a
* segregated free list – buys throughput this kernel does not need in exchange
* for state that is harder to verify.
```

10.3 The magic word earns its four bytes

```
/* Magic values chosen to be implausible as data, and distinct from each other
 * so a block's state is unambiguous even in a raw memory dump. */
#define BLK_FREE 0xF2EEB10Cu
#define BLK_USED 0x05EDB10Cu
```

Two properties, both deliberate. They are implausible as data — so a header is either valid or *obviously* not — and they are distinct from each other, so the free/used flag needs no separate field and a raw dump is readable.

The consequence:

A double free, an interior pointer, or a wild pointer is therefore **counted and refused** rather than linked into the list.

And the justification, which is a memory-protection argument rather than a defensive-programming one:

On a kernel with no memory protection this is not defensiveness for its own sake. A corrupted free list surfaces as a fault in unrelated code much later, and M2 already demonstrated how expensive it is to debug a symptom that appears far from its cause.

The check happens inside the critical section, and the comment says why that matters:

```
/* A live allocation is the only thing that may be freed. Anything else –
 * double free, interior pointer, wild pointer – is counted and refused.
 * Checked inside the section: two tasks freeing the same pointer must not
 * both see BLK_USED. */
if (b->magic != BLK_USED) {
    g_bad_free++;
    crit_exit(crit);
    return;
}
```

Coalescing poisons the absorbed header for the same reason:

```
n->magic = 0;          /* poison, so a stale pointer is not mistaken
 * for a live header */
```

10.4 Allocation

First fit. A block is split only when the remainder can hold a header plus a useful payload:

```
/* Splitting only pays if the remainder can hold a header plus a useful
 * payload. Below this the leftover is left attached to the allocation, which
 * wastes a few bytes but avoids manufacturing unusable slivers. */
#define SPLIT_MIN (HDR_BYTES + HEAP_ALIGN)
```

```
void *heap_alloc(uint32_t bytes)
{
    if (bytes == 0u) {
        return 0;          /* a zero-size allocation has no useful answer */
    }

    uint32_t want = align_up(bytes);

    /* The free list is walked and rewritten here, and a tick landing mid-split
     * would let another task observe – or allocate from – a half-linked block.
     * A critical section rather than a mutex: these are a handful of memory
     * operations, and a mutex would make the allocator depend on the scheduler
     * that may itself want to allocate. */
    uint32_t crit = crit_enter();

    for (block_t *b = g_first; b; b = b->next) {
        if (b->magic != BLK_FREE || b->size < want) {
            continue;
        }

        /* Split only when the tail can stand on its own. */
        if (b->size - want >= SPLIT_MIN) {
            block_t *tail = (block_t *)((char *)b + HDR_BYTES + want);
            tail->size = b->size - want - HDR_BYTES;
            tail->magic = BLK_FREE;
            tail->prev = b;
            tail->next = b->next;
            if (tail->next) {
                tail->next->prev = tail;
            }
            b->next = tail;
            b->size = want;

            /* The new header consumes payload that used to be allocatable. */
            g_total -= HDR_BYTES;
        }

        b->magic = BLK_USED;
        g_used += b->size;
        if (g_used > g_high_water) {
            g_high_water = g_used;
        }
        crit_exit(crit);
        return payload_of(b);
    }

    g_fail++;
}
```

```

    crit_exit(crit);
    return 0;
}

```

The lock choice is argued in one clause and it is the same argument Chapter 11 makes at length: *a mutex would make the allocator depend on the scheduler that may itself want to allocate.*

heap_total() moves, and that is honest

Note `g_total -= HDR_BYTES` on split and `g_total += HDR_BYTES` on coalesce. Splitting consumes payload to create a header; coalescing returns it. So the *usable* total moves as the heap fragments:

`heap_total()` tracks that honestly rather than reporting a fixed figure.

That is a small decision with a real consequence for §10.6: the no-leak test checks that `heap_total()` returns to its baseline, which would be trivially true if the figure were a constant.

10.5 heap_check(): ten ways to be wrong

The structural check walks the list and returns a non-zero code identifying the *first* violated invariant:

```

int heap_check(void)
{
    uint32_t lo = align_up((uint32_t)&_heap_start);
    uint32_t hi = (uint32_t)&_heap_end & ~(HEAP_ALIGN - 1u);

    if (!g_first) {
        return g_total == 0u ? 0 : 1;
    }
    if ((uint32_t)g_first != lo) {
        return 2;
    }

    uint32_t span = 0, used = 0;
    block_t *prev = 0;

    for (block_t *b = g_first; b; prev = b, b = b->next) {
        if (b->magic != BLK_FREE && b->magic != BLK_USED) {
            return 3; /* corrupt header */
        }
        if (b->prev != prev) {
            return 4; /* broken back link */
        }
        if ((b->size % HEAP_ALIGN) != 0u) {
            return 5;
        }
        /* Blocks must tile the region exactly: the next header has to begin
         * where this block's payload ends. A gap or overlap means the split or
         * coalesce arithmetic is wrong. */
        char *end = (char *)b + HDR_BYTES + b->size;
        if (b->next && (char *)b->next != end) {

```



```

        return 6;
    }
    if (!b->next && (uint32_t)end != hi) {
        return 7;
    }
    /* Two adjacent free blocks mean a coalesce was missed, which is how a
     * heap fragments itself to death while reporting plenty free. */
    if (b->magic == BLK_FREE && b->next && b->next->magic == BLK_FREE) {
        return 8;
    }

    span += b->size;
    if (b->magic == BLK_USED) {
        used += b->size;
    }
}

if (used != g_used) {
    return 9;
}
if (span != g_total) {
    return 10;
}
return 0;
}

```

Ten distinct codes: corrupt header, wrong first block, broken back link, misalignment, gap or overlap between blocks, list ending short of the region, missed coalesce, and two accounting disagreements between the walk and the running totals.

Two of those deserve highlighting.

Code 8 — missed coalesce. "That is how a heap fragments itself to death while reporting plenty free." It is the one failure a byte-count leak test cannot see.

Codes 9 and 10 — accounting versus the walk. The running counters are maintained incrementally; the walk recomputes them. Any disagreement means the incremental bookkeeping and the structure have diverged, which is exactly the class of bug that produces a heap that looks fine until it does not.

The check is called after *every phase* of the self-test rather than only at the end, "so a failure names the operation that caused it".

10.6 The three criteria, and what each was designed to catch

Criterion 1 — no leak

10,000 allocate/free cycles across 8 rotating slots with pseudo-random sizes from 16 to 515 bytes, so blocks are split and coalesced continuously.

The test is deliberately stronger than "free bytes return to baseline":

the **largest free block** and the **block count** must also return, because a heap that has fragmented into unusable slivers still reports the correct number of free bytes. That is

the failure a naive leak test misses.

```
[1] no leak : PASS after 10000 cycles free=166432 largest=166432
      blocks=1 check=0
```

Returning to one block is the strong form of the result: it says every one of the ~10,000 splits was undone by a matching coalesce, not merely that the byte totals happen to balance.

Criterion 2 — arena bounds

Ten cases, chosen to include the ones a naive check gets wrong: exact fit, last byte, zero-length access, one past the end, one before the base, one byte too long, **a length chosen to wrap the address space**, an address belonging to a different arena, and a bogus id.

```
[2] arenas : PASS live=2 committed=5120 B base=0x3ffb27e0 len=4096
```

Criterion 3 — exhaustion and refusal

```
[3] oom safe : PASS oversize=NULL fails=1 bad_frees=2 check=0
arenas freed : check=0 free=166432/166432 rejects=1 high_water=5120 B
```

An oversized request returned `NULL` and incremented the failure counter, leaving the heap byte-identical and structurally clean. The double free and the stack-address free were both counted and neither perturbed the list.

The test runs before the tick

The self-test runs single-threaded in ``kmain`` before the tick is armed. Nothing in it can be disturbed by a context switch, so an M3 failure cannot be blamed on M2 — and vice versa.

That is a small structural decision with a large debugging payoff: it removes an entire class of "maybe it's the scheduler" from any failure in this test.

10.7 Arenas

An arena is a contiguous block of DRAM with a recorded base and length. Four slots, statically:

```
#define ARENA_MAX 4
```

```
typedef struct {
    uint32_t base;      /* 0 when the slot is unused */
    uint32_t len;
} arena_t;
```

Not resizable, and why

An arena's base is the value the interpreter holds and bounds-checks against. Allowing it to move would mean every bytecode program must survive its memory being relocated underneath it — a much harder property to guarantee than telling a program its size once, at start.

Zeroed at creation

```
/* Zero before handing over. An arena is application-visible memory and
 * must not leak the previous occupant's contents. */
uint32_t *w = (uint32_t *)p;
uint32_t n = bytes / 4u;
for (uint32_t i = 0; i < n; i++) {
    w[i] = 0;
}
uint8_t *tail = (uint8_t *)p + (n * 4u);
for (uint32_t i = 0; i < (bytes % 4u); i++) {
    tail[i] = 0;
}
```

Word-at-a-time with a byte tail, because `-fno-builtin` plus `-nostdlib` means there is no `memset` unless `kstring.c` supplies one, and this file predates it.

10.8 arena_contains(), and the offset domain

This is the most important function in the kernel that is four lines long.

```
int arena_contains(int id, uint32_t addr, uint32_t len)
{
    if (id < 0 || id >= ARENA_MAX || g_arenas[id].base == 0u) {
        return 0;
    }

    uint32_t base = g_arenas[id].base;
    uint32_t size = g_arenas[id].len;

    /* A zero-length access is inside any live arena provided its start is.
     * Treating it as out of bounds would reject legitimate empty copies. */
    if (addr < base) {
        return 0;
    }

    uint32_t offset = addr - base;
    if (offset > size) {
        return 0;
    }

    /* Compare in the offset domain, where the arena length bounds both terms.
     * Testing `addr + len` directly can wrap past the end of the address space
     * and report success for an access that is wildly out of range — the exact
     * case an attacker-supplied length would aim for. */
    if (len > size - offset) {
        return 0;
    }
}
```

```

    return 1;
}

```

The offset-domain comparison is the whole point:

The wrap case is why `arena_contains()` works in the offset domain. Testing `addr + len` directly overflows and reports success for an access far outside the arena — precisely what a hostile length would target. Since this function is the only thing standing between a bytecode program and the rest of DRAM, it is implemented once and exported rather than reimplemented per caller.

That last clause was honoured for exactly one milestone. The VM duplicates it for performance (`vm_in_bounds`, Chapter 14), and the duplication is *tested* rather than trusted — 35 cases cross-checked. The same offset-domain reasoning then propagates to three more places:

Where	What wraps if done naively
<code>arena_contains()</code>	<code>addr + len</code> past the address space
<code>vm_in_bounds()</code>	same, on cached base/size
<code>vp_fill()</code> (Chapter 17)	<code>x + w</code> where <code>x</code> is a "negative" coordinate arriving as <code>0xFFFFFFFF</code>
<code>SYS TOUCH</code> (Chapter 17)	<code>t.x - vm->vx</code> underflowing to a huge value

The header declares the property as part of the contract:

```

/* The bounds check itself, exported so there is exactly one implementation of
 * it rather than one per caller. Returns non-zero when [addr, addr+len) lies
 * entirely inside the arena. Overflow-safe: an addr+len that wraps is refused,
 * which is the case a naive check gets wrong. */
int arena_contains(int id, uint32_t addr, uint32_t len);

```

10.9 The DRAM budget, measured

UM-NATOS-007 §5 named arena sizing as M3's principal risk and asked for measurements rather than guesses.

Of 180,736 B of DRAM: 10,160 B is kernel data, `.rodata` and task stacks; 4,096 B is the boot stack reservation; **166,432 B is allocatable**.

Consumer	Bytes	Share of heap
Full 240×320 16-bit framebuffer	153,600	92.3%
Remaining for all arenas	12,832	7.7%

That table is the argument of Chapter 1 §1.4 and it landed here, at M3, on the strength of a real allocator rather than an estimate.

The figure has moved three times since, and the reasons are recorded:

Figure	When	Why
166,432 B	M3	Baseline

Figure	When	Why
167,680 B	Flash cache	.rodata moved out of DRAM — grew by <i>exactly</i> the 1,248 B of the section that moved
158,048 B	M5	TASK_MAX 4 → 8, adding 8 KB of static stacks, plus the app table and shell buffers
lower, unmeasured	now	TASK_MAX is 12 and three more drivers exist

The status summary is honest about the last row:

Measured at M3 — 167,680 B allocatable then, 158,000 B after TASK_MAX rose to 8; it has since risen to 12 and three drivers have been added, so the current figure is lower and unmeasured.

A live figure is available at any time from the shell:

```
mem -> heap free=155952 largest=155952 blocks=4 high_water=5120 check=0
```

```
static void cmd_mem(void)
{
    uart_puts("  heap free=");
    uart_put_dec(heap_free_bytes());
    uart_puts(" largest=");
    uart_put_dec(heap_largest_free());
    uart_puts(" blocks=");
    uart_put_dec(heap_blocks());
    uart_puts(" high_water=");
    uart_put_dec(heap_high_water());
    uart_puts(" check=");
    uart_put_dec((unsigned int)heap_check());
    uart_puts("\n");
}
```

largest beside free is the fragmentation indicator; check beside both is the structural one. Three numbers that together say something no one of them can.

10.10 The heap under concurrency

M3 shipped with an explicit gap:

No allocator concurrency. heap_alloc / heap_free are task-context only. There is no lock, because the kernel has no locking primitive yet. An allocation from an interrupt handler would corrupt the list, and nothing currently prevents one.

Chapter 11 closed the first half — the critical sections shown in §10.4 are that fix. The second half is still open: nothing prevents an allocation from an interrupt handler, and a critical section taken *inside* a handler is a no-op because interrupts are already masked.

The design responds by keeping allocation off the hot paths entirely. kmain creates the VM's arena before the scheduler starts:

```

/* The VM's arena is created here, on the boot path, because the heap has no
 * locking and this is the last moment at which exactly one context exists
 * (UM-NATOS-010 §8). The task only ever runs an already-initialised VM. */
g_vm_arena = arena_create(1024);

```

and the framebuffer is allocated on the boot path too, after the self-tests, with its own ordering note:

```

/* After the self-tests, not before: heap_init() runs inside m3_selftest(),
 * and the leak test there checks free memory returns to its baseline – an
 * 80 KB allocation made earlier would fail for want of a heap and, once the
 * ordering was fixed, would break the baseline instead. */
if (raycast_set_framebuffer(1) != 0) {
    uart_puts(" raycast fb : allocation failed, using direct columns\n");
}

```

Application arenas are the one exception: `app_start()` allocates from task context. That is safe under the current critical-section protection and is recorded rather than hidden.

10.11 Metrics

Quantity	Value
Image size at M3	6,720 B
Heap, usable at M3	166,432 B
Header overhead	16 B per block
Boot stack reserved	4,096 B
Alloc/free cycles tested	10,000
Blocks after test	1 (baseline)
High-water mark	5,120 B
Allocation failures	1 (the deliberate one)
Refused frees	2 (both deliberate)
<code>heap_check()</code> failures	0
Arena bounds cases	10 at M3, 35 cross-checked at M4

10.12 What M3 does not establish

- **No allocator concurrency** beyond critical sections — §10.10.
- **No arena enforcement.** M3 provides `arena_contains()`; nothing *calls* it yet. "Until then arenas are bookkeeping, not protection." Closed in Chapter 14.
- **No fragmentation characterisation under realistic load.** The test's size distribution is uniform and its lifetimes are uniform. Real workloads are neither, and a first-fit allocator's worst case is workload-shaped.
- **No allocation latency measurement.** First fit is $O(n)$ in block count.

- **Arena count is fixed** at `ARENA_MAX = 4`, statically.
-

Next: the two locking primitives, and why the cost of contention turned out to be a count rather than a duration.

11. Locking, and Why Contention Cost Is a Count and Not a Duration

Sources: docs/UM-NATOS-014-locking.md Code: kernel/critical.h, kernel/mutex.c, kernel/mutex.h, kernel/console.c, kernel/display.c, kernel/vm.c

11.1 Two primitives, because the two problems have opposite shapes

nat-os had no way to make anything mutually exclusive until this work. Two outstanding items forced it: the heap was task-context-only with no locking, and console output interleaved so a telemetry report could land in the middle of a typed line.

Two primitives were built, not one:

the two motivating problems have opposite shapes: the heap needs a handful of memory operations to be indivisible, and the console needs a 13-millisecond write to be indivisible. Solving both with the same tool would mean either an unsafe heap or a wrecked scheduler.

	Critical section	Blocking mutex
Mechanism	Raise PS.INTLEVEL to mask the tick	Block the task, remove it from the rotation
Cost while held	Scheduling latency, for the whole duration	Two context switches per handoff
Correct for	A few memory operations	Anything longer
Usable from an ISR	Yes	No — a handler cannot block
Used by	heap_alloc, heap_free, ipc.c, task.c	Console, display, application-shared state

The asymmetry that decides it:

The critical section is the wrong tool for the console. Writing a 150-character line at 115200 baud takes about 13 ms. Masking the tick for that long to solve a cosmetic interleaving problem would degrade every scheduling deadline in the system.

11.2 Critical sections

The entire implementation:

```
/* Masks everything up to and including the timer's level 3. */
#define CRIT_LEVEL 3u
```



```
static inline uint32_t crit_enter(void)
{
    uint32_t ps = xt_get_ps();
    xt_set_intlevel(CRIT_LEVEL);
    return ps & 0xFu;
}

static inline void crit_exit(uint32_t saved_level)
{
    xt_set_intlevel(saved_level);
}
```

Two design points, both in the header comment.

It returns and restores the previous level rather than forcing 0.

so nesting works and a section inside an interrupt handler does not silently re-enable interrupts on the way out.

The header shouts about misuse, in capitals, which is unusual for this codebase and therefore deliberate:

```
* THIS IS THE WRONG TOOL FOR ANYTHING LONG. Interrupts are masked for the whole
* section, so scheduling latency is degraded by exactly its duration. Writing a
* 150-character line to UART0 at 115200 baud takes about 13 ms – masking the
* tick for that long would wreck the scheduler to solve a cosmetic problem. Use
* a mutex (mutex.h) for anything that is not a handful of memory operations.
```

The correctness argument is a single-core one and is stated as such: "On a single core with no other bus master, that is sufficient for mutual exclusion: nothing else can be running." If APP_CPU were ever started, every use of this primitive would need re-examining.

11.3 Blocking, and the ownership sentinel

MUTEX_FREE is -2, not -1, and the reason is a one-line bug that would have been very hard to find:

Before the scheduler starts, `task_current()` returns -1, so using -1 for "unheld" would make an unowned mutex indistinguishable from one held by the boot context, and the free and recursive tests would both match. A one-line problem that would have produced a mutex quietly granting itself to everyone during startup.

The mutex is **recursive**:

A console lock is exactly the kind of thing that acquires a nested acquisition by accident, and self-deadlock on a kernel with no debugger is an expensive way to discover it.

That decision paid off immediately in the display driver, where `display_clear()` draws through `display_fill_rect()` — a non-recursive lock would deadlock on the first clear.

11.4 The starvation defect

The first working mutex released the lock and woke every waiter to race for it. The reasoning recorded in the source was that with at most eight tasks a thundering herd was not worth managing.

That reasoning was wrong, and the instrumentation said so precisely:

```
lock owner=1 waiters=0x00000004 acq=238542 contended=137 err=0 states=1121111
work a/b=238541/0
```

Worker-a had acquired the lock **238,542 times**. Worker-b — `waiters` bit 2, state 2, `TASK_BLOCKED` — had acquired it **zero times**, and had not completed a single iteration of its loop.

And the diagnosis, which rules out the obvious explanation:

This was not a lost wakeup. Worker-b was being woken correctly and re-blocking every time. Worker-a's loop is tight enough that it re-acquired the lock long before the woken waiter was next scheduled. **Race-on-release provides no ordering guarantee at all**, and against a tight loop that is not merely unfair — it starves the waiter indefinitely.

The fix: direct handoff

`mutex_unlock()` now transfers ownership to a waiter rather than freeing the lock. The successor is the owner *before* it runs again, so nothing can cut in front:

```
/* Hand the lock to the longest-waiting candidate rather than releasing it.
 * Ownership transfers here, atomically with the wakeup, so a task still
 * spinning in its own lock/unlock loop cannot take it first. */
if (m->waiters != 0u) {
    int next = -1;
    for (int id = 0; id < TASK_MAX; id++) {
        if (m->waiters & (1u << id)) {
            next = id;
            break;
        }
    }
    m->waiters &= ~(1u << next);
    m->granted |= 1u << next;
    m->owner = next;
    m->depth = 1;
    m->acquisitions++;
    task_unblock(next);
    crit_exit(s);
    return;
}
```

This reintroduces a complication the first design was avoiding: `owner` names a task that is not yet on the CPU. A `granted` bitmask resolves it, and the *order of the tests* in `mutex_lock` is load-bearing:

```
void mutex_lock(mutex_t *m)
{
```

```

for (;;) {
    uint32_t s = crit_enter();
    int      me = task_current();

    /* Checked first: a handoff already made this task the owner while it
     * was blocked, so it must return holding the lock without touching
     * depth. Testing owner == me first would read the handoff as a
     * recursive acquisition and leave depth permanently too high. */
    if (me >= 0 && (m->granted & (1u << me))) {
        m->granted &= ~(1u << me);
        crit_exit(s);
        return;
    }

    if (m->owner == MUTEX_FREE) {
        m->owner = me;
        m->depth = 1;
        m->acquisitions++;
        crit_exit(s);
        return;
    }
    if (m->owner == me) {
        m->depth++;
        crit_exit(s);
        return;
    }

    /* Held by someone else. Join the wait list and mark this task
     * unrunnable, both while still masked so a tick cannot land between
     * them – that split is the classic lost-wakeup: the holder releases,
     * sees an empty wait list, and this task blocks forever. */
    m->contentions++;
    if (me >= 0) {
        m->waiters |= 1u << me;
        task_block();
    }
    crit_exit(s);

    /* Only now allow the switch. If this runs before the scheduler has
     * started – me < 0 – there is no context to block, so the loop simply
     * spins, which is correct during single-threaded boot. */
    task_yield();
}
}

```

Three things in that function are worth naming separately:

1. **granted before owner == me** — otherwise a handoff reads as a recursive acquisition and `depth` climbs forever.
2. **waiters |= and task_block() inside one critical section** — the protocol Chapter 9 §9.2 demands, honoured exactly.
3. **me < 0 spins instead of blocking** — correct during single-threaded boot, and the reason the `MUTEX_FREE == -2` sentinel matters.

Non-owner unlock is refused, not obeyed

```
/* Releasing a lock this task does not hold is a bug in the caller. Counted
 * and refused rather than acted on: clearing another task's ownership would
 * let two tasks into the section and corrupt whatever it protects, far from
 * where the mistake was made. */
if (m->owner != task_current()) {
    m->errors++;
    crit_exit(s);
    return;
}
```

Refusing a non-owner unlock matters more than it looks: clearing another task's ownership would admit two holders to the protected section, corrupting whatever it guards far from where the mistake was made.

11.5 The cost of fairness

With the handoff in place and the lock taken on *every* worker iteration:

```
work a/b=2062/2061      (was 238541/0)
```

Fair, and throughput collapsed roughly thirtyfold.

A handoff costs a full scheduling round-trip — the successor cannot run until the next tick — so a lock contended on every iteration becomes bounded by the tick rate rather than by the work.

That is a real property of a blocking lock, not a defect, but it is a pathological workload. Contending once every 32 iterations is representative:

```
work a/b=50143/50143    acq=3133  contended=200  err=0  skew=0
```

Both workers now advance **exactly** in step.

`kmain.c` carries the constant and its justification:

```
/* Contend every Nth iteration rather than every one. Taking the lock on every
 * pass means a worker holds it for most of its iteration, so contention is
 * near-certain and each handoff costs a full scheduling round-trip – measured
 * at roughly a thirtyfold throughput collapse. That is a real property of a
 * blocking lock, but it is a pathological workload, not a representative one. */
#define BUMP_EVERY 32u
```

The lesson, stated at M6 and rediscovered twice afterwards:

a blocking mutex is the wrong instrument for a lock contended at high frequency; that case wants a critical section, or a design that does not share.

11.6 How mutual exclusion is actually tested

The contention target is deliberately hostile:

```

/* Contention target. The workers hammer this from two tasks, and the reporter
 * checks it against their own iteration counts. The read-modify-write below is
 * deliberately slow and deliberately non-atomic: without the mutex it would be
 * reliably corrupted rather than occasionally, which is what makes the result
 * meaningful instead of lucky. */
static mutex_t      g_shared_lock;
static volatile uint32_t g_shared;

static void shared_bump(void)
{
    mutex_lock(&g_shared_lock);
    uint32_t v = g_shared;
    for (volatile int i = 0; i < 6; i++) {
        /* Widen the window between read and write. A tick landing here is the
         * whole point: unprotected, the other worker's increment is lost. */
    }
    g_shared = v + 1;
    mutex_unlock(&g_shared_lock);
}

```

Each worker increments a shared counter under the lock through a deliberately non-atomic read-modify-write with a widened window; the kernel checks that counter against the workers' own independently maintained tallies. Unprotected, this drifts by thousands within a second.

`skew=0` throughout. That is a *cross-check*, not an assertion: two independently maintained counters agreeing is much stronger evidence than one counter having the value it was supposed to.

11.7 The self-test that could only pass by accident

This is a small story with a large moral, and it is the first appearance in this book of a pattern Chapter 28 catalogues.

The critical-section test holds a section across two full tick periods and checks the tick count is frozen, then resumed:

```
[6a] critical : PASS  ticks held at 1 across 2 periods, resumed at 7
```

Both halves matter: freezing proves interrupts were masked, and resuming proves the tick was **deferred rather than lost**. Masking that silently dropped ticks would keep the scheduler running while making every timeout wrong.

And then:

This test originally ran in `m6_selftest()` before `timer_start()`, where `timer_ticks()` is 0 and stays 0 regardless of what masking does — **a test that could only ever pass by accident**. It reported FAIL, which is the one outcome that made the flaw visible. It now runs from the reporter task with the tick live.

The test was saved by *failing*. Had it passed in its original position, it would have been a permanent green light on a mechanism it could not observe. The deferred call

is still visible in `kmain.c`:

```
/* Defined below with the other self-tests, but called from the reporter, which
 * is the only context where a running tick makes it meaningful. */
static void m6_critical_test(void);
```

```
static void task_report(void)
{
    /* ... */
    /* Deferred until here: it needs a running tick to mean anything. */
    m6_critical_test();
}
```

This test is also the *only* instrument that noticed the 183 ms comparator stall of Chapter 7 §7.9:

Only the M6 critical-section self-test noticed, because it is the one test that asserts something about the tick *resuming* rather than about work getting done.

11.8 Console arbitration

Locking is at **message granularity**, not per character:

```
if (str_eq(line, "help"))    { cmd_help(); }
```

is wrapped by:

```
/* One command, one uninterrupted response. */
console_lock();
```

and the two deliberate exclusions:

`uart_putc()` stays lock-free, and the panic handler ignores this layer entirely — it runs after a fault, possibly with the lock held by the task that faulted, and a panic that deadlocks trying to report a panic is the worst failure mode available to a kernel with no debugger.

Result: `ps` output arrived as four unbroken lines with no report interleaved, where previously a report would land mid-table.

One gap remains and is recorded: **keystroke echo is not arbitrated**, because "locking each keystroke would serialise the console against a human typing speed".

11.9 The same defect from the other side, and the finding that matters

This section is UM-NATOS-014 revision 1.2, and it is the most transferable result in the report.

§11.5 established that a blocking mutex is the wrong instrument for a high-frequency lock, and fixed the raycaster by batching: one acquisition per frame instead of 240.

That fix held. What it did not anticipate is that the **other** party would arrive later and reintroduce the same shape from the opposite direction.

Applications draw through `vm.c`, which took the draw lock **per primitive** — exactly what the raycaster used to do to itself. The renderer holds the lock for a whole frame; the applications interrupt it constantly with short takes.

Blocked time is not lock-busy time

The mutex counted acquisitions and contentions from the beginning. Neither is a duration, and the question was a duration, so `display.c` was instrumented to time both the hold and the interval between asking and getting.

Measured over 8.08 s with applications running:

quantity	value
lock held	979 ms — 12% of wall clock
aggregate blocked	13,051 ms — 1.6 tasks blocked at all times
acquisitions	100
contentions	202 — two blocked waiters per acquisition

The lock was free 88% of the time, and thirteen seconds of blocking accumulated in eight seconds of wall clock. That is only possible because "blocked" is not "lock busy": a task that cannot have the lock is descheduled, and the interval includes being selected again afterwards. 13,051 ms across 202 contentions is **63 ms per contention against a 24 ms hold** — most of it spent getting back onto the CPU.

The accessors are named for exactly this reason, and `display.h` says so:

```
/* Lock timing.
 *
 * "Blocked" is time from asking for the lock to holding it. It is NOT time the
 * lock was busy: a task that cannot have it is descheduled, so the figure also
 * covers being selected again afterwards, and measurement showed that is the
 * larger part — 63 ms blocked against a 24 ms hold. The name matters, because
 * "wait" would imply the panel was the bottleneck and send the next person to
 * shorten holds, which was tried and changed nothing. */
uint32_t display_lock_blocked_ms(void);
uint32_t display_lock_hold_ms(void);
uint32_t display_lock_takes(void);
uint32_t display_lock_contentions(void);
```

Naming a counter after what it measures rather than after what it feels like is a small discipline that here prevented a whole wrong investigation.

The obvious fix, and why it failed

Composition writes to a private buffer and touches no shared hardware, so the lock was narrowed to cover only the blit:

	before	after
lock held	979 ms	734 ms (~25%, as designed)
blocked	13,051 ms	13,085 ms (unchanged)
frames/s	3.0	3.2

The change did precisely what it was built to do and moved the outcome by nothing. **Hold duration was never the cost.** The narrowed hold was kept anyway, because a lock should cover the shared thing rather than the whole operation that happens to use it — but it is not an optimisation and is not recorded as one.

That last clause is a small act of bookkeeping honesty that recurs in Chapter 18: a change kept on its merits, explicitly *not* credited with a result it did not produce.

Attribution decided the fix

The aggregate says the system is blocked and cannot say *on whom*, and the two readings imply opposite remedies. Per-task attribution answered it:

blocked, display task	3,880 ms	65% of wall clock
blocked, application host	5,894 ms	98% of wall clock

Both, mutually — contending not for the panel, which was idle, but for the CPU afterwards.

```
/* Milliseconds a specific task has spent blocked on the draw lock. The
 * aggregate says the system is blocked; this says which task, which is what
 * decided the fix – both the renderer and the applications were blocked most of
 * the time, on a lock that was free 91% of it. */
uint32_t display_lock_blocked_of(int task_id);
```

Stop blocking

`vp_fill`, `vp_text` and `SYS_BLIT` now take the lock with `display_try_lock()` and skip the primitive when it is busy:

```
/* Best effort. See display_try_lock(): an application that blocks here
 * costs the whole system a scheduling round trip, and the pixel it wanted
 * to draw is worth far less than that. */
if (!display_try_lock()) {
    g_draw_skipped++;
    return;
}
display_fill_rect(ax, ay, w, h, colour);
display_unlock();
```

	before	after
frames/s	3.0	9.9
blocked, applications	5,894 ms	0
contentions	202 per 8 s	10 total

	before	after
primitives skipped	—	45 of 1,325 = 3.4%

Within 11% of the 11.1 fps measured with every application killed, so nearly all the lost frame rate is recovered with the applications still running.

And skips are counted:

Skips are counted rather than hidden. A best-effort policy that silently discards work is indistinguishable from a broken one, and the ratio is the only thing that distinguishes "reasonable" from "starving applications of the screen".

The finding

The cost of a contended blocking lock is dominated by the number of blocking events, not the time the lock is held. Each one costs a scheduling round-trip whether the lock was held for 24 ms or 24 μ s. So the levers are batching (fewer takes) or not blocking at all (`try_lock`) — and shortening holds, the intuitive move, does nothing.

The two levers were reached for in that order: §11.5 batched (the raycaster's 194× improvement); §11.9 stopped blocking. And the reason batching could not be used the second time is worth noting:

the renderer and the applications cannot be batched together: they are different tasks with no common boundary to batch across.

`display.h` carries the batching interface with the frequency argument attached:

```
/* Hold the draw lock across a batch of primitives.
 *
 * Every drawing call already locks, and the mutex is recursive, so this is only
 * about FREQUENCY. A caller issuing hundreds of small primitives pays a full
 * scheduling round-trip per contended acquisition — a raycaster taking the lock
 * once per column spent 25 seconds per frame on 37 ms of work. Holding it for
 * the batch turns that into one acquisition. */
```

This also explains an otherwise-odd optimisation in Chapter 24: the launcher draws each icon row as one rectangle per *run* of set pixels rather than one per pixel, and the reason is not micro-optimisation but lock count.

11.10 Metrics

Quantity	Value
Primitives	2
Critical section cost	RSR/WSR on PS, no memory
Mutex size	24 B
Lock acquisitions measured	3,133
Contentions	200

Quantity	Value
Non-owner unlocks	0
Lost updates	0
Throughput, pathological contention	~2,060 iterations
Throughput, 1-in-32 contention	~50,143 iterations
Draw lock held, applications running	12% of wall clock
Blocked per contention	63 ms, against a 24 ms hold
Effect of narrowing the hold 25%	none
Frames/s, blocking → best-effort drawing	3.0 → 9.9
Primitives skipped under best-effort	3.4%

11.11 What this does not establish

- **No deadlock detection.** Two tasks taking two mutexes in opposite orders will hang, and the kernel will not say so. The idle task will simply run.
- **No timeouts.** `mutex_lock()` waits forever.
- **No condition variables or semaphores.** There is no way to wait for an event, only for a lock. (The WiFi OSI layer builds its own on top of `task_sleep` — Chapter 27 §27.3.)
- **Priority inheritance drops a boost on the first release**, so nested holds of two boosted mutexes lose it early.
- **Keystroke echo is not arbitrated.**
- **Critical sections are unbounded by convention only.** Nothing measures or enforces how long one is held.
- **Untested against an ISR.** The mutex is documented as unusable from an interrupt handler and nothing enforces that.
- **Best-effort drawing has no fairness of its own.** A skipped primitive is simply lost. At 3.4% that is invisible, but the policy contains no bound on how unlucky one application can be.
- **The skip ratio is measured under one workload** — four small test programs drawing into strips. An application doing sustained full-viewport blits would contend far harder.
- **`try_lock` does not compose with batching.** An application wanting several primitives drawn atomically has no way to ask for that.

Next: the three mechanisms that were supposed to catch failures, and had never been observed to catch anything.

12. Failure Handling: Three Mechanisms That Had Never Fired

Sources: docs/UM-NATOS-019-failure-handling.md Code: kernel/panic.c, kernel/watchdog.c, kernel/task.c, kernel/uart.c, kernel/store.c, kernel/display.c, kernel/shell.c

12.1 The premise

The kernel had three mechanisms for surviving its own failure: stack guards, a panic handler, and a hang detector. Two of the three had never been observed doing anything, and the third had been silently broken by the second.

The unifying failure is not technical. Each of these was verified by observing that it *existed* — a guard word was written, a banner was printed, a watchdog register was armed — rather than by observing it *work*.

12.2 Recovery and evidence are in conflict

The hang detector feeds on the scheduler switching between distinct tasks. That is the right signal for a hang. It is also exactly what a **deliberately halted** kernel looks like, and `panic.c` ended with:

```
for (;;) {
    /* Deliberately spin rather than reset: a reset loop would scroll the
     * evidence off the terminal. */
}
```

Two correct decisions, taken months apart, that combine into a wrong one. The comment states the panic handler's entire purpose, and arming the watchdog defeated it without touching that file.

Measured, not assumed

The obvious move is to disarm the watchdog in the panic path. The less obvious one is to first confirm there is anything to fix — the interaction depends on whether the timer interrupt still fires with `PS.EXCM` set, which is a question about silicon rather than about this code.

With the watchdog left armed and `fault` invoked:

```
*** KERNEL PANIC ***
exccause : 0 (IllegalInstruction)
epc      : 0x40083fd0 <- faulting instruction

halted. reset the board to continue.
```

```
rst:0x7 (TG0WDT_SYS_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
[task 0 entered]
```

The report is produced and then destroyed about three seconds later. With `watchdog_disarm()` in the panic path, output stops at `halted.` and stays there through a fourteen-second capture.

And the reason the measurement mattered more than the fix:

Had the tick continued to fire during a panic, the fix would have been wrong *and* would have hidden a worse problem — a scheduler switching away from a faulted context and resuming normal operation on it. The result rules that out: no reporter output appears after the panic banner in either configuration, so scheduling genuinely stops.

The resulting rule

The distinction is whether the kernel can say **why** it stopped:

- it cannot explain a hang, so recovery is the only useful response
- it can explain a fault or a broken guard, and a reset would destroy the explanation

`watchdog_disarm()` exists solely to express that, and the header says it is not a general-purpose control.

12.3 The watchdog

Three watchdogs matter at boot: one in the RTC controller and one in each timer group. The bootloader arms the RTC watchdog and expects the application to take ownership.

```
* nat-os did neither for three milestones. The consequence was RTCWDT_RTC_RESET
* roughly every second once M2 kept the CPU busy, which presented as a "stuck
* scheduler" and cost three build cycles chasing bugs that did not exist. The
* reset reason said so on the first line of every boot.
```

That last sentence is the whole indictment. The diagnostic was printed, on every boot, on the first line, and was not read.

Disabling

Each configuration register is write-protected behind a key:

```
/* Write-protect key, shared by all three watchdog blocks. */
#define WDT_WKEY                0x50D83AA1u

static void disable_one(unsigned int wprotect, unsigned int config0)
{
    REG(wprotect) = WDT_WKEY;    /* unlock */
    REG(config0)   = 0;          /* clear enable and every stage action */
    REG(wprotect) = 0;          /* re-lock */
}
```

```

void watchdog_disable_all(void)
{
    disable_one(RTC_CNTL_WDTWPROTECT, RTC_CNTL_WDTCONFIG0);
    disable_one(TIMG0_WDTWPROTECT,    TIMG0_WDTCONFIG0);
    disable_one(TIMG1_WDTWPROTECT,    TIMG1_WDTCONFIG0);
}

```

Re-locking afterwards "prevents a stray store from silently re-enabling one".

`kmain` does this first, before anything can take long enough to trip it, and reads the result back:

```

/* Before anything can take long enough to trip it. The bootloader arms the
 * RTC watchdog and expects the application to take ownership. */
watchdog_disable_all();
uart_puts("  rtc wdt      : ");
uart_puts(watchdog_rtc_config() == 0u ? "disarmed\n" : "STILL ARMED\n");

```

A read-back rather than an assumption. Chapters 22 and 23 show three cases where a read-back was *not* sufficient, but it is still strictly better than nothing.

Re-arming as a hang detector

TIMG0 is used, and the choice is justified in units:

```

/* TIMG0 watchdog, used as the hang detector. Chosen over the RTC watchdog
 * because its timeout is expressed in APB-clock ticks through a prescaler,
 * which is a number this kernel already knows, rather than in RTC slow-clock
 * cycles whose frequency is only approximately known. */

```

```

void watchdog_arm(unsigned int ms)
{
    REG(TIMG0_WDTWPROTECT) = WDT_WKEY;

    /* APB is 80 MHz; a prescaler of 40,000 gives a 2 kHz tick, so the timeout
     * is milliseconds times two. Chosen so the timeout fits comfortably in the
     * 32-bit stage register at any duration worth using. */
    REG(TIMG0_WDTCONFIG1) = 40000u << 16;
    REG(TIMG0_WDTCONFIG2) = ms * 2u;

    REG(TIMG0_WDTCONFIG0) = WDT_EN | WDT_STG0_RESET_SYSTEM |
                           WDT_SYS_RESET_LEN | WDT_CPU_RESET_LEN;

    REG(TIMG0_WDTFEED)     = 1u;
    REG(TIMG0_WDTWPROTECT) = 0;
}

```

The liveness signal, and its limitation

```

/* Called every tick from the scheduler. `switched` is non-zero if this tick
 * resumed a DIFFERENT task than the one it interrupted.
 *
 * The window is deliberately much shorter than the watchdog timeout: a healthy
 * system switches many times a second, so requiring one switch per window is a
 * low bar that only a genuine monopoly fails. */
#define LIVENESS_WINDOW_TICKS 100u    /* ~1 s */

```

```

void watchdog_liveness(int switched)
{
    static unsigned int ticks;
    static unsigned int seen;

    if (switched) {
        seen++;
    }

    if (++ticks >= LIVENESS_WINDOW_TICKS) {
        ticks = 0;
        if (seen) {
            seen = 0;
            watchdog_feed();
        } else {
            /* No distinct switch for a whole window. Deliberately does NOT
             * feed: the watchdog is the only thing that can recover this, and
             * feeding on the way past would defeat the entire mechanism. */
            g_starved++;
        }
    }
}

```

Fed by the scheduler at the decision point:

```

/* Liveness for the hang detector: a tick that resumes the SAME task is not
 * evidence the system is healthy – it is exactly what a monopoly looks
 * like. Only a switch between distinct tasks counts. */
watchdog_liveness(next != g_current);

```

The limitation is exactly what Chapter 9 §9.6 ran into:

The hang detector asks whether ANY distinct switch happened, not whether every ready task got a turn.

A task starved to complete silence produced no crash, no watchdog reset, and no symptom other than the absence of its own output. Ageing was the answer, not a better hang detector — the detector is correct for the question it asks.

Arming is deliberately last in `kmain`:

```

/* Armed last, immediately before the scheduler takes over. Arming earlier
 * would have the single-threaded boot path - which never switches tasks and
 * so never feeds - reset the board partway through its own self-tests. */
watchdog_arm(3000u);
uart_puts(" hang detector: armed, 3000 ms, fed on distinct task switches\n");

```

12.4 Stack guards: reported, not enforced

`task.c` filled every stack with a pattern and planted a guard word at its base from the beginning. `task_stack_headroom()` was written to walk that pattern.

It was never called — both `high_water` figures anywhere in the tree were the *heap*'s.

Three separate weaknesses, each individually reasonable:

1. **Coverage.** `kmain` checked guards for three of eight tasks. The display task, which carries the raycaster's call chain, was not among them.
2. **Timing.** The check ran in the reporter, roughly every two seconds. "A guard word is broken *after* the damage; a check that runs seconds later reports a corruption whose cause is long gone."
3. **Response.** A break printed `BROKEN` beside the telemetry and execution continued — "scheduling tasks whose stacks had already written into a neighbour's, and printing numbers that were by then meaningless."

The fix: check every slot, every switch, and panic

```
/* A broken guard is fatal, not cosmetic.
 *
 * It used to print "BROKEN" beside the telemetry and carry on, which means
 * continuing to schedule tasks whose stacks have already written into a
 * neighbour's. Every number printed after that point is suspect, including
 * the ones that would be used to diagnose it. Stopping at the switch that
 * noticed keeps the damage bounded and the report honest. */
int broken = task_stack_broken();
if (broken >= 0) {
    kernel_panic_msg("stack guard overwritten", (unsigned int)broken);
}
```

```
int task_stack_broken(void)
{
    for (int id = 0; id < TASK_MAX; id++) {
        if (g_tasks[id].stack_base && g_tasks[id].stack_base[0] != STACK_GUARD) {
            return id;
        }
    }
    return -1;
}
```

A word load per slot per tick is not a cost worth weighing against continuing to run on corrupted memory.

The prose-versus-code lesson attached to it is small and good:

Written as "all eight tasks" when `TASK_MAX` was 8. It is now 12, and the check is written against `TASK_MAX` rather than a literal, so it followed. The prose did not — which is the argument for describing a loop by its bound rather than by the bound's value on the day.

The margins, which were never known

id	name	free B	of 2048			
0	report	1844	4	app-host	1604	<- tightest
1	worker-a	1796	5	shell	1828	
2	worker-b	1796	6	display	1668	
3	vm-host	1732	7	touch	1716	

Every task retains at least 78% of its stack; the worst uses 444 B of 2048. The 2 KB figure in `task.h` was a guess and turns out to be a comfortable one.

And the finding that justifies measuring all of them:

The tightest task is `app-host`, **not** `display` — which is where the deepest call chain was assumed to be when deciding which three tasks were worth checking. That assumption picked the wrong tasks to watch, and is the argument for measuring all of them rather than the interesting-looking ones.

The reporting helper exists for the same reason:

```
/* The task closest to overflowing. Reported so the margin is a number somebody
 * has seen, rather than an assumption that 2 KB was enough. */
int task_stack_tightest(void)
```

12.5 The shell had never worked

This is the most consequential finding in the report, and it was found by accident while building a test harness for something else.

While building a harness to send `fault` over the serial line, `mem\r` echoed `mem` and did nothing. `mem\r\n` worked.

That difference is diagnostic. If `\r` triggered execution, then `\r\n` would trigger twice — the second on an empty line, which returns immediately. It working *only* with both bytes means the terminator was not being consumed at all until something followed it.

Confirmed directly:

```
send "mem\r", wait 2.5 s  -> reply BEFORE the extra byte: no
send " ",      wait 2.5 s  -> reply AFTER  the extra byte: YES
```

The receive path ran exactly one byte behind. `uart_getc_nb()` read the RX FIFO through the APB address at `0x3FF40000`; it now reads the AHB alias at `0x60000000`.

```
/* The RX FIFO is read through its AHB alias, not through the APB address used
 * for transmit.
 *
 * Measured, not inherited from documentation: with the APB address, the receive
 * path ran exactly one byte behind. Writing "mem\r" echoed `mem` and did
 * nothing; a single further byte then delivered the `\r` and the command ran.
 * Sending "mem\r\n" appeared to work, which is why this survived — every test
 * of the shell had been run from a sender that emitted both.
 *
 * The consequence was worse than a test artefact. A person at a terminal
 * presses Enter and sees nothing happen, because the Enter itself is the byte
 * left waiting. The shell has never been usable interactively, and nothing
 * caught it: it was verified by reading its banner rather than by using it. */
#define UART_FIFO_AHB_REG      0x60000000u
```

```
int uart_getc_nb(void)
{
```



```

    if (!uart_rx_ready()) {
        return -1;
    }
    return (int)(REG(UART_FIFO_AHB_REG) & 0xFFu);
}

```

Why it survived

Two reasons, and both are the same mistake:

- every automated test drove it from a sender emitting **both** CR and LF, which is the one input shape that masks the defect
- the shell was verified by observing that its **banner printed**, not by observing that a command **ran**

A banner proves the task was created and the transmit path works. It says nothing whatsoever about receive. The same substitution — a startup artefact standing in for the behaviour it introduces — is what let the guard checks and the panic handler go unexercised for as long as they did.

This is one of the standing rules in Chapter 29, and it is arguably the most broadly applicable thing this project has produced.

One caveat is recorded in Chapter 30: the AHB alias is validated *behaviourally*, not from documentation. The measurement establishes that the APB address lags and the alias does not; it does not establish *why*, and no erratum is cited.

12.6 Triggering failure on purpose

Three shell commands exist solely to make these paths reachable.

Command	Induces	Expected
hang	masks interrupts, spins	watchdog resets the board
fault	executes ill	panic, halt, evidence retained
smash	clobbers the running task's guard word	panic at the next switch, naming the task

```

void task_smash_guard(void)
{
    if (g_current >= 0 && g_tasks[g_current].stack_base) {
        g_tasks[g_current].stack_base[0] = 0xDEADBEEFu;
    }
}

```

Verified on hardware:

```

smash -> *** KERNEL PANIC ***
        reason   : stack guard overwritten
        detail    : 5                      (task 5 is the shell)

fault  -> *** KERNEL PANIC ***
        exccause  : 0 (IllegalInstruction)
        halted.   still halted after 14 s

```

hang	-> rst:0x7 (TG0WDT_SYS_RESET)	recovered, rebooted
mem\r	-> heap free=71792 ...	no trailing byte needed

smash correctly identified task 5, which is the shell — the task that invoked it. That is a stronger result than a bare panic, because it shows the scheduler attributing the break to the right task rather than to whichever it happened to be examining.

And the policy decision:

These are deliberately not compiled out. A destructive command in a development shell is a smaller risk than a recovery path nobody can reach to test.

The project README puts it more bluntly:

The last three exist on purpose. A recovery path that has never been observed to fire is confidence without evidence.

12.7 Recording the reason for the next boot

Everything above still assumes a serial cable. A panic prints and halts, which helps only somebody attached at the moment it happens — and this board is normally attached to nothing at all.

The persistent record (Chapter 20) carries it:

```
uint32_t fault_kind;    /* exception, guard break, or none */
uint32_t fault_detail; /* exccause, or the task id */
uint32_t fault_epc;     /* faulting instruction */
uint32_t fault_boot;    /* which boot it happened on */
```

Recorded before it is reported

Both panic entry points write the record **before** printing anything:

```
void kernel_panic_msg(const char *why, unsigned int detail)
{
    #if FLASH_ENABLE
        /* Written BEFORE anything is printed. A handler that reports first and
         * records second loses the record if it dies while reporting, and that is
         * not far-fetched — the UART is the more complicated of the two paths and
         * the system is already in an unknown state. */
        g_record_rc = store_record_fault(STORE_FAULT_GUARD, detail, 0);
    #endif
}
```

The reverse ordering would also be self-concealing — a panic that died during its own report would leave no record AND no output, which is indistinguishable from a fault that never happened.

The store side is written to assume as little as possible:

```
/* Runs inside a panic, so it assumes as little as possible about the state of
 * the system. It does not read the existing record first: g_rec already holds
```

```
* what was loaded at boot, and re-reading flash here would add a failure mode
* to a path that exists precisely because something has already gone wrong. */
int store_record_fault(uint32_t kind, uint32_t detail, uint32_t epc)
```

Not cleared by being read

A fault stays on the record until a different one replaces it. Clearing it after one report would mean that power-cycling past the message destroys it, and the user who most needs this feature is exactly the one who reboots twice before thinking to attach a cable.

`fault_boot` is what keeps that honest. Without it, a fault from thirty boots ago reads identically to one from the last run.

The boot-path reporter:

```
if (store_fault_kind() != STORE_FAULT_NONE) {
    uart_puts("  LAST FAULT  : ");
    if (store_fault_kind() == STORE_FAULT_GUARD) {
        uart_puts("stack guard overwritten, task ");
        uart_put_dec(store_fault_detail());
    } else {
        uart_puts("exception, exccause ");
        uart_put_dec(store_fault_detail());
        uart_puts(", epc ");
        uart_put_hex(store_fault_epc());
    }
    uart_puts(" (boot #");
    uart_put_dec(store_fault_boot());
    uart_puts(")\n");
}
```

Verified, with three distinct failure modes covered

```
smash      -> recorded : yes
next boot  -> LAST FAULT : stack guard overwritten, task 5 (boot #6)

fault      -> epc 0x40084071
next boot  -> LAST FAULT : exception, exccause 0, epc 0x40084071 (boot #8)
boot again -> same line, unchanged
```

the guard break survived a reset; the exception **replaced** it rather than being ignored or appended; and the second reboot proves reading does not consume the record. The reported `epc` matches what the panic itself printed, which is what rules out the record being written from stale or default state.

A pleasing side effect:

Bumping the record from version 1 to 2 incidentally exercised the version check that UM-NATOS-018 §2 describes but had never triggered: the old record was rejected and reset cleanly rather than being read with the wrong field layout.

12.8 Putting it on the panel

§12.7 makes a fault legible on the *next* boot. It does nothing for the person holding a frozen device now, and "the kernel panicked" and "the renderer stopped" look identical from across the room.

The panic handler now draws to the display. That is a larger claim than it sounds, because the display driver is built for a healthy system.

Three assumptions suspended at once

`display_enter_panic_mode()` turns off, permanently:

Assumption	Why it cannot hold
the draw lock	Blocking on a mutex means <code>task_block()</code> and a yield, and the scheduler has stopped. Waiting on a lock whose owner will never run again is a hang — and a hang here costs the report.
DMA	The descriptors may describe a transfer already in flight, or a buffer belonging to whatever just died.
the unbounded FIFO wait	<code>spi2_tx()</code> spins until the controller retires a transfer. Correct while the system is healthy; an infinite loop once it is not.

The bound only exists in panic mode, so the healthy path costs one comparison:

```
/* Bounded only in panic mode. A healthy controller retires a 64-byte
 * transfer in microseconds, so the bound is never reached in normal
 * operation and costs a comparison; in a panic an unretired transfer
 * must not be allowed to consume the one chance to report a fault. */
uint32_t spins = 0;
while (GPIO_REG(SPI2_CMD) & SPI_USR_BIT) {
    if (g_panic_mode && ++spins > 4000000u) {
        break;
    }
}
```

The switch is one-way:

There is no `display_leave_panic_mode()`, because a driver that can be talked back out of panic mode invites somebody to try, and the kernel is on its way to a halt in any case.

And `display_ready()` guards the whole thing: a panic before `display_init()` has completed draws nothing rather than writing to an unconfigured controller.

Ordered by decreasing reliability

```
static void halt_forever(void)
{
    watchdog_disarm();

#ifdef FLASH_ENABLE
    /* The record was already written by the caller. Confirm it on the terminal
     * so the two reports can be compared: if the next boot disagrees with what
     * was printed here, the persistence path is what is wrong, not the fault. */
```

```

    uart_puts(g_record_rc == 0 ? " recorded : yes, the next boot will report
this\n"
                                : " recorded : NO – the fault will be lost\n");
#endif

    uart_puts("\n halted. reset the board to continue.\n");

    uint32_t before = display_bytes_written();
    panic_screen();
    uart_puts(" panel      : ");
    uart_put_dec(display_bytes_written() - before);
    uart_puts(" bytes drawn\n");

    for (;;) {
    }
}

```

1. **write the flash record** — fewest moving parts, and the only one that survives the power cycle
2. **print to the UART** — one peripheral, no memory beyond a string literal already resident in DRAM
3. **draw the panel** — needs the SPI controller, the flash-mapped font, and a peripheral whose state nobody has verified

Each step can only cost the steps after it. If drawing wedges despite the bounds, the record and the serial report have both already happened. The reverse order would put the most fragile step first and risk everything on it.

Measured, because a screen cannot be queried

```

/* Report how many bytes the panel actually took.
 *
 * Nobody can query a halted board, and "the screen looks wrong" and "the
 * driver never ran" are indistinguishable by eye. A byte count crossing the
 * wire separates them: roughly 150 KB means a full repaint happened and the
 * question is what was drawn; a number near zero means the panic never
 * reached the panel at all. */

```

```

fault -> panel : 175955 bytes drawn
smash -> panel : 176704 bytes drawn

```

A full repaint is $320 \times 240 \times 2 = 153,600$ bytes; the remainder is the title bar and the text.

This converts an unfalsifiable visual impression into a claim that fails loudly — the same move as the boot counter in Chapter 20 §20.3.

The screen itself

```

static void panic_screen(void)
{
    char buf[12];

```

```

    if (!display_ready()) {
        return;
    }
    display_enter_panic_mode();

    display_clear(COLOR_BLUE);
    display_fill_rect(0, 0, 320, 20, COLOR_WHITE);
    display_text(6, 6, "KERNEL PANIC", COLOR_BLUE, COLOR_WHITE, 1);

    display_text(6, 36, g_panic_what, COLOR_WHITE, COLOR_BLUE, 1);

    /* Numbers as hex without labels: at this size a label costs more width
     * than it buys, and the two values are positional in the same order the
     * UART report prints them. */
    display_text(6, 56, "cause", COLOR_YELLOW, COLOR_BLUE, 1);
    hex8(buf, g_panic_a);
    display_text(70, 56, buf, COLOR_WHITE, COLOR_BLUE, 1);
    if (g_panic_has_b) {
        display_text(6, 72, "pc", COLOR_YELLOW, COLOR_BLUE, 1);
        hex8(buf, g_panic_b);
        display_text(70, 72, buf, COLOR_WHITE, COLOR_BLUE, 1);
    }

    display_text(6, 104, "halted - reset the board", COLOR_WHITE, COLOR_BLUE, 1);
    display_text(6, 120, "reason is saved; next boot", COLOR_GREY, COLOR_BLUE, 1);
    display_text(6, 136, "will report it over serial", COLOR_GREY, COLOR_BLUE, 1);
}

```

Even the hex formatter is local, and the reason is a dependency argument:

```

/* Eight hex digits into a caller-supplied buffer. Local to this file because
 * the panic path must not depend on anything it does not have to: no printf,
 * no heap, no shared scratch buffer that something else might be using. */
static void hex8(char *out, unsigned int v)

```

The exception decode

```

/* Xtensa EXCCAUSE values worth naming. The rest print as a bare number rather
 * than carrying a table that would be mostly dead weight. */
static const char *cause_name(unsigned int cause)
{
    switch (cause) {
        case 0: return "IllegalInstruction";
        case 1: return "Syscall";
        case 2: return "InstructionFetchError";
        case 3: return "LoadStoreError";
        case 4: return "Level1Interrupt";
        case 5: return "Alloca";
        case 6: return "IntegerDivideByZero";
        case 8: return "Privileged";
        case 9: return "LoadStoreAlignment";
        case 20: return "InstFetchProhibited";
        case 28: return "LoadProhibited";
        case 29: return "StoreProhibited";
        default: return "unknown";
    }
}

```

`StoreProhibited` (29) is the one Chapter 27 spends a session on.

The handler also reports the PHY stack high-water mark, and is candid about when that number means anything:

```
/* How deep the PHY got before dying. Meaningless unless a PHY call was in
 * flight, but when one was, this is the difference between a stack that
 * ran out and a fault that merely happened to land in a spill. */
uart_puts("  phystack : ");
uart_put_dec(phy_stack_used());
```

12.9 A hang in the code that prevents hangs

The last defect in this chapter is a two-line comedy with a serious lesson.

Redirecting the driver's internal `mutex_lock(&g_lock)` calls to the new panic-aware `draw_lock()` was done with a regular expression, which also rewrote the body of `draw_lock` itself:

```
static void draw_lock(void)
{
    if (!g_panic_mode) {
        draw_lock();          /* <- was mutex_lock(&g_lock) */
    }
}
```

Infinite recursion, inside the function whose entire purpose is to avoid a hang. The board wedged in `display_init()` before printing a single self-test.

Two findings:

It was found by bisecting — reverting `display.c` alone with `git checkout` and rebuilding — rather than by reading the diff, which is the faster route once a symptom is "nothing happens at all".

It is also the **fourth scripted edit in this project to fail silently**, and the reason those edits now carry assertions that stop on a missed match rather than writing an unchanged file.

12.10 Metrics

Quantity	Value
Tasks with guards checked, before → after	3 → 8 (now 12, by <code>TASK_MAX</code>)
Guard check frequency, before → after	~2 s (reporter) → every context switch
Guard check cost	8 word loads per tick
Stack headroom, worst case	1,604 B free of 2,048 (app-host)
Stack headroom, best case	1,844 B free of 2,048 (report)
Minimum margin across all tasks	78%
UART rx lag, before → after	1 byte → 0

Quantity	Value
Sessions the shell was unusable interactively	every one since M5
Failure paths now triggerable on demand	3
Fault fields added to the persistent record	4
Record version	1 → 2 (now 3)
Panic paths that record before reporting	2 of 2
Driver assumptions suspended in panic mode	3
Bytes drawn by a panic screen	~176,000
Reporting steps, ordered by reliability	3

12.11 What this does not establish

- **No pre-emptive overflow detection.** A guard word is found *after* it has been overwritten. A guard page would be a barrier; this hardware has no MMU to provide one.
- **No wild-pointer protection.** Guards catch growth past a stack's own base; nothing catches a stray write into the middle of a neighbour.
- **The panic screen has no failure path of its own.** If the panel is wedged, the byte count says so — but only to somebody with a serial cable, which is the audience the screen exists to replace.
- **Panic mode is untested against a genuinely broken controller.** Every test so far ran on a display that was working perfectly at the moment of the fault, so the bounds have never actually been reached.
- **A fault during the recording itself is not survivable.** The checksum would reject the torn record on the next boot, correctly, and the reason would be lost.
- **Only two fault kinds are recorded.** A hang recovered by the watchdog writes nothing at all, "so the most common recoverable failure is the one the record cannot describe".
- **No stack sizing per task.** All twelve slots get 2 KB regardless of measured use.
- **The AHB alias is validated behaviourally, not from documentation.**

Part II ends here. The kernel is complete: it boots, it interrupts, it switches, it schedules fairly, it allocates and checks its own heap, it has two locking primitives with measured contention behaviour, and it can survive, explain and report its own failures three separate ways.

Part III is what all of that exists to host.

13. Why a Bytecode VM Is the Only Isolation Available

Sources: docs/UM-NATOS-001-architecture.md §4, docs/UM-NATOS-012-m4-verification.md §2
Code: kernel/vm.h, kernel/arena.h

13.1 The argument, restated in one place

Chapter 1 introduced the constraint. This chapter is the design that follows from it, gathered in one place because the VM's shape is almost entirely determined by that single hardware fact and it is easy to lose the thread across five reports.

The premise. The ESP32's MMU translates flash addresses for execute-in-place caching. It does not implement per-process page tables. There is one address space and any code can write any address.

The consequence. A native-code application can corrupt the kernel and every other application, and no kernel design prevents this.

The response. Run applications in an interpreter. Every memory access executes as a VM instruction, so the interpreter can bounds-check it in software. That recovers, in the interpreter, the guarantee the silicon will not give.

The decision sentence, from UM-NATOS-001 §4.2:

An operating system whose applications can silently corrupt each other is firmware with extra steps, and on hardware without an MMU the VM is the only mechanism that can deliver the property.

13.2 The four properties the VM buys

The isolation is the reason. Three more properties come with it, and two of them turned out to matter as much.

1. Isolation, enforced per access

Stated in `vm.h` in the strongest terms the file can manage:

- * THE VM IS THE ISOLATION MECHANISM. The ESP32 has no MMU paging, so there is
- * no hardware that can confine an application (UM-NATOS-001 §4.2). Every memory
- * access a program makes is bounds-checked against its arena in software, on
- * every instruction, and that check is not optional and must never be compiled
- * out for speed. If it is removed, this kernel has no isolation of any kind.

`arena.h` says the same thing from the other side:

```
* This is the ONLY isolation mechanism this kernel will have. ... the
* interpreter must do it in software on every access, and it must not be
* compiled out for speed.
```

Two files, two independent statements of the same non-negotiable. That redundancy is deliberate: a future optimiser reading either file is told.

2. Preemption at a safe boundary, for free

Preemption becomes trivial. The dispatch loop can check a quantum counter between instructions; an application can always be interrupted at a known-safe boundary, with no native context switch required for application tasks.

The consequence for the kernel is large and is worth stating separately, because it is the reason there are only nine native tasks:

native preemptive context switching is needed only for kernel and driver tasks, not for applications. The number of native tasks is expected to stay small.

`vm.h` restates it as an interface property:

```
* Execution is bounded by an instruction quantum rather than run to completion,
* so a task hosting a VM can be preempted at an instruction boundary – the
* safe-boundary preemption model of UM-NATOS-001 §4.2. A program that never
* terminates costs its quantum and no more.
```

3. A small, auditable instruction set

The instruction set is ours, so it can be kept small. UM-NATOS-007 §6 set a ceiling of "roughly 40 opcodes — and resist convenience additions". The result is 35, and it has not moved since it was fixed. Chapter 14 §14.2 covers why twelve syscalls were added without adding a single opcode.

4. Faults that stop a program rather than the system

```
* - A faulting program stops the program, never the kernel. Every fault is a
* recorded code and a halt, and vm_run() returns normally afterwards.
```

13.3 The two costs, and where they landed

Slower than native. Real, and at the time of the decision unmeasured. Later derived at roughly **109 cycles per bytecode instruction** including full dispatch, operand decode, register-range validation and — for two of the three instructions in the measured loop — a bounds-checked memory access (Chapter 14 §14.8).

Requires a producer. A compiler or assembler must exist. That cost was deliberately moved off the device:

It runs on the **host**. Nothing about a producer needs to live in 176 KB of DRAM, and keeping it off the device means the on-device side stays a pure interpreter with no parsing, no symbol table, and no attack surface from untrusted text.

That last clause is a security property obtained as a side effect of a memory decision, and it is worth noticing: the device cannot be attacked through malformed *source* because it never sees source.

13.4 Register machine, not stack machine

Recorded as pending in the roadmap and settled at M4.

	Stack machine	Register machine (selected)
Instructions per unit of work	High — operands pushed and popped	Low — operands named directly
Encoding	Compact, often 1 byte	4 bytes fixed
Decode	Trivial	One byte plus three fields
Producer complexity	Lower — expression trees map directly	Higher — needs register allocation
Dispatch overhead per useful operation	Paid several times	Paid once

The deciding argument:

Dispatch is the inner loop of everything this kernel will ever run. A stack machine spends a large share of its instructions moving operands rather than computing with them, and each of those pays the full dispatch cost. A register machine executes the same program in materially fewer instructions, which more than repays a wider encoding.

The cost is a more demanding producer. That cost lands on the host, where there is no memory constraint and no consequence for being slow.

The concrete number: `sum(1..10)` compiles to **70 executed instructions** including the loop, syscalls, and a call/return.

13.5 Return addresses live in kernel memory

This is the design decision that most clearly shows what "unrepresentable rather than refused" buys.

`CALL` pushes to a kernel-side stack of 32 entries in the `vm_t` structure, not into the arena:

```
typedef struct {
    uint32_t reg[VM_REGS];
    uint32_t pc;                                /* byte offset into the arena */

    /* Return addresses, deliberately outside anything the program can
     * address. See the header comment. */
    uint32_t call[VM_CALL_DEPTH];
    uint32_t call_sp;
    /* ... */
} vm_t;
```

The header comment:

```
* - Return addresses live in KERNEL memory, not in the arena. A program
*   cannot reach its own call stack, so it cannot corrupt a return address
*   however wrong or hostile it is. This is the one structure where being
*   unable to express something is worth more than the flexibility of a
*   conventional in-memory stack.
```

And the report's version:

A program therefore **cannot address its own return addresses**. No amount of wild pointer arithmetic within its arena can corrupt where a function returns to, because that data is not in the arena at all. The entire family of stack-smashing bugs is unexpressible rather than defended against.

The cost is a fixed call depth and no stack-allocated locals in the conventional sense. For a kernel whose isolation is entirely software-enforced, being unable to express the attack is worth more than the flexibility.

A depth of 32 is a real limit and it is a *diagnosed* one: `VM_FAULT_CALL_DEPTH` exists and is checked on every `CALL`. Chapter 30 notes it is one of four fault classes implemented but never exercised on hardware.

13.6 No condition codes, and why that is a scheduling decision

Comparisons produce 0 or 1 into a register rather than setting flags, so branches need only test a register for zero:

```
/* comparison - produce 0 or 1 so branches stay simple */
VM_OP_SEQ = 0x20,
VM_OP_SNE = 0x21,
VM_OP_SLT = 0x22, /* signed */
VM_OP_SLTU = 0x23,
VM_OP_SLE = 0x24,
VM_OP_SLEU = 0x25,
```

The reason is not aesthetic:

That removes a whole class of state — condition codes with their own save/restore obligations across a context switch — from a kernel that has already paid once for forgetting to save processor state.

The "already paid once" is Chapter 8: `EPC3`, `EPS3`, `SAR`, and the three loop registers all had to be added to the frame, one of them after a full debugging session. Designing the VM so its architectural state is *only* what is already in `vm_t` means a VM context switch is a struct that is never copied at all — the task switch saves the interpreter's C stack, and `vm_t` simply stays where it is.

13.7 Choices that close failure modes

Four small decisions in the ISA, each closing a specific class of bug:

Register indices are validated, not masked.

An index of 16 or more faults. Masking would turn a producer bug into wrong answers instead of a diagnosed stop.

Shift counts are masked to 5 bits.

A shift of 32 or more is undefined in C; leaving it undefined would let a program's behaviour depend on the host compiler, which is precisely what a VM exists to prevent.

That is a subtle and good point. The VM's job is to give a program *defined* behaviour; inheriting C's undefined behaviour would defeat it at exactly the place a hostile program would probe.

Word accesses must be 4-byte aligned.

Faulting is deterministic; the alternative is an unaligned-access exception in kernel context.

Branch displacements count instructions, so a branch cannot target a misaligned address. The assembler enforces the same thing at the other end:

```
def rel(self, tok, here):
    """Branch displacement, counted in instructions from the NEXT one."""
    target = self.value(tok)
    if target % INSN_BYTES:
        raise AsmError(f"branch target '{tok}' is not instruction-aligned")
```

13.8 The asymmetry that is the whole security model

Stated in `app.c`:

```
/* Reads the progress word out of a live arena. The kernel can see into an
 * arena; a program cannot see out of one. That asymmetry is the whole design. */
uint32_t app_published(int id)
```

and in `kmain.c` for the kernel's own hosted VM:

```
/* The counter the bytecode publishes into its arena. Reading it from kernel
 * code is not a special capability – an arena is ordinary DRAM, and the
 * asymmetry is deliberate: the kernel can see into an arena, a program cannot
 * see out of one. */
static uint32_t vm_counter(void)
```

UM-NATOS-013 §3 makes the same point about *observability* rather than access:

An application publishes a progress word at a known offset in its own arena. The kernel reads it directly. **The kernel can see into an arena; a program cannot see out of one** — that asymmetry is the entire security model, and the progress word makes it visible rather than theoretical.

This is why the kernel can report `published=1270666` in `ps` without any cooperation from the program, and why a faulted program's progress can be read one last time

before its arena is released.

13.9 What the model does not claim

Worth stating before Chapter 14 measures what it does claim.

- **Native tasks are not isolated and never will be.** Drivers, the shell, the renderer and the kernel share one address space with nothing between them.
- **A program can overwrite its own instructions.** Code and data share the arena with no separation and no relocation. `vm_run` re-checks the PC on every fetch for exactly this reason (Chapter 14 §14.6).
- **A program can consume its whole quantum forever.** That is a scheduling cost, not a hang, and it is measured rather than asserted (Chapter 14 §14.7).
- **Nothing limits how much a program draws.** A program that draws constantly costs the display mutex and everyone's frame rate, and nothing measures or limits that per-application. Chapter 11's best-effort policy bounds the damage to the system without bounding it per program.
- **There is no device model.** Twelve syscalls, all hardcoded. An application cannot read the light sensor; scan the I²C bus, make a sound, save state, or receive a keypress. Chapter 31 argues this is the single most valuable thing left to build.

Next: the instruction set, the dispatch loop, and the six deliberately malformed programs that could not escape.

14. The Instruction Set and the Interpreter

Sources: docs/UM-NATOS-012-m4-verification.md Code: kernel/vm.h, kernel/vm.c, kernel/kstring.c

14.1 Encoding

Fixed 4-byte instructions. No variable length, no prefixes.

byte 0 opcode
byte 1 a destination register, or the sole operand
byte 2 b source register, or immediate low byte
byte 3 c source register / offset, or immediate high byte

/* ---- instruction encoding -----
*
* R-format reads a, b, c as register indices. I-format reads a as a register
* and (b, c) as a little-endian 16-bit immediate. A register index of 16 or
* more is a fault rather than a masked-down index: silently truncating turns a
* producer bug into wrong answers instead of a diagnosed stop.
*/

Sixteen registers, `r0` – `r15`. A 32-entry kernel-side call stack. Four-byte instructions and 4-byte alignment enforced on both the PC and word accesses.

#define VM_REGS 16
#define VM_CALL_DEPTH 32 /* kernel-side return stack */
#define VM_INSN_BYTES 4

14.2 The 35 opcodes

Group	Opcodes
Control	HALT NOP MOV LDI LDIH
Arithmetic / logic	ADD SUB MUL DIV MOD AND OR XOR SHL SHR SAR NOT NEG ADDI
Comparison	SEQ SNE SLT SLTU SLE SLEU
Control flow	JMP BRZ BRNZ CALL RET
Memory	LDW LDB STW STB
Services	SYS

The full enumeration, with the semantics recorded inline — this is the authoritative ISA definition and Appendix B is derived from it:

enum {
 /* control */
 VM_OP_HALT = 0x00,

```

VM_OP_NOP   = 0x01,
VM_OP_MOV   = 0x02,    /* a <- b                                */
VM_OP_LDI   = 0x03,    /* a <- imm16 (zero-extended)           */
VM_OP_LDIIH = 0x04,    /* a <- (a & 0xFFFF) | (imm16 << 16)    */

/* arithmetic and logic */
VM_OP_ADD   = 0x10,    /* a <- b + c                            */
VM_OP_SUB   = 0x11,
VM_OP_MUL   = 0x12,
VM_OP_DIV   = 0x13,    /* c == 0 faults                         */
VM_OP_MOD   = 0x14,    /* c == 0 faults                         */
VM_OP_AND   = 0x15,
VM_OP_OR    = 0x16,
VM_OP_XOR   = 0x17,
VM_OP_SHL   = 0x18,    /* shift count taken modulo 32          */
VM_OP_SHR   = 0x19,    /* logical                               */
VM_OP_SAR   = 0x1A,    /* arithmetic                            */
VM_OP_NOT   = 0x1B,    /* a <- ~b                               */
VM_OP_NEG   = 0x1C,    /* a <- -b                               */
VM_OP_ADDI  = 0x1D,    /* a <- a + sign_extend(imm16)          */

/* comparison – produce 0 or 1 so branches stay simple */
VM_OP_SEQ   = 0x20,
VM_OP_SNE   = 0x21,
VM_OP_SLT   = 0x22,    /* signed                                */
VM_OP_SLTU  = 0x23,
VM_OP_SLE   = 0x24,
VM_OP_SLEU  = 0x25,

/* control flow – immediates count INSTRUCTIONS, relative to the next one */
VM_OP_JMP   = 0x30,
VM_OP_BRZ   = 0x31,
VM_OP_BRNZ  = 0x32,
VM_OP_CALL  = 0x33,
VM_OP_RET   = 0x34,

/* memory – every access bounds-checked; c is an unsigned byte offset */
VM_OP_LDW   = 0x40,    /* a <- u32 at [b + c]; must be 4-byte aligned */
VM_OP_LDB   = 0x41,    /* a <- u8  at [b + c]                      */
VM_OP_STW   = 0x42,    /* u32 at [b + c] <- a                     */
VM_OP_STB   = 0x43,    /* u8  at [b + c] <- a                     */

VM_OP_SYS   = 0x50    /* imm16 selects the service; see below    */
};

```

Note the opcode *numbering*: groups start at multiples of 0x10 with gaps. That is not decoration — it leaves room to add within a group without disturbing the existing values, which matters once a producer and programs exist.

One opcode for every service, and why that mattered

SYS is one opcode of 35. Twelve services hang off it via an immediate. M4 shipped five; seven were added afterwards, and:

The ISA itself is unchanged: SYS is still one opcode of 35, and every addition is a service number rather than an instruction. That was the point of spending an opcode on a

dispatch number instead of encoding services directly — **the instruction set has not had to move since it was fixed.**

The roadmap's principal risk for M4 was "instruction set design is difficult to revise once a producer and programs exist". This is the design decision that retired that risk.

14.3 State

Everything the VM needs is in one struct, which is what makes `vm_run()` resumable and multiplexing free:

```
typedef struct {
    uint32_t reg[VM_REGS];
    uint32_t pc;                                /* byte offset into the arena */

    uint32_t call[VM_CALL_DEPTH];
    uint32_t call_sp;

    /* Viewport, in panel coordinates. Assigned by the kernel; the program can
     * read its size but never its position, and cannot draw outside it. */
    uint32_t vx, vy, vw, vh;

    int      arena;                            /* owning arena id */

    /* Which application this VM is. Messaging addresses applications, not
     * arenas or tasks, so the identity has to be here. -1 for a VM the kernel
     * hosts directly, which therefore cannot be sent to. */
    int      app_id;
    uint32_t base;                             /* cached kernel address */
    uint32_t size;                             /* cached arena length */

    int      fault;
    uint32_t fault_pc;                         /* pc of the faulting insn */
    uint32_t fault_detail;                    /* offending offset, opcode, ... */

    int      yield_now;

    uint32_t executed;                         /* instructions retired */
    uint32_t exit_status;

} vm_t;
```

`base` and `size` are cached from the arena so the inner loop does not make a function call per access. That duplication is the subject of §14.5.

`fault_pc` and `fault_detail` are separate from `pc` deliberately: a faulted VM retains the *state at the fault* while `pc` is wherever the fetch got to. The diagnostic in `app.c` prints all three.

14.4 The fault taxonomy

Ten codes. `VM_FAULT_NONE` is the only non-terminal value:

```
enum {
    VM_FAULT_NONE = 0,
```

```

VM_FAULT_OPCODE,      /* unknown opcode          */
VM_FAULT_REG,          /* register index >= VM_REGS */
VM_FAULT_BOUNDS,      /* access outside the arena  */
VM_FAULT_ALIGN,        /* misaligned word access    */
VM_FAULT_DIV0,
VM_FAULT_PC,           /* pc outside the arena, or misaligned */
VM_FAULT_CALL_DEPTH,  /* call stack exhausted      */
VM_FAULT_RET,          /* return with an empty call stack */
VM_FAULT_SYSCALL,     /* unknown syscall number    */
VM_FAULT_STRING        /* PUTS string unterminated inside arena */
};

```

Recording a fault captures three things at once:

```

static void fault(vm_t *vm, int code, uint32_t detail)
{
    vm->fault      = code;
    vm->fault_pc    = vm->pc;
    vm->fault_detail = detail;
}

```

and every code has a name, because a number in a `ps` table is not a diagnosis:

```

const char *vm_fault_name(int f)
{
    switch (f) {
        case VM_FAULT_NONE:      return "none";
        case VM_FAULT_OPCODE:    return "bad opcode";
        case VM_FAULT_REG:       return "bad register";
        case VM_FAULT_BOUNDS:    return "out of bounds";
        case VM_FAULT_ALIGN:     return "misaligned";
        case VM_FAULT_DIV0:      return "divide by zero";
        case VM_FAULT_PC:        return "bad pc";
        case VM_FAULT_CALL_DEPTH: return "call depth";
        case VM_FAULT_RET:       return "return underflow";
        case VM_FAULT_SYSCALL:   return "bad syscall";
        case VM_FAULT_STRING:    return "unterminated string";
        default:                 return "unknown";
    }
}

```

14.5 The bounds predicate, and its cross-check

```

/* Bounds predicate. Deliberately identical in meaning to arena_contains(), but
 * expressed against the cached base/size so the inner loop does not make a
 * function call per access.
 *
 * Comparison happens in the OFFSET domain. Testing `off + len` against `size`
 * directly can wrap and admit an access far outside the arena, which is exactly
 * what a hostile length would aim for. m4_selftest() cross-checks this against
 * arena_contains() so the duplication cannot drift unnoticed. */
int vm_in_bounds(const vm_t *vm, uint32_t off, uint32_t len)
{
    if (off > vm->size) {
        return 0;
    }
}

```

```

    return len <= vm->size - off;
}

```

Three lines. The whole isolation guarantee reduces to them.

Duplicated logic drifts, so the agreement is **tested rather than trusted**:

```
[4d] predicate: PASS  vm_in_bounds agrees with arena_contains over 35 cases
```

35 combinations of offset and length, including zero lengths, the exact end, one past the end, and lengths chosen to wrap the address space. All agree.

The function is exported from `vm.h` specifically so the test can reach it:

```

/* Exposed so the bounds predicate can be cross-checked against
 * arena_contains() rather than assumed to agree with it. */
int vm_in_bounds(const vm_t *vm, uint32_t off, uint32_t len);

```

That is the "samples must straddle" discipline of Chapter 21 §21.4 applied in advance: the 35 cases are chosen to *span* the boundary rather than to agree with it.

The checked accessors

Four functions, each of which is the only way memory is touched:

```

static uint32_t load_u32(vm_t *vm, uint32_t off, int *ok)
{
    if (!vm_in_bounds(vm, off, 4)) {
        fault(vm, VM_FAULT_BOUNDS, off);
        *ok = 0;
        return 0;
    }
    if (off & 3u) {
        fault(vm, VM_FAULT_ALIGN, off);
        *ok = 0;
        return 0;
    }
    *ok = 1;
    return *(volatile uint32_t *) (vm->base + off);
}

static int store_u32(vm_t *vm, uint32_t off, uint32_t v)
{
    if (!vm_in_bounds(vm, off, 4)) {
        fault(vm, VM_FAULT_BOUNDS, off);
        return 0;
    }
    if (off & 3u) {
        fault(vm, VM_FAULT_ALIGN, off);
        return 0;
    }
    *(volatile uint32_t *) (vm->base + off) = v;
    return 1;
}

```

Bounds *before* alignment. Both orders would be safe, but this one means an out-of-range misaligned access reports `BOUNDS` rather than `ALIGN`, which is the more useful diagnosis.

`volatile` on the access is deliberate: the compiler must not cache or reorder a read of memory that another agent — the kernel reading a progress word, or a `SYS_RECV` writing into the arena — can change.

14.6 The dispatch loop

```
int vm_run(vm_t *vm, uint32_t quantum)
{
    if (vm->fault != VM_FAULT_NONE) {
        return VM_RUN_FAULTED;
    }

    while (quantum-- > 0) {
        /* Instruction fetch is bounds-checked like any other access. Code and
         * data share the arena, so nothing here may be assumed valid merely
         * because the pc got there by executing. */
        if (!vm_in_bounds(vm, vm->pc, VM_INSN_BYTES) || (vm->pc & 3u)) {
            fault(vm, VM_FAULT_PC, vm->pc);
            return VM_RUN_FAULTED;
        }

        const volatile uint8_t *ins = (const volatile uint8_t *) (vm->base + vm->pc);

        uint8_t op = ins[0];
        uint8_t a = ins[1];
        uint8_t b = ins[2];
        uint8_t c = ins[3];
        uint32_t imm = (uint32_t)b | ((uint32_t)c << 8);
        int32_t simm = (int32_t)(int16_t)imm;

        uint32_t next = vm->pc + VM_INSN_BYTES;
    }
}
```

The comment on the fetch is the important one. A program can overwrite its own code, so the PC being valid one instruction ago says nothing about now.

Register validation, once per instruction

```
/* One register-range check for the whole instruction. Operands that an
 * opcode does not use are still indices in a well-formed program, so
 * checking uniformly costs nothing and closes the gap where an unused
 * field carries garbage. Immediate formats exempt b and c. */
int imm_form = (op == VM_OP_LDI || op == VM_OP_LDIH || op == VM_OP_ADDI ||
                op == VM_OP_JMP || op == VM_OP_BRZ || op == VM_OP_BRNZ ||
                op == VM_OP_CALL || op == VM_OP_SYS);
int off_form = (op == VM_OP_LDW || op == VM_OP_LDB ||
                op == VM_OP_STW || op == VM_OP_STB);

if (op != VM_OP_HALT && op != VM_OP_NOP && op != VM_OP_RET) {
    if (a >= VM_REGS) {
        fault(vm, VM_FAULT_REG, a);
        return VM_RUN_FAULTED;
    }
}
```

```

    }
    if (!imm_form && b >= VM_REGS) {
        fault(vm, VM_FAULT_REG, b);
        return VM_RUN_FAULTED;
    }
    if (!imm_form && !off_form && c >= VM_REGS) {
        fault(vm, VM_FAULT_REG, c);
        return VM_RUN_FAULTED;
    }
}

```

"Checking uniformly costs nothing and closes the gap where an unused field carries garbage" — a well-formed program's unused fields are zero, so validating them is free and turns a producer bug into a diagnosis.

The switch

```

switch (op) {
case VM_OP_HALT: vm->executed++; return VM_RUN_HALTED;
case VM_OP_NOP: break;

case VM_OP_MOV: r[a] = r[b]; break;
case VM_OP_LDI: r[a] = imm; break;
case VM_OP_LDIH: r[a] = (r[a] & 0xFFFFu) | (imm << 16); break;

case VM_OP_ADD: r[a] = r[b] + r[c]; break;
case VM_OP_SUB: r[a] = r[b] - r[c]; break;
case VM_OP_MUL: r[a] = r[b] * r[c]; break;

case VM_OP_DIV:
    if (r[c] == 0u) { fault(vm, VM_FAULT_DIV0, 0); return VM_RUN_FAULTED; }
    r[a] = r[b] / r[c];
    break;
/* ... */

/* Shift counts are masked to 5 bits. A shift of 32 or more is undefined
 * in C and would let a program's behaviour depend on the host compiler,
 * which is precisely what a VM exists to prevent. */
case VM_OP_SHL: r[a] = r[b] << (r[c] & 31u); break;
case VM_OP_SHR: r[a] = r[b] >> (r[c] & 31u); break;
case VM_OP_SAR: r[a] = (uint32_t)((int32_t)r[b] >> (r[c] & 31u)); break;

/* ... */

case VM_OP_JMP:
    next = (uint32_t)((int32_t)next + simm * VM_INSN_BYTES);
    break;
case VM_OP_BRZ:
    if (r[a] == 0u) { next = (uint32_t)((int32_t)next + simm *
VM_INSN_BYTES); }
    break;

case VM_OP_CALL:
    if (vm->call_sp >= VM_CALL_DEPTH) {
        fault(vm, VM_FAULT_CALL_DEPTH, vm->call_sp);
        return VM_RUN_FAULTED;
    }
}

```

```

        vm->call[vm->call_sp++] = next;
        next = (uint32_t)((int32_t)next + simm * VM_INSN_BYTES);
        break;
    case VM_OP_RET:
        if (vm->call_sp == 0u) {
            fault(vm, VM_FAULT_RET, 0);
            return VM_RUN_FAULTED;
        }
        next = vm->call[--vm->call_sp];
        break;

    case VM_OP_LDW: r[a] = load_u32(vm, r[b] + c, &ok); break;
    case VM_OP_LDB: r[a] = load_u8 (vm, r[b] + c, &ok); break;
    case VM_OP_STW: ok = store_u32(vm, r[b] + c, r[a]); break;
    case VM_OP_STB: ok = store_u8 (vm, r[b] + c, r[a]); break;

    /* ... SYS ... */

    default:
        fault(vm, VM_FAULT_OPCODE, op);
        return VM_RUN_FAULTED;
    }

    if (!ok) {
        return VM_RUN_FAULTED;
    }

    vm->pc = next;
    vm->executed++;
}

return VM_RUN_QUANTUM;
}

```

Branch displacements are multiplied by `VM_INSN_BYTES` and computed relative to `next`, so a branch cannot target a misaligned address by construction.

Why a switch and not a computed goto

```

* The dispatch loop is a switch. A computed-goto table would be faster, but it
* is also the kind of construct where a missing entry is a silent jump rather
* than a diagnosed fault, and correctness of the bounds checks matters more
* here than dispatch overhead. Revisit only with a measurement showing dispatch
* is actually the bottleneck.

```

The `default:` case is the payoff: an unknown opcode is `VM_FAULT_OPCODE`, not undefined behaviour.

14.7 The three exits

```

/* Why vm_run() returned. */
enum {
    VM_RUN_HALTED = 0,          /* HALT or SYS_EXIT – program finished */
    VM_RUN_FAULTED,            /* see vm_fault() */
}

```

```

VM_RUN_QUANTUM      /* budget spent; call vm_run() again      */
};

```

`vm_run(vm, quantum)` executes at most `quantum` instructions and returns `VM_RUN_QUANTUM` if the budget runs out. State lives entirely in `vm_t`, so execution resumes exactly where it stopped.

This is the safe-boundary preemption: a task hosting a VM can be descheduled between any two bytecode instructions without the program being able to observe it. **The interpreter never has to be reentrant, and the timer interrupt never has to understand bytecode.**

That last sentence is the payoff for the whole design.

14.8 Results

```

vm says sum(1..10) = 55
[4a] program   : HALTED ok  insns=70 resumes=4 status=0 fault=none
[4b] faults    : bounds=ok div0=ok opcode=ok reg=ok ret=ok align=ok PASS
[4c] runaway   : PASS  bounded at 500 insns, no fault, kernel alive
[4d] predicate: PASS  vm_in_bounds agrees with arena_contains over 35 cases

```

The program, and what makes the result meaningful

The demo exercises arithmetic, a counted loop, a bounds-checked store and reload through arena memory, a call and return, and four syscalls. It ran to `HALT` with exit status 0 in 70 instructions.

But:

It was run with a quantum of 16, so it was **suspended and resumed 4 times** mid-execution and produced the correct answer regardless — which is the property that matters, not the answer itself.

Containment: six malformed programs

Probe	Attempts	Expected fault	Result
bounds	Store to offset <code>0xFFF0</code> in a 2 KB arena	<code>VM_FAULT_BOUNDS</code>	ok
div0	Divide by zero	<code>VM_FAULT_DIV0</code>	ok
opcode	Byte <code>0xFF</code> as an instruction	<code>VM_FAULT_OPCODE</code>	ok
reg	<code>MOV r16, r0</code>	<code>VM_FAULT_REG</code>	ok
ret	<code>RET</code> with an empty call stack	<code>VM_FAULT_RET</code>	ok
align	Word load at offset 1	<code>VM_FAULT_ALIGN</code>	ok

Each was **hand-encoded as raw bytes rather than assembled**, and the reason matters:

Routing them through the assembler would only have demonstrated that well-formed programs behave; the claim under test is about malformed ones.

The assembler would refuse to emit `MOV r16, r0`:

```
def reg(tok):
    m = re.fullmatch(r"[rR](\d+)", tok)
    if not m:
        raise AsmError(f"expected a register, got '{tok}'")
    n = int(m.group(1))
    if not 0 <= n < 16:
        raise AsmError(f"register out of range: {tok}")
    return n
```

So a test that used it would be testing the producer, not the interpreter.

And the strongest evidence is indirect:

every line of output after this test exists only because the kernel survived all six.

Runaway

A one-instruction program branching to itself. Executed exactly 500 instructions under a 500-instruction quantum, reported `VM_RUN_QUANTUM`, raised no fault, and left the kernel running.

An unterminating program is a scheduling cost, not a hang.

Under the scheduler: the arithmetic is the proof

```
vm arena      : id=0 base=0x3ffb23f0 program=28 B
tasks         : report=0 a=1 b=2 vm=3

t=1801  switches r/a/b=451/450/450  work a/b=10585776/11610220  guards=ok
        freew a/b=480/480  corrupt=0
        | vm sw=450  insns=3291313  counter=1097103  fault=none
```

The hosted program increments a counter in its own arena forever, performing a bounds-checked store every iteration, "so the isolation path is on the hot loop rather than exercised once at startup".

The two preemption mechanisms compose:

The timer interrupt suspends the VM task wherever it happens to be — almost always somewhere inside the interpreter's own C code, mid-instruction from the bytecode's point of view — and `vm_run()` separately returns at a bytecode instruction boundary when its quantum expires. Neither is aware of the other, and the program cannot observe either.

And then the check that makes this a measurement rather than an observation:

The arithmetic is the proof. The loop body is exactly three instructions (`add`, `stw`, `jmp`), so instructions retired must be three times the counter plus the three-instruction preamble:

```
1,097,103 × 3 + 3 = 3,291,312 observed 3,291,313
```


The residual of one is the sample being taken mid-iteration, between the increment and the store. Across **450 preemptions** and 3.29 million bytecode instructions, not one instruction was lost, replayed, or half-executed. A context switch that dropped or repeated an interpreter step would show here as drift, and there is none.

This is the single best-designed measurement in the project. Two independently maintained quantities — a counter the *program* writes and an instruction count the *interpreter* keeps — related by a known constant. Any error in either shows as a residual.

Dispatch cost, derived

Between consecutive reports 200 ticks apart, the VM retired 365,704 instructions. 200 ticks is 160,000,000 CCOUNT cycles, of which the VM task receives roughly one quarter under an even four-way round robin:

```
40,000,000 cycles ÷ 365,704 instructions ≈ 109 cycles per bytecode instruction
```

That figure covers full dispatch, operand decode, register-range validation, and — for two of the three instructions in the loop — a bounds-checked memory access. It is a **ceiling** rather than a precise cost, since the quarter-share assumption ignores time spent in the interrupt handler itself.

109 cycles is high enough that a computed-goto dispatch table would be worth measuring if the VM ever becomes the bottleneck, and low enough that nothing here justifies trading away the diagnosability of a `switch` today.

14.9 `kstring.c`, and the trap it nearly created

The link failed on an undefined reference to `memcpy` from a function whose source never mentions it.

GCC synthesises calls to `memcpy` / `memset` from ordinary C — byte-copy loops, struct assignments, large local initialisers — and does so even under `-fno-builtin`, which only governs the treatment of those names when written explicitly. Under `-nostdlib` nothing supplies them.

`kernel/kstring.c` provides `memcpy`, `memset`, `memmove` and `memcpy`. And then:

The trap worth recording is what happens next: GCC will recognise the copy loop *inside* `memcpy` as a `memcpy` and rewrite it into a call to itself. That bug is silent, infinitely recursive, and would surface as a stack overflow in whatever unrelated code first copied a struct.

`-fno-tree-loop-distribute-patterns` is the supported way to prevent it, and it is on the build's command line with that explanation attached (Chapter 5 §5.4).

14.10 Metrics

Quantity	Value
Opcodes	35 (roadmap ceiling ~40)
Instruction width	4 B fixed
Registers	16
Call depth	32, in kernel memory
Demo program	120 B, 6 labels
Instructions to compute <code>sum(1..10)</code>	70
Quantum resumptions during demo	4
Fault classes exercised	6 of 10 defined
Bounds predicate cases cross-checked	35
Bytecode instructions under the scheduler	3,291,313
Preemptions survived	450
Instruction-accounting drift	1 (mid-iteration sample)
Dispatch cost	~109 cycles/instruction (ceiling)
Image size at M4	10,560 B

14.11 What M4 does not establish

- **No arena-relative program loading.** A program is copied to offset 0 and addresses everything relative to its arena. There is no relocation, no linking of separately assembled units, and no code/data separation within the arena — a program can overwrite its own instructions.
- **Only 6 of 10 fault classes are exercised.** `VM_FAULT_PC`, `VM_FAULT_CALL_DEPTH`, `VM_FAULT_SYSCALL` and `VM_FAULT_STRING` are implemented but untested on hardware.
- **Dispatch cost is bounded, not precisely measured.** A direct measurement would need `CCOUNT` sampled either side of a `vm_run()` call — and, given Chapter 9 §9.8, `task_cpu_cycles()` rather than `xt_ccount()`.
- **No multiple concurrent VMs at M4**, though nothing in the design prevented more. Chapter 16 added four.
- **The hosted program is not replaced when it stops.** A halted or faulted VM yields its task forever rather than loading something else.
- **Syscall argument validation is per-call, not systematic.**

Every syscall added since checks its own arguments, and each was reasoned about individually. Nothing enforces that a future one will, and there is no shared harness that would catch an unchecked length in a new service.

That last one is the most serious open item in this chapter and is in Chapter 30.

Next: the producer that makes the ISA usable, and the programs written in it.

15. The Producer: `vasm.py` and the Programs

Sources: docs/UM-NATOS-012-m4-verification.md §4 Code: tools/vasm.py, tools/*.vasm, kernel/generated/*.h, build.ps1

15.1 Why the producer is on the host

An instruction set with no producer is a specification, not a tool. `vasm.py` assembles a text source into a C header containing a byte array and label offsets.

```
"""vasm – assembler for the nat-os bytecode VM.

Runs on the host, not the device. A producer has to exist for the ISA to be
usable (UM-NATOS-007 §6), but nothing about it needs to live in 176 KB of DRAM,
and keeping it here means the on-device side stays a pure interpreter.

    python tools/vasm.py prog.vasm -o kernel/generated/prog.h --name vm_prog
"""
```

Three consequences of that placement, one of which is a security property:

- **No memory constraint.** 381 lines of Python with a two-pass assembler, string escapes, and label resolution costs the device nothing.
- **No consequence for being slow.** It runs once per build.
- **No attack surface from untrusted text.** The device never parses anything. There is no symbol table, no tokeniser, and no error path on the device at all.

15.2 Syntax

<code>; comment</code>	<code>(also #)</code>
<code>label:</code>	<code>byte offset within the arena</code>
<code>ldi r1, 10</code>	
<code>add r2, r1, r3</code>	
<code>ldw r1, r2, 4</code>	<code>; r1 <- u32 at [r2 + 4]</code>
<code>brz r4, done</code>	<code>; label operands are instruction-relative</code>
<code>sys puts</code>	
<code>halt</code>	
<code>.string "text\n"</code>	<code>; NUL-terminated</code>
<code>.byte 1, 2, 3</code>	
<code>.word 0xdeadbeef</code>	<code>; 4-byte aligned automatically</code>
<code>.align 4</code>	
<code>.space 64</code>	<code>; n zero bytes</code>

Immediates accept decimal, `0x` hex, `'c'` character literals, and `@label` for a label's byte offset — which is what makes string arguments to `SYS PUTS` expressible:

```
ldi    r0, @msg
sys    puts
```

15.3 The opcode and syscall tables

```
# mnemonic -> (opcode, format). Formats:
# none no operands          ab    a, b
# abc  a, b, c              ai    a, imm16
# i     imm16 (relative label ok)  air  a, relative label
# abo   a, b, byte offset      sys  syscall name or number
OPS = {
    "halt": (0x00, "none"), "nop": (0x01, "none"),
    "mov": (0x02, "ab"), "ldi": (0x03, "ai"), "ldih": (0x04, "ai"),

    "add": (0x10, "abc"), "sub": (0x11, "abc"), "mul": (0x12, "abc"),
    "div": (0x13, "abc"), "mod": (0x14, "abc"), "and": (0x15, "abc"),
    "or": (0x16, "abc"), "xor": (0x17, "abc"), "shl": (0x18, "abc"),
    "shr": (0x19, "abc"), "sar": (0x1A, "abc"), "not": (0x1B, "ab"),
    "neg": (0x1C, "ab"), "addi": (0x1D, "ai"),

    "seq": (0x20, "abc"), "sne": (0x21, "abc"), "slt": (0x22, "abc"),
    "sltu": (0x23, "abc"), "sle": (0x24, "abc"), "sleu": (0x25, "abc"),

    "jmp": (0x30, "i"), "brz": (0x31, "air"), "brnz": (0x32, "air"),
    "call": (0x33, "i"), "ret": (0x34, "none"),

    "ldw": (0x40, "abo"), "ldb": (0x41, "abo"),
    "stw": (0x42, "abo"), "stb": (0x43, "abo"),

    "sys": (0x50, "sys"),
}
```

The syscall table carries a warning about drift, and a note on where the error lands:

```
# Must stay in step with the VM_SYS_* enum in kernel/vm.h. A name missing here
# is caught at assembly time ("cannot parse value"), which is the right place -
# the alternative is a numeric syscall that assembles cleanly and faults on the
# device with VM_FAULT_SYSCALL.
SYSCALLS = {
    "exit": 0, "putc": 1, "puts": 2, "putd": 3, "ticks": 4,
    "fill": 5, "text": 6, "dims": 7, "touch": 8, "blit": 9, "send": 10, "recv": 11,
}
```

This is the one place in the system where a definition is genuinely duplicated across the host/device boundary and *not* cross-checked at runtime. The mitigation is that the failure lands at assembly time with a comprehensible message rather than on the device with a fault code.

15.4 Two passes

Pass 1 — lay out, recording label offsets. Every item is recorded with its offset, kind, payload and source line:

```
def _layout(self, line, lineno):
    parts = line.split(None, 1)
    head = parts[0].lower()
    rest = parts[1] if len(parts) > 1 else ""
```

```

if head == ".string":
    data = parse_string(rest.strip()) + b"\x00"
    self.items.append((self.offset, "raw", data, lineno))
    self.offset += len(data)
elif head == ".byte":
    /* ... */
elif head == ".word":
    self._pad_to(4)
    /* ... */
elif head in OPS:
    # Instructions must be 4-byte aligned; the VM faults otherwise, so
    # pad rather than emit something that cannot execute.
    self._pad_to(INSN_BYTES)
    self.items.append((self.offset, "insn", (head, split_operands(rest)),
lineno))
    self.offset += INSN_BYTES
else:
    raise AsmError(f"unknown mnemonic or directive '{head}'")

```

The `_pad_to(INSN_BYTES)` before every instruction is the producer half of the `VM_FAULT_ALIGN` contract: the assembler makes misalignment *unrepresentable* rather than leaving it for the interpreter to diagnose.

Pass 2 — encode, resolving labels. Errors carry the source line:

```

except AsmError as e:
    where = f"line {lineno}: " if lineno else ""
    raise AsmError(f"{where}{e}") from None

```

Branch displacements

```

def rel(self, tok, here):
    """Branch displacement, counted in instructions from the NEXT one."""
    target = self.value(tok)
    if target % INSN_BYTES:
        raise AsmError(f"branch target '{tok}' is not instruction-aligned")
    delta = (target - (here + INSN_BYTES)) // INSN_BYTES
    if not -32768 <= delta <= 32767:
        raise AsmError(f"branch to '{tok}' out of range ({delta})")
    return delta & 0xFFFF

```

Two checks the interpreter would otherwise have to make, or a program would otherwise get wrong: alignment and range.

Operand encoding

```

def insn(self, mnem, ops, here):
    opcode, fmt = OPS[mnem]
    a = b = c = 0

    def need(n):
        if len(ops) != n:
            raise AsmError(f"{mnem} takes {n} operand(s), got {len(ops)}")

    if fmt == "none":
        need(0)

```

```

elif fmt == "ab":
    need(2); a, b = reg(ops[0]), reg(ops[1])
elif fmt == "abc":
    need(3); a, b, c = reg(ops[0]), reg(ops[1]), reg(ops[2])
elif fmt == "ai":
    need(2)
    a = reg(ops[0])
    v = self.value(ops[1])
    if not -32768 <= v <= 65535:
        raise AsmError(f"immediate out of range: {v}")
    b, c = v & 0xFF, (v >> 8) & 0xFF
/* ... */
elif fmt == "abo":
    need(3)
    a, b = reg(ops[0]), reg(ops[1])
    off = self.value(ops[2])
    if not 0 <= off <= 255:
        raise AsmError(f"offset out of range 0..255: {off}")
    c = off
elif fmt == "sys":
    need(1)
    key = ops[0].strip().lower()
    v = SYSCALLS[key] if key in SYSCALLS else self.value(ops[0])
    b, c = v & 0xFF, (v >> 8) & 0xFF

return bytes((opcode, a, b, c))

```

Every operand class is range-checked. The register parser refuses 16 and above (Chapter 14 §14.8 notes why the containment tests had to bypass it entirely).

String handling done properly

Comment stripping and operand splitting are both string-aware, which matters because `.string "a;b"` and `.string "a,b"` are legal:

```

def split_operands(text):
    """Split on commas that are not inside a quoted string."""
    parts, cur, in_str, esc = [], "", False, False
    for ch in text:
        if in_str:
            cur += ch
            if esc:
                esc = False
            elif ch == "\\":
                esc = True
            elif ch == "'":
                in_str = False
        elif ch == "'":
            in_str = True
            cur += ch
        elif ch == ",":
            parts.append(cur.strip())
            cur = ""
        else:
            cur += ch
    if cur.strip():

```

```
parts.append(cur.strip())
return parts
```

15.5 The output

A C header, with the bytes and every label as a `#define`:

```
/* Generated by tools/vasm.py - do not edit.
 * Source: tools/spin.vasm
 */
#ifndef NATOS_GENERATED_VM_SPIN_H
#define NATOS_GENERATED_VM_SPIN_H

#include <stdint.h>

static const uint8_t vm_spin[] = {
    0x03, 0x01, 0x18, 0x00, 0x03, 0x02, 0x00, 0x00, 0x03, 0x09, 0x01, 0x00,
    0x10, 0x02, 0x02, 0x09, 0x42, 0x02, 0x01, 0x00, 0x30, 0x00, 0xfd, 0xff,
    0x00, 0x00, 0x00, 0x00,
};

#define VM_SPIN_LEN 28u

#define VM_SPIN_AT_START 0u
#define VM_SPIN_AT_LOOP 12u
#define VM_SPIN_AT_COUNTER 24u

#endif
```

Twenty-eight bytes. Seven instructions and a word.

The exported label offsets are what let the kernel read a program's published progress without any agreement beyond the source file:

```
static uint32_t vm_counter(void)
{
    if (g_vm_arena < 0) {
        return 0;
    }
    return *(volatile uint32_t *) (g_vm_base + VM_SPIN_AT_COUNTER);
}
```

`VM_SPIN_AT_COUNTER` is generated from `counter:` in the `.vasm` file. Move the label and the kernel follows automatically.

Decoding that array by hand

Worth doing once, because it demonstrates that the encoding really is as simple as claimed:

Bytes	Decode	Meaning
03 01 18 00	LDI r1, 0x0018	r1 = 24 — @counter
03 02 00 00	LDI r2, 0	iteration count

Bytes	Decode	Meaning
03 09 01 00	LDI r9, 1	the constant to add
10 02 02 09	ADD r2, r2, r9	r2 += r9
42 02 01 00	STW r2, r1, 0	store r2 at [r1 + 0] — bounds-checked
30 00 fd ff	JMP -3	back three instructions to loop
00 00 00 00	—	the counter word

fd ff is -3 as a little-endian 16-bit value. loop is at offset 12; the JMP is at offset 20, so next is 24, and $24 + (-3 \times 4) = 12$. Correct.

15.6 The programs

Twelve .vasm sources currently assemble. They fall into four groups.

Progress demonstrators

app_a.vasm — three instructions per iteration:

```
; Application A – counts. Three instructions per iteration.
start:
    ldi    r1, @counter
    ldi    r2, 0
    ldi    r9, 1
loop:
    add    r2, r2, r9
    stw    r2, r1, 0        ; publish progress where the kernel can see it
    jmp    loop

    .align 4
counter:
    .word  0
```

app_b.vasm — four instructions per iteration, and the comment explains why that difference is the point:

```
; Application B – publishes squares. Four instructions per iteration, so it
; advances at a visibly different rate from A under the same quantum. Two
; applications sharing a core should not advance in lockstep, and the differing
; rates make that observable rather than assumed.
start:
    ldi    r1, @square
    ldi    r2, 0
    ldi    r9, 1
loop:
    add    r2, r2, r9
    mul    r4, r2, r2
    stw    r4, r1, 0
    jmp    loop
```

That design is what makes the M5 result checkable to the digit (Chapter 16 §16.5): A publishes an iteration count, B publishes its square, and both must agree with their

instruction totals.

The demonstration program

demo.vasm exercises every mechanism M4 claims, and says so:

```
; nat-os - M4 demonstration program.
;
; Exercises every mechanism the milestone claims: arithmetic, a counted loop,
; a bounds-checked store and reload, a call/return through the kernel-side
; return stack, and syscalls that produce UART output.
;
; Computes sum(1..10) = 55, round-trips it through arena memory so the load and
; store paths are actually used rather than assumed, and prints it.

start:
    ldi    r1, 0           ; sum
    ldi    r2, 1           ; i
    ldi    r3, 11          ; limit (exclusive)
    ldi    r9, 1           ; a constant 1 to add with

loop:
    slt    r4, r2, r3      ; r4 = i < limit
    brz    r4, done
    add    r1, r1, r2      ; sum += i
    add    r2, r2, r9      ; i++
    jmp    loop

done:
    ; Round-trip through memory. @total is a byte offset into the arena, so
    ; both of these go through the bounds check like any other access.
    ldi    r5, @total
    stw    r1, r5, 0
    ldi    r6, 0
    ldw    r6, r5, 0

    ldi    r0, @msg
    sys    puts
    mov    r0, r6
    call   print_num       ; return address lives in kernel memory
    ldi    r0, 10          ; '\n'
    sys    putc

    ldi    r0, 0
    sys    exit

print_num:
    sys    putd
    ret

    .align 4
total:
    .word  0

msg:
    .string "  vm says sum(1..10) = "
```

The round-trip through memory is not decoration: without it, `LDW` and `STW` would be untested by the demonstration and the bounds path would be exercised only by the deliberately malformed probes.

The escape attempts

app_rogue.vasm — the memory escape. Its comment is the clearest statement of the isolation model anywhere in the tree:

```
; Application ROGUE – deliberately written to escape its arena.
;
; UM-NATOS-007 §7 names this as the milestone's principal risk: "an application
; deliberately written to escape its arena must fail to do so."
;
; It walks a store upward through memory four bytes at a time, starting above
; its own code so it does not destroy the loop doing the walking. Every store
; inside the arena succeeds; the first store past the end must fault, and the
; reported offset should equal the arena size exactly – which makes this a test
; of WHERE the boundary is, not merely that one exists.
;
; Note what it cannot do. A VM address is an offset into its own arena, so there
; is no value it could load into r5 that names another application's memory. The
; escape is not refused, it is unrepresentable.
start:
    ldi    r1, @counter
    ldi    r2, 0
    ldi    r9, 1
    ldi    r5, 128                ; start probing well above this program
    ldi    r6, 4

probe:
    add    r2, r2, r9
    stw    r2, r1, 0             ; keep publishing so progress is visible
    stw    r2, r5, 0             ; the probe – succeeds until it does not
    add    r5, r5, r6
    jmp    probe
```

Note the second store: it keeps publishing progress *while* probing, so the kernel can see the program was alive and working right up to the fault.

app_gfx_rogue.vasm — the same question asked of the screen:

```
; Application GFX-ROGUE – deliberately tries to paint the entire panel.
;
; UM-NATOS-013 §5.2 established that an application cannot reach another's
; MEMORY. This is the same question asked of the SCREEN: given a syscall that
; draws, can a program scribble over the kernel's status area, the colour strip,
; or a neighbour's output?
;
; It asks for a 240x320 fill – the whole display – in red, forever. If viewport
; clipping works, only its own 26-pixel strip turns red and everything else is
; untouched. If it does not, the entire panel goes red and the failure is
; impossible to miss.
;
; It also tries a wildly out-of-range origin, which arrives at the kernel as a
; huge unsigned number. That is the case offset-domain clipping exists for.
```

```

start:
    ldi    r0, 0
    ldi    r1, 0
    ldi    r2, 240          ; full panel width
    ldi    r3, 320          ; full panel height
    ldi    r4, 0xF800        ; red
    sys    fill
    /* ... white ... */
    ldi    r0, 60000         ; an origin far outside any viewport
    ldi    r1, 60000
    /* ... */
    jmp    start

```

The design of the failure mode is deliberate: red then white in a loop, so that an escape *flickers* and is "impossible to miss".

The interactive programs

app_paint.vasm — the first program that responds to a person:

```

; Application PAINT – the first interactive program.
;
; It asks the kernel how big its canvas is and is never told where that canvas
; sits. Every coordinate it receives is relative to its own viewport, and a
; touch anywhere else on the panel is reported as no touch at all – so it cannot
; observe input aimed at its neighbours, nor reconstruct an absolute position
; from one it is allowed to see.

start:
    sys    dims                ; r0 = (width << 16) | height
    ldi    r8, 16
    shr    r6, r0, r8          ; r6 = width
    ldi    r9, 0xFFFF
    and    r7, r0, r9          ; r7 = height

    ldi    r0, 0                ; paint the whole canvas: "touch here"
    ldi    r1, 0
    mov    r2, r6
    mov    r3, r7
    ldi    r4, 0x001F            ; blue
    sys    fill

loop:
    sys    touch                ; r0 = touched, r1 = x, r2 = y
    brz    r0, loop

    mov    r10, r1              ; the syscall results are about to be
    mov    r11, r2              ; overwritten by the fill arguments

    mov    r0, r10
    mov    r1, r11
    ldi    r2, 5
    ldi    r3, 5
    ldi    r4, 0xFFE0            ; yellow
    sys    fill

```

```
jmp    loop
```

The `mov r10, r1 / mov r11, r2` pair is a small illustration of a real ABI consequence: syscall results and syscall arguments share `r0 – r5`, so a program must save what it wants to keep. `app_blit.vasm` makes the same point explicitly:

```
ldi    r12, @img          ; kept in a high register: the touch syscall
                          ; overwrites r0..r2 every iteration
```

app_blit.vasm — builds an image and blits it, exercising the only syscall that takes a pointer *and* a length:

```
; The image lives entirely inside the application's arena, and the offset it
; hands to SYS BLIT is an arena offset like any other pointer. The kernel bounds
; checks it against the arena AND clips the destination to the viewport, so
; neither the source nor the destination can reach past what this program owns.

start:
    ldi    r12, @img
    ldi    r2, 0            ; byte offset into the image
    ldi    r3, 128          ; 8 x 8 pixels x 2 bytes
    ldi    r4, 0xFFE0       ; low half-word -> first pixel, yellow
    ldih   r4, 0x001F       ; high half-word -> second pixel, blue
    ldi    r5, 4            ; one word = two pixels

build:
    add    r6, r12, r2
    stw    r4, r6, 0
    add    r2, r2, r5
    sltu   r7, r2, r3
    brnz   r7, build
    /* ... */
    .align 4
img:
    .space 128
```

LDIH earns its place here: `0x001FFFE0` is a 32-bit constant and the ISA's immediates are 16 bits, so it takes `LDI` then `LDIH` to build. Two pixels per store is a real optimisation in a machine where every store is bounds-checked.

The messaging pair

app_ping.vasm, whose comment states the addressing property:

```
; Application PING – sends a counter to application 1, forever.
;
; The message body is four bytes in PING's own arena. It names the destination
; by APPLICATION ID; there is no argument here that could denote memory
; belonging to the receiver, so the message cannot be placed anywhere PING
; chooses – only handed over.
;
; Most sends are refused, because the mailbox holds one message and PONG has not
; drained it yet. That is expected and is what the refused counter measures.
```

```

start:
    ldi    r10, @buf
    ldi    r11, 0           ; the value being sent
    ldi    r12, 1

loop:
    add    r11, r11, r12
    stw    r11, r10, 0     ; body: one 32-bit counter

    ldi    r0, 1           ; destination application id
    mov    r1, r10         ; arena offset of the body
    ldi    r2, 4           ; length
    sys    send            ; r0 = 1 delivered, 0 refused

    jmp    loop

```

The last comment is doing real work: it tells the reader that 157,957 refusals in the telemetry (Chapter 16 §16.7) are the *expected* result of this program's design, not a fault.

15.7 The build integration

```

$vasm = Get-ChildItem "$root\tools\*.vasm" -ErrorAction SilentlyContinue
if ($vasm) {
    Write-Host "== assembling bytecode ==" -ForegroundColor Cyan
    foreach ($src in $vasm) {
        $hdr = Join-Path $gen ($src.BaseName + ".h")
        & $python "$root\tools\vasm.py" $src.FullName -o $hdr --name ("vm_" +
$src.BaseName)
        if ($LASTEXITCODE -ne 0) { throw "vasm failed: $($src.Name)" }
    }
}

```

Every `.vasm` in `tools/` is assembled to `kernel/generated/<name>.h` with the array named `vm_<name>`. Adding a program is adding a file; the shell's program table (Chapter 16 §16.4) is the only other place that needs to know.

The assembler reports what it produced:

```

print(f"    vasm: {args.source} -> {args.output} "
      f"({len(data)} bytes, {len(asm.labels)} labels)")

```

15.8 What the producer is not

- **Not a compiler.** There is no higher-level language. Every program in this book is hand-written assembly, which is why they are all short and why the longest does one thing.
- **No linking.** A program is a single translation unit. There is no way to assemble two files and combine them.
- **No relocation.** A program is copied to arena offset 0 and addresses everything relative to that. `@label` offsets are absolute-within-the-arena, which works

precisely because the base is always 0.

- **No separate code and data sections.** `.word` and `.string` land wherever they appear in the source, in the same address space as instructions. A program can overwrite its own code, and Chapter 14 §14.6's PC re-check is the interpreter's answer to that.
- **No macros, no include, no conditional assembly.**

None of those is a defect at this scale; all of them would be needed before a program got much past fifty lines. Chapter 31 discusses what a real toolchain would need.

Next: the layer that turns a VM into several applications with lifecycles.

16. Applications: Lifecycle, Three Levels of Scheduling, Messaging

Sources: docs/UM-NATOS-013-m5-verification.md Code: kernel/app.c, kernel/app.h, kernel/ipc.c, kernel/ipc.h, kernel/shell.c, kernel/kmain.c

16.1 Three levels of scheduling, and only the newest knows

The system now preempts at three independent levels:

Level	Mechanism	Granularity	Introduced
1	Timer interrupt preempts native tasks	Any machine instruction	M2
2	vm_run() quantum returns control	Bytecode instruction boundary	M4
3	app_tick() round-robins that quantum across applications	One quantum per application	M5

The design point:

only the newest one knows the others exist.

Adding level 3 required **no change to the scheduler and no change to the interpreter**. `app_tick()` is an ordinary function calling `vm_run()` in a loop, hosted by an ordinary native task. Levels 1 and 2 remain unaware that applications exist.

That is the payoff for the boundary drawn in M4: because `vm_t` carries all execution state and `vm_run()` is resumable, multiplexing applications is bookkeeping rather than surgery.

The whole of level 3:

```
void app_tick(uint32_t quantum)
{
    for (int id = 0; id < APP_MAX; id++) {
        app_t *a = &g_apps[id];
        if (a->state != APP_RUNNING) {
            continue;
        }

        int r = vm_run(&a->vm, quantum);
        if (r == VM_RUN_QUANTUM) {
            continue;                /* still going; next one gets a turn */
        }
        /* ... halt or fault reporting, then retire ... */
    }
}
```

and the task that hosts it:


```

/* Hosts every application. One native task drives the third scheduling level;
 * the applications inside it are preempted by their quantum, and this task is
 * itself preempted by the timer. */
static void task_apps(void)
{
    for (;;) {
        app_tick(2000);
        if (app_live_count() == 0) {
            task_yield();          /* nothing to run – do not spin at full tilt */
        }
    }
}

```

Eight lines for a whole scheduling level.

16.2 The application record

```

#define APP_MAX 4

typedef enum {
    APP_FREE = 0,
    APP_RUNNING,
    APP_HALTED,          /* ran to completion */
    APP_FAULTED,         /* violated a rule and was terminated */
    APP_KILLED           /* stopped from the shell */
} app_state_t;

typedef struct {
    app_state_t state;
    const char *name;
    int arena;
    uint32_t base;
    uint32_t bytes;
    uint32_t publish_off;
    vm_t vm;
} app_t;

```

Three terminal states rather than one, so `ps` can distinguish a program that finished from one that was killed from one that broke a rule. Chapter 24 §24.9 notes the consequence: a faulted program keeps its slot and shows no close button, because it is not running.

16.3 Lifecycle: one release path, not three

`app_start()` allocates an arena, copies the image in, and initialises a VM:

```

int app_start(const char *name, const uint8_t *img, uint32_t len,
              uint32_t arena_bytes, uint32_t publish_off)
{
    if (len > arena_bytes) {
        return -1;          /* image would not fit its own arena */
    }

    for (int id = 0; id < APP_MAX; id++) {
        app_t *a = &g_apps[id];

```

```

    if (a->state == APP_RUNNING) {
        continue;
    }

    int arena = arena_create(arena_bytes);
    if (arena < 0) {
        return -1;
    }

    uint32_t base = 0;
    if (arena_bounds(arena, &base, 0) != 0) {
        arena_destroy(arena);
        return -1;
    }

    uint8_t *dst = (uint8_t *)base;
    for (uint32_t i = 0; i < len; i++) {
        dst[i] = img[i];
    }

    if (vm_init(&a->vm, arena) != 0) {
        arena_destroy(arena);
        return -1;
    }

    /* One horizontal strip per slot. Assigned by the kernel and never by
     * the application: a program can ask how large its canvas is, but the
     * only coordinates it can express are inside it. Same property as its
     * arena, applied to pixels. */
    vm_set_viewport(&a->vm, 0u, APP_VIEW_Y0 + (uint32_t)id * APP_VIEW_PITCH,
                   APP_VIEW_W, APP_VIEW_H);
    vm_set_app_id(&a->vm, id);

    /* A fresh application must not inherit mail addressed to whoever held
     * this slot before it. */
    ipc_clear(id);

    a->state      = APP_RUNNING;
    /* ... */
    return id;
}

return -1;
}

```

Every failure path releases the arena it had just created. That is easy to get wrong, and the *termination* path is where it was made structural:

```

/* Releases the arena and records why the application stopped. Kept in one place
 * because "terminating an application releases its arena completely" is an exit
 * criterion, and three separate paths reach it – halt, fault, and kill. Three
 * copies of a release is three chances to leak one. */
static void retire(app_t *a, app_state_t why)
{
    if (a->arena >= 0) {
        arena_destroy(a->arena);
        a->arena = -1;
    }
}

```

```

/* Wipe the strip on the way out, so a dead application does not leave its
 * last frame on the panel looking like a live one. */
display_fill_rect(a->vm.vx, a->vm.vy, a->vm.vw, a->vm.vh, COLOR_BLACK);

/* Undelivered mail dies with the recipient. */
for (int i = 0; i < APP_MAX; i++) {
    if (&g_apps[i] == a) {
        ipc_clear(i);
    }
}
a->base = 0;
a->state = why;
}

```

Three concerns in one function, each with its own reason: the arena, the pixels, and the mail. "A dead application does not leave its last frame on the panel looking like a live one" is the same class of decision as the advancing marker block in Chapter 18 — making a stopped thing *look* stopped.

Diagnose before releasing

```

if (r == VM_RUN_FAULTED) {
    /* Diagnose before releasing: the arena is about to go back to the
     * heap, and the offending offset is only meaningful alongside the
     * size it exceeded. */
    console_lock();
    uart_puts("\n [app ");
    uart_put_dec((unsigned int)id);
    uart_puts(" ");
    uart_puts(a->name);
    uart_puts("' TERMINATED] ");
    uart_puts(vm_fault_name(vm_fault(&a->vm)));
    uart_puts(" at offset ");
    uart_put_dec(a->vm.fault_detail);
    uart_puts(" of ");
    uart_put_dec(a->bytes);
    uart_puts(" B arena, pc=");
    uart_put_dec(a->vm.fault_pc);
    uart_puts(", after ");
    uart_put_dec(a->vm.executed);
    uart_puts(" instructions\n");
    console_unlock();
    retire(a, APP_FAULTED);
}

```

Producing, for the rogue:

```

[app 2 'rogue' TERMINATED] out of bounds at offset 256 of 256 B arena,
pc=28, after 167 instructions

```

"*offset 256 of 256 B arena*" is the whole result in one phrase — the offending offset next to the size it exceeded, which is why the diagnostic is printed before the arena goes back to the heap.

16.4 The shell, and launching by name

The shell is a native task polling UART0, and holds no privilege the rest of the kernel lacks:

it is a front end to `app_start()` and `app_kill()`.

Programs are registered by the caller rather than referenced directly, so the shell has no dependency on which images exist.

```
static const shell_program_t PROGRAMS[] = {
    { "counter", vm_app_a,      VM_APP_A_LEN,      512u, VM_APP_A_AT_COUNTER },
    { "squares", vm_app_b,      VM_APP_B_LEN,      512u, VM_APP_B_AT_SQUARE },
    { "rogue",   vm_app_rogue,   VM_APP_ROGUE_LEN,  256u, VM_APP_ROGUE_AT_COUNTER },
    { "draw",    vm_app_draw,    VM_APP_DRAW_LEN,   512u, VM_APP_DRAW_AT_NAME },
    { "gfxrogue", vm_app_gfx_rogue, VM_APP_GFX_ROGUE_LEN, 256u, 0u },
    { "paint",   vm_app_paint,   VM_APP_PAINT_LEN,  512u, 0u },
    { "blit",    vm_app_blit,    VM_APP_BLIT_LEN,   512u, 0u },
    { "ping",    vm_app_ping,    VM_APP_PING_LEN,   512u, 0u },
    { "pong",    vm_app_pong,    VM_APP_PONG_LEN,   512u, 0u },
};
```

Note the arena sizes: 512 bytes for most, and **256 for both rogues**. A smaller arena makes the escape test faster and the boundary figure easier to read.

Never by index

```
/* Start a registered program by NAME.
 *
 * The table is indexed by position everywhere else and that is exactly how the
 * paint application failed to launch: PROGRAMS grew from three entries to six,
 * a hard-coded [4] silently became gfxrogue instead of paint, and the symptom
 * was a screen flickering red and white with no obvious connection to the
 * indexing. A name cannot drift when the table is reordered. */
static int start_program(const char *name)
{
    for (int i = 0; i < PROGRAM_COUNT; i++) {
        if (str_same(PROGRAMS[i].name, name)) {
            return app_start(PROGRAMS[i].name, PROGRAMS[i].img, PROGRAMS[i].len,
                             PROGRAMS[i].arena_bytes, PROGRAMS[i].publish_off);
        }
    }
    uart_puts(" [boot] no such program: ");
    uart_puts(name);
    uart_puts("\n");
    return -1;
}
```

The symptom is worth dwelling on because it is a perfect example of a defect whose *appearance* has no relationship to its *cause*: a strip flickering red and white, caused by an array index. UM-NATOS-017 §8.4 records that the user's description — *"touching in the red/white thing was making black dots"* — identified both the wrong application and a second bug, and was "a faster route to the cause than the counters, which correctly reported `g/w = 0/0` without indicating why".

Boot order also matters and is documented:

```
/* Order matters: ping addresses application 1, so pong must take that
 * slot. Slots are handed out lowest-free-first. */
start_program("ping");
start_program("pong");
```

That is a genuine coupling and it is recorded rather than hidden. It is the one place in the system where application identity is hard-coded into a program.

shell_poll() never blocks

UART receive was added for it — polled rather than interrupt-driven, because the shell is the only consumer and a console that drops a keystroke under load is a better outcome than a second interrupt source competing with the scheduler tick.

shell_poll() never blocks, so a user holding a key cannot starve the system.

```
static void task_shell(void)
{
    shell_begin();
    for (;;) {
        shell_poll();
        task_yield();
    }
}
```

16.5 The M5 results

```
[1] interleave: PASS  a insns=60000 count=19999 | b insns=60000 square=224970001

    [app 2 'rogue' TERMINATED] out of bounds at offset 256 of 256 B arena,
                             pc=28, after 167 instructions

[2] isolation : PASS  rogue faulted at offset 256 = arena size;
                             neighbours still running and advancing
[3] release   : PASS  heap 158048/158048 B, live=0, check=0

tasks: report=0 a=1 b=2 vm=3 apps=4 shell=5
```

The self-test runs before the scheduler starts, for the same reason M3's did:

```
/*
 * Runs single-threaded before the scheduler starts, so the results are
 * deterministic and a failure cannot be blamed on task switching. The live,
 * interactive version of the same thing runs afterwards under the shell.
 *
 * Each block is one exit criterion from UM-NATOS-007 §7.
 */
```

Criterion 1 — interleaving, checked by arithmetic

Both applications received an identical instruction budget: **60,000 each**. `app_tick()` hands out the same quantum per round, and equal totals confirm neither starved nor

over-ran.

Their *progress* differs, which is the more informative result:

```
A: 19,999 iterations × 3 instructions + 3 preamble = 60,000
B: 14,999 iterations × 4 instructions + 3 preamble = 59,999 (sampled mid-iteration)
14,9992 = 224,970,001 – the published square, exactly
```

B does more work per iteration and therefore advances more slowly in its own terms while consuming the same CPU. Two applications sharing a core should not advance in lockstep, and the differing rates make that observable rather than assumed. That B's published value is exactly 14,999² also confirms its arithmetic survived every preemption intact.

The pass condition is written to require *all* of that:

```
/* Both must have run, and both must have run the SAME amount: app_tick
 * hands out an identical quantum to each. Equal instruction counts with
 * unequal published values is exactly right – B does more work per
 * iteration, so it advances more slowly in its own terms. */
int c1 = (app_state(a) == APP_RUNNING) && (app_state(b) == APP_RUNNING) &&
        (ia == ib) && (ia > 0u) && (pa > 0u) && (pb > 0u) && (pa != pb);
```

`pa != pb` is the clause that makes this a test rather than a formality: two applications reporting the *same* progress would mean the quantum was not being shared, or that one program was not running at all.

Criterion 2 — the rogue is terminated, alone

Every store inside the arena succeeded. The first store past the end faulted, **at offset 256 of a 256-byte arena** — the boundary is exactly where it should be, not approximately. This is a test of *where* the wall stands, not merely that one exists.

The neighbours were unaffected: "both still `running`, and both with published values strictly greater than before the rogue was introduced, so they were still making progress rather than merely still alive."

And the deeper point, quoted in Chapter 1 and worth having in place:

A VM address is an offset into its own arena, so there is no value it could load that names another application's memory. Reaching a neighbour is not refused — it is **unrepresentable**. The bounds check exists for the weaker case of a program walking off its own end, which is precisely what was observed.

Relaunching `rogue` from the shell reused slot 2 and terminated it again identically, "confirming the lifecycle is repeatable and not a one-shot".

Criterion 3 — the arena comes back exactly

After killing both applications, free heap returned to **158,048 B** — **exactly** its pre-test value, with zero live applications and a clean structural check. An exact match rules out a

partial release; an approximate one would have indicated a leaked header or a missed coalesce.

The shell, exercised

Command	Result
help	command list
progs	three programs with image and arena sizes
ps	table with live counters and the faulted rogue's diagnosis
run rogue	started id=2, followed by the termination message
kill 0	killed 0, arena released
mem	free=155952 largest=155952 blocks=4 high_water=5120 check=0
bogus	rejected, not silently ignored

id	name	state	arena	insns	published
0	counter	running	512 B	3812000	1270666
1	squares	running	512 B	3811635	1795557008
2	rogue	faulted	256 B	167	0 [out of bounds @256]

That table is the security model made legible: three programs, three arena sizes, three independent instruction counts, and a fault that names both the offending offset and the boundary it crossed.

16.6 Regression

Six native tasks running concurrently — reporter, two M2 workers, the M4 VM host, the application host, and the shell. Workers' guards intact and `corrupt=0` throughout; stack headroom 483 words. M2, M3 and M4 self-tests all still passing.

The heap number moved, and the report explains it rather than hiding it:

Heap fell from 167,680 B because `TASK_MAX` rose from 4 to 8, adding 8 KB of statically allocated task stacks, plus the application table and shell buffers. That is a deliberate trade: six concurrent tasks needed the slots.

16.7 Messaging

M5 originally shipped with no way for applications to communicate, and called that "simple but not ultimately useful". Messaging was added afterwards without weakening anything.

<code>ipc s/d/r = 278/277/157957</code>	<code>badbuf = 0</code>
---	-------------------------

278 sent and 277 delivered — every accepted message reached its recipient, one in flight when sampled. No application ever offered a buffer outside its own arena.

Copied, never shared

ipc.h's header is the design statement:

```
* Applications have no shared memory and will not be given any. An arena is the
* unit of isolation (UM-NATOS-013 §5.2), and mapping one arena into another
* would dissolve the single property everything above it depends on.
*
* So messages are COPIED, twice: out of the sender's arena into a kernel
* mailbox, and later out of that mailbox into the receiver's arena. The cost is
* two copies of a small buffer; what it buys is that neither application ever
* holds a reference to the other's memory, and neither can observe the other's
* layout.
*
* A sender names a DESTINATION APPLICATION, never an address. There is no
* argument to this interface that could denote memory belonging to somebody
* else, which is the same shape as the arena, the viewport and the pointer: the
* unwanted operation is not refused, it is unexpressible.
```

Four resources now share that property: memory, pixels, input, and messaging. Chapter 17 §17.7 tabulates them.

Refusal over queueing

```
* One mailbox per application, holding one message. A second message to a full
* mailbox is refused and counted rather than queued: a queue needs a policy for
* what to drop when it fills, and there is no consumer yet whose requirements
* would decide that policy. Refusing is the honest placeholder.
```

The 157,957 refusals are the *expected* result of ping's design — it writes in a tight loop while pong drains occasionally:

Queueing would need a policy for what to drop when the queue fills, and no consumer exists yet whose requirements would settle that policy. Refusing is the honest placeholder, and the counter makes the back pressure visible rather than hiding it in a buffer.

The implementation

```
int ipc_send(int from, int dst, const uint8_t *data, uint32_t len)
{
    if (dst < 0 || dst >= APP_MAX || len == 0u || len > IPC_MSG_MAX) {
        g_refused++;
        return -1;
    }

    /* Delivering to a slot that is not running would leave a message for
     * whoever occupies it next, which is a channel between two applications
     * that never agreed to talk. */
    if (app_state(dst) != APP_RUNNING) {
        g_refused++;
        return -1;
    }

    /* The mailbox is shared between the sending and receiving applications,
     * which run in the same task but at different times, and with the reporter
```



```

    * reading the counters. Short enough for a critical section. */
    uint32_t crit = crit_enter();

    if (g_box[dst].len != 0u) {
        g_refused++;                /* occupied; sender must retry */
        crit_exit(crit);
        return -1;
    }

    for (uint32_t i = 0; i < len; i++) {
        g_box[dst].data[i] = data[i];
    }
    g_box[dst].len = len;
    g_box[dst].from = from;
    g_sent++;

    crit_exit(crit);
    return 0;
}

```

Receive copies at most `max` bytes and reports the true length:

```

/* Takes the pending message for `id`, if any. Returns its length and writes the
 * sender id through `from`; returns 0 when the mailbox is empty. Copies at most
 * `max` bytes and reports the true length, so a receiver offering too small a
 * buffer loses the tail rather than overrunning. */
uint32_t ipc_rcv(int id, uint8_t *out, uint32_t max, int *from);

```

Mail that must not outlive its context

Two cases, both of which would otherwise create a channel nobody asked for:

- **Sending to a slot that is not running is refused.** Otherwise the message waits for whoever occupies that slot next — two applications connected without either having agreed to it.
- **Retiring an application clears its mailbox**, so a successor in the same slot cannot read its predecessor's mail.

And a third, on the other side: `app_start()` clears the mailbox too, so a fresh application cannot inherit mail addressed to its predecessor even if the predecessor was never retired cleanly.

Where the buffer is checked

The bounds check for a message body is in `vm.c`, not `ipc.c`, and the reason is a good general principle:

SEND and RCV check the buffer in `vm.c`, not in `ipc.c`. Only the VM knows which arena an offset belongs to; the messaging layer receives a kernel pointer and has no way to tell where it came from. Putting the check where the information is, rather than where the operation happens, is the whole reason that split exists.

16.8 Metrics

Quantity	Value
Native tasks	6 (of 8 slots) at M5; 9 of 12 now
Applications	4 slots
Scheduling levels	3
Instruction budget per app, criterion 1	60,000 each
Accounting drift	0 for A, 1 for B (mid-iteration sample)
Rogue fault offset	256, of a 256 B arena
Heap after release	158,048 B, exactly baseline
Image size at M5	14,464 B
Messages sent / delivered / refused	278 / 277 / 157,957
Bad buffers offered	0
Message size limit	64 B
Registered programs	9

16.9 What M5 does not establish

- **No memory protection between native tasks.** Only *applications* are isolated.
- **No per-application CPU accounting or priority.** Every application gets the same quantum, and there is no way to say one matters more.
- **Messaging is one slot deep and unidirectional per send.** No broadcast, no queue, and no way to *wait* for a message — a receiver polls.
- **No program loading from storage.** Images are compiled into the kernel. There is no filesystem, so "install an application" has no meaning yet.
- **The shell has no history, editing beyond backspace, or completion.**
- **Fault recovery is termination only.** There is no way for an application to handle its own fault, and no restart policy.
- ~Console output interleaves.~ Closed by Chapter 11.

Next: the twelve syscalls, and the four resources an application cannot observe outside its own allocation.

17. Syscalls and the Confinement Model

Sources: docs/UM-NATOS-016-display-syscalls.md, docs/UM-NATOS-017-touch.md §8, docs/UM-NATOS-012 §10 Code: kernel/vm.c, kernel/vm.h, kernel/app.h

17.1 Twelve services, one opcode

```
/* Syscalls. Arguments in r0..r3, result in r0. */
enum {
    VM_SYS_EXIT = 0, /* r0 = status */
    VM_SYS_PUTC = 1, /* r0 = character */
    VM_SYS_PUTS = 2, /* r0 = offset of a NUL-terminated string */
    VM_SYS_PUTD = 3, /* r0 = value, printed as unsigned decimal */
    VM_SYS_TICKS = 4, /* r0 <- timer_ticks() */

    VM_SYS_FILL = 5, /* r0=x r1=y r2=w r3=h r4=colour */
    VM_SYS_TEXT = 6, /* r0=str offset r1=x r2=y r3=fg r4=bg r5=scale */
    VM_SYS_DIMS = 7, /* r0 <- (width << 16) | height */
    VM_SYS_TOUCH = 8, /* r0 <- touched, r1 <- x, r2 <- y */
    VM_SYS_BLIT = 9, /* r0=offset r1=x r2=y r3=w r4=h */
    VM_SYS_SEND = 10,
    VM_SYS_RECV = 11
};
```

They fall into three groups by *what crosses the boundary*, and each group needed a different kind of check:

Syscall	Added in	Crosses the boundary
EXIT PUTC PUTS PUTD TICKS	M4	Scalars, and one string read out of the arena
FILL TEXT DIMS	Chapter 18's syscalls	Coordinates, clipped to a viewport
TOUCH	Chapter 19	Coordinates, <i>inward</i> , filtered by viewport
BLIT		A pointer and a length , both program-supplied
SEND RECV	Chapter 16	A buffer, copied through a kernel mailbox

17.2 Viewports

Each application slot owns a horizontal strip. The geometry lives in `app.h` because the close button depends on the exact boundary:

```
#define APP_VIEW_Y0    224u
#define APP_VIEW_PITCH 16u
#define APP_VIEW_H     14u
#define APP_NAME_W     44u
#define APP_CLOSE_W    16u
```

```
#define APP_CHROME_W  (APP_NAME_W + APP_CLOSE_W)
#define APP_VIEW_W    (DISP_W - APP_CHROME_W)
```

Above the strips the kernel keeps its status area; below them the colour strip. Neither is reachable from an application.

Coordinates are relative, and position is never disclosed

```
/* Display. Coordinates are VIEWPORT-relative and clipped to it, so an
 * application cannot name a pixel outside its own region – the same
 * property arenas give it for memory. There is no syscall to move or
 * resize a viewport; that is the kernel's to assign. */
```

`SYS DIMS` returns **size only**:

```
case VM_SYS_DIMS:
    /* Size only. A program is told how much canvas it has, never where on
     * the panel it sits – position is not its business and knowing it would
     * only tempt a producer into absolute coordinates. */
    vm->reg[0] = (vm->vw << 16) | (vm->vh & 0xFFFFu);
    return 0;
```

A program is told how much canvas it has and nothing about where that canvas sits, which removes the temptation for a producer to compute absolute coordinates and removes the possibility of it naming one.

The report states the parallel with memory explicitly:

This is deliberately the arena model applied to a second resource. UM-NATOS-013 §5.2 established that an application cannot reach another's memory because an address outside its arena is *unrepresentable*. The same holds for pixels: a coordinate outside the viewport is clipped before it can exist.

17.3 Clipping belongs in the VM, not the driver

`display_fill_rect()` clips to the **panel**, which is the wrong boundary:

A program allowed to paint anywhere on the panel could erase the kernel's status area, the colour strip, and every other application's output.

So `vp_fill()` clips to the **viewport**, in the offset domain, for exactly the reason `arena_contains()` does:

```
static void vp_fill(vm_t *vm, uint32_t x, uint32_t y, uint32_t w, uint32_t h,
                  uint16_t colour)
{
    if (x >= vm->vw || y >= vm->vh) {
        return; /* origin outside – nothing to draw */
    }
    if (w > vm->vw - x) { w = vm->vw - x; }
    if (h > vm->vh - y) { h = vm->vh - y; }
    if (w == 0u || h == 0u) {
        return;
    }
}
```

```

/* Re-derive the absolute rectangle and confirm it lies wholly inside the
 * viewport. This duplicates the clipping above on purpose: the check that
 * matters is on what actually reaches the panel, not on what the clipping
 * intended. */
uint32_t ax = vm->vx + x, ay = vm->vy + y;
if (ax < vm->vx || ay < vm->vy ||
    ax + w > vm->vx + vm->vw || ay + h > vm->vy + vm->vh) {
    g_vp_escapes++;
    return; /* refuse it as well as count it */
}
g_vp_calls++;
if (ay + h > g_vp_max_y) {
    g_vp_max_y = ay + h;
}

if (!display_try_lock()) {
    g_draw_skipped++;
    return;
}
display_fill_rect(ax, ay, w, h, colour);
display_unlock();
}

```

The offset-domain argument, from the file's own comment:

```

* Arithmetic is in the offset domain for the same reason arena_contains() is
* (UM-NATOS-010 §5.2). Coordinates arrive as uint32_t, so a program passing a
* "negative" value hands over something near 0xFFFFFFFF; comparing it against
* the viewport width rejects it, whereas computing x + w first would wrap and
* admit it.

```

`app_gfx_rogue.vasm` passes `60000`, `60000` for exactly this case, and the program's own comment says so: *"an origin far outside any viewport... That is the case offset-domain clipping exists for."*

The deliberate duplication

The re-check after clipping is not redundancy by accident:

`vp_fill()` re-derives the absolute rectangle *after* clipping and re-checks it against the viewport before handing it to the driver, counting and refusing any that would fall outside. The duplication is deliberate: **what matters is what actually reaches the panel, not what the clipping intended**. If the two ever disagree, `escapes` stops being zero and says so.

17.4 Containment, measured

```
vp calls=136  escapes=0  maxy=250/320
```

- **escapes = 0** — no rectangle reaching the driver ever fell outside the viewport it came from, across 136 fills.
- **maxy = 250** — the lowest row any application has painted. Slot 2's viewport is $168 + 2 \times 28 = 224$, height 26, bottom edge **exactly 250**. "Not one row below it,

and nowhere near the panel's 320."

(Those are the M5-era strip constants; the geometry moved in Chapter 24 §24.7, and the counters moved with it because they are computed from `vm->vy` and `vm->vh` rather than from literals.)

The counters are declared with their rationale:

```
/* Containment, measured rather than observed.
 *
 * Whether a hostile program's drawing stays inside its viewport is visible on
 * the panel, but "it looks right" is a judgement and this is the claim the
 * whole design turns on. These counters make it a number: every rectangle that
 * actually reaches the driver is re-checked against the viewport it came from,
 * and g_vp_escapes counts any that would fall outside. It must stay at zero no
 * matter what an application asks for. */
static uint32_t g_vp_escapes;
static uint32_t g_vp_max_y;      /* highest absolute row any app has painted */
static uint32_t g_vp_calls;
```

Why a counter and not a photograph

This is the methodological point of the whole chapter:

Containment was visible on the panel, and the report back was *"I think it looks good."*

That is a judgement, and this is the claim the entire syscall design rests on. Given [the freeze that was missed by looking at the panel], a hedged glance was not the evidence it deserved. The counter is also the more durable form: **it keeps checking after everyone has stopped looking, and it distinguishes "confined" from "confined so far"**.

And the honest limit of what the counter proves:

The escape counter is not a proof of the clipping logic, only evidence that it and the re-check agree on everything tried so far. A case both get wrong identically would pass.

17.5 The first pointer to cross the boundary

`SYS TEXT` takes an arena offset, making it the first syscall to carry a pointer:

```
#define VM_STR_MAX 48

/* Copies a NUL-terminated string out of the arena into kernel memory, one
 * bounds-checked byte at a time. The copy is not paranoia: display_text() takes
 * a kernel pointer, and handing it an arena address would let a program's own
 * writes change the string mid-render. A string that runs to the end of the
 * arena without a terminator faults rather than reading past it. */
static int copy_string(vm_t *vm, uint32_t off, char *dst, uint32_t max)
{
    for (uint32_t n = 0; n < max - 1u; n++) {
        int ok = 0;
        uint32_t ch = load_u8(vm, off + n, &ok);
        if (!ok) {
            return 0;          /* load_u8 recorded the fault */
        }
        dst[n] = (char)ch;
    }
}
```

```

        if (ch == 0u) {
            return 1;
        }
    }
    dst[max - 1u] = 0;
    return 1;
}
/* truncated, not a fault */

```

Two properties: every byte goes through `load_u8`, which bounds-checks; and truncation at 48 characters is *not* a fault, because a long string is a legal thing to ask for.

`SYS_PUTS` does the same walk directly, with a different termination condition, and the comment names the class of bug:

```

case VM_SYS_PUTS: {
    /* Walked one byte at a time, each bounds-checked. A string that runs
     * to the end of the arena without a terminator is a fault, not a read
     * past the end — this is the classic way a "harmless" print primitive
     * becomes an information leak. */
    uint32_t off = vm->reg[0];
    for (uint32_t n = 0; n < vm->size; n++) {
        int ok = 0;
        uint32_t ch = load_u8(vm, off + n, &ok);
        if (!ok) {
            return 1;
        }
        if (ch == 0u) {
            return 0;
        }
        uart_putc((char)ch);
    }
    fault(vm, VM_FAULT_STRING, off);
    return 1;
}

```

The loop bound is `vm->size` — a string cannot be longer than the arena — so `VM_FAULT_STRING` is reachable and means precisely "unterminated within the arena".

Text overrun handling

```

static void vp_text(vm_t *vm, uint32_t x, uint32_t y, const char *s,
                  uint16_t fg, uint16_t bg, uint32_t scale)
{
    if (scale == 0u || scale > 4u) {
        scale = 1u;
    }
    if (x >= vm->vw || y >= vm->vh) {
        return;
    }
    /* Reject rather than clip vertically: half a line of text is worse than
     * none, and the glyph renderer has no partial-row mode. */
    if (8u * scale > vm->vh - y) {
        return;
    }
}

```

```

/* Truncate to what fits horizontally, so a long string cannot run out of
 * the viewport and across a neighbour's. */
char buf[VM_STR_MAX];
uint32_t room = (vm->vw - x) / (6u * scale);
/* ... */
}

```

Horizontal overrun truncates; vertical overrun refuses. Two different answers for two different reasons, both stated.

17.6 `SYS_BLIT`: the first program-controlled length

```

/* Blit an image the application holds in its own arena.
 * r0 = arena offset of RGB565 pixels, row-major, no padding
 * r1 = x, r2 = y (viewport-relative)
 * r3 = w, r4 = h
 * The first syscall to take a pointer AND a length from the program, so it
 * is the first where the size itself is attacker-controlled. */
VM_SYS_BLIT = 9,

```

```

case VM_SYS_BLIT: {
    uint32_t off = vm->reg[0];
    uint32_t x = vm->reg[1], y = vm->reg[2];
    uint32_t w = vm->reg[3], h = vm->reg[4];

    /* Dimensions are bounded BEFORE they are multiplied. w*h*2 on
     * unchecked 32-bit values overflows readily – 65536x65536 wraps to
     * zero, which would pass a byte-length check and then blit whatever
     * followed. This is the first syscall where the length is supplied by
     * the program rather than implied by the operation. */
    if (w == 0u || h == 0u || w > DISP_W || h > DISP_H) {
        vm->reg[0] = 0;
        return 0;
    }

    /* Pixels are 16-bit; an odd offset would mean unaligned loads. Faults
     * rather than silently reading a shifted image. */
    if (off & 1u) {
        fault(vm, VM_FAULT_ALIGN, off);
        return 1;
    }

    uint32_t bytes = w * h * 2u; /* <= 240*320*2, cannot wrap */
    if (!vm_in_bounds(vm, off, bytes)) {
        fault(vm, VM_FAULT_BOUNDS, off);
        return 1;
    }

    /* Clip to the viewport before drawing, exactly as vp_fill does. The
     * source stride stays the ORIGINAL width so a clipped blit still walks
     * the caller's rows correctly. */
    if (x >= vm->vw || y >= vm->vh) {
        vm->reg[0] = 0;
        return 0;
    }

    uint32_t cw = (w > vm->vw - x) ? (vm->vw - x) : w;
    uint32_t ch = (h > vm->vh - y) ? (vm->vh - y) : h;

```



```

/* A skipped blit is not a fault: the program asked for something legal
 * and the panel was busy. It returns success with nothing drawn, the
 * same contract as a blit clipped entirely outside the viewport above. */
if (!display_try_lock()) {
    g_draw_skipped++;
    vm->reg[0] = 0;
    return 0;
}
display_blit(vm->vx + x, vm->vy + y, cw, ch,
             (const uint16_t *) (vm->base + off), w);
display_unlock();

g_blits++;
vm->reg[0] = 1;
vm->yield_now = 1;           /* milliseconds, not instructions */
return 0;
}

```

The ordering is the interesting part and the report states the rule:

The dimensions are therefore each bounded against the panel *before* they are multiplied, after which the product provably cannot wrap.

$65536 \times 65536 \times 2$ wraps to zero on 32 bits. A zero-byte length would satisfy a naive bounds check and then copy whatever followed the buffer. Bounding $w \leq 240$ and $h \leq 320$ first makes $w * h * 2 \leq 153,600$, which cannot wrap — and the comment says so in one clause: `/* <= 240*320*2, cannot wrap */`.

Note also the source stride: `display_blit(..., w)` passes the **original** width as the stride while drawing `cw` columns, so a clipped blit still walks the caller's rows correctly. `display.h` documents that parameter for this reason:

```

/* Copies a caller-supplied RGB565 image to the panel. `src_stride` is the
 * source row pitch in PIXELS, so a clipped blit can walk the original rows
 * without the caller having to repack. */

```

17.7 Input, and the fourth confined resource

```

case VM_SYS_TOUCH: {
    touch_state_t t;
    touch_latest(&t);

    /* Absolute panel coordinates translated into this viewport, in the
     * offset domain so a touch above or left of the viewport underflows to
     * a huge value and is rejected rather than wrapping into range. */
    uint32_t dx = t.x - vm->vx;
    uint32_t dy = t.y - vm->vy;
    int inside = t.down && (t.x >= vm->vx) && (t.y >= vm->vy) &&
                (dx < vm->vw) && (dy < vm->vh);

    if (t.down && !inside) {
        g_touch_withheld++;
    }
    if (inside) {

```

```

        g_touch_given++;
    }

    vm->reg[0] = inside ? 1u : 0u;
    vm->reg[1] = inside ? dx : 0u;
    vm->reg[2] = inside ? dy : 0u;
    return 0;
}

```

The confinement is stronger than refusal:

Coordinates are **viewport-relative**, and a touch anywhere else on the panel is reported to the asking application as **no touch at all**.

That is stronger than refusing to answer. An application is never told where its viewport sits — `SYS_DIMS` returns size only — so it cannot reconstruct an absolute position even from a touch it is permitted to see, and it has no way to learn that a touch happened elsewhere.

The four confined resources

Resource	Confined by	Outside its allocation
Memory	Arena bounds check	Unrepresentable — an address outside the arena cannot be formed
Pixels	Viewport clipping	Clipped before it exists
Input	Viewport test on delivery	Reported as no touch
Messages	Destination is an application id	No argument can denote another's memory

Four resources, one property. Every one of them was designed by asking "what would an application have to be able to say in order to misbehave?" and then removing the vocabulary.

Measured

```
touch g/w = 81/109211      last = ...->191,174
```

81 touches delivered to the paint application and **109,211 withheld** for landing outside its strip, with the last accepted touch at y=174 — inside it.

Both halves matter. Delivery alone would not show confinement, and withholding alone would be indistinguishable from a broken syscall. The withheld count is large because the application polls continuously: it asked for the pointer tens of thousands of times while a finger was demonstrably on the glass elsewhere, and was told there was none every time.

A snapshot, not the bus

```

/* Pointer input, viewport-relative like everything else an application
 * sees. Returns r0 = touched, r1 = x, r2 = y. A touch outside this
 * application's viewport reports as no touch at all: an application must

```

```
* not be able to observe input directed at its neighbours, any more than it
* can read their memory or draw on their pixels. */
```

The syscall reads a state published by the polling task rather than driving the controller itself. Doing its own SPI would cost milliseconds per call and contend with the touch task for the bus.

That decision is why `TOUCH` does not end the time slice — see §17.8.

17.8 The quantum distinction

`vm_run()`'s quantum bounds **instructions**. A display fill is one instruction and milliseconds of SPI:

```
/* Set by a syscall whose cost is measured in milliseconds rather than
 * instructions. The quantum bounds INSTRUCTIONS; a display fill is one
 * instruction and ~31 ms of bit-banged SPI, so without this a 2,000
 * instruction budget can be minutes of drawing inside a single vm_run()
 * and the preemption guarantee is worthless. */
int    yield_now;
```

The dispatch loop honours it:

```
case VM_OP_SYS:
    if (do_syscall(vm, imm)) {
        vm->executed++;
        return vm->fault == VM_FAULT_NONE ? VM_RUN_HALTED : VM_RUN_FAULTED;
    }
    /* An expensive syscall ends the slice regardless of how much of the
     * instruction budget is left, so wall-clock time per vm_run() stays
     * bounded by roughly one display operation instead of by the
     * quantum. Without this the host task disappears for minutes and
     * every other application starves. */
    if (vm->yield_now) {
        vm->yield_now = 0;
        vm->pc = next;
        vm->executed++;
        return VM_RUN_QUANTUM;
    }
    break;
```

`FILL`, `TEXT` and `BLIT` set it. `TOUCH`, `SEND` and `RECV` do not.

The rule is the cost of the work rather than the fact of being a syscall. A fill is one instruction and milliseconds of SPI, so without ending the slice a 2,000-instruction quantum becomes minutes of drawing. `TOUCH` reads a snapshot the polling task already published and costs nothing, so charging it a slice would only make an application slower for asking.

And the generalisation, which is the transferable part:

This is a general hazard, not a display one: **any future syscall whose cost is measured in time rather than instructions needs the same treatment.**

17.9 The freeze, and how it survived three commits

This is the second half of UM-NATOS-016, and it belongs here because it was caused by the display work and hidden by how the display work was checked.

The symptom

Boot completed. Every task entered. The shell printed its banner. The reporter ran the critical-section test and printed `PASS` at tick 8. Then nothing, ever again — no output, no scheduling, a frozen panel retaining its last frame.

```
before: 0 report lines in 35 s, ticks frozen at 8
after:  16 report lines, t=3246, all tasks healthy
```

The cause

`task_yield()` set the comparator unconditionally:

```
xt_set_ccompare1(xt_ccount() + 64u);
```

The display task's frame delay was:

```
uint32_t until = timer_ticks() + 25u;
while (timer_ticks() < until) { task_yield(); }
```

`timer_ticks()` is a single load, so that loop runs in far fewer than 64 cycles. **Every pass pushed the comparator deadline further ahead of `CCOUNT`, so it was never reached and the timer interrupt stopped firing.**

That is not a slowdown. The tick drives every context switch in this kernel, so when it stops nothing else can run — and the task doing the yielding spins forever waiting for a clock it is itself preventing. A yield loop, the most innocuous-looking construct in the system, starved the mechanism that makes yielding mean anything.

The fix, still in `task.c`:

```
uint32_t soon = xt_ccount() + 64u;
if ((int32_t)(soon - xt_get_ccompare1()) < 0) {
    xt_set_ccompare1(soon);
}
```

A yield can bring preemption forward. It can never defer it. The signed comparison handles `CCOUNT` wrapping.

How it survived three commits

This is the part worth keeping.

The display driver was verified by **looking at the panel** and asking whether it looked right, with the serial capture filtered down to boot-time lines. Sustained operation — the check applied to every previous milestone, and the one that produced every number in reports 008 through 015 — was not run. On that basis three commits were reported as sound, including one that said in as many words "regression: all self-tests still pass".

And the instrument that existed, and was not read:

The status task draws a marker block that advances every frame, and UM-NATOS-015 §5 states its purpose explicitly: *"a display whose numbers all look plausible but never change is otherwise indistinguishable from a working one."* The instrument was built, documented, and then not read. **It was frozen the entire time.**

And the worst part:

the question asked to confirm the driver — whether the colour bars were in the right order — is one a **frozen screen answers identically to a live one**. A static image was treated as evidence of a running system.

Bisection then briefly implicated the display syscall work, which was innocent: reverting to the previously "known-good" commit reproduced the freeze identically, which is what showed the defect predated it.

The summary sentence:

The common thread is not carelessness about any one fact. It is reporting on whatever evidence was nearest rather than on the evidence that would settle the question.

The rule

A yield must never defer the clock it depends on.
More generally: any routine that adjusts a deadline the scheduler depends on must only ever move it earlier. Deferring it, in a loop, disables preemption entirely — and does so silently, since the symptom appears wherever the system happened to stop rather than at the line responsible.

And the observation that ties it to Chapter 8:

`_handler_level13`'s `PS.EXCM` rule is the other standing rule of this kind. Both share a shape: **a single unconditional register write, correct in isolation, catastrophic under repetition.**

Chapter 7 §7.9 is the third member of that family, from the other side: a write that was correct for its own purpose and invalidated somebody else's cached shadow of the same register.

17.10 Metrics

Quantity	Value
Syscalls	12
Opcodes spent on them	1
Application viewport	240×26 at M5; 180×14 now
Fills audited	136
Viewport escapes	0

Quantity	Value
Lowest row painted	250, of a 320-row panel
String buffer	48 B, per-byte bounds-checked
Touches delivered / withheld	81 / 109,211
Confinement failures	0
Commits the freeze survived	3

17.11 What this does not establish

- **No pixel-level drawing.** Rectangles, text and bitmaps only; no lines or circles.
- **No viewport negotiation.** Sizes are fixed by slot; an application cannot request more, and there is no way to grant a full-screen application anything.
- **No read-back.** A program cannot see what is on screen, including its own output.
- **No per-application draw accounting.** A program that draws constantly costs everyone's frame rate, and nothing measures or limits that per program.
- **The escape counter is not a proof of the clipping logic.**
- **No syscall for anything else.** No ADC, no I²C, no audio, no keypress, no persistence. Chapter 31.
- **Syscall validation is per-call, not systematic.** Nothing enforces that a future syscall will check its arguments, and there is no shared harness that would catch an unchecked length in a new service.

Part III ends here. The isolation claim is complete: an application is confined in memory, in pixels, in input and in messaging, and every one of those confinements has a counter attached that has been read on hardware.

Part IV is the hardware underneath it.

18. The Display: From 387 ms to 43 ms, and a Stall Still Open

Sources: docs/UM-NATOS-015-display.md Code: kernel/display.c, kernel/display.h, kernel/gpio.h

18.1 The claim

nat-os drives the CYD's ILI9341: 240×320, 16-bit colour, filled regions, text and bitmaps, from a native task showing live kernel state.

There is **no framebuffer anywhere in the system**. UM-NATOS-010 §7.2 argued that one should not exist — the panel holds the image in its own GRAM, so a host copy is a redundant 153,600 B against a 158,000 B heap. This report is that argument implemented: every pixel is rendered through a **480-byte** span buffer.

That claim held for the driver and was later relaxed for one specific consumer: the 3D view acquired an 80,640 B framebuffer, which was measured as worthless, defaulted off, and then — §18.7 — measured again and defaulted on. The general statement remains true: nothing composites a full screen in DRAM.

18.2 Pin map, read rather than remembered

Signal	GPIO	Note
SCLK	14	
MOSI	13	
MISO	12	unused — the driver never reads the panel
CS	15	
DC	2	low = command, high = data
BL	21	active high
RST	—	not wired to a GPIO ; follows the ESP32's own reset

Taken from the vendor project's active `TFT_eSPI/User_Setup.h` rather than from recollection. On a board where a wrong pin and a wrong init sequence produce an identical symptom, the difference between reading and remembering is the difference between a first-try success and an afternoon.

Because RST is not under software control, the only reset available after boot is the panel's `0x01 SWRESET`.

18.3 Bit-banged first, deliberately and temporarily

Hardware SPI2 requires DPORT peripheral clock-enable bits and GPIO matrix signal routing. The staging argument:

Every mistake in either produces **exactly the same symptom** as a wiring error, a wrong pin, or a bad init sequence: a black screen with no diagnostic. Bringing up four uncertain subsystems at once and then bisecting them from a single bit of output is how M2 consumed nine build cycles.

Bit-banging touches only GPIO registers, which `gpio.h` covers completely. A failure could therefore only be the panel, the pins, or the commands — three candidates instead of seven.

The bit-banged transport is eleven lines:

```
static void spi_write(uint8_t b)
{
    for (int i = 7; i >= 0; i--) {
        if (b & (1u << i)) {
            gpio_set(PIN_MOSI);
        } else {
            gpio_clear(PIN_MOSI);
        }
        gpio_set(PIN_SCLK);
        gpio_clear(PIN_SCLK);
    }
    g_bytes++;
}
```

with a comment that records a wrong estimate rather than deleting it:

```
* MEASURED: 153,600 bytes in 387 ms, about 397 kB/s, an effective clock near
* 3.2 MHz. The estimate written here first was ~150 ms – wrong by 2.6x, because
* a peripheral-register store costs rather more than the few cycles assumed.
* 2.6 full screens per second is ample for a status display and far too slow
* for animation; that is the number that decides whether SPI2 is worth bringing
* up, so it is measured rather than asserted.
```

The estimate being wrong by 2.6× is the point:

The figure is now measured at init and reported over UART, because throughput is the number that decides whether SPI2 is worth the risk, and an assertion would have decided it wrongly.

The measurement is taken during `display_init()` itself:

```
/* Measured rather than estimated: throughput is the first thing anyone
 * asks of a bit-banged driver, and it decides whether the SPI peripheral is
 * worth the risk of bringing up. */
uint32_t t0 = xt_ccount();
display_clear(COLOR_BLACK);
g_last_fill_cycles = xt_ccount() - t0;
```


18.4 Rendering without a framebuffer

```
#define LINE_MAX DISP_W /* one full-width span of pixels */
static uint16_t g_line[LINE_MAX]; /* 480 B – the whole "framebuffer" */
```

`display_fill_rect()` composes one row, sets the panel's address window once, then pushes that span `h` times:

```
void display_fill_rect(uint32_t x, uint32_t y, uint32_t w, uint32_t h, uint16_t
colour)
{
    if (x >= DISP_W || y >= DISP_H || w == 0u || h == 0u) {
        return;
    }
    draw_lock();
    if (x + w > DISP_W) { w = DISP_W - x; }
    if (y + h > DISP_H) { h = DISP_H - y; }

    for (uint32_t i = 0; i < w && i < LINE_MAX; i++) {
        g_line[i] = colour;
    }

    set_window(x, y, x + w - 1u, y + h - 1u);
    for (uint32_t row = 0; row < h; row++) {
        push_pixels(g_line, w); /* one span at a time – no framebuffer */
    }
    push_end();
    draw_unlock();
}
```

The panel's GRAM is the framebuffer. Setting a window and streaming into it is not a workaround for having too little RAM; it is how the device is designed to be driven, and the 153,600 B figure was only ever the cost of declining to use it.

The window protocol

```
/* Sets the rectangle subsequent pixel writes fill, then leaves the panel in
 * RAMWR with CS asserted so the caller can stream pixels. */
static void set_window(uint32_t x0, uint32_t y0, uint32_t x1, uint32_t y1)
{
    uint8_t col[4] = { (uint8_t)(x0 >> 8), (uint8_t)x0, (uint8_t)(x1 >> 8),
(uint8_t)x1 };
    uint8_t row[4] = { (uint8_t)(y0 >> 8), (uint8_t)y0, (uint8_t)(y1 >> 8),
(uint8_t)y1 };

    write_cmd_data(0x2A, col, 4); /* CASET – column address */
    write_cmd_data(0x2B, row, 4); /* PASET – page address */

    gpio_clear(PIN_DC);
    gpio_clear(PIN_CS);
    static const uint8_t ramwr = 0x2C;
    spi_tx(&ramwr, 1); /* RAMWR – memory write */
    gpio_set(PIN_DC); /* everything after is data; CS stays low */
}
}
```

CS stays asserted from the RAMWR until `push_end()` raises it. That shape — "a window command *and* its entire pixel stream under one CS" — is why the peripheral's CS automation is not used, and, much later, the mechanism behind the open fault in §18.9.

Byte order

```
/* Byte-swapped copy of the span. RGB565 goes out high byte first, the opposite
 * of how it sits in memory. Sent as one transfer: with a hardware FIFO the
 * per-call overhead dominates if this is done a byte at a time. */
static uint8_t g_txbuf[LINE_MAX * 2u];

static void push_pixels(const uint16_t *px, uint32_t n)
{
    for (uint32_t i = 0; i < n; i++) {
        g_txbuf[i * 2u]      = (uint8_t)(px[i] >> 8);
        g_txbuf[i * 2u + 1u] = (uint8_t)px[i];
    }
    spi_tx(g_txbuf, n * 2u);
}
```

The font is free

A 5×8 column-encoded font, 95 glyphs, 475 bytes, in `.rodata`:

```
/* ---- text -----
 * A 5x8 column-encoded font: each glyph is five bytes, each byte one column,
 * bit 0 at the top. A sixth blank column separates characters. Only 32..126 are
 * present; anything else prints as a space.
 *
 * It lives in .rodata, which since UM-NATOS-011 is mapped from flash and costs
 * no DRAM at all.
 */
static const uint8_t FONT5X8[95][5] = {
```

This is the first consumer of that work, and the reason it was scheduled before M4 rather than after.

Guarded by a recursive mutex

```
/* The span buffer and the panel's window state are both shared, so two tasks
 * drawing at once would interleave pixel streams into one window. Recursive,
 * because display_clear() draws through display_fill_rect(). */
static mutex_t g_lock;
```

Recursive matters here: `display_clear()` draws through `display_fill_rect()`, so a non-recursive lock would deadlock on the first clear.

Chapter 11 §11.3 records that recursion was chosen for a *different* reason — "a console lock is exactly the kind of thing that acquires a nested acquisition by accident" — and this is the case that would have found it immediately.

18.5 Hardware SPI2

The deferral paid off

§3 argues for bit-banging first because a wrong DPORT clock bit or a wrong IOMUX selection produces a black screen. ... That argument is what made this change cheap when it came. Panel, pins, command sequence and colour order were already known good, so the only thing under test was the transport. **A black screen could mean exactly one thing.**

What it took

The CYD's display pins are the ESP32's **native HSPI pads**, so IOMUX routes SCLK and MOSI straight to the peripheral — no GPIO matrix, and none of the 40 MHz ceiling the matrix imposes.

CS and DC stay ordinary GPIOs, because the driver holds CS asserted across a window command *and* its entire pixel stream, which is not a shape the peripheral's CS automation expresses.

```
static void spi2_init(void)
{
    GPIO_REG(DPORT_PERIP_CLK_EN) |= DPORT_SPI2_BIT;
    GPIO_REG(DPORT_PERIP_RST_EN) &= ~DPORT_SPI2_BIT;

    GPIO_REG(IO_MUX_GPIO14) = IO_MUX_HSPI_FUNC; /* SCLK */
    GPIO_REG(IO_MUX_GPIO13) = IO_MUX_HSPI_FUNC; /* MOSI */

    GPIO_REG(SPI2_SLAVE) = 0; /* master */
    GPIO_REG(SPI2_PIN) = 0x7u; /* peripheral drives no CS */
    GPIO_REG(SPI2_USER) = SPI_USR_MOSI_BIT; /* write phase only, mode 0 */
    GPIO_REG(SPI2_USER1) = 0;
    GPIO_REG(SPI2_USER2) = 0;
    GPIO_REG(SPI2_CTRL) = 0; /* MSB first */
    GPIO_REG(SPI2_CTRL2) = 0;
    GPIO_REG(SPI2_CLOCK) = SPI2_CLKDIV;

    /* Read back rather than assume. A clock register that did not take, or a
     * DPORT bit that did not stick, is the difference between a fast display
     * and a black one. */
    spi2_dma_init();

    g_spi2_clk_reg = GPIO_REG(SPI2_CLOCK);
    g_spi2_dport = GPIO_REG(DPORT_PERIP_CLK_EN);
}
```

Two details worth keeping:

The DPORT write is read-modify-write, never a plain store.

```
/* DPORT peripheral clock gating. SPI2 is bit 6. Bit 1 in the same register
 * clocks the flash controller this code executes from, so the write is
 * read-modify-write and never a plain store. */
```

A plain store to that register would stop the flash controller the executing code depends on.

Both configuration registers are read back, and reported at boot:

```
display : init ok, bytes=153634 fullscreen=387 ms clk=0x00001001 dport=0x...
```

`clk=0x00001001` confirms the `sysclk/2` divisor took; `dport` bit 6 confirms the peripheral clock gate stuck.

The bit-banged path is retained behind `DISPLAY_USE_SPI2 0`, and both backends route through a single egress point:

```
/* Single egress point. Both backends implement this and nothing else touches
 * the transport, so switching them cannot leave a stray path behind. */
static void spi_tx(const uint8_t *data, uint32_t n)
```

The result, and why the estimate was wrong

```
387 ms  ->  78 ms      full-screen fill
```

5×, against the ~12× the report predicted.

At 40 MHz the theoretical floor for 153,600 bytes is about 31 ms, so roughly 2.5× of what remains is per-transaction overhead. The FIFO holds 64 bytes, so a 480-byte span costs eight transactions — sixteen register writes, a start and a poll each — and a full screen is **2,560 of them**.

The original estimate treated the clock rate as the only variable. It is the transaction count that dominates, and that is precisely what DMA exists to remove. **The gap between the estimate and the measurement is more useful than either number alone**, which is why both are recorded.

The clock ceiling is a person looking at the screen

```
/* 80 MHz / 2. This is the ceiling on THIS board, established by trying the
 * next step up and looking at the panel.
 *
 * Full APB (SPI_CLK_EQU_SYSCLK, 0x80000000) works electrically – the driver
 * reports, the DMA completes, no timeouts, every self-test passes, and the
 * full-screen fill drops from 44 ms to 29 ms. It also puts visible noise on the
 * glass. Nothing in the kernel can see that: every counter says success while
 * the pixels are wrong.
 *
 * That is the whole reason the conservative divisor was taken first. The limit
 * here is the panel and the flex, not the controller, and the only instrument
 * that can measure it is a person looking at the screen. */
#define SPI2_CLKDIV      0x00001001u  /* pre=0 n=1 h=0 l=1 -> sysclk/2 */
```

That comment is the most quotable thing in the driver, and Chapter 28 is built on its observation. The clock was later made runtime-selectable for exactly this reason:

```

/* Runtime-selectable panel clock.
 *
 * The note above records that clocking this panel too fast puts visible noise
 * on the glass while every counter in the kernel reports success. That is
 * exactly the symptom being chased: identical code rendering cleanly at one
 * moment and torn the next, with corrupt=0, dma=N/0 and no timeouts. The limit
 * is the panel and the flex, and the only instrument that can measure it is a
 * person looking at the screen -- so it needs to be changeable while they do. */
uint32_t display_spi_clock_preset(uint32_t which)

```

18.6 DMA

The FIFO path costs 2,560 transactions per full screen. DMA takes a whole 480-byte span in one, so a screen becomes 320 transfers.

```

387 ms   bit-banged GPIO
 78 ms   SPI2, CPU-driven FIFO
 43 ms   SPI2 + DMA          dma=320/0
~31 ms   theoretical floor at 40 MHz

```

320 transfers and zero timeouts is exactly one per row, which confirms every span went through a descriptor rather than falling back.

Written so failure is diagnosable rather than fatal

A DMA engine that never asserts completion would hang the display task forever and take the system with it — the failure mode this project has already paid for twice, in the `task_yield` freeze and the zero-overhead `LOOP` defect, both of which presented as a stopped kernel.

So:

- every wait is bounded by `CCOUNT` and counted, never a bare `while`
- a timeout **disables DMA permanently** and falls through to the FIFO path
- the configuration registers are read back and reported beside the counters

```

/* Returns 1 if the transfer completed, 0 if it timed out. A timeout disables
 * DMA permanently rather than retrying: a DMA engine that missed one completion
 * has no reason to be trusted with the next, and the FIFO path still works. */
static int spi2_dma_tx(const uint8_t *data, uint32_t n)
{
    GPIO_REG(SPI2_DMA_INT_CLR) = 0xFFFFFFFFu;

    g_desc.flags = (n & 0xFFFFu)          /* size */
                  | ((n & 0xFFFFu) << 12) /* length */
                  | (1u << 30)           /* eof */
                  | (1u << 31);          /* owned by the DMA engine */
    g_desc.buf   = (uint32_t)data;
    g_desc.next  = 0;

    GPIO_REG(SPI2_DMA_OUT_LINK) = ((uint32_t)&g_desc & 0xFFFFFu) |
DMA_OUTLINK_START;

```

```

GPIO_REG(SPI2_MOSI_DLEN) = n * 8u - 1u;
GPIO_REG(SPI2_CMD)      = SPI_USR_BIT;
/* ... bounded wait ... */
}

```

Descriptors as words, and buffers in DRAM

```

/* Hardware descriptor. Written as explicit words rather than bitfields: the bit
 * order of a C bitfield is implementation-defined, and this layout is the
 * silicon's. */
typedef struct {
    uint32_t flags;      /* size:12 | length:12 | offset:5 | sosf:1 | eof:1 |
owner:1 */
    uint32_t buf;
    uint32_t next;
} dma_desc_t;

```

The descriptor and the transmit buffer are both in `.bss`, which the linker places in DRAM — a buffer in IRAM would be silently unreachable by the DMA engine.

That is a real hazard on this part and it is closed by construction rather than by convention: `g_desc` and `g_txbuf` are ordinary statics.

Small transfers stay on the CPU

```

/* Short transfers stay on the FIFO path: below roughly a FIFO's worth, the
 * descriptor setup costs more than it saves. Commands and their few
 * parameter bytes are all in that range. */
if (g_dma_ok && !g_panic_mode && n > 64u) {
    if (spi2_dma_tx(data, n)) {
        return;
    }
    /* Fell through: DMA timed out and is now disabled. The FIFO path
     * below still works, so the display degrades rather than stopping. */
}
spi2_tx(data, n);

```

18.7 Two claims that did not survive

The blitter drew row by row

Driving the display from a raycaster rather than a status screen put a very different load on this layer:

display_blit drew row by row. A 1-pixel-wide column therefore became 168 separate two-byte transfers. When the source is contiguous the whole rectangle is one linear stream, because the panel walks the address window itself.

```

if (src_stride == w) {
    /* Contiguous source: the whole rectangle is one linear stream, because
     * the panel walks the window itself. Sent in buffer-sized chunks rather
     * than row by row.
     *
     * This matters most where it looks least likely to. A 1-pixel-wide

```

```

    * column has 168 rows of one pixel each, so the row-by-row path issues
    * 168 two-byte transfers and a raycaster column costs ~170 ms. The same
    * data as one stream is a single transfer. */
    uint32_t total = w * h;
    while (total) {
        uint32_t chunk = (total > LINE_MAX) ? LINE_MAX : total;
        push_pixels(src, chunk);
        src += chunk;
        total -= chunk;
    }
} else {
    for (uint32_t row = 0; row < h; row++) {
        /* push_pixels byte-swaps into the transmit buffer, so the source is
        * never modified and need not be aligned to anything but a pixel. */
        push_pixels(src + (uint32_t)row * src_stride, w);
    }
}
}

```

The strided path remains for genuinely strided sources — which is what a *clipped* blit produces (Chapter 17 §17.6).

The lock, per column

```
blit time    25,075,460 us  ->  129,000 us      194x
```

That is UM-NATOS-014 §5.2 exactly — *a blocking mutex is the wrong instrument for a lock contended at high frequency* — **rediscovered by walking into it after writing it down.**

The framebuffer that did not pay, and then did

A framebuffer for the view region was added on the reasoning that 120 address windows per frame must dominate. It was made **switchable at runtime**, and the switch is what settled it:

```
fb off:  blit 126,650 us
fb on:   blit 131,411 us
```

No measurable difference. 80,640 B for nothing, so it is implemented, kept, and defaulted **off**.

The reasoning behind it was wrong twice over:

`xt_ccount()` deltas taken across preemptible code measure **wall clock**, including every moment the task spent descheduled — so the "450 us per window" that motivated the framebuffer was mostly other tasks running. Killing the applications moved the frame rate from 8 per 6 s to 19, a 2.4× change from freeing CPU alone, which located the real limit.

And then — revision 1.4 — the conclusion was overturned:

Superseded. The switch was rechecked and the conclusion did not survive. Both figures above were taken on a clock that intermittently stalled (Chapter 7 §7.9) and under draw-lock contention that dominated the frame (Chapter 11 §11.9). With both fixed the

framebuffer is worth having and is now the default. The paragraph above is left standing because the *method* — keep the switch, recheck the claim — is what produced the correction.

§5.7's *conclusion* was wrong; its *method* was right, and is why the error was findable — **a switch that can be flipped is a claim that can be rechecked.**

That is the most valuable single sentence in the display chapter, and it generalises: `spiclk`, `touchcfg`, `fb` and `dfreeze` all exist because of it.

Where the frame goes now

march	2.6 ms	one ray per screen column
compose	13.9 ms	240 x 168 pixels into DRAM
blit	41.9 ms	one 80,640-byte window ~9.1 fps

The frame is bus-bound: 72% of it is pushing pixels down SPI at 40 MHz, which this board's ceiling. So detail is cheap here and pixels are expensive, and that shapes what is worth adding.

Chapter 25 is what was added on the strength of that.

18.8 Verification

Visual confirmation plus two designed-in diagnostics.

The colour strip is an instrument, not decoration

Eight primaries in a known order make a pixel-format or byte-order fault immediately visible; a garbled or monochrome strip would have said which of the two was wrong, without inference from a photograph.

Colour order confirmed. Red renders leftmost, matching the intended RED, GREEN, BLUE, YELLOW, CYAN, MAGENTA, WHITE, GREY. That validates the BGR bit in `MADCTL 0x48`: had the panel's red and blue channels been transposed, the strip would have read blue-first and every colour in the system would have been mirrored while still looking entirely plausible in isolation. **Distinct primaries alone would not have caught it — only their ORDER does**, which is why the strip is drawn in a known sequence rather than as arbitrary swatches.

The init sequence carries the confirmation:

```
/* MADCTL 0x48: column order flipped, BGR panel. CONFIRMED on hardware by
 * the colour strip rendering red-leftmost. Had the BGR bit been wrong, red
 * and blue would be transposed system-wide and every screen would still
 * look entirely plausible in isolation — only the strip's known ORDER
 * catches it. */
static const uint8_t madctl[] = { 0x48 };
```


The advancing marker

The advancing marker block is the same idea applied to liveness: a display whose numbers all look plausible but never change is otherwise indistinguishable from a working one.

Chapter 17 §17.9 is the session where that instrument was built, documented, and then not read while it sat frozen.

The byte count that is exactly right

```
display : init ok, bytes=153634 fullscreen=387 ms
```

153,634 is $240 \times 320 \times 2$ for the initial clear plus 34 command bytes — the exact figure a correct full-screen fill produces, which is what made it usable as a check **before anything was known to be visible**.

That is the best kind of instrument: a number derivable in advance, so it can be checked on a dark screen.

18.9 The DMA stall, and three failed fixes

Status: unresolved and deliberately left alone. This section is the evidence, so the next attempt starts from it rather than from a fresh guess.

What happens

DMA disables itself within about eight seconds of the raycaster starting. After that every transfer takes the 64-byte FIFO path — 2,400 transactions per full screen instead of 320 — and the blit costs 55.9 ms instead of a possible ~22 ms. **The 3D view has run this way for its entire existence.** Nothing reported it; the fallback is silent by design.

The stall is spurious

```
stall after 63910 good transfers, asking 480 B
cmd      = 0x00000000      <- USR bit already CLEAR
dma_status = 0x00000000
after that : FINISHED late after 0 us
waited     = 30332 us
```

The transfer had completed. `cmd` reads zero, and the follow-up poll found it done in 0 μ s.

The mechanism is a race between two adjacent lines:

```
while (GPIO_REG(SPI2_CMD) & SPI_USR_BIT) {      /* A: still running? */
    if ((xt_ccount() - start) > 2000000u) {      /* B: too long?      */
```

A preemption landing between A and B times out a transfer that has already finished. `xt_ccount()` is wall-clock, so descheduled time counts against the bound. Rare per

transfer — 63,910 succeeded first — and certain over the ~2,900 waits a second the raycaster issues.

The shipped source now carries the full analysis, and it is the clearest statement anywhere of the wall-clock hazard:

```
/* Bounded wait, and the bound has to survive PREEMPTION.
 *
 * xt_ccount() is wall clock: it keeps running while this task is not. The
 * old bound was 2,000,000 cycles -- ~25 ms at 80 MHz -- justified as "far
 * beyond the ~100 us a 480-byte transfer needs", which is true of the
 * TRANSFER and false of the WAIT. One scheduling round trip is longer than
 * that, so a display task descheduled mid-transfer times out on a DMA
 * engine that is working perfectly.
 *
 * The consequence is not a dropped frame. A timeout disables DMA
 * PERMANENTLY and falls back to the FIFO path, so a single spurious trip
 * breaks rendering until reboot -- and it presents as the blit getting
 * FASTER (55.8 ms to 31.3 ms), because the fallback does different work.
 * A performance number improving was the symptom.
 *
 * This is the same class of error as the frame timings earlier in this
 * project: wall clock measured where work was meant. Raising the bound
 * rather than switching clocks keeps the guard intact -- a genuinely stuck
 * engine is still caught, it just costs half a second once instead of
 * 25 ms. task_cpu_cycles() would be the principled fix, but it only
 * advances at context switches and so cannot bound a spin inside one. */
```

"A performance number improving was the symptom" is worth reading twice.

The corruption is the recovery, not the timeout

`spi2_dma_tx()` returns 0 on timeout, and `spi_tx()` then falls through to `spi2_tx()` — **re-sending bytes the DMA has already sent**. The panel takes the span twice, its window advances one span too far, and every row after it shifts.

This is a latent bug in the shipped driver. It fires once per boot and is invisible, because the same timeout that causes it also disables DMA so it cannot happen again.

What was tried

attempt	result
Overlap conversion with DMA (two buffers)	no measured gain; broke the view once DMA actually ran
Bound by CPU time, reset and retry instead of disabling	blit 21.9 ms — and the view rendered wrong
Bound by CPU time, re-read USR, never re-send	view still wrong

Three attempts, three breakages. Each had a plausible mechanism and a good number **before anyone looked at the screen**, which is the whole lesson: this driver's only correctness instrument is a person looking at the panel.

And a separate finding about a change that was kept for the wrong reason:

The pipelined path deserves its own note. It measured as noise (55.9 → 55.5 ms) and was kept anyway as "correct and free". It had also **never actually run** — DMA disabled itself before it mattered — so it sat in the tree for a day looking like tested code. **An optimisation with no measured gain has no claim on the tree, and code that only executes when another bug is absent has been tested by nothing.**

If this is picked up again

- The 55.9 ms blit is correct. It is slow, and it works. That trade has already been made three times in the other direction and lost every time.
- Fix the duplicate re-send first. It is a real defect, it is independent of the timing question, and it makes any later timeout change safe rather than destructive.
- Do not remove the guard to study the failure. That was the first mistake.
- Verify on the glass before reporting a number.

18.10 `display_resync()`: a diagnostic for an open question

Chapter 27 §27.10 ends with a hypothesis with a mechanism, and this function is the instrument built to test it:

```
/* Re-establish the controller's window and end the transaction, WITHOUT
 * writing a single pixel.
 *
 * The ILI9341 holds a window set by CASET/PASET, and RAMWR starts a stream
 * that continues for as long as CS stays asserted. A stream that ends short,
 * or a CS left low, leaves the controller mid-window -- and the next pixels
 * sent land at whatever offset it had reached rather than at the top left.
 * That is what a garbled image is, and no amount of correct pixel data fixes
 * it, because the data is not what is wrong.
 *
 * This is the test for that: it issues CASET, PASET and RAMWR for the full
 * panel, then raises CS. Nothing is drawn. If a garbled view comes good after
 * calling it, the fault was the controller's idea of where pixels go, not the
 * pixels -- which would also explain why starting an application repairs the
 * view, since every draw it makes issues a fresh window of its own. */
void display_resync(void)
{
    draw_lock();
    set_window(0, 0, DISP_W - 1u, DISP_H - 1u);
    push_end();                /* CS high: ends the write cleanly */
    draw_unlock();
}
```

A five-line function that is a *falsifiable experiment*: if the view repairs, the hypothesis is right; if it does not, the remaining suspect is the SPI/DMA stream itself. That is the correct shape for the ninth theory about a fault, after eight have been eliminated by measurement.

18.11 Metrics

Quantity	Value
Resolution	240×320, RGB565
Framebuffer	none in the driver; 80,640 B optional for the 3D view
Span buffer	480 B
Font	475 B, in flash
Full-screen fill	43 ms via SPI2 + DMA (78 ms FIFO, 387 ms bit-banged)
SPI clock	40 MHz ($\text{sysclk}/2$); ~3.2 MHz bit-banged
Transfers per full screen	320 with DMA (2,560 on the 64-byte FIFO)
Theoretical floor at 40 MHz	~31 ms
Bytes for init + clear	153,634
Frame cost, march / compose / blit	2.6 / 13.9 / 41.9 ms
Share of frame spent on the bus	72%
SPI clock ceiling on this board	40 MHz (80 MHz is electrically fine and visibly noisy)
Blit improvement from one lock per frame	194×
Image size at this milestone	18,896 B

18.12 What this does not establish

- **No descriptor chaining.** Each span is a single descriptor started and waited on individually. Chaining whole frames would remove most of the gap to the 31 ms floor.
- **No 80 MHz.** One register bit, electrically fine, visibly noisy.
- **No read phase.** The driver is write-only; MISO is wired but unused.
- **No orientation control.** MADCTL fixed at 0x48.
- **No clipping of text.** A string running past the right edge stops at the panel boundary rather than wrapping.
- **No damage tracking.** Callers decide what to redraw; nothing computes dirty regions. This is the interesting gap: the frame cost is dominated by redrawing everything every frame, not by the transport.
- **No gamma correction.** Gamma tables left at panel defaults.
- **The DMA stall is open,** §18.9.
- **Nothing measures whether the shading or icons are legible.** Judged by one person on one panel.

Next: the input side, four attempts, and an axis that was backwards for three months behind a calibration that could only ever give one answer.

19. Touch: Four Attempts, and an Axis Inverted for Three Months

Sources: docs/UM-NATOS-017-touch.md Code: kernel/touch.c, kernel/touch.h, kernel/gpio.h, kernel/calib.c

19.1 The summary

The XPT2046 resistive touchscreen works: contact is detected through PENIRQ *and* pressure, and both axes are mapped to panel coordinates by measurement rather than by reasoning.

The driver took four attempts, and **only one of the four failures was in the driver**. Two were in `gpio.h`, and the fourth — the one worth reading this report for — was in **how the results were being read**.

And then, three months later, a fifth:

The horizontal axis had been **inverted since the driver was written**, and the calibration not only missed it but concluded the opposite — because it inferred direction from the last sample of a drag, which is the one sample guaranteed to be invalid.

19.2 Hardware

Signal	GPIO	Note
CLK	25	
MOSI	32	second GPIO bank
MISO	39	input-only pin, second bank
CS	33	second bank
IRQ (PENIRQ)	36	input-only pin, second bank

A separate bus from the display, and that matters:

The display driver leaves CS asserted between a window command and its pixel stream, so a shared bus would mean either interleaving touch traffic into an open window or serialising every touch read behind a 387 ms full-screen fill. Neither is acceptable for a pointer.

Bit-banged, for the same reason the display is: it needs only GPIO registers, so a failure can only be the panel, the pins, or the protocol.

`gpio.h` notes why MISO and IRQ are where they are:

```
/* Touch. GPIO 34-39 are INPUT ONLY on this part, which is why MISO and IRQ sit
 * on 39 and 36 – they can never be driven by mistake. */
```

19.3 Two defects in `gpio.h` that made a broken driver look plausible

`gpio.h` only ever handled pins 0–31

Pins 32–39 have entirely separate registers — `OUT1`, `ENABLE1`, `IN1` — and **nothing complains when the wrong bank is used**. A shift of 39 on a 32-bit register is undefined and reads zero in practice.

So MISO on GPIO 39 read a constant 0, and MOSI and CS on 32 and 33 were never driven at all.

The scar is recorded in the header:

```
/* TWO BANKS. Pins 0-31 and pins 32-39 have entirely separate registers, and
 * nothing complains if the wrong one is used – a shift of 39 on a 32-bit
 * register is undefined and in practice reads zero. That is indistinguishable
 * from a device wired correctly but not answering, which is exactly how the
 * touch controller first presented: MISO on GPIO 39 read a constant 0, and
 * MOSI and CS on 32 and 33 were never driven at all. */
#define GPIO_OUT_W1TS_REG    0x3FF44008u /* pins 0-31 */
#define GPIO_OUT_W1TC_REG    0x3FF440Cu
#define GPIO_OUT1_W1TS_REG   0x3FF44014u /* pins 32-39 */
#define GPIO_OUT1_W1TC_REG   0x3FF44018u
```

Every accessor now selects its bank from the pin number:

```
/* Every accessor below picks its bank from the pin number, so a caller never
 * has to know the split exists. */

static inline void gpio_set(uint32_t pin)
{
    if (pin < 32u) {
        GPIO_REG(GPIO_OUT_W1TS_REG) = 1u << pin;
    } else {
        GPIO_REG(GPIO_OUT1_W1TS_REG) = 1u << (pin - 32u);
    }
}
```

This bug was latent from the moment `gpio.h` was written for the display, whose pins are all below 32. It would have broken **any** peripheral above pin 31.

And the file's opening comment is candid about the claim it originally made:

```
* The claim that this file needed no pins above 31 was already false when it
* was written; the touch controller sits on 32, 33, 36 and 39, and the two-bank
* split below is the scar from discovering that.
```

The interrupt registers have the same split, and there it is *dangerous* rather than merely annoying:

```

* Two banks again, and here the split is dangerous rather than merely annoying:
* a status bit left set holds the shared source asserted, so failing to clear
* the right bank's bit hangs the machine in the handler instead of dropping an
* event.

```

Silence read as maximum pressure

Pressure is computed as $z = z1 + 4095 - z2$. A controller returning zeros on both channels yields **4095** — full scale.

A dead bus therefore presented as a permanent hard press, and the first run reported $s/e=10/10$: every sample a touch, on a screen nobody was touching.

An all-zero reading is now rejected outright:

```

/* Pressure from the two Z channels. A light touch gives a small z1 and a
 * large z2; the difference is what separates contact from noise.
 *
 * A controller that is not answering returns 0 on every channel, and that
 * formula turns 0,0 into 4095 — maximum pressure. Silence would read as a
 * permanent hard press, which is how a dead bus first presented here. An
 * all-zero reading is therefore rejected outright: no touch produces a
 * z1 of exactly zero. */
int answered = (z1 != 0u) || (z2 != 0u);
uint32_t z = answered ? ((z1 + 4095u) - z2) : 0u;

```

The general rule, which recurs in the I²C driver (Chapter 22 §22.7):

a formula whose failure mode is a plausible extreme is worse than one that fails to an obvious value, because the first requires interpretation and the second announces itself.

The first conversion is the one that matters

```

/* Control bytes. Bit 7 starts a conversion; bits 6:4 select the channel; bit 3
 * clear selects 12-bit; bit 2 clear selects differential mode, which cancels
 * the supply variation single-ended mode is sensitive to.
 *
 * Bits 1:0 are PD1:PD0, the power-down mode, and they were the defect. With
 * PD=00 the ADC and its reference power down between every conversion, and the
 * first conversion after power-up is specified as inaccurate. Z1 is the first
 * conversion after CS goes low on every single read, so the one channel that
 * decides whether a touch exists was always the throwaway sample: z1 never
 * exceeded 1 across 150 samples including a firm held press.
 *
 * PD=11 keeps the reference and ADC powered for the whole burst. The final
 * command uses PD=00 to power down again and re-enable PENIRQ, which is what
 * makes the IRQ line usable later. */
#define PD_ON      0x03u
#define CMD_Y      (0x90u | PD_ON)
#define CMD_X      (0xD0u | PD_ON)
#define CMD_Z1     (0xB0u | PD_ON)
#define CMD_Z2     (0xC0u | PD_ON)
#define CMD_IDLE   0x90u          /* PD=00: power down, PENIRQ back on */

```


The burst absorbs the inaccurate reading with an explicit throwaway:

```
/* Throwaway. Powers the reference up and absorbs the inaccurate first
 * conversion so it cannot land on a channel whose value is load-bearing. */
(void)convert(CMD_Z1);

uint32_t z1 = convert(CMD_Z1);
uint32_t z2 = convert(CMD_Z2);
```

and again per axis:

```
/* Four conversions per axis, kept individually. */
uint32_t sx[4], sy[4];
for (int i = 0; i < 4; i++) {
    sx[i] = convert(CMD_X);
}
for (int i = 0; i < 4; i++) {
    sy[i] = convert(CMD_Y);
}
(void)convert(CMD_IDLE);          /* power down, re-enable PENIRQ */
uint32_t rx = (sx[1] + sx[2] + sx[3]) / 3u; /* first discarded */
uint32_t ry = (sy[1] + sy[2] + sy[3]) / 3u;
```

Eleven conversions per read, of which two are deliberately discarded.

19.4 The capture was destroying its own data

This is the section the whole report is named for.

The symptom

Three consecutive runs, across different builds and different user actions, reported **exactly 150 samples**.

A counter that lands on an identical value every time is not accumulating. It is measuring a fixed window.

The cause

Opening a serial port asserts DTR/RTS, which is wired to the ESP32's reset and boot pins. Configuring them *after* opening still glitches EN. Every capture described as "no reset" was rebooting the board, waiting for it to boot, and then reading roughly twelve seconds of a system nobody was touching.

Setting `dtr` and `rts` **before** `open()` fixes it. The same read then reports 647,406 samples and no boot banner.

What it invalidated

Two statements were recorded as findings on data that had already been erased:

- *"pressure never exceeds 1 under a firm press"* — actual value **1123**
- *"PENIRQ never asserts across two full drags"* — actual count **17**

Both measured on a board that had rebooted seconds earlier. The PD=11 fix had already worked when it was declared a failure.

And the worst part:

the user reported that dots were following their finger, and that report was overridden as "almost certainly leftovers" on the strength of the broken measurement. **The human observation was the better evidence.**

Why it was hard to see

The reboot was invisible in every way that was being checked. Output looked normal, because a booting kernel prints a full banner and then healthy report lines. Values looked plausible, because they were real readings — of the wrong interval. **Only the sample counter carried the contradiction, and it was printed in the same line as the values that were being trusted.**

It happened again, with new tools

Investigating the inverted axis needed a way to read board state while a finger was on the glass. Two scripts were written for it, both described as "no reset", and **both reset the board.**

So the same failure this section documents was reproduced, by someone who had just read this section, in tools written specifically to avoid it. Two rounds of four-corner taps were destroyed between being recorded and being read.

The fix was known; what was missing was a *check*:

What is new is the verification — uptime is now sampled twice across a connection and confirmed to increase, rather than the absence of a reset being assumed. **Knowing the failure mode was not enough; a check was needed.**

19.5 The finding

Three times across two sessions, an instrument reported its own reading invalid and the value beside it was believed anyway:

Instrument	What it said	What was believed instead
Marker block	Frozen — the system is halted	The colour bars, which a frozen screen renders identically
s/e=150, three runs	This counter is not accumulating	The pressure values printed beside it
Boot banner in the capture	The board just restarted	The counters, as though continuous

The peripherals were largely fine. **The verification method was what kept failing,** and it failed in the same shape each time: a signal that the measurement was invalid, sitting next to a number that looked reasonable.

What broke the deadlock, every time, was the same move — **latch the quantity so timing cannot lie about it**, then feed the system a controlled input rather than interpret an uncontrolled one.

Latching is now a habit in this driver:

```
/* Extremes since boot. Confirming what the pressure channels do under a finger
 * needs a capture to coincide with the press, which is not something that can
 * be arranged reliably. Latching the extremes makes the question answerable at
 * any later moment instead. */
```

and in the press log:

```
/* Record the FIRST sample of each press. A held finger yields hundreds
 * of samples and only the moment of contact is a known screen
 * position; recording every sample would fill the log with the drift
 * of a finger settling. */
if (!g_was_down && g_log_count < TOUCH_LOG_MAX) {
    touch_log_t *e = &g_log[g_log_count++];
    e->raw_x = rx;
    e->raw_y = ry;
    e->x      = out->x;
    e->y      = out->y;
    e->z      = out->z;
}
```

Chapter 28 counts these alongside four more.

19.6 PENIRQ answers a different question than it was asked

The original reasoning, which was sound

```
irq=17    z1max=1123    z2min=2992    zmax=2065
```

PENIRQ asserted 17 times, matching the event count exactly, and computed pressure peaked at 2065 against a threshold of 300.

PENIRQ is used because it is a direct electrical indication from the panel, needing no ADC, no threshold and no calibration. It is read *before* CS is asserted, since the pin is only meaningful while the controller is idle.

Pressure was kept anyway:

A channel that never moves is itself evidence, and hiding it would only make the next person repeat the measurement.

And the conclusion was still wrong

The reasoning above is sound and the conclusion was still wrong, which is an uncomfortable combination worth stating precisely.

PENIRQ reports whether a finger is **there**. It says nothing about whether the position sample taken alongside it is **valid**. Those are different questions, and this section

answered the second by measuring the first.

Measured on the launcher, within a single tap:

```
z=7      raw_x=4095    -> x=0    approach, input floating
z=2025   raw_x=1280    -> x=167   the actual finger
z=20     raw_x=4095    -> x=0     release
```

A factor of a hundred separates the populations, and `Z_THRESHOLD` — 300, defined in `touch.c` and unused since it was written — sits cleanly between them.

And the consequence, invisible until something depended on horizontal position:

4095 is near the top of the range, so a rail reading maps to one edge of the screen.

Every spurious sample votes for the same place. In the launcher that place was the leftmost column, and it looked exactly like a broken axis rather than a valid-sample problem.

The fix is one expression, and the source records both halves of the reasoning:

```
/* PENIRQ says a finger is NEAR; pressure says it has actually landed.
 *
 * Both are now required, and the reason is not that PENIRQ was wrong.
 *
 * UM-NATOS-017 §4 chose PENIRQ over pressure with pressure working fine –
 * it measured a peak of 2065 against a threshold of 300 – on the grounds
 * that PENIRQ is a direct electrical signal needing no ADC, no threshold
 * and no calibration. That reasoning is still correct for the question it
 * answered.
 *
 * It answered the wrong question. PENIRQ reports whether a finger is
 * THERE; it says nothing about whether the position sample taken alongside
 * it is VALID. ...
 *
 * The rail maps to x=0, so every spurious sample votes for the leftmost
 * column – which is exactly what the launcher did once the axis inversion
 * stopped masking it. Two populations separated by a factor of a hundred,
 * and a threshold that has been sitting unused in this file the whole
 * time. */
out->down = pen && (z > Z_THRESHOLD);
```

A constant defined at the start of the project and first used in revision 1.2 — which means the *knowledge* was there and the *connection* was not.

19.7 Calibration by controlled input

Watching a trail follow a finger can say “*the dots go the wrong way*”. It cannot distinguish a **swapped** axis from a **flipped** one, and several build cycles were spent guessing between those two before that was noticed.

Instrumenting the raw span of each channel answers both at once. Two drags, each along one screen axis:

Drag	rx span	ry span
Left → right	486–3536 (3050)	2499–2862 (363)
Top → bottom	675–1518 (843)	432–3204 (2772)

The mirror image is the result. Each drag swept one channel across most of its range while the other barely moved, which is precisely what a correct axis assignment looks like.

The original code had the axes transposed, "on the reasoning that a portrait display must swap a landscape-calibrated panel. **That reasoning was wrong**, and no amount of looking at the screen would have said so."

19.8 The direction test was decided by its worst sample

The conclusion above about direction is wrong, and the method that produced it is why.

Raw X does not increase left to right. It decreases. Measured by tapping four labelled corners:

corner	raw_x	raw_y
top-left	3360	416
top-right	591	376
bottom-left	3258	3518
bottom-right	376	3462

Left reads ~3300, right reads ~480. The axis assignment is correct — `raw_x` really is horizontal — but its **sign** is backwards, and has been since the driver was written.

The span measurement was never the problem. The *inference* was:

"the horizontal drag ended at `rx=3527` against a maximum of 3536, so raw X increases left to right"

That reads the **last sample of a drag**, which is the release sample — and release samples read the ADC rail. A rail reading is 4095, near the top of any range, so a drag that ends anywhere at all "ends near its maximum", and **every axis appears to increase**. The test could only ever return one answer.

And:

The same corrupted sample later decided the launcher's icon selection, three months apart, in a different file. **One defect, two symptoms**.

The rule

Never infer direction from the endpoint of a gesture. A drag produces a cloud of samples whose first and last are the two least trustworthy, because both sit at a contact transition. Inferring direction from precisely those two is the method's flaw, not a slip in applying it.

Four **labelled** taps have no such moment. Each is a known screen position paired with a raw reading, and the direction of every axis falls out of comparing labels rather than trusting an endpoint.

The corner table is now recorded in `touch.c` *as the calibration*, rather than the two derived limits it produces, so a future re-calibration can check the derivation instead of repeating the guess.

Why nothing noticed for three months

A backwards horizontal axis had no consumer:

- the application strips are full width, so horizontal position never mattered
- the raycaster steers from the left and right thirds of the view, so a reversed axis makes it turn the other way — **which reads as a control preference**
- the containment tests check that a touch is confined to the right viewport, which is a question about `y`

The launcher is the first thing in this project that asks **where across the screen** a finger is. It failed on the first tap.

19.9 The corners were the wrong four points

The labelled-points method is right, and it still chose the four places on the panel where a labelled point is least trustworthy.

A finger cannot reach the extreme corner of a bezelled panel. Every corner reading was therefore short of the true extreme, the derived range came out narrower than reality, and — because the range is the *divisor* — every raw step counted for too many pixels. The mapping magnified position about the centre of the screen.

	corner-derived span	measured span	error
X	3000	3694	23%
Y	3140	3484	11%

The consequence is what made it hard to characterise:

true screen x	old mapping returned	error
30	~1	-29 px
120	~112	-8 px
210	~222	+12 px

The error is proportional to distance from centre **and changes sign**. UM-NATOS-022 §3.2 described this as "about 24 px low on X", which is a fair reading of one symptom at one place on the screen and is not what was wrong. **No fixed offset could have corrected a magnification**, which is why every attempt to nudge the constants moved the fault somewhere else rather than removing it.

Chapter 26 §26.4 records the note pad's own correction of that description, and it is the sharpest statement of the trap:

The number 24 was real. It was measured in one place, on keys near the middle, and then written down as though it were a property of the panel rather than of where it happened to be measured. **A single sample of a position-dependent quantity looks exactly like a constant.**

19.10 The fix: four inset targets

`kernel/calib.c` — four targets inset 30 px from the edges, tapped on the device.

```
/* nat-os – touch calibration by tapping targets. See calib.h.
 *
 * Four targets, INSET from the edges. That inset is the entire point.
 *
 * ... A finger cannot reach the extreme corner of a
 * bezelled panel, so every corner reading was short of the true extreme, the
 * derived range came out narrower than reality, and every mapped coordinate
 * landed inward of the finger – about 24 px on X by the time it reached the
 * middle of the screen. That was enough to make a 24 px keyboard key type its
 * neighbour.
 *
 * Targets at 30 px in are reachable, so the readings are real positions rather
 * than the closest a fingertip could get to an unreachable one. The mapping is
 * then interpolated outward to the edges instead of extrapolated inward from a
 * guess.
 */

#define INSET 30u
#define TARGETS 4u

/* Screen positions of the targets, in the order they are asked for. Two on each
 * side of both axes, so each axis is fitted from an average of two readings and
 * a single bad tap cannot define the whole scale. */
static const uint32_t TX[TARGETS] = { INSET, DISP_W - INSET, INSET,          DISP_W
- INSET };
static const uint32_t TY[TARGETS] = { INSET, INSET,          DISP_H - INSET, DISP_H
- INSET };
```

The pair check, which caught the first run

Two targets share each screen coordinate, so every reading has a partner it should nearly match:

```
/* The four pairs that must agree: two targets share a screen x, two share a
 * screen y, so each reading has a partner that should nearly match it. */
static const uint32_t PAIR_A[4] = { 0u, 1u, 0u, 2u };
static const uint32_t PAIR_B[4] = { 2u, 3u, 1u, 3u };
static const char *const PAIR_AXIS[4] = { "x", "x", "y", "y" };

/* How far two readings that should match may differ before the run is refused.
 * The usable raw span is about 3000 and a fingertip is worth a few hundred of
 * it, so 900 admits a sloppy tap and rejects a tap on the wrong cross. */
#define PAIR_TOLERANCE 900u
```

```

/* Check the pairs before fitting anything.
 *
 * Averaging two readings guards against one sloppy tap, but it also hides a
 * tap on the wrong cross: 3453 and 353 average to 1903, which looks like a
 * perfectly ordinary number and is a measurement of nothing. The averaged
 * fit then fails its own sanity check and reports "readings too close
 * together", which is true of the averages and says nothing about the
 * mistake that produced them.
 *
 * Two targets share each screen coordinate, so every reading has a partner
 * it should nearly match. Comparing them first turns a vague rejection into
 * a specific one. */
g_disagree = -1;
for (int p = 0; p < 4; p++) {
    const uint32_t *v = (p < 2) ? g_raw_x : g_raw_y;
    uint32_t a = v[PAIR_A[p]], b = v[PAIR_B[p]];
    uint32_t d = (a > b) ? a - b : b - a;
    if (d > PAIR_TOLERANCE) {
        g_disagree = p;
        break;
    }
}

```

The first run on hardware was refused by this check — two targets had been tapped on the wrong cross, and 3453 and 353 average to 1903, an entirely ordinary-looking number that measures nothing.

The rejection message is *specific*, which is the difference between a check and a complaint:

```

    uart_puts("REJECTED - targets ");
    uart_put_dec(PAIR_A[g_disagree] + 1u);
    uart_puts(" and ");
    uart_put_dec(PAIR_B[g_disagree] + 1u);
    uart_puts(" share a screen ");
    uart_puts(PAIR_AXIS[g_disagree]);
    uart_puts(" but read far apart.\n");
    uart_puts("                tap the cross that is SHOWING, one at a
time.\n");

```

The outcome is held, not just printed

```

/* The outcome of the last run, kept rather than only printed.
 *
 * The result line is emitted the instant the fourth target is tapped, which is
 * exactly when nobody has a capture attached – the same way the touch log's
 * values were lost before it was changed to hold them. A number that has
 * already scrolled past is not a measurement. */
static int    g_last_ok = -1;    /* -1 never run, 0 rejected, 1 installed */

```

That is §19.4's failure in a new costume — **the third time in this project a measurement was written into a stream nobody was reading** — so `calshow` reports the four readings, the outcome, and the calibration in use, and can be asked at any later time.

The result is read back, not assumed

```
touch_set_calibration(xmin, xmax, ymin, ymax);

/* Read back what was actually installed rather than what was computed:
 * touch_set_calibration() refuses a degenerate range, and reporting the
 * request instead of the result would hide that. */
uint32_t ax, bx, ay, by;
touch_get_calibration(&ax, &bx, &ay, &by);
/* ... */
uart_puts(ax == xmin ? "    (installed)\n" : "    (REFUSED, kept previous)\n");
```

And the refusal itself has a self-preservation argument:

```
void touch_set_calibration(uint32_t xmin, uint32_t xmax,
                          uint32_t ymin, uint32_t ymax)
{
    /* Refuse a degenerate or inverted range rather than installing it. A bad
     * calibration makes the panel unusable, which also makes it impossible to
     * run the calibration routine again – the failure would be
     * self-perpetuating. */
    if (xmax <= xmin + 100u || ymax <= ymin + 100u) {
        return;
    }
    /* ... */
}
```

Persisted and restored

```
touch cal    : restored x 141..3835 y 243..3727
```

Record version 3 (Chapter 20 §20.7), restored at boot before anything can be touched.

19.11 The mapping

```
static uint32_t map_axis(uint32_t raw, uint32_t lo, uint32_t hi, uint32_t span)
{
    if (raw <= lo) {
        return 0;
    }
    if (raw >= hi) {
        return span - 1u;
    }
    return ((raw - lo) * span) / (hi - lo);
}
```

```
        out->x = map_axis(rx, g_cal_x_min, g_cal_x_max, DISP_W);
#ifdef TOUCH_X_INVERTED
        out->x = (DISP_W - 1u) - out->x;
#endif
        out->y = map_axis(ry, g_cal_y_min, g_cal_y_max, DISP_H);
```

The calibration values are runtime, not compile-time:

```

/* Runtime, not compile-time, so a calibration routine can replace them and a
 * saved result can be restored at boot. The values below are the defaults from
 * the corner measurement described above – usable, and wrong by about 24 px on
 * X for the reason in the next paragraph. */
static uint32_t g_cal_x_min = 380u;      /* right edge – the LOW end of raw_x */
static uint32_t g_cal_x_max = 3380u;     /* left edge */
static uint32_t g_cal_y_min = 380u;     /* top */
static uint32_t g_cal_y_max = 3520u;     /* bottom */

```

Note the comments on `x_min` and `x_max`: *right edge* is the LOW end. The inversion is documented at the point where somebody would otherwise misread the names.

19.12 Input reaches applications

Covered in Chapter 17 §17.7. The measurement:

```
touch g/w = 81/109211      last = ...->191,174
```

Two defects it exposed are recorded there and in Chapter 16 §16.4: launching by index, and the kernel drawing a cursor over application viewports.

The second is worth repeating here because it is an ownership argument rather than a safety one:

A kernel-drawn touch cursor painted over whatever strip the finger was in — exactly what the kernel forbids applications from doing to each other. Not unsafe, since the kernel is trusted, but **wrong about ownership**: those pixels belong to whoever owns the strip.

The cursor is removed. An application that wants pointer feedback draws it itself, through `SYS_TOUCH` and `SYS_FILL`, in its own coordinates and inside its own viewport — which is the entire point of the syscall.

19.13 Metrics

Quantity	Value
Detection	PENIRQ (GPIO 36) and pressure
Poll rate	100 Hz, at HIGH priority, real 10 ms sleep
Conversions per read	11, including two throwaways
Peak pressure measured	2065 (threshold 300)
z1 under touch	1123, against 0 idle
z2 under touch	2992, against ~4090 idle
Raw span, horizontal drag	3050 counts
Raw span, vertical drag	2772 counts
raw_x at the left edge	~3300
raw_x at the right edge	~480

Quantity	Value
Direction of raw_x	decreases left → right
Rail samples per tap, before the pressure gate	~2 of 3
Pressure, real contact vs approach/release	~2000 vs ~10
Months the X axis was inverted	~3
Attempts before working	4
Conclusions retracted	2
Tools written to avoid §19.4's failure that reproduced it	2 of 2
Calibration error, corner-derived	23% on X, 11% on Y
Touch samples per reporter interval, before / after the sleep fix	~15 / ~150

19.14 What this does not establish

- **No multi-touch.** A resistive panel cannot report two points.
- **No IRQ-driven wakeup — but no longer for want of an interrupt.** PENIRQ is routed to CPU line 23 and its handler runs (Chapter 22). What has never been observed is a *finger* causing that to happen.

The gap moved from "not wired" to "wired and unexplained", which is progress of a sort and is not the same as working. - **No debounce or gesture recognition.** Every sample is independent. - **No pointer-up event.** An application cannot distinguish a held finger from a rapid sequence of contacts. - **One pointer, shared.** All applications read the same snapshot; nothing arbitrates focus. - **The calibration is this board's, but only this board's,** run once, with one finger. The vendor extremes remain the compiled-in defaults and are known to be wrong by 23% on X. - **The calibration is still linear.** Nothing measures panel non-linearity *between* the target positions. - **Nothing detects that a calibration has gone stale.** - **Z_THRESHOLD is a single number chosen from one measurement,** on this panel, on this day, with this finger. A lighter touch has not been tested. - **No filtering beyond the gate.** No median-of-N, no hysteresis. - **Nothing re-checks the axis at runtime.**

Next: state that survives a power cycle, and a read defect that pointed at the wrong register for a day.

20. Persistence: A Read Defect That Looked Like the Wrong Thing

Sources: docs/UM-NATOS-018-persistence.md Code: kernel/flash.c, kernel/flash.h, kernel/store.c, kernel/store.h, kernel/linker.ld

20.1 What was built

Component	Role
kernel/flash.c	SPI1 user-mode read, sector erase, page program
kernel/store.c	One checksummed record: magic, version, boots, frames, the last fault, and the touch calibration
kernel/linker.ld	.dram.rodata extended to flash.o and store.o

The mechanism is small. The report is mostly about a defect that sat in front of it for the whole of the work:

every flash read returned the true value shifted right by one bit, consistently, on every transaction. The chip ID read `0x34200B` where the part reports `0x684016`, which is exactly that value shifted once.

20.2 The record

```
#define STORE_MAGIC    0x59444F53u    /* "SODY" little-endian */
#define STORE_VERSION 3u    /* 3 adds the touch calibration */
```

Validated whole, on every load:

```
int store_load(void)
{
    store_t r;
    g_valid = 0;

    if (flash_read(FLASH_DATA_ADDR, &r, sizeof r) != 0) {
        goto defaults;
    }
    if (r.magic != STORE_MAGIC || r.version != STORE_VERSION) {
        goto defaults;    /* first run, or an erased sector reading 0xFF */
    }
    if (r.checksum != checksum(&r)) {
        goto defaults;    /* torn by a reset mid-write */
    }

    g_rec  = r;
    g_valid = 1;
    return 0;
}
```

```
defaults:
    /* ... reset every field ... */
    return -1;
}
```

A first run and a corrupt sector are deliberately indistinguishable — both reset to defaults — because **a partially-believed record is worse than no record**.

The checksum is deliberately trivial and says so:

```
/* Sum over every field but the checksum itself. Deliberately trivial: this
 * detects a torn or blank sector, which is what actually happens here. It is
 * not a defence against a determined corruption, and pretending otherwise would
 * be worse than the simple version. */
static uint32_t checksum(const store_t *r)
{
    return r->magic + r->version + r->boots + r->frames +
        r->fault_kind + r->fault_detail + r->fault_epc + r->fault_boot +
        r->cal_x_min + r->cal_x_max + r->cal_y_min + r->cal_y_max +
        0x9E3779B9u;
}
```

Naming what a mechanism does *not* defend against is a recurring habit in this codebase, and it is what stops a checksum being cited later as integrity protection.

20.3 A boot counter first, deliberately

```
* Persistence is unusually easy to *believe* you have: the value read back is
* the value just written, whether or not it ever reached the chip. A RAM
* variable passes that test perfectly.
*
* A counter that survives a power cycle cannot.
```

And the second field is stronger still:

`frames` accumulates raycaster frames **across** boots, so it can only be correct if the previous value was read back correctly *and added to*. A boot counter proves a write reached the chip; a running total proves the read path too. That distinction is what made the one-bit shift visible at all — the counter stuck at 1 was the symptom that started the investigation.

Two fields, three distinct claims, each with its own failure mode. That is the same design as the touch counters in Chapter 17 and the instruction/counter cross-check in Chapter 14.

20.4 Where it lives, and the constant that protects the bootloader

```
/* Bounded so a chip that never answers costs a delay rather than the system.
 * A sector erase is tens of milliseconds; this allows roughly a second. */
#define BUSY_TIMEOUT_CYCLES 80000000u
```

The data region starts at 2 MB, past the bootloader, the partition table and the kernel image. Both `flash_erase_sector()` and `flash_write()` refuse any address below `FLASH_DATA_ADDR`, and the reason is stated in terms of recovery cost:

This is not defence against a subtle bug. It is defence against the specific failure where a wrong constant erases the bootloader and the board stops enumerating over USB — which turns a five-minute fix into a recovery job. **Every failure in this driver stays recoverable over serial.**

20.5 The cache hazard, closed differently than planned

UM-NATOS-011 §4 predicted this would break first, and it did — on the first version of the driver, with a memorable presentation:

every string literal in the kernel began printing as `0xFF` immediately after the first flash operation. The console filled with `????????`.

But the cause was not the predicted one:

It was worse and simpler — **the driver reconfigured SPI1 and walked away**. SPI1 shares the flash bus with the cache's SPI0, and those registers are how the cache issues its own reads. Leaving them altered left the cache unable to read flash at all.

The fix is to save and restore all controller registers — `USER`, `USER1`, `USER2`, `CTRL`, `CTRL2`, later `CLOCK` — **unconditionally, including on the failure path**:

A driver that restores state only when it succeeds has simply moved the hazard to the error case, where it is harder to find.

Why the cache is never disabled

The planned mitigation was to disable the cache around flash writes, as ESP-IDF does. `nat-os` makes the dangerous reads impossible instead:

- all kernel code executes from IRAM, so instruction fetch never touches flash
- `flash.o`, `store.o`, `panic.o`, `uart.o` and `watchdog.o` have their `.rodata` placed in DRAM by the linker (Chapter 4 §4.5)
- interrupts are masked for the whole operation, so no handler can run and touch one either

A cache *hit* is harmless — it never reaches the chip. Only a miss would, and with no flash-mapped address referenced there is nothing to miss on.

The residual risk is stated plainly, and the mitigation is structural:

this argument depends on every function reachable from the driver keeping its read-only data out of flash. The linker rule selects **by object file** rather than by annotation precisely so that adding a string to one of these files cannot quietly break it. An annotation can be forgotten on a new string; a file-scoped rule cannot.

20.6 The read defect

The symptom

flash id	: 0x0034200b	expected 0x00684016	
store	: initialised, boot #1, frames 0, saved		
store	: initialised, boot #1, frames 0, saved		← after a reset

0x684016 >> 1 == 0x34200B. Every read, every time, shifted right one bit with a zero entering at the top.

Why it pointed at the command phase

A leading zero followed by the real data means the first sample was taken before the chip drove anything. Two readings:

1. **framing** — the command phase is one clock short, so data sampling begins a clock early
2. **timing** — sampling is misaligned against the data by a clock

Reading 1 was pursued first, and the reason is the transferable part:

the shift was *perfectly consistent*. Timing faults are expected to be marginal — to vary with temperature, or produce occasional rather than uniform corruption. A defect that reproduces bit-exactly on every transaction reads as structural.

That instinct is wrong here. A sampling point misplaced by a *whole clock* is not marginal at all; it is as deterministic as a framing error and looks identical from the outside.

Consistency does not distinguish framing from timing. It only rules out marginality.

The measurement that eliminated a class

Rather than settle the encoding question, the command and address were moved out of `USR_COMMAND` / `USR_ADDR` entirely and sent as ordinary MOSI bytes:

A SPI NOR part cannot tell the difference; the phase split is a convenience of this controller, not something on the wire.

The result was **byte-for-byte identical**: 0x0034200b.

Two structurally different transactions producing the same shift is strong evidence, and it is the single most useful measurement in this report. It did not identify the cause, but it eliminated an entire class — the extra bit arrives regardless of how the command is framed, so no command-phase register can be responsible.

The measurement that settled it

With framing eliminated, the remaining candidates were the sampling edge and the clock. Both are register fields with several plausible values, and testing them one at a time costs a build-and-flash cycle per hypothesis.

Instead, all of them were tested in a single boot — four clock dividers against four sampling-edge combinations, with the known-good answer printed alongside so the right row identifies itself:

```
flash probe : want 0x00684016
clk=0x00000000 edge=0x00000000 -> 0x0034200b
clk=0x00000000 edge=0x00000040 -> 0x0034200b
clk=0x00000000 edge=0x00000080 -> 0x00b4200b
clk=0x00000000 edge=0x000000c0 -> 0x00b4200b
clk=0x001c9109 edge=0x00000000 -> 0x00684016 <== MATCH
clk=0x001c9109 edge=0x00000040 -> 0x00684016 <== MATCH
...
clk=0x0009109 edge=0x000000c0 -> 0x00684016 <== MATCH
```

`clk=0` means *inherited*. The reading is unambiguous: **every explicit divider reads correctly at every edge setting; the inherited one fails at all of them**. Sixteen measurements, one flash cycle, no interpretation required.

This is the same technique as `touchcfg` and `spiclk` in Chapter 27 §27.9, and it is named there as a method note: *make the variable runtime-tunable before forming a theory about it*.

The fix, with the whole history preserved in the source:

```
/* ~8 MHz for user commands, explicitly set rather than inherited.
 *
 * This was the whole of the read defect. The bootloader leaves SPI1's divider
 * configured for its own fast cache reads, and at that rate a user transaction
 * sampled MISO one clock early: every byte came back as the true value shifted
 * right once, with a spurious leading zero. RDID read 0x34200B where the part
 * reports 0x684016.
 *
 * Two things made this hard to see. The shift was perfectly consistent, so it
 * looked like a framing error in the command phase rather than a timing one –
 * and moving the command out of USR_COMMAND into the MOSI stream changed
 * nothing at all, which is what finally ruled that out. A sweep across four
 * dividers and four sampling edges settled it in one boot: every explicit
 * divider read correctly at every edge, and the inherited one failed at all of
 * them.
 *
 * These operations are rare and small, so there is nothing to be gained by
 * [running the bus near its limit] ... */
```

There is a second inherited-state hazard in the same file, and it is documented beside the first:

```
/* Fast-read mode bits in SPI_CTRL. The bootloader leaves the controller in dual
 * or quad IO for cache reads; a user command issued in that mode goes out on
 * more than one line and the chip sees nonsense. */
#define CTRL_FASTRD_MODE (1u << 13)
#define CTRL_FREAD_DUAL (1u << 14)
/* ... */
```


Two independent ways for inherited controller state to corrupt a user transaction, on one peripheral. The general lesson is in Chapter 29: **inherited configuration is not neutral configuration**.

20.7 Two hypotheses tested against stale firmware

This is the part of the investigation that cost the most and taught the most.

`build.ps1` builds; `build.ps1 -Flash` builds *and* flashes. For a stretch of the investigation the build step ran alone, and the results were read off a board still running older firmware. The invocation used was `flash.ps1` — a script that **does not exist**. PowerShell's error went to a stream that was being filtered out of the captured output.

Two conclusions were drawn from those runs:

Hypothesis	Verdict recorded	Actually
SPI_CTRL fast-read mode bits	no change	genuinely no change
SPI_CTRL2 MISO sampling delay	disproven	untested — never ran

The CTRL2 result was retested properly once the flash path was fixed. It *is* disproven — the conclusion was right. But it had been **right by accident**, and a comment in `flash.c` had already been written attributing the shift to CTRL2 as established fact. That comment was corrected.

The cost was not the wrong conclusion:

during the stale window, **every hypothesis returned the same answer** — the board could not have responded differently, because it was running the same firmware throughout. A sequence of identical negative results was read as "none of these are the cause" when it actually meant "no experiment has run yet".

The general form, and the standing rule

This is the third time in this project that an instrument reported its own reading invalid and the reading beside it was believed anyway.

The distinguishing signature is the same each time: **a run of results that do not vary when the input does**. Three different register configurations returning bit-identical output is not evidence about registers. It is evidence that the configuration is not reaching the device.

Standing rule. A negative result is only informative if the experiment demonstrably ran. When consecutive hypotheses produce identical output, suspect the harness before the theory — and verify the change reached the target, rather than inferring it from the absence of an error message.

The mitigation is deliberately unglamorous:

the flash step now always shows its `Hash of data verified.` line, and that line is checked before any capture is interpreted.

20.8 Verification

Persistence verified across hard resets, with the frame count accumulating:

```
--- run ~130 s so two saves land ---
store : loaded, boot #14, frames 256, saved
--- reset 1 ---
store : loaded, boot #15, frames 512, saved
--- reset 2 ---
store : loaded, boot #16, frames 512, saved
flash id : 0x00684016
```

Three things asserted, each with a distinct failure mode:

- **boots increments on every reset** — writes reach the chip
- **frames grew 256 → 512 across reset 1** — the previous value was read back correctly and added to; this cannot pass on a RAM variable or a broken read
- **frames stayed 512 across reset 2** — the 6-second window did not reach the 256-frame save point, so nothing was written. A counter that advanced here would indicate a write not tied to the work it claims to measure

That third assertion is the subtle one and it is the best-designed part of the test: a *negative* result that would have been positive if the save were firing spuriously.

The chip ID now matches what `esptool` independently reports for the part — an external oracle rather than an internal one, which is the pattern Chapter 27 §27.9 names as a method note.

20.9 The save interval was set by measurement

The periodic save initially fired every 4096 frames, then 512, on an assumed ~8 fps. Neither ever fired inside a test window.

The renderer actually runs at **4.4 fps** with the framebuffer on — 81 frames in 18.3 seconds, read from the existing telemetry. At 512 frames that is a save every two minutes, which is why a 75-second verification run saw nothing and was briefly mistaken for a broken save path.

The general point:

an interval nobody exercises is an interval nobody has tested. A save path that fires every eight minutes would have shipped unverified, and its first real execution would have been in the field.

And an honest note that the constant has since drifted from its justification:

The save cadence figure also moved. 256 frames was ≈ 60 s at the 4.4 fps measured here; after the lock contention fix the renderer runs several times faster, so the same 256 frames is now well under half that. The constant was chosen from a measured rate, and the rate changed underneath it — the flash sees more erases per hour than this report's wear analysis assumed.

20.10 The record has grown twice

Revision 1.0 described a four-field record of 20 B. It is now **13 words, 52 B, at record version 3**:

words	added by
magic, version, boots, frames	this chapter
fault kind, detail, EPC, boot	Chapter 12 §12.7 — the panic recorded to flash
cal x-min/max, y-min/max	Chapter 19 §19.10 — the on-device touch calibration
checksum	this chapter

Nothing broke as it grew, because the version field does exactly what it is for: an older record fails validation and resets to defaults rather than being read with the wrong layout.

With a caveat:

no upgrade path was ever written — a version bump silently discards the boot counter, the frame total and the calibration, which is why the panel had to be recalibrated after the field was added.

One writer, by construction

The calibration is persisted through a function that lives in `store.c` rather than in `calib.c`, and the reason is a discipline argument:

```
/* Declared in calib.h. Writing the record here keeps every write to it in one
 * file, which is what makes "the record is only changed in store.c" a property
 * rather than a habit. */
void calib_persist(uint32_t xmin, uint32_t xmax, uint32_t ymin, uint32_t ymax)
{
    g_rec.cal_x_min = xmin;
    /* ... */
    (void)store_save();
}
```

Same for the boot counter:

```
/* Bumped by kmain once the record has been loaded. Kept here so the only code
 * that writes the counter is the code that owns it. */
void store_count_boot(void) { g_rec.boots++; }
```

This is Chapter 7 §7.9's standing rule applied prospectively: **one writer, or every writer maintains the shadow**. Here, one writer, enforced by where the function lives.

20.11 Load order in `kmain`

```
/* Persistence. Loaded before the scheduler starts so the boot count is
 * settled before anything can race it, and saved immediately so a power
 * cut a second later still records that this boot happened. */
```

```

extern void store_count_boot(void);
#if FLASH_ENABLE
    uart_puts(" flash id    : ");
    uart_put_hex(flash_read_id());
    uart_puts("\n store      : ");
    int found = (store_load() == 0);
    store_count_boot();
    int saved = (store_save() == 0);
    /* ... */

```

The chip ID is printed first, every boot. If the read path ever regresses, the first line says so before any record is interpreted.

Then the calibration is restored before anything can be touched, and the last-fault line is printed (Chapter 12 §12.7).

20.12 Metrics

Quantity	Value
Record size	52 B, record version 3
Sector used	4,096 B at 0x200000
Flash user clock	~8 MHz (80 MHz / 1 / 10)
Bootloader clock (inherited)	fast cache-read divider — unusable for user commands
Max transaction	64 B; page program capped at 60 B + 4 B header
Save cadence	every 256 frames
Erase cost	tens of ms, interrupts masked
Registers saved/restored per transaction	6
Boots survived in test	16
Hypotheses tested against stale firmware	2
Chip ID	0x684016, matching esptool

20.13 What this does not establish

- **No wear levelling.** One sector, erased and rewritten in place.
- **No power-cut testing.** The checksum is *designed* to reject a torn record; no test has interrupted a write mid-erase to confirm the rejection path executes. "The mechanism is reasoned, not measured."
- **No application access.** Persistence is kernel-only. No syscall exposes it.
- **No filesystem.** One fixed-layout record, not named storage.
- **No verification against a second board.** The clock finding is from one part. A different flash chip may tolerate the inherited divider, "which would make this

defect appear board-specific to anyone who hits it".

- **The cache argument remains an argument.** §20.5 reasons that no flash-mapped read can occur during an operation. That has not been instrumented — there is no assertion that would fire if a future edit broke it.

Next: the storage a person can carry away, and a pad table that is not in pin order.

21. microSD, and a Pad Table That Is Not in Pin Order

Sources: docs/UM-NATOS-020-sdcard.md Code: kernel/sd.c, kernel/sd.h, kernel/gpio.h

21.1 The first storage a person can carry away

Flash holds what the kernel was built with; this holds what the user brought.

The driver was straightforward. One constant was not, and it is the reason this chapter exists:

the ESP32's IO_MUX pad table is **not in pin order**, and the two UART0 pads sit between GPIO22 and GPIO23. Counting up from a known entry put GPIO23 at the address of U0RXD, so initialising the SD bus reconfigured the console's receive pin as a GPIO output. Transmit kept working, so the board went on printing telemetry and simply stopped answering.

21.2 Why SPI mode

The card supports a 4-bit parallel protocol several times faster. This driver uses 1-bit SPI mode, for three reasons stated in `sd.h`:

- * - it is the mode every card must support, so a card that fails here is
- * faulty rather than unsupported
- * - the CYD wires the slot to ordinary GPIOs, not to the ESP32's dedicated
- * SDMMC pins, so the fast path is not available on this board anyway
- * - this project has now brought up three SPI devices, and reusing a shape
- * that is understood beats learning a protocol whose failures are new

Bit-banged first, again

- * As with the display (UM-NATOS-015 §3), the first version bit-bangs. Card
- * initialisation is a one-time cost measured in milliseconds and is required to
- * run at 400 kHz or slower, which no hardware peripheral makes easier. Sector
- * reads can move to SPI3 once something is proven to be reading the right
- * bytes; moving first would mean debugging a protocol and a peripheral at the
- * same time, which is how the display cost three commits to one defect.

Third application of the same staging argument (Chapter 18 §18.3, Chapter 22 §22.7), and by this point it is a stated project convention rather than a case-by-case judgement.

Pins, and a caveat attached

- * Taken from the ESP32-2432S028R board design, which puts the slot on the VSPI
- * default pads. These are ASSUMED until a card answers CMD0 – a wrong pin map

```

* and an absent card are the same silence, so sd_init() reports which stage
* failed rather than a single boolean.
*/

#define SD_PIN_CS      5u
#define SD_PIN_SCK     18u
#define SD_PIN_MISO    19u
#define SD_PIN_MOSI    23u

```

"These are ASSUMED until a card answers CMD0" is exactly the kind of labelling Chapter 0b describes: a transcribed value marked as transcribed, with the condition that would upgrade it to measured.

21.3 One error code per stage

```

/* Distinct codes per failure stage. "No card" and "wrong wiring" and "card
 * present but refuses the voltage range" are three different problems, and a
 * driver that returns -1 for all of them makes the user guess. */
typedef enum {
    SD_OK                = 0,
    SD_ERR_IDLE          = -1, /* no answer to CMD0 – absent card, or bad wiring */
    SD_ERR_IFCOND        = -2, /* CMD8 rejected – pre-2.0 card, or a bad bus */
    SD_ERR_READY         = -3, /* ACMD41 never completed initialisation */
    SD_ERR_OCR           = -4, /* CMD58 failed, so addressing mode is unknown */
    SD_ERR_BLOCKLEN      = -5, /* CMD16 refused on a byte-addressed card */
    SD_ERR_READ          = -6, /* CMD17 refused */
    SD_ERR_TOKEN         = -7, /* card never sent a data token */
} sd_err_t;

```

Seven codes for a driver whose whole job is "read a block". Each one names a distinct thing to check next.

Every wait is bounded, and the reason is not defensive habit

Every wait is bounded, and that is not defensive habit — the slot is normally *empty*. A probe that hangs on an absent card would be worse than no driver at all, because the normal case would be the fatal one.

Verified:

```

sd_init FAILED type=none last R1=0x000000ff
stage: CMD0 - no card, or MISO/CS/SCK wrong
reporter lines after a failed probe: 4      <- kernel still running

```

That last line is the assertion. A failed probe that leaves the kernel running is the requirement; a driver that reports the failure and then wedges would have passed a naive reading of the first two lines.

R1 = 0xFF is unambiguous: bit 7 of a real R1 is always zero, so "no answer" cannot be confused with any legal response.

Another instance of *choose a sentinel that cannot be a legal value* — the same reasoning as the heap's magic words and `MUTEX_FREE == -2`.

`kmain` probes at boot and treats failure as non-fatal, with the reasoning recorded:

```
/* Probe the card at boot, so storage is ready before anything asks for it
 * and so an absent card is reported once rather than discovered later by
 * whatever needed it. A failure here is not fatal: the slot is normally
 * empty, and every wait inside sd_init() is bounded for exactly that
 * reason. */
uart_puts(" sd          : ");
{
    int sd_rc = sd_init();
    if (sd_rc == SD_OK) {
        uart_puts(sd_type() == SD_TYPE_SDHC ? "SDHC ready" : "SDSC ready");
    } else if (sd_rc == SD_ERR_IDLE) {
        uart_puts("no card");
    } else {
        uart_puts("present but init failed at stage ");
        uart_put_dec((unsigned int)(-sd_rc));
    }
    uart_puts("\n");
}
```

21.4 The pad table, and a check that could not fail

The table

`gpio_out_init()` takes a pin number *and* its IO_MUX register address, because the table is not in pin order:

```
/* IO_MUX pad registers are not in pin order – the table is the silicon's, not
 * ours, so it is written out rather than computed. Only the pins used here. */
#define IO_MUX_GPIO2      0x3FF49040u
#define IO_MUX_GPIO12     0x3FF49034u
#define IO_MUX_GPIO13     0x3FF49038u
#define IO_MUX_GPIO14     0x3FF49030u
#define IO_MUX_GPIO15     0x3FF4903Cu
#define IO_MUX_GPIO21     0x3FF4907Cu
```

The relevant stretch of the real table:

0x7C GPIO21	0x80 GPIO22	0x84 U0RXD(GPIO3)	0x88 U0TXD(GPIO1)	0x8C GPIO23
-------------	-------------	-------------------	-------------------	-------------

Counting up from GPIO21 while forgetting the two UART pads puts GPIO23 at 0x84, which is the console's **receive** pad. Configuring it as a GPIO output kills serial input.

The symptom named the fault

Transmit is a different pad (U0TXD at 0x88, untouched), so the board kept printing its telemetry at full rate and simply stopped responding to anything typed. A shell that echoes nothing looks like a hung shell; the reporter output proved the kernel was fine, which narrowed it to the input path immediately.

That asymmetry was luck. Had the mistake landed on 0x88 instead, the board would have gone silent and looked like a boot failure.

The verification that agreed with the wrong answer

This is the most transferable part of the chapter.

The pin map was checked before flashing, against the four IO_MUX entries already in `gpio.h`:

entry	offset
GPI02	0x40
GPI012	0x34
GPI014	0x30
GPI021	0x7C

All four confirmed the indexing. **All four are below the UART pads**, so every one of them agreed with the wrong answer. The check was real, it was performed, and **it could not have failed**.

A cross-check only tests what its samples can distinguish. Four entries that all sit on the same side of the anomaly confirm the rule and say nothing about the exception. The samples must **straddle** the thing being verified.

The report explicitly links it to its sibling:

This is the same shape as UM-NATOS-017 §7.1, where a calibration inferred axis direction from a sample that could only ever give one answer.

Both are cases where a test was *run, passed*, and was *incapable of failing*. Chapter 28 groups them with a third — the audio self-test that ran on the one pin structurally unable to exhibit the fault.

The positive form of the rule is visible in the VM's bounds cross-check (Chapter 14 §14.5), where the 35 cases are chosen to include *the exact end, one past the end, and lengths chosen to wrap the address space* — samples that straddle the boundary rather than agreeing with it.

21.5 Verification: what block 0 does and does not prove

```
sdread 0 -> FA 33 C0 8E D0 BC 00 7C 8B F4 50 07 50 1F FB FC
             BF 00 06 B9 00 01 F2 A5 EA 1D 06 00 00 BE BE 07
             signature=0x55 0xAA
             partition 0: type=0x06 start=240 sectors=490000
```

FA 33 C0 8E D0 BC 00 7C is the textbook MBR bootstrap — `cli, xor ax, ax, mov ss, ax, mov sp, 0x7C00` — and BE BE 07 points at the partition table. Real data, correctly framed.

And then the caveat, which is the reason the section exists:

It proves nothing about the **addressing mode**, and that is worth stating because it looks like a complete result. Block 0 is byte 0 on a byte-addressed card and block 0 on a block-

addressed one. The read succeeds either way and discriminates nothing.

The read that does discriminate

```
sdread 240 -> signature=0x55 0xAA
              fs type: FAT16
```

Those five characters require the pin map, the clock, the addressing mode and the block framing to all be correct **simultaneously**. A wrong addressing mode would land 512 times too far into the card and return something that is not a filesystem header.

A single read at a non-zero LBA is worth more than any number of reads at zero, because it is the first one whose *result depends on* the property under test. That is the same principle as the straddling samples in §21.4, applied to a different question.

And a note on luck:

The card is ~250 MB and genuinely **SDSC**, which was luck: it exercised the byte-addressing path that a modern card would have skipped entirely.

The corresponding gap is recorded: SDHC is untested, the **CCS** bit is read and the branch exists, and the path a modern card would take has never executed.

21.6 Metrics

Quantity	Value
Mode	SPI, 1-bit, bit-banged
Identification clock	~250 kHz (spec requires ≤400 kHz)
Error codes	7, one per stage
Unbounded waits	0
Card under test	~250 MB, SDSC, FAT16
Partition found	type 0x06, start LBA 240, 490,000 sectors
IO_MUX entries cross-checked	4
IO_MUX entries that could have caught the fault	0

That last row is the chapter in one line.

21.7 What this does not establish

- **No write path.** `sd_read_block()` only.
- **No filesystem.** The MBR is decoded far enough to find a partition and confirm a FAT signature. There is no directory walk, no file lookup, no FAT chain traversal. This is the largest missing piece in the storage story and the reason "install an application" still has no meaning (Chapter 16 §16.9).
- **No card-removal detection.** Pull the card mid-read and the result is a token timeout rather than a clean "card gone" error. Nothing polls for insertion.

- **SDHC is untested.**
 - **Still slow.** Bit-banged at every stage including bulk reads. Moving to SPI3 is deliberate future work, "but nothing here measures what the current throughput actually is".
 - **The pin map is confirmed only for this board.**
-

Next: three peripherals brought up in one session, and four distinct ways to deliver an interrupt to something that could not act on it.

22. The Interrupt Matrix, the ADC, and I²C

Sources: docs/UM-NATOS-023-interrupts.md, docs/UM-NATOS-024-adc.md, docs/UM-NATOS-025-i2c.md Code: kernel/intr.c, kernel/intr.h, kernel/adc.c, kernel/i2c.c, kernel/gpio.h

These three drivers were built in one session and belong in one chapter, because they share a theme that the display and touch chapters only touched: **a register that reads back correctly is not evidence that anything happened.** Between them they produced three distinct instances of it, and Chapter 23 adds a fourth.

Part A — The Interrupt Matrix

22.1 Everything was polled, and nothing said so

Until this work, **every peripheral in this kernel was polled**, and nothing said so. The Level-3 vector served exactly one source — CCOMPARE1, which is internal to the Xtensa core and reaches the CPU without passing through any routing at all.

That is a hard wall for anything serviced on the hardware's schedule rather than ours.

22.2 Sources are not lines

```
* On the ESP32 a peripheral cannot reach the CPU by itself. Each of ~70
* peripheral interrupt SOURCES is routed, through a DPORT map register, onto
* one of 32 CPU interrupt LINES. Those are different numbering spaces and
* confusing them is the classic failure: writing the source number into
* INTENABLE enables an unrelated line, and the peripheral stays silent while
* every register involved reads back exactly as intended.
*
*   source  (0..69)  what raised it      - GPIO, I2S0, UART1, TG0_T0 ...
*   line    (0..31)  what the CPU sees  - fixed priority level, fixed type
*
* A line's priority level and edge/level type are properties of the silicon,
* not choices. Line 23 is used here because it is level-triggered at level 3,
* so it shares the existing Level-3 vector and its proven context save rather
* than needing a second one.
*/
```

Reusing the existing vector is the same economy as choosing CCOMPARE1 in Chapter 7: fewer new things to be wrong about.

The map register address is computed, and the derivation is checked against two independently known points:

```
/* The PRO CPU's map registers are one word per source, in the silicon's source
* order, starting at the MAC source. The address is therefore computed rather
* than tabulated - but the INDEX still has to be right, which is why intr.h
* names the sources instead of letting callers pass a number.
*
* Checked against two independently known registers: TG0_T0 is source 14 and
* maps to 0x3FF0013C, UART0 is source 34 and maps to 0x3FF0018C. Both fall out
* of this base. */
#define DPORT_PRO_MAP_BASE 0x3FF00104u
#define DPORT_PRO_MAP(src) (DPORT_PRO_MAP_BASE + 4u * (src))
```

Two samples, on either side of the sources this kernel uses — which is the straddling discipline of Chapter 21 §21.4, applied correctly this time.

22.3 The vector could only ever have been right by accident

`_handler_level3` called `timer_isr` unconditionally.

Correct while CCOMPARE1 was the only source that could reach level 3, and a **clock corruption** the moment a second one is routed: `timer_isr()` advances the tick deadline by a whole interval every time it runs, so any GPIO interrupt would silently push the clock into the future.

That is the same defect as the `task_yield()` bug, arrived at from the opposite direction, and it would have been just as quiet — sleeps running long, timeouts running loose, and every frame rate measured against a clock that drifts.

The fix is one line of assembly — `call0 intr_dispatch` instead of `call0 timer_isr` — and the vector records why:

```
/* Service whichever level-3 sources are actually pending.
 *
 * This called timer_isr directly until the interrupt matrix existed, which
 * was correct while CCOMPARE1 was the only source that could reach level 3.
 * It stops being correct the moment a second one is routed: timer_isr
 * advances the tick deadline by a whole interval every time it runs, so a
 * GPIO interrupt arriving here would silently push the clock forward — the
 * same corruption as the yield bug in timer.c, from the other direction.
 *
 * intr_dispatch reads the INTERRUPT register and dispatches on fact. */
call0 intr_dispatch
```

"Dispatches on fact" rather than on an assumption about who could have fired.

22.4 The defence against a hang

```
void intr_dispatch(void)
{
    uint32_t pending = xt_get_interrupt() & xt_get_intenable();

    /* The tick first, and by name. It is not a matrix source — CCOMPARE1 is
     * internal to the core — so it has no handler in the table. */
    if (pending & (1u << INTR_LINE_TIMER1)) {
        g_count[INTR_LINE_TIMER1]++;
        timer_isr();
        pending &= ~(1u << INTR_LINE_TIMER1);
    }

    while (pending) {
        uint32_t line = 31u - (uint32_t)__builtin_clz(pending);
        pending &= ~(1u << line);

        if (g_handler[line]) {
            g_count[line]++;
            g_handler[line]();
            continue;
        }

        /* An enabled, pending, unhandled LEVEL-triggered line is a hang, not a
         * glitch: nothing clears the condition, so returning re-enters this
         * handler immediately and the machine never runs task code again. There
         * is no output from that state and no watchdog reach — the watchdog is
         * fed by a task, and tasks have stopped running.
         */
    }
}
```

```

    *
    * So the line is masked instead. The kernel loses that interrupt and
    * keeps running, which is recoverable and inspectable ('intr' in the
    * shell reports it) rather than a silent brick. */
    g_spurious++;
    g_disabled |= (1u << line);
    xt_disable_interrupt(line);
}
}

```

Note the masking with `INTENABLE` — the warning attached to `xt_get_interrupt()` in Chapter 7 §7.6 is honoured here.

And routing installs the handler *first*:

```

/* Handler first. The matrix write can make the line fire immediately if the
 * peripheral is already asserting, and a line that fires before its handler
 * exists is a spurious interrupt that the defence in intr_dispatch() would
 * then permanently disable – the routing would appear to have failed. */
g_handler[line] = fn;

DPORT_REG(DPORT_PRO_MAP(source)) = line;
xt_enable_interrupt(line);

```

A two-line ordering that prevents the safety mechanism from sabotaging the thing it protects.

22.5 Four ways to deliver an edge to nobody

The theme: at every stage, an edge was generated correctly, routed correctly, and consumed by something that could not act on it. **No register ever read back wrong.** Only counters distinguished these.

1. Delivered to a halted CPU

`GPIO_PINn_INT_ENA` is a 5-bit field at 17:13 selecting which CPU and which kind of interrupt receives the pin. Bit 13 was chosen as the low bit of the field, matching the vendor header's `PRO_CPU_INTR_ENA` being `BIT(0)`.

Real taps latched `GPIO_STATUS1` bit 4. The map register read 23. `INTENABLE` had bit 23. The CPU's `INTERRUPT` register never showed it.

The pin was being delivered to the **APP CPU, which this kernel never starts** — arriving perfectly at a core that was halted.

The answer was *measured*, not read:

```

void intr_selftest(void)
{
    uint32_t saved = DPORT_REG(GPIO_PIN36_REG);
    int found = -1;

    uart_puts("    injecting an edge for each INT_ENA bit:\n");

    for (uint32_t bit = 13u; bit <= 17u; bit++) {

```

```

DPORT_REG(GPIO_PIN36_REG)    = 0u;                /* disable */
DPORT_REG(GPIO_STATUS1_W1TC_A) = 1u << IRQ_PIN_BIT; /* clear stale */

uint32_t before = g_count[INTR_LINE_GPIO];

DPORT_REG(GPIO_PIN36_REG)    = (2u << 7) | (1u << bit);
DPORT_REG(GPIO_STATUS1_W1TS_A) = 1u << IRQ_PIN_BIT; /* inject */

for (volatile int i = 0; i < 2000; i++) {
}

uint32_t after = g_count[INTR_LINE_GPIO];

uart_puts("    bit ");
uart_put_dec(bit);
uart_puts(after != before ? " -> SERVICED\n" : " -> nothing\n");
if (after != before && found < 0) {
    found = (int)bit;
}
/* ... restore ... */
}

if (found >= 0) {
    uart_puts("    PRO CPU enable is bit ");
    uart_put_dec((uint32_t)found);
    uart_puts("; restoring with it\n");
    DPORT_REG(GPIO_PIN36_REG) = (2u << 7) | (1u << (uint32_t)found);
}
/* ... */
}

```

The measured answer is bit 15. `intr_selftest()` settles it by injecting an edge through `GPIO_STATUS1_W1TS` once per candidate bit and reporting which the handler actually saw. Running `irqtest` re-derives the constant on any board rather than trusting the comment that records it.

`gpio.h` carries the constant with the whole story attached:

```

/* GPIO_PINn_REG: INT_TYPE is bits 9:7, and INT_ENA is a 5-bit field at 17:13
 * selecting which CPU and which kind of interrupt the pin is delivered to.
 *
 * Bit 15 is THIS CPU's ordinary interrupt, and that number is measured rather
 * than read. The obvious choice – bit 13, the low bit of the field, matching
 * the vendor header's PRO_CPU_INTR_ENA being BIT(0) – produced a pin that
 * latched its edge in GPIO_STATUS1 and never reached the processor. It was
 * being delivered to the APP CPU, which this kernel never starts, so the edge
 * arrived correctly at a core that was halted.
 *
 * That failure is worth naming because every register involved read back
 * exactly as intended. */
#define GPIO_PIN_INT_ENA_PRO (1u << 15)

```

Software injection mattered for a second reason:

it isolates the status→CPU path from the pad→status path. The alternative was asking someone to tap a panel five times while watching a serial log.

2. The driver interrupting itself

`touch.c` already knew that "a conversion in progress drives PENIRQ regardless of whether anything is touching". That is as true of the *interrupt* as of the *level*, and the driver had not been read that way.

Every `touch_read()` manufactured its own edges — **3,917 across a handful of taps, none caused by a finger**. Each one set the "edge arrived with nobody waiting" latch, so the idle wait returned immediately, forever. The driver had interrupted itself into a busy loop.

Fixed by masking the pin for the duration of the burst:

```
/* Mask PENIRQ for the duration of the burst.
 *
 * The comment above says a conversion drives this pin regardless of whether
 * anything is touching, and that is exactly as true of the INTERRUPT as of
 * the level. Every read therefore manufactured its own edges: 3900 of them
 * across a handful of taps, none caused by a finger, each one indexing the
 * "an edge arrived with nobody waiting" latch and making the idle wait
 * return immediately forever. The driver was interrupting itself into a
 * busy loop.
 *
 * Masking here means the only edges that survive are the ones a finger
 * caused. An edge genuinely missed during the burst costs nothing: the burst
 * only runs while the task is awake and already sampling. */
gpio_int_disable(PIN_IRQ);
```

The knowledge was already in the file. What was missing was reading it as a statement about the *interrupt* rather than about the *level*.

3. Re-arming against a pin a finger is holding down

The mask was then released at the end of `touch_read()`, which looked right and was the same bug moved. The pad is still **LOW** at that point if a finger is on it, and arming falling-edge detection against an already-low pin latches an edge immediately.

Result: 100 edges, 0 wakes — every one self-inflicted, every one while the task was awake with no waiter registered.

```
/* Deliberately does NOT re-arm. Arming happens in touch_irq_wait().
 *
 * Re-arming here looked right and was the second version of this bug. The
 * pad is still held LOW by a finger at this point, and enabling falling-edge
 * detection against an already-low pin latches an edge immediately – so
 * every read produced an interrupt, always while the task was awake and had
 * no waiter registered. The counters said 100 edges and 0 wakes, which is
 * exactly what a self-inflicted edge looks like from the outside.
 *
 * The interrupt is only ever useful while the task is idle and the pen is
 * up. Arming it anywhere else can only manufacture events. */
```

The generalisation — *arm an edge detector only in the state where an edge is meaningful* — is now enforced by having exactly one arming site.

`gpio_int_enable()` clears stale status first, for the third variant of the same hazard:

```
/* Enables interrupts on a pin. ...
 *
 * Clears any stale status first. A pin that was already asserting before its
 * enable bit was set has a status bit set, and enabling into that state fires
 * the handler for an edge that happened before anyone was listening. */
static inline void gpio_int_enable(uint32_t pin, uint32_t type)
{
    gpio_int_clear(pin);
    GPIO_REG(GPIO_PIN_REG(pin)) = GPIO_PIN_INT_TYPE(type) | GPIO_PIN_INT_ENA_PRO;
}
```

4. A diagnostic that was itself wrong

The first register dump read `0x3FF44078` for the PRO CPU's masked status and reported a confident zero. The per-CPU status registers sit between `STATUS1` and `PIN0` in an order taken from memory, and that address is most likely the *other* CPU's copy.

Printing the whole range and letting the value identify the register found bit 4 set at `0x3FF44074` — which is what pointed at §1.

A diagnostic derived from the same faulty recollection as the code it is checking will agree with the code and both will be wrong. Dump the range, not the address you believe in.

The dump now does exactly that:

```
/* The per-CPU masked status registers sit between STATUS1 and PIN0, and
 * their exact order is the thing in question — the first attempt at this
 * dump read 0x78 and reported a confident zero from what is probably the
 * APP CPU's copy. Printing the range and letting the value identify the
 * register is the same move as the calibration's pair check: do not ask the
 * hardware to confirm a guess, ask it what is there.
 *
 * GPIO 36 is bank-1 bit 4, so the register showing 0x10 is the one that
 * matters, and whether it shows it at all is the actual finding. */
for (uint32_t a = 0x3FF44064u; a <= 0x3FF44084u; a += 4u) {
    uart_puts("    ");
    uart_put_hex(a);
    uart_puts("] = ");
    uart_put_hex(DPORT_REG(a));
    uart_puts("\n");
}
uart_puts("    expect: one of the above = 0x10 if the pin reaches the PRO
CPU\n");
```

Printing the *expectation* alongside the data is what makes the output self-interpreting.

22.6 What is verified, and what is switched off

<code>irqtest</code>	-> bit 15 SERVICED	injected edge reaches the handler
<code>intr</code>	-> tick line 15 serviced N	dispatch does not disturb the clock
	spurious 0	nothing masked defensively

```
armed rb 0x00008100
```

```
INT_TYPE=falling + enable, read back  
from inside the armed window
```

That last line is a methodological point:

sampling `GPIO_PIN36` from the shell returned zero three times running against a window covering 93% of the cycle. That is not chance — **the shell's sampling is correlated with the touch task being awake**. Only the armed task could answer without the measurement being correlated with its own subject.

An observer whose scheduling is coupled to the thing it observes cannot sample it.

And a finger has never once woken the touch task

With everything above confirmed, real taps produced 24 touch events and 30 low PENIRQ readings while the armed edge detector latched nothing. **That gap is unexplained.**

The touch task is therefore back to polling — byte for byte its previous behaviour — and `touch_irq_wait()` is intact, one line from reinstatement.

Shipping touch input on an interrupt whose end-to-end path has never been observed to work would trade a mechanism that demonstrably works for one that only should.

The remaining suspects, all untested:

- the pad→status edge path may need `FUN_IE` or a GPIO-matrix input selection that plain reads do not require — *`gpio_read()` working does not prove the edge detector is fed*
- the arm/disarm boundary may be clearing a finger's edge
- GPIO 34–39 are input-only RTC pads and may route their edges differently

22.7 Why the matrix was built first, and whether that was right

The alternative was ADC — smaller, and with an onboard LDR to prove it. The matrix was chosen because it **cannot be bolted on later without revisiting every driver**, and because PENIRQ offered a real, low-rate, forgiving consumer already on the board.

That reasoning half survives. The infrastructure is real and the next peripheral inherits it. But the "forgiving consumer" turned out to be the hardest part, and an ADC would have proved the same matrix against something with no edge semantics to get wrong.

Part B — The ADC

22.8 A driver whose output cannot be checked for plausibility

Every driver before this one talked to a device that answers in protocol. A display either shows the pattern or it does not; a flash read either returns the magic number or it does not; a touch controller either responds on the bus or is silent. Each of those has an internal check.

This one returns a number meaning "how much light", and nothing in the system can check it for plausibility. 619 is a perfectly good ADC reading. So is 351. So is the same value on all eight channels, which is what a converter that is not converting anything returns.

That is what shapes this report: the driver is ordinary, and the verification is the work.

22.9 ADC1, and a constraint that became an advantage

ADC2 is unusable on this part whenever WiFi is running, which is the standard reason to avoid it. **That reason does not apply here.** The `-mabi=call0` build cannot link Espressif's precompiled radio libraries at all, so WiFi will never run and ADC2 is permanently free.

That reasoning was correct when written and was overtaken two days later by Chapter 27. ADC2 remains untouched.

ADC1 is still first "because the board's own light sensor is on it and a sensor already soldered down needs no wiring to be wrong about".

The file opens by declaring its own risk:

```
* Register offsets below are the risky part of this file. The last driver this
* kernel gained lost an afternoon to a five-bit field whose order was recalled
* rather than measured (UM-NATOS-023 §5.1), so adc_dump() exists from the first
* commit rather than being added when something goes wrong, and the shell reads
* every channel rather than the one that is supposed to be interesting.
```

Building the diagnostic *first*, on the strength of yesterday's lesson. It still was not enough — §22.10.

22.10 The defect was one bit

`SENS_SAR1_EN_PAD_FORCE` is `BIT(31)`. It was written as bit 27, from memory.

Bit 27 is not a spare bit. `SAR1_EN_PAD` is **twelve** bits at shift 19, so 27 sits inside it: setting it selected a channel that does not exist, and left pad selection with the hardware FSM. The FSM then measured whatever it liked, identically, forever.

The symptom was a complete set of reassurances:

observation	what it seemed to prove
all 8 channels $\approx$ 619	plausible mid-scale readings
DONE asserted every time	conversions completing
0 timeouts	the converter is alive
every register read back as written	the configuration is correct

A wrong bit in the right register is invisible to a read-back. That is the one failure mode the read-back discipline cannot catch, and it is worth naming because that discipline had been working well enough to feel sufficient.

The constant now carries the whole story:

```
/* BIT(31), and this was the whole defect.
 *
 * It was written as bit 27 from memory. Bit 27 is not a separate field:
SAR1_EN_PAD
 * is TWELVE bits at shift 19, so 27 is inside it, and setting it selected a
 * channel that does not exist while leaving pad selection with the FSM. The FSM
 * then measured whatever it liked, identically, forever -- which is why all eight
 * channels read ~619 and no pad mux bit changed anything.
 *
 * Every other offset and field in this file was recalled correctly. This one was
 * not, and nothing in the read-back could show it, because a wrong bit in a
 * correct register reads back exactly as written. The value came from the
 * vendor's sens_reg.h, fetched rather than remembered. */
#define SAR1_EN_PAD_FORCE (1u << 31)
```

"Every other offset and field in this file was recalled correctly" is the uncomfortable part: recall has a high hit rate, which is exactly what makes it dangerous.

There is a second trap in the same driver, closed in advance:

```
/* DATA_INV is not optional and is the classic way this comes out looking
 * broken-but-alive: without it every reading is its own complement, so a
 * dark sensor reads high and covering it makes the number go up. */
```

22.11 Three probes, none of which found it

Three probes were built before the answer arrived, and each is kept because each answers a question that would otherwise be asked again:

- **adconv** — proves a conversion *actually runs*. **DONE** goes low when **START** is cleared and high 13 spins later. This killed the plausible theory that the poll was returning stale latched data.
- **adcdri** — sweeps against **GPIO32**, a pin this kernel drives. Every earlier probe depended on **GPIO36** idling high, which is a belief about the board; this commands the input voltage instead. It also settled **DATA_INV** empirically: driving the pin high *raises* the reading, so the inversion as configured is correct.
- **ldrscan** — watches all eight channels at once.

None of them found it. What found it was **fetching the vendor's `sens_reg.h` instead of recalling it**, at which point every other offset and field in the file turned out to be correct. It was one line.

The lesson is a taxonomy of tools:

This was the third register-layout error in a day and the first fixed by reading the definitions rather than probing around them. **Probing is the right tool when the question is *what does this hardware do*. It is the wrong tool when the question is *what is this constant*, and those are easy to confuse when both present as "it does not work".**

`adcdribe` is worth noting separately: it is the only probe that *commands* the input rather than observing it. Chapter 19 §19.5 identified that move as the one that broke every previous deadlock — "feed the system a controlled input rather than interpret an uncontrolled one".

22.12 The diagnostic that hid the answer

The RTC IO dump skipped registers reading zero, to make the block's shape legible. The two registers that mattered — the pad muxes at `+0x7c` and `+0x80` — **were zero precisely because they were the thing not yet configured.**

The filter removed exactly the registers the dump existed to find. A bit sweep then ran against `+0x00`, which is `RTC_GPIO_OUT` and governs nothing here, and reported thirty-two failures that meant nothing at all.

A diagnostic that suppresses the uninteresting cannot find something whose signature is *being uninteresting*.

It prints zeros now. And a nice recovery of the information the filter was introduced to provide:

The block's shape was recoverable anyway: the ten-register run at `+0x94..+0xb8` can only be the ten touch pads, which fixes every other offset in the block by counting.

22.13 Verification, with a control group

A hand passed over the board, all eight channels watched at once:

ch	gpio	min	max	spread	
0	36	176	188	12	(touch pin)
1	37	552	560	8	
2	38	0	0	0	
3	39	226	240	14	(touch pin)
4	32	0	0	0	(touch pin)
5	33	1777	1824	47	(touch pin)
6	34	273	538	265	<- the sensor
7	35	621	625	4	

265 counts against a control group that never exceeded 47. The four touch pins are kept in the scan for exactly this reason: **they are pins that should not respond to light**,

| so they measure the noise floor while the experiment runs.

A control group measured *simultaneously with* the experiment, on the same hardware, in the same conditions. That is the strongest form of this project's "one number that must move, one that must not" idiom.

And a claim settled rather than repeated:

| "The LDR is on GPIO34" was an assertion about the board, propagated through comments; `ldrscan` tested it. It happened to be right. **The point is that it is no longer an assumption.**

Part C — I²C

22.14 Two pins, most sensors

The ADC turned one pin into one sensor. This turns two pins into **most** sensors — temperature, humidity, pressure, accelerometers, magnetometers, real-time clocks, small displays — and they share the bus, so the pin cost does not grow with the sensor count. On a board with three spare pins that matters more than it would elsewhere.

Signal	GPIO	Why
SDA	22	brought out to a header, unclaimed
SCL	27	brought out to a header, unclaimed

The pin budget, which is the reason there are only two:

Everything else on this board is spoken for: display (2, 12–15, 21), touch (25, 32, 33, 36, 39), SD (5, 18, 19, 23), light sensor (34), speaker (26), RGB LED (4, 16, 17).

22.15 Bit-banged, and why that is *safe* rather than merely consistent

This part has two hardware I²C controllers. Neither is used.

The weak reason is consistency: the display and touch drivers are both bit-banged, so the technique is proven in this kernel and its failure modes are understood.

The real reason is that **clock stretching is in the I²C specification**. A slave may hold SCL low to buy itself time, and every master is required to tolerate it. A bus that is legal to stall is therefore a bus on which a *preempted master* is legal too — a task switch in the middle of a transaction stretches the clock rather than corrupting the transfer.

That is a guarantee from the protocol, not a consequence of this code being careful.

This is a genuinely elegant argument and it is the only place in the project where a *protocol property* substitutes for a *kernel mechanism*. A bit-banged SPI transaction preempted mid-byte would be fine too (SPI has no timing floor), but a bit-banged I²S or UART would not be — which is why Chapter 23 puts audio behind hardware PWM rather than bit-banging a waveform.

The property lives in one function:

```
/* Releases SCL and waits for it to actually be high.
 *
 * This is the clock-stretch handler, and it is the reason a preempted
 * transaction survives: the master does not assume the clock rose because it
 * let go of it. A slave holding SCL down is legal and expected. A line that
 * never rises is a stuck bus or a missing pull-up, and that is a timeout rather
 * than an infinite wait inside a shell command. */
static int scl_release_and_wait(void)
{
```



```

line_release(I2C_PIN_SCL);
for (uint32_t spin = 0; spin < 100000u; spin++) {
    if (line_read(I2C_PIN_SCL)) {
        half_bit();
        return I2C_OK;
    }
}
return I2C_ETIMEOUT;
}

```

22.16 Open drain, emulated

```

/* Open drain, emulated.
 *
 * A line is pulled low by ENABLING the output driver (which is already set to
 * output zero) and released by DISABLING it, letting the pull-up do the rest.
 * Never driven high: two masters, or a slave mid-ACK, would then be shorted
 * against each other rather than merely contending. */
static inline void line_low(uint32_t pin)
{
    if (pin < 32u) {
        GPIO_REG(GPIO_ENABLE_W1TS_REG) = 1u << pin;
    } else {
        GPIO_REG(GPIO_ENABLE1_W1TS_REG) = 1u << (pin - 32u);
    }
}

static inline void line_release(uint32_t pin)
{
    if (pin < 32u) {
        GPIO_REG(GPIO_ENABLE_W1TC_REG) = 1u << pin;
    } else {
        GPIO_REG(GPIO_ENABLE1_W1TC_REG) = 1u << (pin - 32u);
    }
}

```

The output register is written once at init and never again; only the *enable* is toggled. Nothing is ever driven high.

FUN_IE, and the third instance of "silence reads as data"

```

/* FUN_IE because the master must READ these lines – for the ACK bit, for
 * incoming data, and for clock stretching. An open-drain pin whose input
 * buffer is off reads a constant zero, which would look like every device
 * ACKing everything. */
GPIO_REG(mux_reg) = MCU_SEL_GPIO | FUN_IE | FUN_PU;

```

Same class as the touch controller reading silence as maximum pressure. Third appearance of the pattern in this book, and the general rule from Chapter 19 §19.3 applies unchanged.

The constants are fetched, not recalled, with an explicit reference to yesterday:

```

/* IO_MUX pad registers, and the pull-up bit. Both taken from the vendor's
 * io_mux_reg.h rather than recalled – FUN_PU is BIT(8), FUN_IE is BIT(9), and

```

```
* an afternoon went into learning that this file's kind of constant is not
* something to remember (UM-NATOS-023 §5.1, UM-NATOS-024 §4). */
```

22.17 The pull-up compromise

The internal pull-ups are used: roughly **45 kΩ**, against the 2.2–10 kΩ a real bus wants.

This is a stated compromise, not a design. Rise times are slow, so the clock is slow to match — **a master that clocks faster than the line can rise reads its own transition times as data.**

```
/* Half a bit period.
 *
 * Deliberately slow. The internal pull-ups are ~45 kOhm, so a released line
 * rises through an RC that a proper 4.7 kOhm bus would not have, and a master
 * that clocks faster than the line can rise reads its own transition times as
 * data. Nothing here needs speed: a sensor read is a few dozen bytes and the
 * caller is a task that sleeps anyway. */
static inline void half_bit(void)
{
    for (volatile int i = 0; i < 60; i++) {
    }
}
```

22.18 The failure that looks like success

A bus with no pull-up reports every address as present.

SDA floats, reads low, and a floating low is indistinguishable from an ACK. A scan then returns 112 devices, which is not a subtle wrong answer — but the individual result at any one address is entirely plausible, and code that probes a single expected sensor would find it and proceed.

The self-test checks this **before any scan is believed**, per line and in both directions:

```
void i2c_selftest(void)
{
    static const uint32_t pin[2] = { I2C_PIN_SDA, I2C_PIN_SCL };
    static const char *name[2] = { "SDA (gpio22)", "SCL (gpio27)" };
    int ok = 1;

    for (int i = 0; i < 2; i++) {
        line_release(pin[i]);
        half_bit();
        uint32_t released = line_read(pin[i]);

        line_low(pin[i]);
        half_bit();
        uint32_t driven = line_read(pin[i]);

        line_release(pin[i]);
        half_bit();

        /* ... print ... */
    }
}
```

```

        if (released && !driven) {
            uart_puts("    ok\n");
        } else if (!released) {
            uart_puts("    NO PULL-UP or shorted low\n");
            ok = 0;
        } else {
            uart_puts("    cannot drive low\n");
            ok = 0;
        }
    }

    uart_puts(ok ? "    bus looks sane; a scan can be believed\n"
              : "    bus is NOT sane; ignore any scan result\n");
}

```

```

SDA (gpio22) released=HIGH driven=LOW    ok
SCL (gpio27) released=HIGH driven=LOW    ok
bus looks sane; a scan can be believed
scanning 0x08..0x77
0 device(s)
(nothing attached is the expected result on a bare board)

```

Both halves matter. A line that reads high when released and high when driven means the driver is not working; a line that reads low both times means there is no pull-up.

Only one combination is a working line, and it is the one that gets the word `ok`.

Two readings, four outcomes, three of which are named failures. That is a properly designed test.

The scan also interprets its own result rather than reporting it neutrally:

```

    if (found == 0u) {
        uart_puts("    (nothing attached is the expected result on a bare
board)\n");
    } else if (found > 100u) {
        uart_puts("    (that many is a stuck SDA, not that many devices)\n");
    }
}

```

Telling the reader what the expected result *is* turns a number into a conclusion.

22.19 The protocol details that matter

Two, both about not corrupting somebody else's bus.

The last byte must be NAKed:

```

/* The master's ACK tells the slave whether another byte is wanted. The LAST
 * byte must be NAKed: a slave that is ACKed keeps driving the bus and the
 * following STOP is then issued into a bus somebody else owns. */

```

Repeated START, not STOP-then-START:

```

/* Repeated START, not STOP-then-START. Releasing the bus between the
 * register number and the read lets another master interleave, and on a

```

```

    * single-master bus it still costs a slave the right to keep its internal
    * address pointer. */
if (r == I2C_OK) {
    i2c_start();
    r = i2c_write_byte((uint8_t)((addr7 << 1) | 1u));
    /* ... */
}
```

22.20 Metrics

Interrupt matrix

Quantity	Value
Peripheral interrupts routed before this	0
CPU line used	23, level 3, level-triggered
GPIO source	22
PRO CPU enable bit	15 (measured , not read)
Self-inflicted edges before masking	3,917
Assembly changed	1 call
Distinct delivery failures found	4
Finger-driven wakes observed	0

ADC

Quantity	Value
Channels	8, ADC1
Resolution	12-bit
Attenuation	11 dB, all channels
Conversion time	~13 poll iterations
Light sensor	channel 6, GPIO34, confirmed
Sensor swing (hand over board)	265 counts
Control-group maximum	47 counts
Lines of driver	~150
Wrong bits	1

I²C

Quantity	Value
Pins	SDA 22, SCL 27
Pull-ups	internal, ~45 kΩ
Devices found on a bare bus	0
Bytes ever transferred to a real device	0

22.21 What these do not establish

Matrix. No peripheral interrupt has yet driven anything — proven by injection only. `task_wake()` has never woken a task. One line, one handler; no sharing, no priority among sources. Nothing is re-entrant. The APP CPU is still never started.

ADC. The reading is uncalibrated — counts, not volts, with factory eFuse calibration data unread. The sensor's usable range is narrow (273–538 of 0–4095, and the low end was a hand rather than darkness). It is polled. No application can reach it. **Only channel 6 has been proven to track anything** — "the other seven return distinct, stable, plausible values, and plausible is exactly what this whole report is about not trusting." ADC2 is untouched despite being permanently free.

I²C. No byte has ever been transferred. START, STOP, the ACK bit, repeated START and the read path are all correct by inspection and have never moved data.

The first real device will be the first test of §22.19's logic, and the most likely defects are the ones inspection is worst at: a half-bit of timing, or an ACK sampled on the wrong clock edge.

Clock stretching has never happened, so the handler that justifies the whole bit-banging argument is unexercised. Speed is unmeasured. No multi-master arbitration detection. No bus recovery (the nine-clock flush is not implemented). No application access.

Next: the first output that is not the screen, and three faults where each one made the next invisible.

23. Audio, and Three Ways to Be Silent

Sources: docs/UM-NATOS-027-audio.md Code: kernel/audio.c, kernel/audio.h, kernel/gpio.h

23.1 The smallest feature in the book, and the longest debugging story

The first output this system has that is not the screen.

The feature is small: hardware PWM on GPIO26, a tone, a beep, and a click on every keypress. The report is mostly about the debugging, because getting from "a speaker is plugged into the connector labelled SPEAK" to a sound took three separate faults stacked on top of one another, and **each one made the next one invisible**.

23.2 Why it exists at all

Not novelty:

Multi-tap's worst property is that a press registering is invisible — UM-NATOS-022 §3.4 records that the press which "did not register" is usually one that did, and then replaced the letter you wanted. A click resolves that using none of the 224 rows every other part of the interface is competing for.

Feedback that costs no screen space is worth a great deal on a device where Chapter 24 §24.7 records that **every row of the panel is allocated with none spare**.

23.3 Tones, not sample playback

Sample playback needs a clock at 8 kHz against a 100 Hz tick, which means either a dedicated timer interrupt or I²S with DMA. **This kernel's interrupt matrix has never delivered a peripheral interrupt end to end**, so PCM would mean debugging audio and interrupts simultaneously, and silence is a far less legible symptom than a counter that fails to advance.

LEDC needs none of that machinery. Given a divider it generates the waveform in hardware — no CPU, no interrupt, no DMA, no task, no buffer — so tones cost nothing that is not already proven.

The same staging principle as bit-banging the display first, applied to a different axis: build the thing whose failure has *one* candidate.

23.4 The three faults

1. The driver muted its own pin

The first implementation used the DAC's cosine generator, which requires setting `MUX_SEL` on the pad to hand it to the RTC subsystem.

Once RTC owns a pad, the digital GPIO matrix cannot drive it.

So `audio_init()`, running at boot, muted GPIO26 for every subsequent experiment. The DAC did not work *and* the square-wave probes aimed at the same pin were writing to a pad that was not listening. Fifteen pins were swept in two rounds, all silent, and the mute was this file's own initialisation.

A driver's `init()` sabotaging every subsequent diagnostic of itself is a particularly nasty shape, because the diagnostics look independent and are not.

2. The check that could not fail

Fourteen silent pins is also exactly what a broken generator looks like, so the generator was checked — correctly, in principle. `spktest` drives GPIO32, the one pin this kernel can both drive and measure, and confirmed 351 edges against ~400 expected with the pad swinging 1829 against 0.

GPIO32 has no RTC mux. It is structurally incapable of exhibiting §1's fault. The test passed, and its conclusion — *"generator and pad both work; the silence is the hardware"* — was reported with confidence and was wrong, sending the search toward connectors and amplifiers.

Verifying an instrument on the one case that cannot fail is not verification. The limitation is now printed by the probe itself, so the next person to run it is told what it does not cover before they read the result.

This is the third member of the family with Chapter 21 §21.4 (four `IO_MUX` entries all on the same side of the anomaly) and Chapter 19 §19.8 (a direction test that could only return one answer). All three were real tests, correctly performed, structurally unable to fail.

The mitigation here is the most direct of the three: **the probe prints its own blind spot.**

3. A peripheral that was switched off

With the pad released, LEDC still produced nothing — while **every register read back byte-for-byte correct:**

```
timer0_conf = 0x021631ca  matching the computed divider exactly
SIG_OUT_EN  set
duty        50%
GPIO26      routed to signal 71
```

LEDC comes out of reset clock-gated. **A gated peripheral accepts register writes and does nothing.**

The fix is two lines:

```
REG(0x3FF00C0u) |= (1u << 11); /* DPORT_PERIP_CLK_EN: LEDC */
REG(0x3FF00C4u) &= ~(1u << 11); /* DPORT_PERIP_RST_EN: LEDC out of reset */
```

with the observation that made it findable:

```
* LEDC comes out of reset clock-gated, and a gated peripheral ACCEPTS REGISTER
* WRITES and does nothing. ... [SPI2 and GPIO are ungated by]
* the bootloader before this kernel runs. LEDC is the first peripheral here
* [that this kernel has had to switch on itself].
```

The shape, three times in one project

symptom	reality
ADC returning the same value on every channel	converting, attached to nothing
GPIO edge latched, CPU never interrupted	delivered to the APP CPU, which is halted
LEDC configured perfectly, silent	clock gated off

A read-back cannot detect any of them. The display and touch never needed a clock gate touched because SPI2 and GPIO are ungated by the bootloader; LEDC is the first peripheral this kernel has had to switch on itself.

Chapter 27 §27.2 adds a fourth and answers it differently — by looking for free-running counters rather than by reading registers at all.

23.5 And then the frequency

With all three fixed, **440 Hz was still inaudible. 3 kHz was immediately clear.**

```
440 Hz is unremarkable for a speaker in general and beyond this one. A correct tone at
a frequency the transducer cannot move is indistinguishable from a broken driver,
and it cost a full round of debugging after the driver was already working.

Everything meant to be heard now sits near 3 kHz, which is also roughly where human
hearing is most sensitive — so the same drive is louder for free.
```

The shell's help text carries the warning where somebody will actually read it:

```
tone <hz>      tone on gpio26; 'tone 0' stops. try 3000, not 440
```

23.6 What actually broke the deadlock

Not a probe.

The user said "**it works fine on my other project.**"

One sentence eliminated the entire hardware half of the search space — which was precisely the half §2 had concluded was at fault. Every tool built that evening was aimed at the wrong hypothesis, and none of them could have disconfirmed it, because they all tested the same layer.

The question worth asking earlier is not "what else can I measure" but "what already works that this can be compared against". A working reference is stronger evidence than any amount of instrumentation on the broken thing.

That is the second time in this book a human observation outranked the instruments — Chapter 19 §19.4's *"the dots are following my finger"*, overridden as "almost certainly leftovers" and correct — and Chapter 27 §27.6 adds a third where a user contradicted a confident hardware diagnosis and was right.

23.7 The driver

```
#define LEDC_BASE          0x3FF59000u
#define LEDC_HSCH0_CONF0_REG (LEDC_BASE + 0x0000u)
#define LEDC_HSCH0_HPOINT_REG (LEDC_BASE + 0x0004u)
#define LEDC_HSCH0_DUTY_REG (LEDC_BASE + 0x0008u)
#define LEDC_HSCH0_CONF1_REG (LEDC_BASE + 0x000Cu)
#define LEDC_HSTIMER0_CONF_REG (LEDC_BASE + 0x0140u)
#define LEDC_CONF_REG      (LEDC_BASE + 0x0190u)

#define TICK_SEL_APB      (1u << 25)    /* timer counts APB, not REF_TICK */
#define TIMER0_RST        (1u << 24)
#define DIV_NUM_S         5u            /* 18 bits, 10.8 fixed point */
#define SIG_OUT_EN        (1u << 2)
#define DUTY_START        (1u << 31)
#define DUTY_S            4u            /* the duty field carries 1/16ths */

#define LEDC_HS_SIG_OUT0   71u          /* GPIO matrix signal index */

#define SPK_PIN            26u
#define SPK_MUX_REG        0x3FF49028u

/* 10-bit period: deep enough that 50% duty is exactly 50%, shallow enough that
 * the divider stays inside 18 bits down to about 80 Hz. */
#define DUTY_RES           10u
#define DUTY_PERIOD        (1u << DUTY_RES)

#define APB_HZ             80000000u
```

Offsets fetched rather than recalled, and the file says so:

```
* Offsets and field positions came from the vendor's ledc_reg.h and
* gpio_sig_map.h, fetched rather than recalled.
```

Starting a tone:

```
REG(LEDC_HSTIMER0_CONF_REG) = TICK_SEL_APB
                             | /* ... divider ... */;
REG(LEDC_HSTIMER0_CONF_REG) |= TIMER0_RST;    /* latch the divider */
REG(LEDC_HSTIMER0_CONF_REG) &= ~TIMER0_RST;
```

```
REG(LEDCHSCH0_DUTY_REG) = (DUTY_PERIOD / 2u) << DUTY_S;
REG(LEDCHSCH0_CONF1_REG) = DUTY_START;
REG(LEDCHSCH0_CONF0_REG) = SIG_OUT_EN;          /* timer 0, output on */
```

Clocked from APB at 80 MHz, which this project has *measured* (Chapter 7 §7.8), so the pitch is exact rather than nominal against an untrimmed RC oscillator. The report makes that point explicitly as a reason to prefer LEDC over the DAC.

And why LEDC was the better choice anyway

```
* LEDC replaces it, and is the better fit regardless. It is hardware PWM: set a
* divider and it generates the waveform continuously with no CPU, no interrupt
* and no DMA – the properties the cosine generator was chosen for – and it is
* clocked from APB at 80 MHz, which this project has measured, so the pitch is
* exact rather than nominal against an untrimmed RC oscillator.
```

An accidental improvement: the workaround for a fault turned out to be a better design than the thing it replaced.

23.8 Verification

```
tone 3000      -> audible
tone 0         -> stops
open shell, tap keys -> a click on every accepted press
open notes, tap keys -> the same
```

The click is 3 kHz for 2 ticks (~20 ms), and it fires on every *accepted* press — which is the property that makes it useful feedback rather than noise. A press rejected as chatter does not click, so the click means "this one counted".

23.9 Metrics

Quantity	Value
Pin	GPIO26 (SPEAK connector)
Generator	LEDC high-speed timer 0, channel 0
Clock	APB, 80 MHz — pitch is exact
Duty resolution	10 bits, fixed 50%
Frequency range	~78 Hz to APB/1024; usable from ~1 kHz on this speaker
Click	3 kHz, 2 ticks (~20 ms)
CPU cost while sounding	none
Pins swept before the fault was found	15
Faults stacked	3

23.10 What this does not establish

- **No sample playback.** Tones only; there is no PCM path.
- **No volume control.** Duty is fixed at 50%. "Varying it changes timbre and loudness together on a square wave, and nothing here has tried."
- **One channel.** LEDC has sixteen; one timer and one channel are configured.
- **The speaker's response is uncharacterised.** 440 Hz is known inaudible and 3 kHz known clear. Nothing between has been measured, so "usable from ~1 kHz" is an estimate.
- **No applications can make a sound.** No syscall.
- **Clicks are unconditional.** Every accepted keypress clicks, with no way to silence it, "which is a preference nobody has been asked for".
- **The DAC is untouched now.** Whether the cosine generator could have been made to work was never established:

it was abandoned once LEDC worked, so §23.4 records a fault in this driver, not a verdict on the peripheral.

A careful distinction, and the right one: the DAC was never shown to be broken, only never shown to work.

Part IV ends here. Nine drivers: display, touch, flash, microSD, the interrupt matrix, ADC, I²C, audio, and — in Chapter 27 — an 802.11 receiver. Four of them produced a register that read back perfectly while the hardware did nothing, and each was caught by a different technique: a control group, an injected edge, a moving-word scan, and a person saying "it works on my other project".

Part V is what a user actually sees.

24. The Launcher, and Four Defects It Found by Existing

Sources: docs/UM-NATOS-021-launcher.md Code: kernel/desktop.c, kernel/desktop.h, kernel/app.h, kernel/shell.c

24.1 The first thing that exists for a user

The first thing in this project that exists for a user rather than for a test. Everything before it was verified by reading serial output; this is verified by someone touching the glass and a program starting.

It is about 250 lines, and its value turned out to be less in what it does than in what it demanded:

exposed	defect	chapter
horizontal position	touch X axis inverted since the driver was written	19 §19.8
position validity	PENIRQ answers presence, not validity; rail samples	19 §19.6
a task slot	nine task_create calls against a TASK_MAX of 8 — no idle task	§24.5
display throughput	applications and renderer blocking each other on the draw lock	11 §11.9

None was found by inspection. All four were found by building something that depended on them.

That is the strongest argument in this book for building the consumer rather than more infrastructure. Four defects, all latent for weeks, all in code that had been reviewed and had passing self-tests, and all surfaced by 250 lines whose own logic is trivial.

24.2 A launcher, not a window manager

- * It is a LAUNCHER: an icon grid, a cursor, and double-tap to start a program.
- * It is not a window manager. Applications still render into the fixed horizontal strips app.c assigns by slot index (UM-NATOS-016 §2), because
- * there is no focus model, no z-order, and no way for an application to own the screen and give it back.
- *
- * That boundary is deliberate rather than unfinished. Dynamic viewports need an
- * arbitration policy for who owns which pixels, and building one before there
- * is a user interface to demand it would be designing against a guess — the
- * same reason focus arbitration was deferred when viewports could not overlap.
- * A launcher is the consumer that makes the question concrete.

"The consumer that makes the question concrete" — but not the one that answers it. That distinction keeps a known gap on the record rather than letting it become a

decision by default.

24.3 Why a cursor on a touchscreen

```
* A cursor is an indirection: you can already point at what you want. It earns
* its place here because this panel is resistive and noisy – a single reading
* can land tens of pixels from the finger – and because an icon large enough to
* hit reliably by direct tap would leave room for very few of them.
*
* So the interaction is hybrid, not modal: a single tap MOVES the cursor to it,
* and a double-tap OPENS whatever the cursor is on. A confident user taps
* twice and it opens; an unsure one taps once, sees where the cursor landed,
* and corrects. **Neither has to be told which mode they are in.**
```

Two measured reasons for a design decision that would otherwise look like inherited desktop convention.

24.4 Two defects of its own

Status text drawn over a control

The launch message was drawn at `DESK_H - 10`, which is inside the bottom-left cell's label. So "started" appeared over `pong`'s name — and `g_msg_sel` was never cleared, so it stayed there permanently.

It was reported as *"it selected Pong, which was a bit off"*, and that reading was entirely reasonable. **The interface was asserting something false about its own state, indefinitely.**

The fix is recorded in the source as a rule rather than a patch:

```
/* The bottom of the region is a status strip, not grid.
 *
 * It used to be neither: the launch message was drawn at DESK_H-10, which is
 * inside the bottom-left cell's label. So "started" appeared over pong's name
 * and stayed there permanently, and the obvious reading – that pong was what
 * had been selected – was wrong but entirely reasonable. Text that overlaps a
 * control is not a cosmetic problem; it makes the interface lie about its own
 * state. */
#define STATUS_H 14u
#define GRID_H (DESK_H - STATUS_H)
```

Plus a second decision on the same line:

```
/* How long a launch message stays up. It expires rather than persisting,
 * because a status line that never clears stops describing the present and
 * becomes decoration. */
#define MSG_TICKS 200u /* ~2 s */
```

The selection was read at the worst possible moment

Selection followed *every* sample while the finger was down, so the **last** sample before release decided it — and release is precisely when a resistive panel produces its worst

data.

Measured, one tap on the top-right icon:

```
first sample -> cell 2      (correct)
last sample  -> cell 3      (wrong)
```

The correct target was read, recorded, and discarded microseconds later.

The fix:

Selection is now latched from the **first** sample of a press and not moved again until the finger lifts. Later samples describe a finger being *lifted*, which is not an instruction. Drag no longer moves the cursor; tap again to correct.

And the note about the instrument:

This was diagnosed by an instrument built for the theory and then nearly deleted when the theory looked wrong. The first/last cell pair is still reported, with a note saying why: **removing an instrument once it has found its bug is how the bug returns unnoticed.**

This is the same defect as Chapter 19 §19.6, seen from the consumer's side: the release sample reads the ADC rail. One cause, and by the time both were found it had produced *three* symptoms across two files and three months.

24.5 There was no idle task

`kmain` makes **nine** `task_create` calls; `TASK_MAX` was **8**. The ninth is the idle task, so it failed: `task_create()` returns `-1` when the table is full, `task_set_idle(-1)` bounds-checks and quietly does nothing, and the return value was never printed or checked.

Two real costs:

`WAITI` never executed, so the CPU never slept. And with every task asleep the scheduler found nothing runnable and fell through to "resume the interrupted context" — **which resumes a SLEEPING task, the exact invariant blocking exists to enforce.**

And the part that stings:

The evidence had been printed and read. The `stacks` table added hours earlier listed ids 0–7 with no idle task; it was looked at and not registered. **That is the failure UM-NATOS-019 is entirely about, committed while writing the report about it.**

The fixes are structural: `TASK_MAX` is now 12, and `must_create()` panics rather than returning `-1` (Chapter 9 §9.9). `task.h` records the whole thing at the constant:

```
/* 12, not 8. The kernel creates nine tasks and TASK_MAX was 8, so the ninth -
 * the idle task - failed to be created and nobody noticed: task_create()
 * returns -1, task_set_idle(-1) quietly does nothing, and the return value was
 * never checked or printed. The system ran without an idle task for as long as
 * priorities have existed, which cost the WAITI power-saving path entirely and
 * meant that with every task asleep the scheduler resumed a SLEEPING task
```

```
* rather than idling. */
#define TASK_MAX      12
```

24.6 Icons, close buttons, and ownership

Icons are a bitmap, not nine drawing routines

8 × 8 monochrome glyphs, one byte per row, drawn at 3× into 24 × 24. Nine bespoke drawing functions would be more code than nine constants and would make every icon a place a bug can hide; a bitmap is also editable by anyone who can count to eight, which matters more than elegance for something whose only requirement is being recognisable at 24 pixels.

And a detail that is not micro-optimisation:

Each row is drawn as one rectangle per **run** of set pixels rather than one per pixel. That is not micro-optimisation: every primitive takes the draw lock, and UM-NATOS-014 §10 established that the cost of that lock is the number of acquisitions rather than the time held.

Chapter 11's finding, applied by a completely unrelated piece of code, correctly.

The close button is outside the viewport, and that is the point

app.h carries the argument at the constant:

```
/* ---- where an application may draw, and where it may not -----
 *
 * Exported because the close button depends on the exact boundary. The kernel
 * reserves a column at the right of every strip and draws the X there, so the
 * viewport handed to the application STOPS short of it.
 *
 * That is isolation, not layout. If the close button lived inside the viewport
 * an application could paint over it, draw a decoy elsewhere, or simply fill
 * its strip and hide the way out. Putting it outside means the one control the
 * user needs in order to escape a misbehaving program is the one control that
 * program cannot touch – the same argument as the viewport itself
 * (UM-NATOS-016 §2), applied to a pixel the user owns rather than one the
 * application does. */
#define APP_VIEW_Y0      224u
#define APP_VIEW_PITCH   16u
#define APP_VIEW_H       14u
#define APP_NAME_W       44u
#define APP_CLOSE_W      16u
#define APP_CHROME_W     (APP_NAME_W + APP_CLOSE_W)
#define APP_VIEW_W       (DISP_W - APP_CHROME_W)
```

the one control a user needs in order to escape a program is the one control that program cannot touch.

The touch path enforces the same boundary:

close buttons see the press first, and a consumed press is not passed on, so closing a program cannot also select an icon underneath it.

A button for something you cannot name

The area below the launcher was reported as *"strange space ... why is there two x boxes in there"*, and the answer was worse than the question implied.

Those were `ping` and `pong`, which `kmain` starts at boot to keep the IPC counters live. They exchange **messages**, not pixels. So two programs were running correctly, producing no visible output, and **the only evidence of their existence was two buttons whose function was to destroy them**. Tapping one would have been a guess about what it deleted.

```
/* The kernel's column: the program's NAME and its close button.
 *
 * The name is there because without it the area is unreadable. Four empty
 * strips with two X floating in them is what a user actually saw, and the
 * honest reading of that is "what are those" – the programs starting at boot
 * exchange messages rather than drawing, so nothing identified them. A close
 * button for something you cannot name is worse than no close button. */
```

The cost is stated rather than hidden:

the kernel column grew from 16 px to 60 px, so applications lost about a fifth of their strip width to a label they can neither draw nor overwrite. If a program later needs the full width, this is the decision to revisit.

The running marker, again

The mark for "this program is running" was a four-pixel dot in the corner of the icon. It is now an underline beneath the icon's label.

The dot had already failed once: it compared only the first character of the program name, so `paint`, `ping` and `pong` marked each other, and **nobody noticed by looking at the screen** — three wrong four-pixel dots read as a rendering artefact. It was found while writing §9 of this report.

A mark too small to be read wrongly is also too small to be read at all.

24.7 Chrome over a repainting view

The problem

The 3D view's close button was first drawn immediately after the view, every frame, by the same routine that draws the application buttons. It strobed badly enough that the view read as broken.

The raycaster **repaints every pixel every frame**. Anything drawn over it therefore survives only until the next frame begins, and drawing it later in the sequence cannot help — "later" ends about sixty milliseconds afterwards. **The button and the frame beneath it are not two draws to be ordered. They have to be one draw.**

The fix

`desktop_overlay_into()` writes the button into the view's framebuffer, in that buffer's own coordinates, and the raycaster calls it immediately before blitting.

It is a pure function: it writes pixels and calls nothing. The frame and the button reach the panel in a single transfer, and there is no moment at which one exists without the other.

```
/* Stamps the close button into a view's framebuffer, in the buffer's own
 * coordinates. Pure: it writes pixels and calls nothing.
```

And the invariant that keeps drawing and hit-testing in step:

The hit test did not change. The stamped button lands on exactly the coordinates `desktop_chrome_touch()` already tested, which is worth stating because **a control drawn by one mechanism and tested by another is a standing invitation for the two to drift.**

Three owners, made explicit

The `fb off` path has no buffer to stamp into, and the note pad has no framebuffer at all. All three cases are now named in one comment:

```
/* Who draws the close button depends on who owns the region:
 *
 * 3D view, framebuffer on   the raycaster stamps it into the buffer, so
 *                           it and the frame arrive in one transfer
 * 3D view, framebuffer off  drawn here, and it flickers; fb off is a
 *                           diagnostic mode
 * note pad                  the app draws it in its own header
 *
 * The middle case is the only one this branch is for. It used to be the
 * only case considered, which is why the note pad's button was invisible:
 * present, hit-testable, and drawn by nobody. */
if (g_mode == MODE_3D && !raycast_framebuffer()) {
    draw_close(DISP_W - 20u, 2u, 18u, 18u, COLOR_WHITE);
}
```

Chapter 26 §26.7 is the note pad's side of that: *"present, hit-testable, and drawn by nobody"* — a button that worked and was invisible.

And the reasoning for accepting the flicker in the diagnostic mode:

An invisible exit is worse than a visible strobing one: the way out of the view would exist and be unfindable.

24.8 The same diagnosis, at twenty times the scope

This section is about a fix that was reverted, and it is here because the mistake was not in the reasoning.

The first attempt reached exactly the conclusion above — chrome cannot be drawn over a view that repaints continuously — and implemented it by **reserving rows** at the foot

of the region for a chrome bar the views would not touch.

That is a correct solution. It also meant moving the region boundary, and therefore the application strips, and therefore the colour strip, and the launcher grid was resized to use the space freed by removing things that had been on screen at boot. **Four files of constants moved together.**

The result was a screen that looked wrong, and it was reverted without the cause being understood. And every measurement said the renderer was fine:

the wall band landed at the right rows, the composed framebuffer held a dark ceiling, an orange wall and a dark floor at the expected offsets, `display_blit` was verified including its contiguous fast path, and eleven self-tests passed throughout. Three rounds of instrumenting the raycaster produced three rounds of evidence that the raycaster was fine.

The comparison with the fix that worked:

The fix in §24.7 does the same job by writing 324 pixels into a buffer that was about to be sent anyway.

A correct diagnosis does not license a fix of arbitrary scope. The two attempts share a diagnosis and differ by a factor of twenty in what they touch.

24.9 The cause, found later — and it was not the layout

Revision 1.3. §24.8 said the cause was never found. It has been, and the conclusion a reader would have drawn from §24.8 was wrong.

The geometry change was applied **on its own** to current code — nothing else those commits touched — and the 3D view rendered correctly: the framebuffer allocated, frames climbed, the camera crossed the map. **Geometry alone breaks nothing.**

What broke was the camera. The relayout raised the frame rate, movement was per-frame at the time, and the camera outran its turn probe and buried itself in a wall. **"Blank screen" and "a bunch of vertical lines" are precisely what a wall at zero distance looks like: full-height columns of flat colour.**

Which is why every instrument insisted the renderer was fine. **It was fine.** It was correctly drawing the inside of a wall. A taller region made the symptom worse rather than better, because wall height scales with `RAY_VIEW_H` — so the layout looked more guilty the more of it there was.

The actual lesson

The revert bundled the layout change **and** the camera fix into one commit. The screen looked right afterwards, which appeared to convict the layout, and the information needed to acquit it had been destroyed by the same action that produced the evidence.

Reverting two changes together destroys the information about which one mattered. If a revert must bundle, the bundle is a hypothesis to test later, not a conclusion — and it should be recorded as one.

| §24.8 recorded it as a conclusion. This section is the correction.

24.10 The screen budget, and six assertions

The screen budget is real and was the other half of the difficulty. Before the relayout it was **312 of 320 rows allocated with 8 spare**, and the four claims on it live in four files:

rows	claimed by	file
launcher and full-region views	DESK_H, RAY_VIEW_H	desktop.h, raycast.h
application strips	APP_VIEW_Y0/PITCH/H	app.h
colour strip	SPEC_Y/SPEC_H	kmain.c

| Growing one forces a cascade through the others with no single owner to check it.

The assertions are the mechanical part of the lesson:

```
/* The screen layout is agreed across four files – desktop.h sizes the launcher
 * region, raycast.h sizes the 3D view, app.h places the application strips, and
 * kmain.c places the colour strip. Nothing enforced that agreement, and a
 * session spent reshuffling it produced a screen that looked broken in ways no
 * test could see: every self-test passed throughout.
 *
 * These do not check that the layout is GOOD. They check that its pieces do not
 * overlap or fall off the panel, which is the part a compiler can know. */
_Static_assert(RAY_VIEW_H <= DESK_H,
               "the 3D view must fit inside the launcher region");
_Static_assert(APP_VIEW_Y0 >= DESK_H,
               "application strips must start at or below the launcher region");
_Static_assert(APP_VIEW_Y0 + APP_MAX * APP_VIEW_PITCH <= DISP_H,
               "application strips must fit on the panel");
_Static_assert(APP_CHROME_W < DISP_W,
               "the kernel column must leave an application something to draw in");
```

plus two in kmain.c for the colour strip:

```
_Static_assert(SPEC_Y >= APP_VIEW_Y0 + APP_MAX * APP_VIEW_PITCH,
               /* ... */);
_Static_assert(SPEC_Y + SPEC_H <= DISP_H,
               /* ... */);
```

"These do not check that the layout is GOOD. They check that its pieces do not overlap or fall off the panel, which is the part a compiler can know" is a precise statement of what a static assertion is for.

They also enabled the experiment that acquitted the layout in §24.9: *"they are what let the experiment build safely instead of silently overlapping."*

The layout that resulted

```
launcher / views    0..223      was 0..167
application strips  224..287    was 168..279 – now 14 rows each
```

colour strip	288..319	unchanged
launcher cells	80 x 70	was 80 x 51
icon glyphs	32 x 32	was 24 x 24
notes text area	10 lines	was about 4
3D view	224 rows	was 168

The cost is the application strips losing more than half their height. ping and pong draw nothing, so it costs nothing today; a program that draws will notice.

24.11 A guard narrowed to an assertion

A late correction, from Chapter 27's session, and it belongs here because it is about this file.

The chrome column was, for a while, believed to be painting over the full-width 3D view (Chapter 27 §27.6 — and it genuinely was, at the earlier geometry). The first fix returned from `desktop_chrome()` outright in `MODE_3D`. Once the geometry changed, that guard became both unnecessary and harmful:

```
void desktop_chrome(void)
{
    /* The strips are BELOW a full-width view, and that is load-bearing.
     *
     * APP_VIEW_Y0 is 224 and RAY_VIEW_H is 224, so slot 0 begins exactly where
     * the 3D view ends and nothing here can touch it. An earlier fix returned
     * from this function outright in MODE_3D, on the theory that the chrome was
     * painting over the view. The geometry says otherwise, and the cost of the
     * blanket guard was that the strip band was never repainted while the view
     * was open -- so a program starting behind the view left a stale band that
     * only looked right once it had painted over every pixel itself. That is
     * the "wrong for ten to thirty seconds, then fine" report.
     *
     * The assertion below is the part worth keeping: it fails the build if the
     * view ever grows tall enough to reach the strips, rather than leaving a
     * future overlap to be rediscovered from the glass. */
    _Static_assert(APP_VIEW_Y0 >= RAY_VIEW_H,
        "app strips must stay below the 3D view, or chrome overwrites
it");
}
```

A runtime guard replaced by a compile-time assertion — *"which is the check that should have been written first, since it answers the overlap question at compile time instead of from the glass."*

24.12 Verification

```
tap left icon   raw_x=3408 -> x=0,   cell 0
tap right icon  raw_x=920  -> x=196, cell 2
seven consecutive taps, z 1662..2261, no rail samples
x spanning 0..187 across all three columns
first and last sample of each press agree
```

Five separate claims in five lines: the axis maps correctly at both ends, taps are firm enough to pass the pressure gate, no rail samples got through, the horizontal range spans the grid, and the latching fix holds.

All three columns selectable, double-tap opens the named program, and the status strip reports which. Confirmed on hardware by the user.

24.13 The launcher costs nothing when idle

* It repaints only when something changed, so an idle desktop pushes **zero**
* pixels – unlike the raycaster, which repaints unconditionally because every
* frame differs. If the screen looks frozen, that is correct: it is waiting for
* input.

A frozen-looking screen that is *supposed* to be frozen, in a project whose worst debugging session was caused by a frozen screen that was not. Hence the last sentence, which exists so nobody re-diagnoses it.

24.14 Metrics

Quantity	Value
Status strip	14 px, message expires ~2 s
Double-tap window	60 ticks (~600 ms)
Minimum press	2 ticks, rejects contact chatter
SPI cost when idle	0 bytes
Grid	3 × 3, cells 80 × 70
Icon glyphs	8 × 8, drawn at 4×
Screen rows allocated, before / after	312 of 320 / 320 of 320
Layout assertions	6, spanning four files
Overlay cost per frame	324 pixels into a buffer already being sent
Reverted attempt	~200 lines, four files
Kernel-owned column	60 px of 240
Application viewport width	180 px, was 240
Defects exposed in other subsystems	4
Defects of its own	2

24.15 What this does not establish

- **The screen is now fully allocated.** Every row is claimed; there is no spare. The next feature wanting rows must take them from something named in §24.10, and the assertions will refuse a build that overlaps.
- **Application strips are 14 rows.** No program currently draws into one, so nothing has tested whether that is enough to be useful.

- **No focus, no windows, no z-order.**
- **No way to stop a program from the launcher itself.** `kill` exists only in the shell — which since Chapter 26 is *on* the grid, so it no longer needs a host, but is still a command typed into another app.
- **The `fb off` path still flickers.**
- **Nothing tests the overlay.** The button's position is agreed between `desktop_overlay_into()` and `desktop_chrome_touch()` by two matching constants, and no assertion ties them together — "the exact drift the layout assertions were added to prevent elsewhere".
- **The running marker is per-name, not per-instance.**
- **Faulted programs cannot be dismissed.** A program that faults keeps its slot and shows no X, because it is not running; it lingers until `kill`.
- **The chrome column is fixed width.** 60 px regardless of name length.
- **Double-tap timing is unmeasured.** 600 ms and the 2-tick minimum press were reasoned, not derived. "The counters exist to settle them and nobody has."
- **The icons are guesses at 24 pixels.** Judged by one person. "`blit` and `rogue` are the two most likely to read as noise."
- **One user.** Every judgement about whether the interaction feels right came from a single person on a single panel.

Next: what the launcher hands the region to.

25. The Renderer

Sources: docs/UM-NATOS-015-display.md §5.8, docs/UM-NATOS-021-launcher.md §6.7 Code: kernel/raycast.c, kernel/raycast.h, tools/gen_sintab.py

25.1 What it is, and why it is in this book

A grid raycaster: one ray per screen column, fixed-point throughout, face shading, wall texture, a camera that wanders a maze on its own.

It is the most demanding consumer in the system and it is the reason four separate defects elsewhere became visible. From Chapter 18:

Driving the display from a raycaster rather than a status screen put a very different load on this layer and found two things.

From Chapter 11:

A raycaster took the display lock once per column and spent 25 seconds per frame on 37 ms of work.

From Chapter 9, it is the only HIGH-priority task besides touch, and the reason strict priority had to be reasoned about at all.

It also carries an admission about its own placement:

That is recorded here for want of a better home. It is not a display-driver property, and if the renderer grows much further it wants its own report rather than a subsection of the driver's.

This chapter is that report.

25.2 The map

```
#define MAP_W 16
#define MAP_H 16

/* 1 is wall, 0 is floor. A ring with a few interior blocks, so walking it
 * produces corridors and openings rather than one empty box. */
static const uint8_t MAP[MAP_H][MAP_W] = {
    {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1},
    {1,0,0,0,0,0,0,1,0,0,0,0,0,0,1},
    {1,0,1,1,0,1,0,1,0,1,1,1,0,1,1},
    /* ... */
    {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1},
};
```

The map shape is a design decision with a stated purpose: corridors and openings rather than one empty box, so the camera's navigation has something to fail at

(§25.6).

25.3 Fixed point, and an overflow avoided by construction

```
#define FP          16
#define ONE        FP_ONE
#define STEP_SHIFT 3                /* march 1/8 of a cell per step */
#define MAX_STEPS  160              /* 20 cells before giving up   */

/* tan(30 deg) in 16.16 – half of a 60 degree field of view. */
#define PLANE_SCALE 37837
```

16.16 fixed point. No floating point anywhere — the ESP32 has an FPU but the kernel is `-nostdlib` and the soft-float helpers would have to come from `libgcc` or the ROM, which Chapter 2 §2.10 shows is a link-order minefield.

The marcher steps incrementally rather than scaling a ray by a growing parameter, and the reason is arithmetic:

```
/* March by a fixed fraction of a cell. Stepping incrementally avoids
 * multiplying the ray by a growing t, which would overflow 32 bits
 * within a few cells. */
int32_t sx = rayX >> STEP_SHIFT;
int32_t sy = rayY >> STEP_SHIFT;
```

The scaling in the camera-plane arithmetic is the same defence, applied differently:

```
int32_t planeX = (-dirY / 256) * (PLANE_SCALE / 256);
int32_t planeY = ( dirX / 256) * (PLANE_SCALE / 256);
/* ... */
int32_t rayX = dirX + (planeX / 256) * (cameraX / 256);
int32_t rayY = dirY + (planeY / 256) * (cameraX / 256);
```

Both operands pre-scaled by 256 so the product stays inside 32 bits. Chapter 19 §19.10 notes the contrast: the calibration arithmetic does *not* need this, "nothing here approaches the limits that forced the scaled multiply in `raycast.c`", and says so at the point where a reader might copy the pattern unnecessarily.

The sine table is generated on the host (`tools/gen_sintab.py`) into `kernel/generated/sintab.h`, and lives in `.rodata` — which since Chapter 4 costs zero DRAM.

25.4 The march

```
for (uint32_t c = 0; c < RAY_COLS; c++) {
    uint32_t x = c * RAY_COLW;
    /* -1 at the left edge, +1 at the right. */
    int32_t cameraX = (int32_t)((2 * x * ONE) / RAY_VIEW_W) - ONE;

    int32_t rayX = dirX + (planeX / 256) * (cameraX / 256);
    int32_t rayY = dirY + (planeY / 256) * (cameraX / 256);
```



```

int32_t sx = rayX >> STEP_SHIFT;
int32_t sy = rayY >> STEP_SHIFT;

int32_t fx = g_px, fy = g_py;
int hit = 0, steps = 0;
/* ... */

t0 = task_cpu_cycles();
while (steps < MAX_STEPS) {
    fx += sx;
    fy += sy;
    steps++;
    if (wall_at(fx, fy)) {
        hit = 1;
        hx = fx >> FP;
        hy = fy >> FP;
        /* ... face and texture recovery ... */
        break;
    }
}

t_march += task_cpu_cycles() - t0;

```

Note `task_cpu_cycles()` rather than `xt_ccount()` — Chapter 9 §9.8's clock, used correctly here.

`MAX_STEPS` bounds the march at 20 cells. A ray that finds nothing produces `hit = 0` and a zero-height wall rather than running forever, which matters because the map is finite and a ray parallel to a corridor can miss everything.

Perspective, without a divide per pixel

```

/* Perpendicular distance, in 16.16 cells. */
int32_t dist = (int32_t)steps << (FP - STEP_SHIFT);
if (dist < (ONE / 8)) {
    dist = ONE / 8;          /* never divide by ~zero */
}

int32_t h = hit ? (int32_t)((RAY_VIEW_H * ONE) / dist) : 0;
if (h > (int32_t)RAY_VIEW_H) {
    h = RAY_VIEW_H;
}

int32_t top = ((int32_t)RAY_VIEW_H - h) / 2;
int32_t bot = top + h;

```

One division per column, not per pixel. `dist` is clamped away from zero — the guard that turns Chapter 24 §24.9's "camera inside a wall" from a divide-by-zero into a full-height column of flat colour, which is exactly what was observed.

Distance falloff

```

/* Distance falloff. Full brightness up close, floor of 40 so a far wall
 * stays visible rather than fading into the ceiling. */
uint32_t shade = 255u;

```

```

if (dist > ONE) {
    uint32_t d = (uint32_t)(dist >> FP);
    shade = (d >= 12u) ? 40u : (255u - d * 18u);
}

```

A floor rather than a linear fade to black, because a wall that fades into the ceiling colour stops being a wall.

25.5 One ray per column, and an estimate wrong by forty

RAY_COLW went from 2 to 1, doubling horizontal detail. The constant carries both the old reasoning and the measurement that overturned it:

```

/* One ray per screen column.
 *
 * Was 2, which halved the cast cost at the price of halving horizontal detail.
 * That trade made sense when the renderer's cost was unknown; measurement says
 * it is not: marching is 0.2 ms of a 50 ms frame and the blit is 41.9 ms, so
 * the frame is bus-bound and casting twice as many rays is close to free. */
#define RAY_COLW 1u
#define RAY_COLS (RAY_VIEW_W / RAY_COLW)

```

The estimate was wrong by a factor of forty — predicted 0.2 ms, cost about 9 ms — for **two reasons worth recording**:

- the 0.2 ms march figure was sampled with the camera **against a wall**, where rays terminate immediately. **A cost measured at the cheapest moment is not a cost.**
- compose doubled although the pixel count did not: the per-row loop now runs once per column, 240×168 iterations instead of 120×168 writing two pixels each. **Same pixels, twice the loop.**

Both are general. The first is a sampling error — the same class as Chapter 19 §19.9's "measured in one place and written down as a property of the panel". The second is a reminder that loop *iterations* and *work* are different quantities.

The superseded comment above it is left in the file, which is how the history survives:

```

/* Two pixels per row, so one composed buffer covers a 2-pixel-wide column.
 *
 * The data volume is identical either way – the panel receives the same 80,640
 * bytes. What halves is the number of address-window setups, and those measured
 * at ~450 us each against only ~94 us of pixel data per column. The overhead was
 * five times the payload. */

```

That "~450 µs each" figure is itself one of the numbers Chapter 18 §18.7 later identified as wall-clock contaminated by preemption. Two superseded measurements stacked in one comment block, both left standing, both labelled by what came after — which is the project's documentation style in miniature.

25.6 Face shading and wall texture

Faces, recovered rather than tracked

```
/* Which face was crossed, recovered by comparing the cell one
 * step back. A DDA would know this for free; a fixed-step march
 * does not, and one subtraction is cheaper than changing the
 * marcher.
 *
 * If both coordinates changed cell in the same step the choice
 * is arbitrary and X wins. At 1/8 of a cell per step that is a
 * corner hit, where either answer is defensible. */
face_x = (((fx - sx) >> FP) != hx);
```

The design problem:

Every wall face was equally bright, so two walls meeting at a corner differed only by cell hue and the corner read as a colour change rather than as geometry. Faces crossed through a vertical boundary now render at full brightness and the others at 5/8.

One subtraction per hit, one multiply per column, "unmeasurable against a 41.9 ms blit". Chapter 18 §18.7's finding — *detail is cheap here and pixels are expensive* — used to decide what was worth adding.

Texture, positioned in world space

```
/* Where along the wall face the ray landed, as a fraction of a
 * cell. The face runs along the axis that did NOT change cell,
 * so that is the coordinate to take the fraction of. */
wall_u = (uint32_t)((face_x ? fy : fx) & (ONE - 1));
```

Four panels per cell with a narrow dark seam between them, positioned in **world space** so perspective compresses them as a wall recedes. **A flat colour gives the eye no scale; evenly spaced marks that converge do.** The same trick as the face shading, one level finer.

Computed once per column, so it costs a shift and a compare.

And a cost decision stated as one:

Vertical only, and that is a cost decision rather than an aesthetic one: horizontal courses would need per-pixel work inside the compose loop, which is the one place in this renderer where a cost multiplies by 168.

25.7 Navigation, and per-cell decisions

The camera's wander was rewritten from reactive probing to a heading chosen once per cell:

no amount of probe tuning could traverse a maze — in corridors one cell wide a look-ahead probe is blocked almost always, so the camera turned continuously and never

committed to a direction. **Distinct cells visited in 20 s went from 4, pacing in one pocket, to 12 across the map.**

The tuning constants carry their own history, which is where two "runs into walls" reports are recorded:

```
/* ---- walk and steer -----
*
* MOVE_SHIFT distance per tick, as a shift of the unit direction vector.
*             6 is 1/64 of a cell per tick, so about 1.5 cells a second once
*             MOVE_CATCHUP_MAX stopped throwing most of the ticks away.
* LOOK_SHIFT how far ahead a wall triggers a turn. 0 is a FULL cell – half a
*             cell was not enough warning at this speed, which is what "runs
*             into walls before turning" meant the second time.
* STOP_SHIFT how far ahead a wall stops the walk. 3 is an eighth of a cell,
*             so the camera keeps advancing through a turn and only halts when
*             it is genuinely about to embed.
* TURN_UNITS angle per tick while a turn is wanted, out of 256 for a full
*             circle. 1 is 1.4 degrees – a sweep, where the 3 it replaced was
*             a 4.2-degree jerk applied only on contact. */
#define MOVE_SHIFT 6
#define LOOK_SHIFT 0
#define STOP_SHIFT 3
#define TURN_UNITS 1
```

Movement per tick, not per frame — and a guard that fired constantly

```
/* Ticks of movement a single frame may apply.
*
* This was 4, and it was the reason the camera crawled AND turned late. At
* 9 fps about eleven ticks pass between frames, so the clamp discarded two
* thirds of both the walking and the turning, every frame – a stall guard that
* fired constantly during normal running.
*
* 20 is 200 ms, longer than any gap between frames the renderer produces, so it
* now only bites on a genuine stall. The guard is still needed: without it, a
* pause of a second would apply a second of movement in one step and walk
* straight through a wall, because the collision probe only checks the
* destination of each step and not the path to it. */
#define MOVE_CATCHUP_MAX 20u
```

Three things in one comment: what the guard is for, why the old value was wrong, and why removing it entirely would be worse. **"A stall guard that fired constantly during normal running"** is a category of bug worth naming — a limit set for an exceptional case that turns out to bind in the ordinary one.

Movement is decoupled from frame rate:

```
for (uint32_t step = 0; step < elapsed; step++) {
    navigate_step();
}
```

which is the fix for Chapter 24 §24.9's actual cause: *"the relay layout raised the frame rate, movement was per-frame at the time, and the camera outran its turn probe and*

buried itself in a wall."

25.8 Where the frame goes

```
/* Split the frame cost three ways: ray marching, column composition, and the
 * SPI transfer. Guessing which dominates has been wrong often enough in this
 * project that it is cheaper to measure. */
static uint32_t g_us_march, g_us_compose, g_us_blit;
```

march	2.6 ms	one ray per screen column
compose	13.9 ms	240 x 168 pixels into DRAM
blit	41.9 ms	one 80,640-byte window
		~9.1 fps

The frame is bus-bound: 72% of it is pushing pixels down SPI at 40 MHz. So detail is cheap here and pixels are expensive, and that shapes what is worth adding.

Three separate timers rather than one total, and the justification — *"guessing which dominates has been wrong often enough in this project that it is cheaper to measure"* — is the project's own record cited as evidence.

25.9 The framebuffer, twice measured

Covered in Chapter 18 §18.7 and worth restating from this side because the switch lives here.

```
/* NULL when the direct path is in use. */
static uint16_t *g_fb;
```

- **First measurement:** fb off 126,650 μ s, fb on 131,411 μ s. No difference. 80,640 B for nothing. Defaulted **off**.
- **Both measurements were contaminated** — by a tick that stalled for up to 183 ms (Chapter 7 §7.9) and by draw-lock contention that dominated the frame (Chapter 11 §11.9).
- **Second measurement, both faults fixed:** one window per frame beats 240. Defaulted **on**.

§5.7's *conclusion* was wrong; its *method* was right, and is why the error was findable — a switch that can be flipped is a claim that can be rechecked.

The framebuffer's existence also enables the overlay of Chapter 24 §24.7: the close button is stamped into it so the frame and the button arrive in one transfer. With fb off there is nothing to stamp into, which is why that path flickers.

25.10 Two accessor names, one of which caused a panic

From Chapter 27 §27.8's tally of instruments that lied:

`raycast_framebuffer()`. Returns a boolean. Named like an accessor. Panicked a diagnostic that trusted the name.

The diagnostic used its return value as a pointer and stored through address `0x00000001`:

(The first attempt at this panicked, because `raycast_framebuffer()` returns a *boolean* and the test used it as a pointer, storing through address `0x00000001`. `raycast_fb_ptr()` now exists so nobody repeats that. **It was a fast and educational panic.**)

Two functions now, with unambiguous names. The lesson is about naming rather than about pointers: an accessor named for a *thing* should return the thing.

25.11 `dfreeze`, and the measurement that finally said something

The renderer's most useful diagnostic is one that stops it.

```
/* Stops every drawer in the display task, leaving whatever is on the panel and
 * in the framebuffer exactly as it stands.
 *
 * This is a measurement tool. The raycaster rewrites the entire framebuffer
 * every frame as the camera moves, so comparing the buffer before and after
 * some event always differs for innocent reasons. With the renderer frozen,
 * ANY change to the buffer was made by something else -- which is precisely
 * the open question about why launching a program repairs a garbled view. */
volatile int g_display_frozen = 0;
```

Chapter 27 §27.10 is the experiment it enabled, and the result is the first positive finding in a nine-theory investigation:

BEFORE launch	equal-to-first=25088	rows 67e0 67e0 67e0 67e0
CONTROL	equal-to-first=25088	rows 67e0 67e0 67e0 67e0
AFTER launch	equal-to-first=25088	rows 67e0 67e0 67e0 67e0

Launching a program does not alter one sampled byte of the framebuffer.

with the control row doing the real work:

The **control** row is the load-bearing one. Taken with nothing done in between, it proves the freeze holds the buffer still and the sampler is stable — without it, "no change" would be indistinguishable from a broken measurement.

A control group measured on the *same* instrument in the *same* run, which is the same discipline as the ADC's four touch pins in Chapter 22 §22.13.

25.12 Metrics

Quantity	Value
Map	16 × 16 cells, walls and corridors
Fixed point	16.16

Quantity	Value
March step	1/8 cell, MAX_STEPS 160 (20 cells)
Field of view	60° (PLANE_SCALE = tan 30° in 16.16)
Rays per frame	240, one per column
View	240 × 224
Framebuffer	80,640 B, on by default
Frame cost, march / compose / blit	2.6 / 13.9 / 41.9 ms
Frame rate	~9.1 fps
Share of frame on the bus	72%
Cost of face shading and wall seams	unmeasurable — 9.1 fps before and after
Distinct cells visited in 20 s, before / after navigation rewrite	4 / 12
RAY_COLW change, predicted / actual cost	0.2 ms / ~9 ms
Overlay cost per frame	324 pixels

25.13 What this does not establish

- **Wall texture is vertical only.** No horizontal courses, because those need per-pixel work where costs multiply by 168.
- **Face shading is by orientation only.** No light source, no falloff across a face. "It is a depth cue, not lighting."
- **The corner case in face recovery is arbitrary.** When a march step crosses both boundaries at once, X is chosen because something must be. Nothing measures how often that happens or whether it is visible.
- **No sprites, no entities, no floor or ceiling texture.**
- **No player control.** The camera wanders; touch steers left/right only, and did so backwards for three months (Chapter 19 §19.8).
- **No damage tracking.** Every frame repaints every pixel unconditionally, which is why the launcher can idle at zero SPI bytes and this cannot.
- **Nothing measures whether the shading is legible.** Judged by one person on one panel.
- **The startup glitch is open.** Chapter 27 §27.10 — a view that is garbled for ten to thirty seconds after opening, repaired by launching an unrelated program, with a mechanism proposed and untested.

Next: the two applications that take text from a person.

26. The Note Pad and the Shell on the Panel

Sources: docs/UM-NATOS-022-notes.md, docs/UM-NATOS-026-onscreen-shell.md Code:
kernel/notes.c, kernel/notes.h, kernel/messages.c, kernel/term.c, kernel/term.h,
kernel/shell.c, kernel/uart.c

Part A — The Note Pad

26.1 The first thing that takes text and keeps it

The first thing in this project that takes **text** from a person and keeps it. Everything before it read a tap or a serial line, and a serial line needs a computer attached — which is the thing the launcher exists to avoid needing.

A message is written on a multi-tap keypad, saved to flash, and read back after a power cycle. **That last part is the whole claim.**

26.2 Why it is native code, and why that is not a win

```
* Every other application is VM bytecode confined to a 26-row strip
* (UM-NATOS-016 §2). A keyboard does not fit in 26 rows, and the VM has no
* syscall that returns a keypress. This owns the launcher's region instead, the
* same way the 3D view does, and is native code for the same reason the
* launcher is: it is part of the interface rather than something running inside
* it.
*
* That is a real limitation and not a design win. It means text entry is a
* kernel feature rather than a service applications can ask for. Giving the VM
* a keyboard syscall is the version that would let any program read text, and
* it is not built.
```

The report is blunter:

text entry is a kernel feature rather than a service applications can request. Any program wanting text input cannot have it.

That is recorded here and in `notes.h` because it is the kind of limitation that becomes invisible once the feature works: the note pad takes text, so text entry appears solved, and **it is solved for exactly one program.**

Recording a limitation *at the point where a reader would assume it was addressed* is a habit worth naming. Chapter 30 collects every instance.

26.3 The keypad is a workaround, and saying so kept it on the record

The layout that failed

The first version was QWERTY: three letter rows, keys **24 × 26**.

It was close to unusable. Tapping `e` produced `w` — one key to the left, consistently, for every key. Reported as *"quite a task to write anything legible"*, which is an accurate description of a keyboard whose letters are reliably wrong.

What was actually broken, and the correction to that diagnosis

The original diagnosis was that the touch mapping read "systematically about 24 px low on X". That was wrong, and the report corrects itself in place:

What that paragraph got wrong. The cause is right; the characterisation is not. "Reads systematically about 24 px low on X" describes a constant offset, and the fault was a **magnification** — the narrow range is a divisor, so the error is proportional to distance from centre and changes sign across the screen (-29 px at the left edge, +12 px at the right).

The number 24 was real. It was measured in one place, on keys near the middle, and then written down as though it were a property of the panel rather than of where it happened to be measured. **A single sample of a position-dependent quantity looks exactly like a constant.**

It also explains something this document treated as unremarkable: **no adjustment of the constants ever helped, because there is no offset that corrects a scale error. That should have been the clue.**

"No adjustment ever helped" is a diagnostic signature in its own right: a fix that moves the fault around rather than removing it is evidence about the *shape* of the fault.

Why 80 px keys work anyway

Twelve keys, 3×4 , each 80×26 — a third of the panel wide. An 80 px key absorbs a 24 px error. A 24 px key is destroyed by it.

The launcher's icon cells are also 80 px and were never affected, which is why this fault survived unnoticed until something needed fine positioning.

The paragraph that kept a fault on the record

This is the most valuable thing in the report:

The keypad is tolerant of the defect, not a correction of it. Those are different, and calling the first one a fix is how a known fault becomes folklore. Anything finer than 80 px will hit it again.

That paragraph is the reason this section survived. **Because the keypad was never called a fix, the fault stayed on the record as unresolved, and calibrating it properly stayed on the list instead of quietly becoming the way things are. A workaround that is honestly labelled is a debt; one that is called a solution is a defect with good manners.**

The fault *was* subsequently corrected (Chapter 19 §19.10), and the keypad stayed — now for a different reason, which the report also insists on recording:

Since correction, 80 px keys are no longer load-bearing. They are kept because multi-tap is the point of this app, not because a smaller key cannot be hit — and that difference is now testable rather than assumed.

It is now chosen — the same keypad, held for a different reason, **which is worth writing down because a decision whose original justification has expired is one nobody re-examines.**

26.4 Multi-tap details that matter in use

Four, each closing a specific usability failure:

- **The live key is drawn back-lit.** Without it, multi-tap is guesswork about whether a press registered — and the press that "did not register" is usually one that did, replacing the letter you wanted.
- **An 800 ms timeout commits.** This is how two letters from one key are typed.
- **Cycling wraps** rather than sticking at the end of a sequence.
- **One press, one action,** latched on the first sample. The last sample before release is the one a resistive panel gets wrong, and typing the *wrong* letter is worse than missing a press.

The first of those is what Chapter 23 exists to reinforce: the audio click was built specifically because *"the press which did not register is usually one that did"*.

notes.h restates the latching rule at the interface:

```
/* Feeds a touch sample. Acts on the FIRST sample of a press, for the reason in
 * UM-NATOS-021 §4.2: the last sample before release is the one a resistive
 * panel gets wrong, and a keyboard that types the wrong letter is worse than
 * one that misses a press. */
void notes_touch(uint32_t x, uint32_t y, int down);
```

26.5 Storage

A separate sector, deliberately

Messages live in the flash sector **after** the kernel's boot record, not in it.

A flash write is erase-then-write over a whole 4 KB sector. Sharing one would mean rewriting the boot counter on every save, and losing every message if a save were interrupted while the counter was the thing being rewritten.

Separate sectors can only damage themselves.

Eight messages of 160 characters

160 is the SMS limit. It suits what this is imitating, and it bounds the store: eight of them plus a header is under 1.4 KB, comfortably inside one sector.

When full, the oldest is dropped. A note pad that refuses to save is worse than one that forgets what you wrote first.

Validation follows the boot record's rule — magic, version, count and checksum, and on any failure the store resets to empty, "so a first run and a corrupt sector behave identically rather than producing a half-believed store".

The buffer size is chosen against the screen rather than against memory:

```
/* Longest note. Deliberately small: this lives in .bss, and the panel can show
 * about seven lines of forty characters at once, so a buffer much larger than
 * the screen would be text the user cannot see or reach. */
#define NOTES_MAX 256u
```

26.6 Reading, and states you can get out of

The **header bar is the navigation control**. Tapping it swaps `WRITE` and `INBOX`, the way a phone with three buttons did it, and it costs no key.

In the inbox, the left half of the text area pages back and the right half forward, with a position indicator so paging has a place rather than being an endless cycle.

Letter keys do nothing while reading. A stray tap must not silently begin composing over a message being read. `del`, `space` and `save` still act, so **there is always a way out of a state**.

26.7 Verification, and the reflash that matters

```
write a message on the panel
save                               -> header shows "saved", box clears
power cycle
reflash
boot                               -> messages : loaded 1 saved
open notes, tap header             -> message reads back in the inbox
```

The reflash matters: it proves the message is in flash rather than in RAM that happened to survive a warm reset.

A warm reset does not necessarily clear DRAM, so a power cycle alone is a weaker test than it looks. Reflashing rewrites the image and cannot leave application state behind.

And a small correctness property:

The box clears **only on success**. A save that failed must not look like one that worked by leaving an empty box behind.

26.8 The close button nobody drew

Worth its own section because it is a fix's blast radius landing on a view that did not exist when the fix was written.

Chapter 24 §24.7 stopped the 3D view's close button flickering by stamping it into the raycaster's framebuffer, and made `desktop_chrome()` skip drawing it whenever the framebuffer is on. Correct for the only full-region view that then existed.

The note pad has no framebuffer to stamp into. It fell through both paths: **present, hit-testable, and drawn by nobody**. The button worked — tapping the corner returned to

| the launcher — and was invisible.

Ownership is now explicit for all three cases:

view	who draws the close button
3D, framebuffer on	the raycaster stamps it into the buffer
3D, framebuffer off	<code>desktop_chrome()</code> , and it flickers
note pad	the app, in its own header

| The note pad draws it at exactly the coordinates `desktop_chrome_touch()` already tests, so the drawing and the hit test sit in one file and cannot drift. That was an open gap in UM-NATOS-021 §9 and is now **half closed** — half, because nothing *enforces* the agreement, it is merely adjacent.

Half-closing a gap and saying which half is the honest form.

Part B — The Shell on the Panel

26.9 One command set, not two

The shell has always been a front end that happened to read a serial port. This gives it a second front end and changes nothing behind it.

The consequence is larger than the feature: **the device no longer needs a computer attached to be inspected.** `mem`, `ps`, `stacks`, `adc`, `i2c`, `intr`, `calshow` and `kill` were reachable only from a host over USB.

The shortcut not taken

The easy version is an on-screen menu of the popular commands — a grid of buttons labelled `mem`, `ps`, `adc`, each calling the function directly. It needs no keyboard, no line buffer and no output capture, and it would have taken an hour.

It would also have created **a second command set**. Every command added to the real shell would be absent from the panel until someone remembered; every command whose output changed would have a stale twin. **The two would diverge in the direction of whichever was used more, and nothing would report the drift.**

So instead:

a line typed on the glass goes through `shell_run_line()` into the same `execute()` a serial line reaches — same parsing, same splitting, same output, same unknown-command message. There is no on-panel command table, because there is no on-panel command set. **help on the glass lists what help lists, because it is help.**

This is the same "delete the second mechanism" principle as Chapter 8 §8.2 (one way for a task to exist), Chapter 8 §8.3 (one switching mechanism), and Chapter 16 §16.3 (one `retire()` path).

The one concession, and why it is also a safety check

```
/* Runs one command line from somewhere other than the UART.
 *
 * execute() splits its argument in place, so a caller's string cannot be passed
 * straight through – it would be modified, and a string literal would fault.
 * The copy is also the length check: a line longer than the shell's own buffer
 * is refused rather than truncated into a different command.
 *
 * This is the whole interface the on-screen shell needs. It deliberately does
 * not bypass execute(): a command typed on the panel takes exactly the same
 * path, with the same parsing and the same output, as one typed over serial. */
void shell_run_line(const char *line)
{
    static char buf[LINE_MAX];
    uint32_t i = 0;

    while (line[i] && i < LINE_MAX - 1u) {
```

```

        buf[i] = line[i];
        i++;
    }
    if (line[i]) {
        uart_puts("    line too long\n");
        return;
    }
    buf[i] = 0;
    execute(buf);
}

```

a truncated command is a different command, and `kill 12` truncated to `kill 1` kills something.

26.10 Output capture by teeing

The shell writes to the UART, because that is where a shell writes. Rather than teach it a second destination, `uart_putc()` grew an optional tee and the app installs itself as one.

```

/* Optional second destination for everything printed.
 *
 * Exists so the on-screen shell can show a command's output on the panel. The
 * shell writes to the UART because that is where a shell writes; teeing here
 * means it does not need to know it is being watched, and no output path grows
 * a second case.
 *
 * Installed only around a single command's execution, never left on: the
 * reporter task prints several lines a second, and a tee left installed would
 * fill the screen buffer with telemetry the user did not ask for. */
static uart_tee_fn g_tee;

void uart_set_tee(uart_tee_fn fn) { g_tee = fn; }

void uart_putc(char c)
{
    if (g_tee) {
        g_tee(c);
    }
    /* ... */
}

```

The tee is installed for exactly the duration of one command and removed immediately. Left on, it would capture the reporter task's telemetry — several lines a second — and the pane would fill with output nobody asked for within about two seconds. `execute()` holds the console lock for its whole run, so while the tee is set the only writer is the command itself.

The console lock (Chapter 11 §11.8) turns out to do double duty: it was added for interleaving, and it is what makes the tee's exclusivity hold.

Three constraints on the capture function, all consequences of where it runs:

The capture function runs inside `uart_putc()`. It must not print, must not lock, and must not be slow, since it is called once per character of output.

26.11 Layout, and three assertions

```
#define HDR_H      22u
#define KEY_ROWS   4u
#define KEY_COLS   3u
#define KEY_H      26u
#define KEY_W      (DISP_W / KEY_COLS)      /* 80 */
#define KB_Y       (DESK_H - KEY_ROWS * KEY_H) /* 224 - 104 = 120 */

#define INPUT_H     11u
#define INPUT_Y     (KB_Y - INPUT_H)         /* 109 */
#define OUT_Y       HDR_H
#define OUT_H       (INPUT_Y - OUT_Y)        /* 87 */

#define CHAR_W      6u
#define LINE_H      9u
#define TERM_COLS   39u                      /* 240/6, less a margin */
#define TERM_ROWS   (OUT_H / LINE_H)        /* 9 */

_Static_assert(KB_Y + KEY_ROWS * KEY_H == DESK_H,
               "keyboard must end exactly at the region boundary");
_Static_assert(OUT_Y + OUT_H == INPUT_Y, "output pane must meet the input line");
_Static_assert(TERM_ROWS >= 6u, "an output pane under six lines is not worth
having");
```

band	rows	
header	22	title and the close button
output	87	9 lines at 9 px
input	11	prompt, line, block cursor
keyboard	104	12 keys at 80 × 26

The keyboard is the fixed cost — four rows of 26 px is 104 rows, nearly half the region — and everything else fits in what is left.

The third assertion is unusual and worth noting: it encodes a *quality* requirement, not a geometric one. "An output pane under six lines is not worth having" refuses a build that would compile and produce a cramped, useless terminal.

That is the same discipline as UM-NATOS-021 §7, arrived at the same way — by having previously broken a layout and not noticed.

Not the note pad's look, deliberately

```
/* A terminal, deliberately not the note pad's LCD. Two full-region native apps
 * that look alike are two apps a user has to read the header to tell apart. */
#define TRM_BG      0x0000u      /* black */
#define TRM_FG      0x07E0u      /* phosphor green */
#define TRM_DIM     0x03E0u      /* half-bright, key faces */
#define TRM_KEY     0x2124u      /* key body */
```


26.12 The output pane is a ring of lines, not a byte stream

The output pane is a **ring of fixed-width lines**, not a byte stream. A stream would need re-wrapping on every draw, and the wrap is already decided at capture time by where the newlines fall.

Scrollbar

Nine visible lines was the wrong size for the two commands most worth running. `help` produces about thirty lines and `adc` about fifteen, so the pane showed the end of a list whose beginning was the part being asked for.

The ring now holds **48 lines** — a little over three screens of `help` — while still showing nine. The bound is deliberate rather than generous: **this is a fixed allocation in a kernel with no paging, so scrollbar that grew with output would be an unbounded allocation driven by whatever the user typed**. 48 lines costs under 2 KB of `.bss`.

Scrolling by tap, in half-screens

Tapping the output pane scrolls it: upper half back, lower half forward, half a screen per tap. A line per tap would be nine taps to move one screen, which on a panel this slow is not a control but a chore; half-screens also leave two lines of overlap so the reader need not remember what the last line was.

A scrollbar appears on the right only when there is something to scroll. It is the affordance as much as the indicator — nothing else on screen says the pane can be scrolled, and **a gesture with no visible cue is a feature only its author knows about**.

Addressed backwards from the write cursor

Lines are addressed by **distance back from the write cursor**, never forward from an origin. A ring has no origin once it has wrapped, and counting forward from one means recomputing where it moved to on every draw.

And the arithmetic was tested rather than reasoned:

The arithmetic was checked by simulating 70 lines through the 48-line ring and comparing the window against a plain list at all 40 scroll positions.

That is the closest thing in this project to a unit test — an exhaustive check of a pure function against a reference implementation. Chapter 31 argues there should be more of them.

It does not follow while you read

Submitting a command snaps to the newest; scrolling stays put while reading. Output only ever arrives because a command was submitted, so a pane that jumped mid-read would be worse than one that does not follow.

26.13 The keypad is a second copy, and that is recorded

```
* This is a SECOND copy of the note pad's cycling logic. That is duplication
* and is recorded as such rather than pretended away: factoring it out means
* changing a working app to serve a new one, which is a worse trade tonight
* than two copies with a comment. If a third consumer appears, factor it then.
*
* The bottom row differs from the note pad's. There is no `save`; the third key
* is `run`, because a shell's terminating key submits.
```

The report's version adds the reason the trade is asymmetric:

Factoring it into a shared widget means modifying a working application to serve a new one, and **the note pad is the app holding the user's saved messages**. Two copies with a comment was judged the better trade against changing working code late at night; **if a third consumer appears, that is when it should be extracted**, because at three the duplication stops being a shortcut and becomes a pattern.

A stated threshold for when to pay a debt is more useful than an intention to pay it.

And the key size, held for a new reason:

The keys are 80 × 26, matching the note pad. There, that size was forced. That error has since been corrected, so here the size is **a choice rather than a workaround**, and the reason is different: shell commands are short lowercase words, and multi-tap types those adequately.

26.14 Verification

```
double-tap the shell icon
type a command on the keypad, press run
-> the command echoes after a "> " prompt
-> its output appears in the pane
-> the same output appears on the serial port, unchanged
close returns to the launcher
```

The serial half of that matters: it confirms the tee **copies** output rather than diverting it, so attaching a host does not change what the panel shows.

A one-line check that distinguishes a tee from a redirect — the two would look identical from the panel alone.

26.15 Metrics

Note pad

Quantity	Value
Keys	12, 80 × 26
Key size that failed	24 × 26
Touch X error absorbed	~24 px near centre, up to 29 px at an edge

Quantity	Value
Calibration status	corrected, Chapter 19 §19.10
Multi-tap commit timeout	800 ms
Messages stored	8 × 160 characters
Store size	~1.4 KB in one 4 KB sector
Compose buffer	256 B in .bss
Flash sectors used	1, separate from the boot record

On-screen shell

Quantity	Value
Output pane	9 lines × 39 columns visible
Scrollback ring	48 lines, under 2 KB of .bss
Input line	40 characters, refused beyond
Keys	12, 80 × 26
Multi-tap commit	80 ticks (~800 ms)
Commands reachable without a host	all 25
Command sets	1
Copies of the keypad in the tree	2

26.16 What these do not establish

Note pad. The calibration is corrected but still linear, run once on one unit; nothing measures panel non-linearity between target positions, so a UI element much finer than the keys "has not been shown to work — only stopped being impossible". The full store has never been exercised, so the oldest-dropped path has never run outside reasoning. Corrupt-sector rejection is reasoned, not tested. A save interrupted by power loss is untested. No editing beyond `del`. No timestamps, "because the kernel has no clock that survives a power cycle — only a tick counter that restarts at zero".

Text entry is not available to applications.

On-screen shell. 48 lines is still a bound and what leaves is gone. Scrolling costs presses — nine taps to reach the top of a full ring. No history, no editing beyond `del`, no recall. Arguments are typeable but tedious. Uppercase is impossible and nothing enforces that no command needs it. The tee is single-slot and not reentrant. And one that is honest about the difference between "never happens" and "cannot happen":

Output during a command from ANOTHER task is captured too, if that task prints without taking the console lock. Nothing in the tree does, so this has never happened — **which is not the same as being prevented.**

Next: the largest single piece of work in the project, and the one that required the kernel's calling convention to be bridged rather than chosen.

27. WiFi: Mixed ABIs, a Vendor Blob, and Frames Off the Air

Sources: docs/UM-NATOS-028-wifi-touch-and-the-chrome-column.md Code:
kernel/window.S, kernel/window.h, kernel/wifi_osi_impl.c, kernel/wifimac.c,
kernel/phyinit.c, kernel/efuse.c, vendor/windowed/, vendor/phy/

27.1 What happened

The report opens with an unusually honest summary of a working day:

This report covers one long day that went: *let us finish the WiFi MAC → hold on, why is task_sleep not sleeping → the touchscreen has gone strange → and now the 3D view is torn → ah, it was a rectangle of black paint the whole time.*

The outcome:

nat-os now **receives 802.11 frames**, decodes beacons, names the networks around it, and keeps doing so continuously while rendering a 3D view. It also **transmits** — in the sense that the MAC accepts frames and reports them complete — but nothing on Earth has yet heard one, which is a distinction §27.4 will insist on at some length.

This is the largest single piece of work in the project and the one that required the calling convention of Chapter 2 to be *bridged* rather than merely chosen.

27.2 The OSI table gets bodies

wifi_osi_funcs_t is Espressif's 116-entry function table: the entire contract between the MAC blob and whatever operating system hosts it.

Previously nat-os provided all 116 entries with the correct types, in the correct order, containing nothing whatsoever. **It linked beautifully and would have collapsed the instant anything called it.**

Thirty-nine entries now do real work: semaphores, mutexes, queues, event groups, software timers, the malloc family, delays, tick conversion and a PRNG.

The shape matters more than the count

The table **must** be windowed ABI, because libpp calls it. The kernel is call0 and always will be. So every windowed entry does no work at all — it forwards through w2c_callN() in window.S into wifi_osi_impl.c, where the heap and scheduler actually live:

```
libpp (windowed) → wifi_osi.c entry (windowed, does nothing)
                  → w2c_callN (window.S)
```

```
→ wifi_osi_impl.c (call0: heap, scheduler, tick)
```

```
* call0, deliberately. The OSI table itself must be windowed because libpp
* calls it, but the table's only job is to forward: the work happens here,
* where the kernel's heap, scheduler and tick can be used directly. Each
* windowed entry crosses back through w2c_callN() in window.S.
*
* That split is the design. Writing these bodies in the windowed file would
* mean either duplicating the kernel's primitives or calling into them across
* a boundary that does not permit it – the fault that put IllegalInstruction
* at 0x4008a810 the first time it was tried.
```

Three design notes

Fixed pools, not the heap:

```
* ---- static pools, not the heap -----
*
* Semaphores, queues, event groups and timers come from fixed arrays. The blob
* creates its objects during init and keeps them, so a pool costs a known
* amount of DRAM and cannot fragment, cannot fail late, and cannot leak.
```

```
#define OSI_SEM_MAX      12u
#define OSI_QUEUE_MAX    8u
#define OSI_EVT_MAX      4u
#define OSI_TIMER_MAX    12u
#define OSI_QUEUE_BYTES 512u      /* item storage per queue */
```

Handles are tagged indices, checked on every use:

```
* A handle is a tagged index rather than a pointer, so a stale or foreign
* handle is detected instead of becoming a wild write. The blob is not code
* this kernel can inspect, and it is the only caller.
*/

/* Tagged handles. The tag is checked on every use. */
#define H_TAG      0x05100000u
#define H_MASK     0xFFFF0000u
#define H_MAKE(i)  ((void *) (uintptr_t) (H_TAG | (i)))
#define H_INDEX(h) (((uintptr_t) (h)) & 0xFFFFu)
#define H_OK(h, n) (((uintptr_t) (h)) & H_MASK) == H_TAG && H_INDEX(h) < (n))
```

The same argument as the heap's magic words (Chapter 10 §10.3), applied to an untrusted caller rather than to a buggy one.

Blocking uses `task_sleep`, not `task_block`:

even for an infinite wait. A lost wake-up on `task_block` never runs again. This costs a tick of latency on a missed wake; the caller re-checks anyway. **Deliberate pessimism.**

That is Chapter 9 §9.3's design applied at the point it was built for: a wait that degrades to polling rather than to deafness.

Waiters are a bitmask of task ids, which `TASK_MAX = 12` makes trivially cheap:

```
/* Waiters are a bitmask of task ids. TASK_MAX is 12, so one word covers every
 * task and "wake everyone waiting" is a loop over set bits. */
```

Verified by driving the table the way libpp will

`ositest` drives the table through its function pointers exactly as libpp will, so each check crosses windowed→call0 and back. All six pass, pools return to zero, nothing leaks.

Calling *through the function pointers* rather than calling the implementations directly is what makes this a test of the bridge rather than of the bodies.

27.3 The MAC wakes up, and a fourth read-back failure answered differently

open-mac's entire MAC initialisation is one masked register write:

```
MAC_CTRL_REG = MAC_CTRL_REG & 0xffffe800;
```

The work was proving it did anything. This kernel has been caught **three separate times** by peripherals whose registers read back perfectly while the hardware sat completely dead — LEDC behind `DPORT_PERIP_CLK_EN` bit 11 being the clearest. **A readback is not evidence.**

So instead of reading registers, look for *motion*:

```
/* Reads the MAC register window twice and reports how many words differ.
 *
 * This is the evidence that the peripheral is RUNNING rather than merely
 * readable. A clock-gated block returns a stable value -- usually zero -- from
 * every address; a live MAC has free-running counters in it, and those show up
 * as words that change between two passes with nothing driving them. */
uint32_t wifimac_liveness(uint32_t *first);
```

stage	moving words
cold boot	0
after phyinit	6
after macinit	13–15

Three measurements, monotonically increasing, on a quantity that *cannot* be faked by a write that went nowhere. This is the best answer in the book to the read-back problem, and it is qualitatively different from the previous three: it does not ask the hardware a question at all.

27.4 Finding the TSF timer by behaviour

Espressif publishes nothing about this register map, and every address in circulation is reverse-engineered. Rather than assert that some offset is the TSF timer, the scan reports the *rate* of every mover:

```
0x3ff73c00 1000 kHz <- stable across every run
0x3ff73c14 1054 -> 1102 kHz
0x3ff73c18 1055 -> 1107 kHz
0x3ff73dd0 1063 -> 1114 kHz
```

0x3ff73c00 is the only word reporting **exactly** 1000 kHz every time; everything else drifts.

Confirmed against an independent clock:

```
tsf advanced 622458 over 622 ms of cpu time -> 1000 kHz
tsf advanced 508686 over 508 ms of cpu time -> 1001 kHz
```

Agreement to 0.1% over half a second is not something a misread address produces. **nat-os now has a 1 MHz timebase**, which it did not have before — the kernel tick is 10 ms.

The header records the identification method rather than just the address:

```
/* The 802.11 TSF timer, identified by behaviour: 0x3ff73c00 is the one word in
 * the MAC window that advances at exactly 1 MHz across repeated samples. */
#define WIFIMAC_TSF_REG 0x3FF73C00u

/* Counts TSF ticks against the kernel's own cycle counter over `ms`
 * milliseconds. Returns the TSF delta; `cycles` receives the cycles elapsed.
 * A genuine 1 MHz counter yields delta ~= ms * 1000 -- a long-interval match
 * no drifting or noisy register can produce by accident. */
uint32_t wifimac_tsf_check(uint32_t ms, uint32_t *cycles);
```

Two clocks with no connection to each other agreeing to 0.1% is the strongest form of evidence available without documentation.

27.5 The MAC address, and a better oracle

A six-byte address looks equally plausible in any byte order, so the decode needed checking against something.

The first attempt compared the top three bytes against a list of Espressif OUIs and reported **failure** for 5c:01:3b:50:3f:64.

The decode was right. The **list** was wrong. That test was measuring a recollection of IEEE registrations, not the hardware.

efuse.c records the whole thing:

```
* The first attempt checked the top three bytes against a list of Espressif
* OUIs, on the reasoning that an OUI is a published, assigned value. It
* reported failure on this board: 5c:01:3b:50:3f:64, no match. The decode was
* correct and the LIST was wrong -- an OUI assigned after the list was written,
* or to the module maker rather than to Espressif. The test was measuring my
* recollection of IEEE registrations, not the hardware.
*
* eFuse stores a CRC8 of the address in the same block. Checking against that
* tests the decode against data on the chip itself: if the bytes are read in
```



```

* the wrong order the CRC cannot match, and no external knowledge is involved.
* Confirmed here -- 5c:01:3b:50:3f:64 gives 0x08, which is the stored byte,
* while the reversed order gives 0x8f.
*
* The lesson is worth keeping: the better oracle was already on the chip.

```

Note also what the file says about the *rest* of the WiFi work by contrast:

```

* Unlike almost everything else in the WiFi path, this is DOCUMENTED. The eFuse
* controller is in the technical reference manual, the block 0 layout is
* public, and the decode below is the same one ESP-IDF performs. Nothing here
* is reverse-engineered.

```

Marking which parts of a subsystem rest on documentation and which on reverse-engineering is exactly the evidence grading of Chapter 0b, applied at file scope.

27.6 The PHY, and a StoreProhibited that was an ABI requirement

`phyinit.c` is candid about the change in risk:

```

/* nat-os – bringing Espressif's PHY up. See vendor/phy/README.md.
*
* This is the first code in the project that touches the radio. Everything
* before it – the window handlers, the ROM calls, phy_version_print – was
* provably inert: it could not affect hardware even if it was wrong. This can.

```

What it needs:

```

* - the WiFi/BT common clock, ungated in DPORT. Without it the PHY's
*   registers are dead and it hangs waiting on a peripheral that is not
*   running – the same shape as the LEDC clock gate in UM-NATOS-027 §3.3,
*   which is the third time that pattern has appeared in this project.
* - 128 bytes of initialisation data. This is Espressif's own default table,
*   copied from phy_init_data.h; the 107 listed values are theirs and the
*   remaining 21 are the zeros C fills in.
* - a calibration buffer it writes into. Normally this is persisted to flash
*   and reused; here it is .bss, so every run pays for a full calibration.
*
* PHY_RF_CAL_FULL is chosen deliberately over PARTIAL or NONE: the other two
* expect calibration data from a previous run, and there has never been one.
*/

```

Three theories, and the two wrong ones were the useful part

The blocker was `chip_v7_set_chan_nomac` panicking with `StoreProhibited`.

1. Stack depth. The PHY got a private 6 KB stack.

The fault moved *forward* without going away — which looked like progress and was not. Priming the stack with a pattern and reporting its high-water mark from the panic handler settled it: **272 bytes of 6144 used**. Depth was never the problem, and **without that number the obvious next move was a bigger stack, which would also have failed**.

That instrument is in `window.h` with its purpose stated:

```

/* Fills the PHY stack with a pattern, and reports how much of it has since
 * been touched. The point is to settle a question rather than argue it: when a
 * PHY call dies inside a window-overflow spill, "the stack was too small" and
 * "something else is wrong" look identical from the fault address alone. If
 * the call consumes the whole stack the cause is depth; if it dies having used
 * a fraction of it, depth was never the problem. */
void    phy_stack_prime(void);
uint32_t phy_stack_used(void);

```

The *priming* is essential and its absence produced one of the seven lying instruments in §27.8: an unprimed `.bss` stack is all zeros, and the fill-pattern scan reads that as fully consumed — reporting 6144 of 6144 bytes used in a real panic, which "looked exactly like a stack exhaustion that had not happened".

2. Stale WINDOWSTART bits. Declaring exactly one live frame: no change.

3. The base frame was not a windowed frame. This was it.

The one store that fixed it

`_WindowOverflow8` — the handler in this very repository — spills `a4..a7` relative to the caller's stack pointer, fetched with:

```
132e    a0, a1, -12        /* a0 <- call[j-1]'s sp */
```

Every windowed frame must carry its caller's stack pointer at `sp - 12`. `ENTRY` does not write it; the *caller's prologue* does. `phy_stack_call` is `call0` code and wrote nothing there, so the handler read a fresh `.bss` zero and spilled to `0 - 32`.

And the reason the fault address was useless:

That also explains why the reported PC was never where the store was: the spill faults *inside* the overflow handler, which runs with `PS.EXCM` set, so the second fault vectors to the double-exception handler while `EPC1` still holds the instruction that caused the *original* overflow — the `call18`. **Hours were spent disassembling the wrong instruction.**

`PS.EXCM` again — Chapter 8's defect from a completely different angle.

The fix in `window.S`, with the whole analysis attached:

```

/* THE BASE FRAME MUST LOOK LIKE A WINDOWED ONE.
 *
 * This is what the StoreProhibited actually was, and it is an ABI
 * requirement that call0 code has no reason to know about.
 *
 * _WindowOverflow8 spills a4..a7 relative to the CALLER'S stack pointer,
 * which it fetches with `132e a0, a1, -12`. Every windowed frame is
 * required to hold its caller's sp in that slot; ENTRY does not write it,
 * the caller's prologue does. phy_stack_call is call0 and wrote nothing
 * there, so the handler loaded a fresh .bss zero and spilled to 0 - 32.
 * ...
 * Two earlier theories were wrong and are recorded because the measurements
 * that killed them were the useful part: it was not stack DEPTH -- the

```

```

* fault came 272 bytes into 6144 -- and it was not stale WINDOWSTART bits,
* since declaring a single live frame did not change it either. */
movi    a9, _phy_stack_top
addi    a10, a8, -12          /* s32i takes no negative offset */
s32i    a9, a10, 0

```

The fix is one store. Everything after it worked first try.

Two more things `phy_stack_call` must do

Mask interrupts, and not for atomicity:

```

/* Mask interrupts for the duration.
 *
 * Not for atomicity – to stop a context switch happening while windowed
 * frames are live. nat-os's level-3 handler saves a0..a15 and nothing else:
 * it does not save WINDOWBASE or WINDOWSTART and does not spill the window.
 * A switch in the middle of blob code therefore leaves another task running
 * with this one's frames still marked live in the register file. */
rsil    a9, 3

```

That is the cost of Chapter 8's minimal context frame, paid here rather than by enlarging every task switch in the system. A good trade: windowed excursions are rare and bounded.

Declare exactly one live frame, and deliberately not restore `WINDOWSTART`:

```

* WINDOWSTART is deliberately NOT restored afterwards. The bits describe
* frames whose registers this excursion has already overwritten, so
* putting them back would re-mark frames whose contents are gone. A call0
* kernel's correct steady state is one live frame, and that is what this
* leaves behind. */

```

27.7 It receives

```

802.11 frame: BEACON  len=348
bssid 44:25:38:19:0d:1a  ssid "TC7NR"

```

A real beacon from a real access point, on a kernel built `-mabi=call0` running Espressif's PHY blob through hand-written window handlers.

And then continuous reception, via descriptor recycling:

```

frames=372  recycled=374  networks=2
44:25:38:19:0d:1a  x199  "TC7NR"
38:88:71:2d:c1:cd   x53  "Verizon_S6QHX4"

```

With four buffers, 372 frames can only come from reuse.

A four-buffer pool and 372 frames is a *derived* proof of recycling — the same style of arithmetic check as Chapter 14 §14.8's instruction accounting.

It also kept receiving *through* a 3D rendering session — the radio and the display share nothing but the scheduler, and neither starved.

27.8 Transmit, and the difference between "sent" and "sent"

The MAC accepts frames and reports every one complete:

```
tx handed to hardware=178  completions reaped=178  forced=0
```

This report initially described a rising completion count as "the MAC saying the frame actually went out". That was an overclaim, and the user checked their phone for the beacon SSID, and it was not there.

The test that settles it

A **probe request**. A beacon is a statement and nothing has to answer it, so silence proves nothing. A broadcast probe request with a wildcard SSID obliges every access point in range to send a probe *response* addressed to this station — and one arriving is **unforgeable**, because a radio cannot hear itself and nobody sends to this MAC without having heard from it first.

```
sent 20 probe requests
completions reaped=20
frames addressed to us=0
```

Twenty frames the hardware called complete, zero answers from two access points loud enough that we receive them continuously. **A completion bit says the frame left the queue, not that it left the antenna.**

This is the best-designed experiment in the book. It converts an unfalsifiable claim ("did the frame transmit?") into a falsifiable one ("did anything answer?"), using a response that cannot be produced by any local fault.

Two causes eliminated by measurement

- **Transmit power.** `most_tpw` already reads `0x28` — 40 quarter-dBm, **10 dBm** — straight out of `register_chipv7_phy`. Never zero. `phy_set_most_tpw(78)` returns success and changes nothing.
- **The MAC hardware init chain.** `ic_mac_init`, `hal_init`, `ic_enable_rx` and `hal_mac_tsfr_reset` all run without a crash. Still zero answers.

Two pieces of good news from a failed hypothesis

The IRAM panic was unfounded — Chapter 4 §4.8's 2,459 bytes against a believed 48 KB. A constraint that had shaped decisions for weeks did not exist.

The blob runs.

This was the first time `nat-os` executed `libpp` at all, and the OSI table, the window handlers and the mixed-ABI bridges all held up under the code they were built for. **That was never certain.**

And an expensive lesson about linking

naming a symbol changes the link. Referencing `ram_tx_pwctrl_bg_init`, which was not previously linked, pulled fresh objects out of `libphy.a`, after which `register_chipv7_phy` died with `IllegalInstruction` inside `set_rx_gain_testchip_70` — a calibration function nobody had touched and which had worked for days.

Rule adopted: **reference only what open-mac references**, and check with `nm` that a symbol is already in the image before calling it.

The strongest untried lead

`periph_module_reset(0x19)`. `nat-os` has only ever *ungated* the WiFi peripheral, never *reset* it, and a MAC left in whatever state the ROM bootloader put it in would plausibly receive while refusing to transmit. **That asymmetry fits the symptom exactly.**

27.9 The `task_sleep` detour, which was not a detour

Covered in Chapter 9 §9.4. Summarised here because of where it was found:

Mid-WiFi, the TSF check reported $\sim 1\ \mu\text{s}$ for a requested 500 ms sleep. **Two independent clocks agreed on that**, so it was not a measurement artefact.

Two bugs, stacked: `timer_ticks()` counted ISR entries rather than time (217/s where 100 is correct), and `task_yield()` arms a switch rather than performing one (`task_sleep(50)` returning in 107 cycles).

Why it mattered *here* specifically:

every OSI queue and semaphore timeout is built on `task_sleep`, so anything the blob did with a timeout was quietly unreliable.

And the general form:

nothing showed a symptom. Sleeps ran short, timeouts ran loose, and every frame-rate figure this project has ever recorded was measured against a clock running fast by a load-dependent factor.

It was found because the WiFi work introduced the first *independent* clock the kernel had ever had. A second clock is the only thing that can catch a first one lying.

27.10 The touchscreen gets worse, twice

A background task outranking the user

The WiFi receive task was raised to `HIGH` to get beacons from 3 Hz to 9.4 Hz. The raycaster blit was checked, found unchanged, and the change declared a clean win.

The touch task is `NORMAL`. With **two** flat-out `HIGH` tasks instead of one it stopped being scheduled except when ageing rescued it — roughly every 300 ms. The panel went

from responsive to intermittent, **which no counter in this kernel reports** and which the user noticed within minutes.

And the merits, reassessed:

It was also a bad trade on the merits: beacon timing was prioritised over input latency, and **the beacons were not reaching the air anyway**, so the 9.4 Hz was entirely theoretical.

The useful measurement was the one that ruled out the obvious suspect:

`update_rx_chain()` spins on a hardware acknowledge and looked exactly like the cost; instrumenting it showed a worst-case wait of **one** iteration. It was never the radio, it was the scheduler.

A "restoration" that was not

Reverting the priority was not enough. The cause was an *earlier* change described in a commit message as a "restoration" — the `task_yield()` for `task_sleep(1)` substitution of Chapter 9 §9.4.

Fixing `task_sleep` had, with perfect irony, broken the thing that was accidentally relying on it being broken.

	before	after
touch samples per reporter interval	~15	~ 150

27.11 The 3D view, and four confident wrong answers

Then the 3D view started tearing, and the close button vanished.

Theory one: DMA had fallen back to the FIFO path. The reasoning was sound (Chapter 18 §18.9 is that reasoning, and it is genuinely a real defect). The counter said `dma=320/0`. **Zero timeouts. Ever.** The bound was raised anyway on principle, then restored when it fixed nothing.

Theory two: touch preempting the display mid-transfer. And here the method changed:

Instead of one flash per guess, both knobs were made runtime-tunable (`touchcfg <prio> <sleep>`), and all four combinations tested in a single build:

`NORMAL + yield 31399 31332 31394 HIGH + yield 31352 31398 31353 NORMAL + sleep1 31399 31346 31385 HIGH + sleep1 31424 31387 31438`

Identical. Touch scheduling does not move it. **This is the single best thing that happened during the display investigation, and it should have happened three rounds earlier.**

Theory three: applications drawing over the view. `ps` showed `ping` and `pong` running and drawing, with `drawskip` climbing. Killed all three applications. `drawskip` froze. No change on screen.

Theory four: the panel clock. The driver has documented since Chapter 18 that clocking this panel too fast puts visible noise on the glass *while every counter reports success* — "a suspiciously exact description of the symptom". A runtime clock selector was added and the panel dropped from 40 MHz to 10 MHz. Different. Still wrong.

The misleading evidence, and the user who was right

At this point the investigation had a genuinely misleading piece of evidence: the same commit rendered correctly earlier in the day and torn later, with no code change in between. That reads as hardware. A reasonable-sounding case was made for a marginal display flex and a sagging USB rail, and the user was asked to reseal connectors.

The user replied, in effect, *no, it is a bug in the code, because the X button is also missing.*

They were right. What differed between the two runs was not the hardware. It was which application slots happened to be occupied.

Third instance in this book of a human observation outranking the instruments.

The test that should have been first

A **static test pattern** through the raycaster's exact blit call: same buffer, same width, same stride, same contiguous path. Six vertical colour bars, plus a white block in the top-right corner where the close button goes.

This splits one question into two, and the two have completely different answers:

- If the pattern is sheared or torn → the **transport** is broken.
- If the pattern is clean but something is missing → the content is fine and something **overwrites it afterwards.**

Result: **clean bars, and a hole exactly where the white block had been written.**

The actual bug

`desktop_chrome()` repaints a right-hand column every frame, one strip per application slot — a name and a red X for running slots, and **solid black** for empty ones:

```
display_fill_rect(CHROME_X, y, APP_CHROME_W, APP_VIEW_H, COLOR_BLACK);
```

That column is documented as reserved *outside every application viewport*, which is entirely true — for applications, whose viewports are narrower.

The 3D view is not an application. `RAY_VIEW_W` is 240. `DISP_W` is 240. The view owns the whole width, including that column, and the chrome runs immediately after `raycast_frame()`, painting over the right edge of a view that had just drawn it.

One cause, every symptom:

- the tearing along the right edge

- the missing close button
- the vanished white test block, in precisely that corner
- and why killing the applications made it **worse** — an empty slot fills the column with solid black, while a running one at least draws text

That last point is also why "the same binary behaved differently" was so convincing and so wrong. **The binary was identical. The application slots were not.**

Compare Chapter 8 §8.7, where one cause explained six observations. The difference between a correct diagnosis and a plausible one is whether it accounts for the observations that looked *arbitrary*.

Chapter 24 §24.11 covers the fix and its subsequent narrowing to a compile-time assertion.

27.12 The seven instruments that lied

This is the part worth keeping. Every one of these cost real time.

1. **The blit timer.** `task_cpu_cycles()` only advances at context switches, so the figure tracks switches, not work. A 55.85 → 31.3 ms shift was read as a serious regression, and reported as a 44% *improvement* an hour earlier. "It was an artifact both times."
2. **`timer_ticks()`.** Counted ISR entries rather than elapsed time. Silent for months.
3. **The transmit completion bit.** Reports that a frame left the *queue*. "A phone was the instrument that caught it."
4. **The OUI list.** Declared a correct MAC decode invalid. "The chip's own CRC8 was sitting right there."
5. **The PHY stack high-water mark.** Reported 6144 of 6144 bytes used — not a large number but a *missing measurement*, because an unprimed `.bss` stack is all zeros.
6. **`raycast_framebuffer()`.** Returns a boolean. Named like an accessor. Panicked a diagnostic that trusted the name.
7. **`fb=on` in the reporter.** Reports the framebuffer *mode*, not whether a buffer exists.

The pattern:

the counters were all clean while the picture was visibly wrong. `corrupt=0`, `dma=320/0`, no timeouts, no skew, no lock contention worth mentioning. Every automated check this kernel has said everything was fine, and every one of them was telling the truth about the narrow thing it measured.

The person looking at the screen was the only instrument that could see the actual fault, and twice in this session that person was right while the counters and the reasoning were wrong.

27.13 Three method notes worth repeating deliberately

Make the variable runtime-tunable before forming a theory about it.

`touchcfg` turned four hypotheses into one flash. `spiclk` did the same for the panel clock. Both were written *after* several reflash-per-guess rounds that they would have eliminated.

The same technique as the flash divider sweep in Chapter 20 §20.6 — sixteen measurements, one flash cycle — and as the interrupt-enable bit sweep in Chapter 22 §22.5.

When something looks corrupted, draw a static test pattern first.

It separates "the content is wrong" from "something overwrites it afterwards" in a single step, and those two have disjoint suspect lists. This was the fifth thing tried and should have been the first.

Prefer an oracle that lives on the chip.

The eFuse CRC beat a hand-maintained OUI list. Two independent clocks agreeing beat one clock asserting. A hardware acknowledge bit clearing beat a register reading back what was written.

And a process note:

when a user says *it worked a minute ago on this exact build*, that is data, but it is not automatically evidence of hardware. It is equally evidence that **something in the run-time state differs** — in this case, which application slots were full. **Enumerate that state before reaching for the screwdriver.**

27.14 The open fault, and the measurement that cracked it open

Symptom. Opening the 3D view sometimes shows a wrong picture for ten to thirty seconds, then it comes good on its own.

Everything ruled out, each by direct measurement rather than argument:

theory	how it died
DMA fell back to the FIFO path	<code>dma=320/0</code> — the guard has never fired
Touch scheduling	identical blit across all four priority/poll combinations
Applications overdrawing the view	killed them all; <code>drawskip</code> froze; no change
Panel signal integrity	clock 40 → 10 MHz changes appearance, does not fix
The chrome column	<code>APP_VIEW_Y0 224 == RAY_VIEW_H 224</code> ; they never overlap
Movement catch-up burst	fixed by <code>raycast_open()</code> ; symptom unchanged
Touch calibration	was genuinely broken and is fixed; symptom unchanged
An arena overlapping the framebuffer	<code>fb 0x3ffbcd70..0x3ffd7170</code> , arenas from <code>0x3ffd7590</code> — clear

The strangest observation: opening `gfxrogue` *repairs* the view.

If a second program starting fixes the first, the view is missing an initialisation step that an application start happens to perform, rather than being actively corrupted by anything.

A contaminated diagnostic

The blittest that produced [one] theory was also contaminated: it called `desktop_set_active(1)` to stop the renderer overwriting the pattern, which put the launcher into repaint mode and erased the test block itself. **A diagnostic that changes the state it is measuring is worse than no diagnostic**, and this one was believed for several rounds.

The measurement that finally said something

The obvious experiment — compare the framebuffer before and after a launch — does not work as stated. The raycaster rewrites **every pixel** of the buffer every frame as the camera moves, so any two samples differ for entirely innocent reasons. **A comparison whose result is "different" no matter what is not a measurement.**

So the renderer is frozen first (`dfreeze`, Chapter 25 §25.11):

BEFORE launch	equal-to-first=25088	rows 67e0 67e0 67e0 67e0
CONTROL	equal-to-first=25088	rows 67e0 67e0 67e0 67e0
AFTER launch	equal-to-first=25088	rows 67e0 67e0 67e0 67e0

The **control** row is the load-bearing one. Taken with nothing done in between, it proves the freeze holds the buffer still and the sampler is stable — without it, "no change" would be indistinguishable from a broken measurement. **That is the same lesson as the contaminated blittest above, applied in advance for once.**

Launching a program does not alter one sampled byte of the framebuffer.

This is the first positive result in the entire investigation, and it eliminates more than the previous eight negatives combined: the renderer, the framebuffer contents, the heap, and all five of `app_start()`'s operations. **Whatever the repair is, it happens downstream of the image.**

A mechanism that fits

The ILI9341 holds a **window**, set by `set_window()`, and pixel data streams into it with CS asserted. If a stream ever ends short, or CS is left asserted, the controller sits mid-window and the *next* pixels land at the wrong offset. That is what a garbled image is. Any other drawer issuing a fresh `set_window()` and `push_end()` **resynchronises the controller** — and applications draw constantly, while the raycaster alone only ever writes one full-screen window per frame.

That accounts for every observation, including the one that looked like magic: code which never touches the image can repair the image, because it is not repairing the image — **it is repairing the panel's idea of where the image goes.**

And it retires an earlier claim:

"Transport is clean, proven with colour bars" tested a single blit **from a fresh state**. It did not test a blit issued after a previous one may have left the controller mid-window, which is a different question and the one that matters.

Status: untested. *"It is a hypothesis with a mechanism, which is more than the previous eight had."* `display_resync()` (Chapter 18 §18.10) is the instrument built to settle it.

27.15 Where things ended up

3D view	fixed, close button present, corrupt=0
Touch	100 Hz at HIGH priority, ~10× the previous sample rate, default
WiFi receive	working — continuous scanning, beacon decode, descriptor recycling
WiFi transmit	MAC accepts and completes frames; nothing hears them
task_sleep	actually sleeps, for the first time
timer_ticks()	actually counts time, for the first time
IRAM budget	not a real constraint; the 48 KB figure was a measurement error

27.16 Metrics

Quantity	Value
OSI table entries	116 declared, 39 implemented
OSI pools	12 semaphores, 8 queues, 4 event groups, 12 timers
Window vectors implemented	6
MAC moving words: cold / after phy / after mac	0 / 6 / 13–15
TSF rate, two independent measurements	1000 kHz / 1001 kHz
MAC address	5c:01:3b:50:3f:64, CRC8 0x08 confirmed
PHY stack	6 KB private, 272 B used
PHY init data	128 B, Espressif's default table
PHY calibration buffer	1,904 B in .bss, full calibration every boot
Frames received	372 across 4 buffers, 374 recycled
Networks decoded	2
Frames transmitted / completed / answered	178 / 178 / 0
Probe requests sent / responses received	20 / 0
IRAM cost of the MAC init chain	2,459 B (not 48 KB)
Text / IRAM	119,151 of 131,072

Quantity	Value
Instruments caught lying	7
Display theories eliminated by measurement	8

27.17 What this does not establish

- **Transmit does not reach the air.** The single largest open item in the project, with a named strongest lead (`periph_module_reset`).
- **No association, no encryption, no IP.** This is a raw 802.11 receiver, not a network stack. There is no DHCP, no ARP, no TCP, nothing above the MAC frame.
- **77 of 116 OSI entries are still stubs.** They are the ones the MAC has not asked for yet, which is a statement about the code path exercised so far rather than about the table.
- **The MAC register map is reverse-engineered** except where §27.5 notes otherwise. Every address other than eFuse and DPORT came from open-mac or from behavioural identification.
- **No power management.** The PHY runs a full calibration every boot because there is nowhere to persist the result — the flash record exists (Chapter 20) and has not been extended to hold 1,904 bytes.
- **The 3D startup glitch is open,** §27.14.
- **The DMA stall is open,** Chapter 18 §18.9.

Part V ends here. The device is complete as a device: it boots, schedules, isolates, draws, listens to a finger, remembers, makes a sound, and hears the networks around it.

Part VI is what the project learned about how to know any of that is true.

28. The Instruments That Lied

Sources: docs/UM-NATOS-017 §6, docs/UM-NATOS-018 §5.1, docs/UM-NATOS-028 §8, and every other report Code: throughout

28.1 Why this chapter exists

Across 138 commits and 28 reports, this project spent more debugging time on faulty *measurement* than on faulty *hardware*. That is not a figure of speech. UM-NATOS-028 §8 catalogues seven instruments that lied in a single session, and ends:

Every automated check this kernel has said everything was fine, and every one of them was telling the truth about the narrow thing it measured.

The person looking at the screen was the only instrument that could see the actual fault, and twice in this session that person was right while the counters and the reasoning were wrong.

This chapter collects every instance in the project, groups them by failure shape, and states what each shape looks like from the outside. It is the chapter to read before diagnosing something.

28.2 Shape 1 — an instrument that reports its own reading invalid, believed anyway

The signature: a signal that the measurement is invalid, sitting next to a number that looks reasonable.

UM-NATOS-017 §6 identified three instances and treated the pattern as the finding:

Instrument	What it said	What was believed instead
The marker block (Ch. 17 §17.9)	Frozen — the system is halted	The colour bars, which a frozen screen renders identically
s/e=150, three runs (Ch. 19 §19.4)	This counter is not accumulating	The pressure values printed beside it
A boot banner in a capture (Ch. 19 §19.4)	The board just restarted	The counters, as though continuous

A fourth arrived later:

| The flash divider sweep (Ch. 20 §20.7) | Identical output across three configurations
| The theories being tested |

The peripherals were largely fine. **The verification method was what kept failing**, and it failed in the same shape each time.

What breaks the deadlock

Latch the quantity so timing cannot lie about it, then feed the system a controlled input rather than interpret an uncontrolled one.

Both halves are now habits in the tree. Latching:

```
/* Extremes since boot. Confirming what the pressure channels do under a finger
 * needs a capture to coincide with the press, which is not something that can
 * be arranged reliably. Latching the extremes makes the question answerable at
 * any later moment instead. */
```

```
/* The outcome of the last run, kept rather than only printed.
 *
 * The result line is emitted the instant the fourth target is tapped, which is
 * exactly when nobody has a capture attached ... A number that has
 * already scrolled past is not a measurement. */
```

Controlled input: `intr_selftest()` injects edges; `adcdriive` commands the input voltage; the flash probe sweeps sixteen configurations in one boot; `touchcfg` and `spiclk` make a variable adjustable while somebody watches.

28.3 Shape 2 — a test that cannot fail

The signature: a real test, correctly performed, structurally incapable of producing a negative result.

Three instances, in three different subsystems:

The direction test (Ch. 19 §19.8)

Direction inferred from the last sample of a drag. Release samples read the ADC rail; a rail reading is near the top of any range; so *every* axis appears to increase.

The test could only ever return one answer.

The IO_MUX cross-check (Ch. 21 §21.4)

Four entries checked, all four below the anomaly.

The check was real, it was performed, and it could not have failed.

A cross-check only tests what its samples can distinguish. The samples must **straddle** the thing being verified, not merely agree with it.

The speaker self-test (Ch. 23 §23.4)

`spktest` drove GPIO32, which has no RTC mux and is therefore structurally incapable of exhibiting the fault under investigation.

Verifying an instrument on the one case that cannot fail is not verification.

And a fourth, which was caught by *failing*:

The critical-section test (Ch. 11 §11.7)

Originally ran before `timer_start()`, where `timer_ticks()` is 0 and stays 0 regardless of what masking does.

a test that could only ever pass by accident. **It reported FAIL, which is the one outcome that made the flaw visible.**

That is worth dwelling on: three of these were discovered by an unrelated failure weeks later; one was discovered because it happened to be positioned where its tautology produced the wrong answer rather than the right one.

The mitigations now in the tree

- The bounds cross-check uses 35 cases chosen to include "zero lengths, the exact end, one past the end, and lengths chosen to wrap the address space" — samples that straddle by construction.
- The speaker probe **prints its own blind spot**, so the next reader is told what it does not cover before they read the result.
- The calibration's pair check compares readings that *should* match, which is a test whose failure mode is specific rather than vague.

28.4 Shape 3 — a register that reads back perfectly while nothing happens

The signature: complete configuration confidence, zero effect.

Four instances:

Symptom	Reality	Chapter
ADC returning ~619 on every channel	Converting, attached to nothing — one wrong bit inside a twelve-bit field	22 §22.10
GPIO edge latched, CPU never interrupted	Delivered to the APP CPU, which is halted	22 §22.5
LEDC configured perfectly, silent	Clock gated off	23 §23.4
WiFi MAC accepting a masked write	Unknown until proven otherwise	27 §27.3

A read-back cannot detect any of them.

The ADC case is the sharpest, because it defeats a discipline that had been working:

A wrong bit in the right register is invisible to a read-back. That is the one failure mode the read-back discipline cannot catch, and it is worth naming because that discipline had been working well enough to feel sufficient.

Four different answers, one per case

Each was caught by a technique the others could not have used:

- 1. **ADC** — a control group. Four pins that *should not* respond to light, measured simultaneously with the one that should. 265 counts against a noise floor of 47.
- 2. **GPIO** — injection. Five candidate bits swept in one boot, with the handler's own counter as the oracle.
- 3. **LEDC** — an external reference. "It works fine on my other project."
- 4. **WiFi MAC** — motion. Scan the register window twice and count words that changed with nothing driving them. A gated block is static; a live one has free-running counters.

The fourth is the most general and the only one that does not ask the hardware a question at all. If a peripheral can be observed to *move*, no write that went nowhere can fake it.

28.5 Shape 4 — a clock measuring the wrong thing

The signature: a duration that varies with system load rather than with work.

Three clocks, each correct for a different question, each used for the wrong one at least once:

Clock	Measures	Where it lied
xt_ccount()	Wall clock	A full-screen fill: 43 ms single-threaded, 249–362 ms from a task. The framebuffer's "450 μ s per address window". The DMA timeout that fired on a working engine.
task_cpu_cycles()	Cycles the task ran	Only advances at context switches, so it cannot time a spin <i>inside</i> a slice. Reported a blit as both a 44% improvement and a serious regression; it was neither.
timer_ticks()	Was: ISR entries. Now: elapsed time	217 ticks per real second where 100 is correct. Every tick-denominated deadline came due early by a load-dependent factor. Silent for months.

Plus the comparator itself, which stalled for up to 183 ms at a time (Ch. 7 §7.9).

The consequences reached everywhere:

Sleeps ran short, timeouts ran loose, and **every frame-rate figure this project has ever recorded was measured against a clock running fast by a load-dependent factor.**

and

Both figures above were taken on a clock that intermittently stalled and under draw-lock contention that dominated the frame. With both fixed the framebuffer is worth having and is now the default.

A conclusion published, defaulted off, and reversed — because the clock it was measured against was wrong.

The mitigation

The comment in `task.c` is the most useful thing written about this:

MEASURED, and the correction is not uniform. A full-screen fill from the SHELL task read 249-362 ms wall-clock and 63-79 ms of work — the shell runs at NORMAL and is preempted constantly. The raycaster's blit did not move at all: 55.5 ms either way, because the display task runs at HIGH and is barely interrupted.

Which means the fix matters most exactly where the old numbers were least trusted, and changes nothing where they were quietly correct. **A conclusion drawn from one task's timings could not have been carried to another's.**

28.6 Shape 5 — a diagnostic that changes what it measures

Two instances, and the second was caught because of the first:

The contaminated blittest (Ch. 27 §27.14) called `desktop_set_active(1)` to stop the renderer overwriting a test pattern — which put the launcher into repaint mode and erased the test block itself.

A diagnostic that changes the state it is measuring is worse than no diagnostic, and this one was believed for several rounds.

The correlated sample (Ch. 22 §22.6): sampling `GPIO_PIN36` from the shell returned zero three times running against a window covering 93% of the cycle.

That is not chance — **the shell's sampling is correlated with the touch task being awake**. Only the armed task could answer without the measurement being correlated with its own subject.

The answer in both cases is a **control**:

The **control** row is the load-bearing one. Taken with nothing done in between, it proves the freeze holds the buffer still and the sampler is stable — without it, "no change" would be indistinguishable from a broken measurement. That is the same lesson as the contaminated blittest above, **applied in advance for once**.

28.7 Shape 6 — a startup artefact standing in for behaviour

The signature: a mechanism confirmed to *exist* rather than observed to *work*.

The canonical case is the shell (Ch. 12 §12.5):

The shell was verified by observing that its **banner printed**, not by observing that a command **ran**. A banner proves the task was created and the transmit path works. It says nothing whatsoever about receive.

The receive path was one byte behind for the shell's entire existence — every interactive session since M5.

The same substitution accounts for three more:

- **Stack guards** — a guard word was written, and `task_stack_headroom()` was never called (Ch. 12 §12.4).
- **The panic handler** — present and linking, never triggered (Ch. 12 §12.6).
- **The hang detector** — a watchdog register was armed, and its interaction with the panic path was never exercised (Ch. 12 §12.2).

And the cruellest variant:

the question asked to confirm the driver — whether the colour bars were in the right order — is one a **frozen screen answers identically to a live one**. A static image was treated as evidence of a running system.

The mitigation

Three shell commands that exist for no other purpose:

Command	Induces
hang	masks interrupts, spins — the watchdog must reset the board
fault	executes <code>ill</code> — the panic handler must report and halt
smash	clobbers the running task's guard — the next switch must panic, naming the task

These are deliberately not compiled out. **A destructive command in a development shell is a smaller risk than a recovery path nobody can reach to test.**

28.8 Shape 7 — the harness did not run

The signature: a run of results that do not vary when the input does.

Two hypotheses about flash timing were recorded as tested against a board that had never been reflashed (Ch. 20 §20.7). The invocation named a script that does not exist; PowerShell's error went to a filtered stream.

Every hypothesis returned bit-identical output, which was read as "none of these are the cause" when it meant "no experiment has run yet".

A negative result is only informative if the experiment demonstrably ran. When consecutive hypotheses produce identical output, suspect the harness before the theory — and verify the change reached the target, rather than inferring it from the absence of an error message.

The near-identical failure in Chapter 19 §19.4 is the same shape from the capture side, and produced the same signature: three runs, exactly 150 samples each.

And a related case in Chapter 12 §12.9: a scripted edit that silently rewrote the body of the function it was redirecting.

It is the **fourth scripted edit in this project to fail silently**, and the reason those edits now carry assertions that stop on a missed match rather than writing an unchanged file.

28.9 Shape 8 — a name that misleads

Two instances, both cheap to fix and both having cost real time:

`raycast_framebuffer()` returns a boolean and is named like an accessor. A diagnostic used it as a pointer and stored through address `0x00000001`.
`raycast_fb_ptr()` now exists.

`display_lock_wait_*` would have been the natural name for the lock timing accessors, and would have been wrong:

- * The name matters, because "wait" would imply the panel was the bottleneck and
- * send the next person to shorten holds, which was tried and changed nothing.

They are `display_lock_blocked_*`. A name that asserts a mechanism is a claim, and a wrong one sends the next reader in the wrong direction.

28.10 What actually worked

Positively, the techniques that broke deadlocks in this project:

A control group measured simultaneously. The ADC's four touch pins. The `dfreeze` control row.

Two independent clocks agreeing. The TSF timer against `CCOUNT`, 0.1% over half a second.

An oracle that lives on the chip. The eFuse CRC8 beat a hand-maintained OUI list. `esptool`'s chip ID beat a belief about the part.

A response nobody local could forge. A probe *response* addressed to this station, which "a radio cannot hear itself" makes unforgeable.

Sixteen measurements in one boot. The flash divider sweep. The interrupt-enable bit sweep. `touchcfg`'s four combinations.

Arithmetic that must balance. $1,097,103 \times 3 + 3 = 3,291,312$ against an observed 3,291,313. $14,999^2 = 224,970,001$. $240 \times 320 \times 2 + 34 = 153,634$.

A pair of counters, one that must be non-zero and one that must be zero. 81/109211. 136 fills / 0 escapes. 265 counts / 47 control. 372 frames / 374 recycled.

A switch that can be flipped. `fb`, `spiclk`, `touchcfg`, `dfreeze`, `DISPLAY_USE_SPI2`.

a switch that can be flipped is a claim that can be rechecked.

Reading the source of truth instead of probing around it.

Probing is the right tool when the question is *what does this hardware do*. It is the wrong tool when the question is *what is this constant*.

A person looking at the screen. Three times, a human observation outranked the instruments and was right:

- *"the dots are following my finger"* — overridden as leftovers, correct (Ch. 19 §19.4)
- *"it works fine on my other project"* — eliminated the entire hardware half of the search space (Ch. 23 §23.6)
- *"no, it is a bug in the code, because the X button is also missing"* — correct against a confident hardware diagnosis (Ch. 27 §27.11)

28.11 The tally

Shape	Instances	Chapters
Instrument reports itself invalid, believed anyway	4	17, 19 ×2, 20
A test that cannot fail	4	11, 19, 21, 23
Register reads back, hardware dead	4	22 ×2, 23, 27
A clock measuring the wrong thing	4	7, 9, 18, 25
Diagnostic changes what it measures	2	22, 27
Startup artefact standing in for behaviour	4	12 ×3, 18
The harness did not run	3	12, 19, 20
A misleading name	2	25, 11

Twenty-seven distinct instances. Against them: four defects that were genuinely in a peripheral's protocol or wiring, and none at all that turned out to be faulty hardware.

That ratio is the single most useful number in this book.

Next: the rules those failures produced.

29. The Standing Rules

Sources: docs/README.md, and the report section that produced each rule Code: the file each rule now lives in

29.1 What a standing rule is here

docs/README.md carries twelve rules in block quotes after the status table. Every one was produced by a specific defect, every one names the report section that produced it, and every one is stated in a form that constrains *future* code rather than describing past code.

They are reproduced here with their origin, the code they now live in, and — where the project has since violated one — a note saying so.

29.2 The kernel rules

Rule 1 — Clear `ps.EXCM` before calling C from an interrupt handler

Hardware sets `EXCM` on interrupt entry, and while it is set the Xtensa zero-overhead loop-back is disabled: a hardware loop body executes once and falls through to whatever sits at `LEND`. GCC emits these loops for ordinary counted C loops, so this silently degrades **every C function reachable from a handler** — drivers and the VM interpreter included, not just the scheduler where it was found. `_handler_level3` now clears it; any future handler must do the same.

Origin: Chapter 8 §8.7. Twelve build cycles, three wrong hypotheses, five instructions in the fix.

Lives in: `kernel/vectors.S`, with a 17-line comment.

Scope note: UM-NATOS-009 §9 records that whether anything *else* silently depended on `EXCM` being set has never been examined systematically. Still open — Chapter 30.

Reappeared as: the `StoreProhibited` in Chapter 27 §27.6, where a fault inside a window-overflow handler running with `EXCM` set vectored to the double-exception handler while `EPC1` still held the *original* faulting instruction. "Hours were spent disassembling the wrong instruction."

Rule 2 — A shared hardware register with a software shadow has one safe shape

either one writer; or every writer maintains the shadow.

`timer.c` kept `g_next` as its idea of the comparator deadline while `task_yield()` wrote `CCOMPARE1` directly. The handler then added a whole interval to a deadline that was only 64 cycles old, so every yield pushed the tick further out — 18 tick periods, 183 ms, before anything noticed.

Origin: Chapter 7 §7.9.

The sharp part: the rule below it (Rule 3) *was obeyed exactly* and did not prevent this.

The rule constrains the **writer**. The defect was in the other party's **bookkeeping**, and a shadow is only as good as its exclusivity. Moving the deadline earlier is safe for the deadline and quietly invalidates every assumption anyone else has cached about it.

Applied prospectively in: `store.c`, where `calib_persist()` and `store_count_boot()` both live in the file that owns the record —

```
/* Declared in calib.h. Writing the record here keeps every write to it in one
 * file, which is what makes "the record is only changed in store.c" a property
 * rather than a habit. */
```

Rule 3 — A yield must never defer the clock it depends on

`task_yield()` originally wrote `CCOMPARE1 = ccount + 64` unconditionally, so any loop yielding faster than 64 cycles pushed the deadline ahead of `CCOUNT` forever, the timer interrupt stopped firing, and the whole kernel froze — the tick is what drives every context switch. Any routine adjusting a scheduler deadline must only ever move it **earlier**.

Origin: Chapter 17 §17.9. Survived three commits and two "verified on hardware" claims.

Lives in: `kernel/task.c`:

```
uint32_t soon = xt_ccount() + 64u;
if ((int32_t)(soon - xt_get_ccompare1()) < 0) {
    xt_set_ccompare1(soon);
}
```

The general form, from the report:

`_handler_level3`'s PS.EXCM rule is the other standing rule of this kind. Both share a shape: **a single unconditional register write, correct in isolation, catastrophic under repetition**.

Rule 2 is the third member of that family, from the bookkeeping side.

Rule 4 — Lock contention cost is the NUMBER of blocking events, not the time held

Each one costs a scheduling round-trip whether the lock was held for 24 ms or 24 μ s: the blocked task is descheduled and must be selected again. Measured here at 63 ms per contention against a 24 ms hold, with the lock free 88% of the time while two tasks sat blocked on it. Narrowing the hold by 25% changed the outcome by nothing. The levers are batching (fewer takes) or not blocking at all (`try_lock`) — shortening holds, the intuitive move, does nothing.

Origin: Chapter 11 §11.9.

Reached for twice: batching gave the raycaster a **194×** improvement; `try_lock` took the frame rate from 3.0 to 9.9 fps.

Applied by an unrelated file: the launcher draws each icon row as one rectangle per *run* of set pixels rather than one per pixel — "not micro-optimisation: every primitive takes the draw lock".

Lives in: `kernel/display.h`, in both the batching interface and the accessor names.

29.3 The verification rules

Rule 5 — When an instrument reports its own reading invalid, that outranks every value printed beside it

Three times across two sessions a frozen marker, a sample counter stuck at an identical value, and a boot banner in a serial capture each said "this measurement is not what you think", and the plausible-looking numbers next to them were believed anyway. Two conclusions were published and later retracted as a result. **Latch the quantity so timing cannot lie about it, then feed the system a controlled input rather than interpreting an uncontrolled one.**

Origin: Chapter 19 §19.5. Four instances by the end of the book.

Lives in: the latching in `touch.c` and `calib.c`, and the controlled-input probes in `intr.c`, `adc.c` and `flash.c`.

Rule 6 — A negative result is only informative if the experiment demonstrably ran

Two flash hypotheses were recorded as tested against a board that had never been reflashed. Every hypothesis returned bit-identical output, which was read as "none of these are the cause" when it meant "no experiment has run yet". The signature to watch for is **a run of results that do not vary when the input does** — suspect the harness before the theory, and verify the change reached the target rather than inferring it from the absence of an error.

Origin: Chapter 20 §20.7.

Mitigation: the flash step always shows its `Hash of data verified.` line, and that line is checked before any capture is interpreted.

Related: Chapter 19 §19.4's capture harness, which produced exactly 150 samples three runs running; and Chapter 12 §12.9's scripted edit, the fourth in the project to fail silently.

Rule 7 — A startup artefact is not evidence the thing it introduces works

The shell was signed off because its banner printed; the banner proves a task was created and the TRANSMIT path works, and says nothing about receive. The receive path was one byte behind for the shell's entire existence, so pressing Enter did nothing until the next keystroke — and every automated test drove it with CR and LF, the one input shape that hides it. Stack guards and the panic handler went unexercised for the same reason: each was confirmed to EXIST rather than observed to WORK. **Trigger the mechanism on purpose, or treat it as untested.**

Origin: Chapter 12 §12.5.

Lives in: the `hang`, `fault` and `smash` commands, and the README's one-sentence defence of them:

The last three exist on purpose. A recovery path that has never been observed to fire is confidence without evidence.

Arguably the most broadly applicable rule in the book.

Rule 8 — Never infer direction from the endpoint of a gesture

The first and last samples of a drag are its two least trustworthy, because both sit at a contact transition where the panel is not bridged and the ADC reads its rail. A rail reading is near the top of the range, so a drag ending anywhere "ends near its maximum" and EVERY axis appears to increase — the test can only return one answer. The touch X axis was backwards for three months behind exactly that. **Calibrate from labelled points, each a known position paired with a reading, and let direction fall out of comparing labels.**

Origin: Chapter 19 §19.8.

And then the follow-up, which is why Chapter 19 has two sections on calibration: labelled points beat endpoints, and the four *corners* were still the wrong four labelled points, because a finger cannot reach the extreme corner of a bezelled panel. `calib.c`'s four inset targets are the correction.

Lives in: `kernel/touch.c`'s recorded corner table and `kernel/calib.c`.

Rule 9 — A cross-check only tests what its samples can distinguish

The SD pin map was verified against four IO_MUX entries already in the tree. All four confirmed the indexing, and all four sat below the anomaly, so every one of them agreed with the wrong answer: GPIO23 was read from the UART's receive pad. The check was real, it was performed, and **it could not have failed. Samples must STRADDLE the thing being verified, not merely agree with it.**

Origin: Chapter 21 §21.4.

Applied correctly in: the VM's 35-case bounds cross-check (Chapter 14 §14.5), whose cases include "the exact end, one past the end, and lengths chosen to wrap the address space"; and the interrupt matrix's two-point derivation check, using sources 14 and 34 on either side of source 22.

Sibling rule: *verifying an instrument on the one case that cannot fail is not verification* (Chapter 23 §23.4) — the same shape with one sample instead of four.

29.4 The engineering-judgement rules

Rule 10 — A correct diagnosis does not license a fix of arbitrary scope

Chrome drawn over a view that repaints every pixel every frame will strobe; that diagnosis was reached twice and was right both times. The first fix reserved rows for it, which moved the region boundary, the application strips, the colour strip and the grid — four files of constants — and produced a screen that looked wrong for reasons never found, while every measurement insisted the renderer was correct. The second wrote 324 pixels into a buffer that was already being sent. **Prefer the fix whose blast radius matches the defect.**

Origin: Chapter 24 §24.8.

Follow-up: Chapter 24 §24.9 later found the cause and it was not the layout — which makes this rule's *illustration* partly wrong while leaving the rule intact. Both are recorded.

Rule 11 — Tolerating a defect is not fixing it

The note pad's keys are 80 px because the touch mapping reads about 24 px low on X; a 24 px key was destroyed by that error and an 80 px key absorbs it. The app works, the fault is untouched, and every future element finer than 80 px will meet it again. **Record which one you did, at the place a reader would otherwise assume the generous version.**

Origin: Chapter 26 §26.3.

The payoff, stated in the report itself:

Because the keypad was never called a fix, the fault stayed on the record as unresolved, and calibrating it properly stayed on the list instead of quietly becoming the way things are. **A workaround that is honestly labelled is a debt; one that is called a solution is a defect with good manners.**

The fault *was* subsequently fixed, and the keypad stayed — now for a stated new reason, "because a decision whose original justification has expired is one nobody re-examines".

Rule 12 — Reverting two changes together destroys the information about which one mattered

A relay layout and a camera fix were reverted in one commit; the screen looked right afterwards, which appeared to convict the layout. Re-applying the geometry alone, later, showed it was innocent — the camera had broken concurrently and "blank screen" was a correct rendering of the inside of a wall. **If a revert must bundle, the bundle is a hypothesis to test later, not a conclusion.**

Origin: Chapter 24 §24.9.

The information needed to acquit the layout "had been destroyed by the same action that produced the evidence".

29.5 Rules the source carries that the README does not

Several rules exist only as comments, and are worth promoting here.

Choose a sentinel that cannot be a legal value

- `BLK_FREE` / `BLK_USED` — "implausible as data, and distinct from each other"
- `STACK_GUARD` `0x57ACC0DE` and `STACK_FILL` `0xEEEEEEEE`
- `MUTEX_FREE` = -2, not -1, because `task_current()` returns -1 before the scheduler starts
- `R1` = `0xFF` from an SD card — "bit 7 of a real R1 is always zero, so 'no answer' cannot be confused with any legal response"
- The interrupt-integrity patterns `0x11111111`, `0x22222222`, ... — "so that a stray zero, or a neighbouring register's value, is unmistakable rather than plausible"

A formula whose failure mode is a plausible extreme is worse than one that fails to an obvious value

From `touch.c`, where `z = z1 + 4095 - z2` turns a dead bus into maximum pressure. Reappeared in `i2c.c`, where an input buffer left off makes every address appear to ACK.

| because the first requires interpretation and the second announces itself.

Fetch a constant; do not recall it

Three register-layout errors in one day (Chapter 22). The fix in every case was to read the vendor header.

Probing is the right tool when the question is *what does this hardware do*. It is the wrong tool when the question is *what is this constant*, and those are easy to confuse when both present as "it does not work".

And the corollary, from `window.S`:

A wrong register number here does not fault — it silently reloads a caller's frame with the wrong values, which is the least debuggable failure this project could construct.

Delete the second mechanism rather than debug it

- Two ways for a task to come into existence → one (Chapter 8 §8.2)
- A separate `switch_to()` → the tick path, "exercised constantly, rather than a second one exercised rarely" (Chapter 8 §8.3)
- Three arena release paths → one `retire()` (Chapter 16 §16.3)
- A second on-panel command set → `shell_run_line()` into the same `execute()` (Chapter 26 §26.9)

Describe a loop by its bound, not by the bound's value on the day

Written as "all eight tasks" when `TASK_MAX` was 8. It is now 12, and the check is written against `TASK_MAX` rather than a literal, so it followed. The prose did not.

An optimisation with no measured gain has no claim on the tree

It measured as noise (55.9 → 55.5 ms) and was kept anyway as "correct and free". It had also **never actually run** — DMA disabled itself before it mattered — so it sat in the tree for a day looking like tested code. **Code that only executes when another bug is absent has been tested by nothing.**

The positive counterpart, from the same project: a change kept for a *stated* reason other than the one it was made for —

The narrowed hold was kept anyway, because a lock should cover the shared thing rather than the whole operation that happens to use it — but it is not an optimisation and is not recorded as one.

Removing an instrument once it has found its bug is how the bug returns unnoticed

From Chapter 24 §24.4. The first/last cell pair is still reported, with a note saying why. The M2 switch tracing, the `TRACE_PROBES` A/B switch, and `task_select_probe()` are all retained on the same reasoning.

A switch that can be flipped is a claim that can be rechecked

From Chapter 18 §18.7, and the reason its wrong conclusion was findable at all. `fb`, `spiclk`, `touchcfg`, `dfreeze`, `DISPLAY_USE_SPI2`.

Prefer an oracle that lives on the chip

The eFuse CRC beat a hand-maintained OUI list. Two independent clocks agreeing beat one clock asserting. A hardware acknowledged bit clearing beat a register reading back what was written.

A best-effort policy that silently discards work is indistinguishable from a broken one

Every skip is counted. `g_draw_skipped`, `g_vp_escapes`, `g_refused`, `g_spurious`, `g_bad_free`, `g_late`, `g_age_rescues`.

`rescues=0` is the only thing that distinguishes "never needed" from "never working".

Record a limitation where a reader would assume it was addressed

`notes.h` on text entry being a kernel feature; `arena.h` on native tasks never being confined; `store.c` on the checksum not defending against determined corruption; `sd.h` on the pin map being assumed until a card answers.

29.6 The rules, as a checklist

For the reader who wants one page:

Before writing: 1. Fetch the constant; do not recall it. 2. Choose sentinels that cannot be legal values. 3. One writer per register, or every writer maintains the shadow. 4. A deadline may only ever move earlier. 5. Clear `PS.EXCM` before calling C from a handler. 6. Delete the second mechanism rather than debug it.

Before believing a measurement: 7. Did the experiment demonstrably run? 8. Could this test have failed? 9. Do the samples straddle the boundary, or merely agree with it? 10. Is the instrument reporting itself invalid? 11. Which clock is this, and is it the right one for the question? 12. Does the diagnostic change what it measures? Where is the control?

Before believing a mechanism works: 13. Was it *triggered*, or merely confirmed to exist? 14. Is there a counter that must be zero *and* one that must be non-zero?

Before committing a fix: 15. Does the blast radius match the defect? 16. Is this a fix or a tolerance? Say which. 17. If this reverts more than one thing, that is a hypothesis, not a conclusion. 18. Did the number improve, and did anyone look at the screen?

Next: everything this system does not establish.

30. What Is Not Established

Sources: the "what this does not establish" section of all 28 reports Code: throughout

30.1 Why this chapter is the most useful one

Every report in this project ends with a section stating what it does *not* prove. The project README explains why:

Every report ends with what it does *not* establish. Those sections are the most useful part. They are where the known gaps live, and several of them correctly predicted the next defect.

Three predictions came true:

Predicted in	Prediction	Came true in
UM-NATOS-011 §4	Writing flash will collide with the data cache	Chapter 20 §20.5 — "It broke exactly as predicted, on the first version of the driver"
UM-NATOS-010 §8	<code>arena_contains()</code> exists and nothing calls it; arenas are bookkeeping, not protection	Chapter 14 — closed by the interpreter
UM-NATOS-009 §9	Nothing has audited the other consequences of <code>PS.EXCM</code>	Still open , §30.2

This chapter consolidates all of them, ordered by what they would cost if someone relied on the system anyway.

30.2 Structural gaps — the ones that shape what can be built next

There is no device model

The largest gap in the system, and the only *structural* item on the post-M5 list that is missing. UM-NATOS-007 §2.1:

Every driver above is reachable only from the kernel. The VM has twelve syscalls, all hardcoded, and no device model — so an application cannot read the light sensor, scan the I2C bus or receive a keypress. Each new peripheral has meant a kernel edit plus a hand-written syscall, which was tolerable at two and is the obvious next piece of *architecture* rather than more drivers.

The consequences appear in six separate reports as the same line: *no application can reach it*. ADC, I²C, audio, persistence, microSD, text entry.

`notes.h` states the specific case most sharply:

text entry is a kernel feature rather than a service applications can request. Any program wanting text input cannot have it. ... **the note pad takes text, so text entry appears solved, and it is solved for exactly one program.**

Syscall validation is per-call, not systematic

Every syscall added since checks its own arguments, and each was reasoned about individually. **Nothing enforces that a future one will, and there is no shared harness that would catch an unchecked length in a new service.**

Given that the whole isolation claim rests on those checks, this is the most dangerous gap in the book. Chapter 17's `SYS_BLIT` shows how much care one syscall needed; nothing guarantees the next one gets it.

No host-side test path

No test target. There is no host-side unit test path; everything is verified on hardware.

Several defects in this book are in *pure functions* that a host test could exercise exhaustively: the bounds predicates, the offset-domain clipping, the ring-buffer arithmetic (which *was* checked that way, by hand — Chapter 26 §26.12), the calibration fit, the fixed-point projection.

No filesystem

The microSD driver reads blocks and decodes an MBR far enough to find a FAT16 signature. There is no directory walk, no file lookup, no FAT chain traversal. Consequently:

No program loading from storage. Images are compiled into the kernel. There is no filesystem, so "install an application" has no meaning yet.

No asset pipeline

Nothing yet converts a font or bitmap into a linkable read-only object. The mechanism exists; the tooling does not.

The launcher's icons are "in the image because there is nowhere else to put them yet".

No execute-in-place for code

Only data is mapped from flash. An IROM segment "has not been attempted, and would interact with the `132r` constraint". The kernel is 119,151 bytes of text against 131,072 of IRAM — enough headroom that it has not been forced, and not much.

30.3 Open faults

WiFi transmit does not reach the air

The single largest open item. The MAC accepts frames and reports 178 of 178 complete; twenty probe requests produced zero responses from two access points that are received continuously.

Strongest untried lead: `periph_module_reset(0x19)`.

nat-os has only ever *ungated* the WiFi peripheral, never *reset* it, and a MAC left in whatever state the ROM bootloader put it in would plausibly receive while refusing to transmit. That asymmetry fits the symptom exactly.

The DMA stall, and the duplicate re-send it causes

DMA disables itself within about eight seconds of the raycaster starting, on a spurious timeout caused by preemption landing between two adjacent lines. The blit costs 55.9 ms instead of a possible ~22 ms, and **the 3D view has run this way for its entire existence.**

The latent defect underneath it is worse than the performance cost:

`spi2_dma_tx()` returns 0 on timeout, and `spi_tx()` then falls through to `spi2_tx()` — **re-sending bytes the DMA has already sent.** ... This is a latent bug in the shipped driver. It fires once per boot and is invisible, because the same timeout that causes it also disables DMA so it cannot happen again.

Three attempted fixes, three breakages. Guidance for the next attempt is in Chapter 18 §18.9.

The 3D view's startup glitch

Garbled for ten to thirty seconds after opening; repaired by launching an unrelated program. Eight theories eliminated by direct measurement; a ninth has a mechanism and is untested. `display_resync()` is the instrument built to settle it.

PENIRQ never fires from a finger

The matrix is verified by injection, the handler runs, and the end-to-end path has never been observed to work.

real taps produced 24 touch events and 30 low PENIRQ readings while the armed edge detector latched nothing. **That gap is unexplained.**

Touch polls at 100 Hz instead, "byte for byte its previous behaviour", with `touch_irq_wait()` intact and one line from reinstatement.

30.4 Isolation: the exact boundary of the claim

Stated in `arena.h`, `task.h`, `vm.h`, and three reports:

- **Native tasks are not isolated and never will be.** Nine tasks share one address space. "A driver bug can still corrupt the kernel."
 - **Stack guards are a post-mortem, not a barrier.** A guard word is found *after* it has been overwritten. "A guard page would be a barrier; this hardware has no MMU to provide one."
 - **Nothing catches a wild pointer** into the middle of a neighbour's stack.
 - **The escape counter is not a proof of the clipping logic**, only evidence that it and the re-check agree on everything tried so far. "A case both get wrong identically would pass."
 - **Four of ten VM fault classes have never been exercised on hardware:**
VM_FAULT_PC, VM_FAULT_CALL_DEPTH, VM_FAULT_SYSCALL, VM_FAULT_STRING.
 - **A program can overwrite its own instructions.** No code/data separation within an arena.
 - **No per-application draw accounting.** A program that draws constantly costs everyone's frame rate; the best-effort policy bounds the damage to the system without bounding it per program.
-

30.5 Concurrency and the scheduler

- **No EXCM audit.** Whether anything else in the kernel silently depended on EXCM being set for the handler's duration "has not been examined systematically". Open since M2.
- **No deadlock detection.** Two tasks taking two mutexes in opposite orders will hang, and the kernel will not say so. "The idle task will simply run."
- **No timeouts on mutex_lock().**
- **No condition variables or semaphores** in the kernel proper. There is no way to wait for an *event*, only for a lock or a deadline. (The WiFi OSI layer builds its own on task_sleep.)
- **Priority inheritance drops a boost on the first release**, so nested holds of two boosted mutexes lose it early. "Nothing in this kernel holds two."
- **Critical sections are unbounded by convention only.** Nothing measures or enforces how long one is held.
- **The mutex is untested against an ISR**, and nothing enforces the documented prohibition.
- **Heap allocation from an interrupt handler would corrupt the list**, and a critical section taken inside a handler is a no-op. Nothing prevents it.
- **No fragmentation characterisation under realistic load.** The leak test's size and lifetime distributions are both uniform; "a first-fit allocator's worst case is workload-shaped".

- **No allocation latency measurement.**
 - **APP_CPU is never started.** Every use of `crit_enter()` assumes single-core and would need re-examining.
 - **`task_wake()` has never woken a task.** "It exists, it is correct by inspection, and no interrupt has ever called it successfully."
-

30.6 Measurement and calibration

- **The DRAM budget is unremeasured.** 167,680 B at M3, 158,048 B after `TASK_MAX` rose to 8; it is now 12 with three more drivers, "so the current figure is lower and unremeasured".
 - **Dispatch cost is a ceiling, not a measurement.** ~109 cycles/instruction, from a quarter-share assumption that ignores interrupt-handler time.
 - **The CPU frequency is derived, not read.** 80 MHz, inferred from tick counts against a host clock. "It should be re-derived rather than assumed if boot configuration changes."
 - **The SRAM region boundaries are transcribed, not verified.**
 - **Cache consumption of SRAM0 under XIP is unmeasured.**
 - **Flash cache-miss latency is unquantified.**
 - **The touch calibration is linear, run once, on one unit, with one finger.** Nothing measures panel non-linearity *between* the target positions, and nothing detects a stale calibration.
 - **`Z_THRESHOLD` is one number from one measurement.** "A lighter touch has not been tested, and a threshold that rejects a real press is worse than one that admits a bad sample."
 - **The SPI clock ceiling was found by eye.** 80 MHz "works electrically and puts visible noise on the glass"; nothing in the kernel can measure that.
 - **Double-tap timing is unmeasured.** 600 ms and the 2-tick minimum press were reasoned. "The counters exist to settle them and nobody has."
 - **The skip ratio is measured under one workload.**
 - **The speaker's response is uncharacterised** between 440 Hz and 3 kHz.
 - **The ADC is uncalibrated** — counts, not volts, with factory eFuse data unread, and only channel 6 proven to track anything.
 - **The save cadence constant has drifted from its justification.** 256 frames was ~60 s at 4.4 fps; the renderer is now several times faster.
 - **The AHB alias for UART receive is validated behaviourally, not from documentation.** No erratum is cited.
-

30.7 Storage and persistence

- **No wear levelling.** One sector, erased and rewritten in place.
 - **No power-cut testing.** The checksum is designed to reject a torn record; nothing has interrupted a write mid-erase. "The mechanism is reasoned, not measured." Same for the message store.
 - **No record upgrade path.** A version bump silently discards the boot counter, the frame total and the calibration.
 - **The cache argument remains an argument.** §20.5's reasoning that no flash-mapped read can occur during an operation "has not been instrumented — there is no assertion that would fire if a future edit broke it."
 - **No verification against a second board.** The flash clock finding is from one part.
 - **SDHC is untested.** The CCS branch exists and has never executed.
 - **No SD write path, no card-removal detection.**
 - **The full message store has never been exercised,** so the oldest-dropped path has never run outside reasoning.
-

30.8 Failure handling

- **Only two fault kinds are recorded.** A hang recovered by the watchdog writes nothing at all, "so the most common recoverable failure is the one the record cannot describe."
 - **A fault during the recording itself is not survivable.** The checksum would reject the torn record on the next boot, correctly, "and the reason would be lost."
 - **Panic mode is untested against a genuinely broken controller.** Every test ran on a display that was working perfectly at the moment of the fault, "so the bounds have never actually been reached."
 - **The panic screen has no failure path of its own.** If the panel is wedged, the byte count says so — but only to somebody with a serial cable, which is the audience the screen exists to replace.
 - **No per-task stack sizing.** All twelve slots get 2 KB regardless of measured use.
 - **No pre-emptive overflow detection.**
-

30.9 Peripherals

- **I²C has never transferred a byte.** START, STOP, the ACK bit, repeated START and the read path are correct by inspection only.

The first real device will be the first test of that logic, and the most likely defects are the ones inspection is worst at: a half-bit of timing, or an ACK sampled on the wrong clock

| edge.

Clock stretching — the property that justifies the whole bit-banging argument — has never happened. No multi-master arbitration detection. No bus recovery. - **ADC2 is untouched** despite being permanently free. - **The display is write-only**. MISO is wired and unused; the panel is never interrogated. - **No orientation control, no gamma correction, no text clipping, no damage tracking**. - **No descriptor chaining** in DMA — most of the remaining gap to the 31 ms floor. - **The DAC was never shown to be broken**, only never shown to work. - **77 of 116 WiFi OSI entries are still stubs**. - **The MAC register map is reverse-engineered** except eFuse and DPORT. - **No association, encryption, or IP**. A raw 802.11 receiver, not a network stack. - **The PHY runs a full calibration every boot** because there is nowhere to persist 1,904 bytes.

30.10 Interface

- **No focus, no windows, no z-order**. Applications occupy fixed strips.
 - **The screen is fully allocated**. 320 of 320 rows. The next feature wanting rows must take them from something named, and the assertions will refuse a build that overlaps.
 - **Application strips are 14 rows and nothing draws into one**, so nothing has tested whether that is enough.
 - **Nothing tests the overlay**. The close button's position is agreed between two matching constants with no assertion tying them — "the exact drift the layout assertions were added to prevent elsewhere."
 - **Faulted programs cannot be dismissed from the launcher**.
 - **The running marker is per-name, not per-instance**.
 - **The fb off path still flickers**.
 - **No multi-touch, no gestures, no pointer-up event, no filtering beyond the pressure gate**.
 - **One pointer, shared**. Nothing arbitrates focus.
 - **The shell has no history, no editing beyond backspace, no completion**. Uppercase is impossible on the panel keypad.
 - **The tee is single-slot and not reentrant**, and would capture another task's output if one printed without the console lock — "which has never happened, which is not the same as being prevented."
 - **48 lines of scrollbar is a bound**, and what leaves is gone.
 - **Two copies of the multi-tap keypad**, with a stated threshold for extracting it (a third consumer).
-

30.11 Build and tooling

- **No dependency tracking.** Every build recompiles every file.
 - **No incremental or parallel build.**
 - **Hard-coded toolchain paths.**
 - **The producer is an assembler, not a compiler.** No linking of separately assembled units, no relocation, no macros, no includes.
 - **The syscall table is duplicated** between `vm.h` and `vasm.py`, with no runtime cross-check. The failure lands at assembly time with a comprehensible message, which is the mitigation rather than the fix.
 - **JTAG was ordered and never arrived.** Every defect in this book was found without one.
-

30.12 The honest bottom line

One board. One panel. One flash chip. One microSD card. One finger. One person's judgement about whether an icon is legible at 24 pixels.

The reports say it themselves, in three places:

The calibration is this board's, but only this board's.

Every judgement about whether the interaction feels right came from a single person on a single panel.

Nothing measures whether the icons or the shading are legible. Both were judged by one person on one panel, **which is the same standard the report set criticises elsewhere.**

That last clause — a report criticising itself by its own standard — is the project's documentation discipline working correctly, and it is the right note to end this chapter on.

Next: what would be worth building on top of all of it.

31. Where It Could Go Next

Sources: docs/UM-NATOS-007-roadmap.md §2.1, and the closing sections of every report

31.1 The one structural item

UM-NATOS-007's revision 1.1 makes the argument, and nothing since has weakened it:

The one structural item on it is missing. Every driver above is reachable only from the kernel. The VM has twelve syscalls, all hardcoded, and no device model — so an application cannot read the light sensor, scan the I2C bus or receive a keypress. Each new peripheral has meant a kernel edit plus a hand-written syscall, which was tolerable at two and is the obvious next piece of *architecture* rather than more drivers.

Seventeen reports later, the list of things applications cannot do has grown to:

- read the light sensor
- scan or talk to the I²C bus
- make a sound
- save state
- read the microSD card
- receive text
- read a key press
- know anything about the network

Every one of those is a kernel edit plus a hand-written syscall away, and the thirteenth syscall would be as ad-hoc as the twelfth.

What a device model would have to do

The confinement discipline of Chapter 17 is the constraint. Whatever replaces hand-written syscalls has to preserve four properties that were expensive to establish:

1. **Offset-domain arithmetic on every program-supplied quantity.** Four places in the current tree do this correctly, and each was reasoned about individually.
2. **Length bounded before it is multiplied.** `SYS BLIT` is the only syscall that has needed this so far, and it is the model.
3. **A pointer copied out of the arena before use,** not handed onward — because "a program's own stores could change the string while it is being rendered".
4. **The quantum distinction.** Any service whose cost is measured in milliseconds must end the caller's slice; any service that reads a published snapshot must not.

The report's own framing is the right one: this is *architecture*, not another driver. It is the difference between a system that has twelve services and a system that can have any number.

And the missing harness

Nothing enforces that a future one will [check its arguments], and there is no shared harness that would catch an unchecked length in a new service.

A device model that came with a validation harness — one place where a program-supplied (offset, length) pair is checked, used by every service — would close the largest safety gap in Chapter 30 at the same time as the largest capability gap.

31.2 The three open faults, in order of tractability

1. The DMA duplicate re-send

The most tractable, and the guidance is already written:

Fix the duplicate re-send first. It is a real defect, it is independent of the timing question, and **it makes any later timeout change safe rather than destructive**.

A timeout currently falls through to the FIFO path and re-sends bytes the DMA already sent, advancing the panel's window by one span. Making a timeout *abandon* the span rather than re-send it would cost one dropped row and remove a class of corruption.

Then, separately, the timeout itself. `task_cpu_cycles()` is the principled clock and cannot bound a spin inside one slice; the current answer is a raised wall-clock bound. A DMA completion *interrupt* would remove the spin entirely, and Chapter 22's matrix is the infrastructure for it — which would also give `task_wake()` its first real consumer.

2. The panel window desynchronisation

A hypothesis with a mechanism and an instrument already built:

with the view visibly garbled, force one clean `set_window()` and full-screen fill from the shell, touching no framebuffer. If the view repairs, it is panel desynchronisation, and the fix belongs in the blit path — re-asserting window and CS state per frame rather than assuming they survived.

One command, one observation, a binary answer. This is the cheapest open item in the project and it is one `display_resync` away.

3. WiFi transmit

`periph_module_reset(0x19)`. nat-os has only ever *ungated* the WiFi peripheral, never *reset* it, and a MAC left in whatever state the ROM bootloader put it in would plausibly

| receive while refusing to transmit. That asymmetry fits the symptom exactly.

And the test is already built and unforgeable: twenty probe requests, and count frames addressed to this station.

31.3 The gaps whose closing would change the shape of the system

A filesystem

The microSD driver stops at a FAT16 signature. A read-only FAT16 implementation — directory walk, FAT chain traversal, file open and read — is perhaps 400 lines and would make "install an application" mean something for the first time.

That in turn makes the assembler's output a *file* rather than a compiled-in array, which makes `PROGRAMS[]` a directory listing, which makes the launcher's grid dynamic.

UM-NATOS-001 §7 called writing FAT "a substantial subproject" and it is. It is also the single change that most alters what the system *is*.

A host-side test path

Several defects in this book are in pure functions:

- `vm_in_bounds()` and `arena_contains()` — already cross-checked on-device against 35 cases, which is the right test in the wrong place
- `vp_fill()`'s clipping and re-check
- the terminal's ring arithmetic — checked by simulating 70 lines through a 48-line ring at all 40 scroll positions, by hand
- `calib.c`'s fit and pair check
- `map_axis()`
- the fixed-point projection in `raycast.c`
- `heap_check()`'s ten invariants, against a synthetic heap

None of those needs an ESP32. A host build of the pure files plus a few hundred lines of test would let them be exercised exhaustively rather than sampled — and Chapter 28's Shape 2 (*a test that cannot fail*) is much harder to construct when the test enumerates its input space.

An asset pipeline

The mechanism has existed since Chapter 4 — `.rodata` mapped from flash at zero DRAM cost. What is missing is tooling: a script that turns a PNG or a font into a linkable object. The icons are hand-written 8×8 constants "because there is nowhere else to put them yet".

Execute-in-place for code

119,151 bytes of text against 131,072 of IRAM. The headroom is real and finite, and an IROM segment is the answer — with the `132r` literal-pool constraint of Chapter 4 §4.4 as the thing to solve first.

31.4 The smaller items, ranked by cost against value

Cheap and valuable:

- Extend the flash record to hold the PHY calibration data (1,904 B). Saves a full RF calibration on every boot.
- An assertion tying `desktop_overlay_into()`'s button position to `desktop_chrome_touch()`'s hit test — the one layout agreement that is adjacent rather than enforced.
- Per-task stack sizing. The margins are measured; the worst task uses 444 B of 2,048 and the best uses 204. Trimming would return several KB of DRAM.
- Read the factory eFuse ADC calibration and report volts rather than counts.
- A `mutex_lock()` timeout, which would turn the undetectable deadlock of Chapter 30 §30.5 into a reported one.

Cheap and uncertain:

- Attach one I²C device. Everything in Chapter 22 Part C is correct by inspection and has never moved a byte; a single sensor would test all of it at once.
- Try 80 MHz on the panel with a person watching, now that `spic1k` makes it a runtime choice.
- Start APP_CPU, if only to stop half of every `INT_ENA` field addressing a core that does not exist.

Expensive and structural:

- The device model (§31.1).
 - The filesystem (§31.3).
 - SMP, which UM-NATOS-001 §7 said should be settled before M2 fixed the scheduler's shape, and which was never settled. Every `crit_enter()` in the tree assumes a single core.
-

31.5 What should not be done

A short list, because "do not build this" is as useful as a roadmap.

Do not remove the bounds checks. Stated twice, in two files, in the strongest terms either can manage:

that check is not optional and must never be compiled out for speed. If it is removed, this kernel has no isolation of any kind.

Do not replace the `switch` dispatch with a computed `goto` without a measurement showing dispatch is the bottleneck.

it is also the kind of construct where a missing entry is a silent jump rather than a diagnosed fault.

Do not remove the destructive shell commands.

A recovery path that has never been observed to fire is confidence without evidence.

Do not delete an instrument because its bug is fixed.

removing an instrument once it has found its bug is how the bug returns unnoticed.

Do not accept a display change on the strength of a number. Three attempts at the DMA stall each had a plausible mechanism and a good number *before anyone looked at the screen*, and all three broke the view.

Do not chase the DMA stall by removing the guard.

Do not remove the guard to study the failure. That was the first mistake.

Do not call a workaround a fix.

A workaround that is honestly labelled is a debt; one that is called a solution is a defect with good manners.

31.6 What this project already demonstrates

Worth stating plainly, because the preceding two chapters are entirely about limits.

An operating system was written from scratch for a \$10 development board, with no RTOS, no C library, no framework, and no debugger. It has preemptive priority scheduling with a bounded starvation guarantee, a checkable heap, software-enforced memory isolation on hardware that offers none, a bytecode virtual machine with a host toolchain, nine device drivers, a touch-driven interface, a 3D renderer, persistent state, three independent failure-reporting channels, and an 802.11 receiver reached through a vendor binary in a different calling convention.

It fits in 37,248 bytes.

Every measurement in it was taken on hardware. Every retracted conclusion is still in the record next to its correction. Every mechanism that has never been observed to work says so.

The gaps in Chapter 30 are long because somebody wrote them down.

Part VI ends here. The appendices follow.

A. File Inventory

Every file in the tree, what it does, and which chapter covers it.

A.1 The kernel

Sizes are source bytes. 671,884 bytes across `kernel/` in total.

Entry, vectors, and the machine

File	Bytes	Contents	Chapter
<code>start.S</code>	1,343	The entry stub: stack, .bss clear, call0 kmain	3
<code>vectors.S</code>	8,807	Vector slots, the level-3 handler and its 21-word frame, the panic entry	7, 8
<code>window.S</code>	18,744	Six register-window handlers, the windowed↔call0 bridges, <code>phy_stack_call</code>	2, 27
<code>window.h</code>	3,376	Bridge declarations, with the verification rationale for each	2, 27
<code>xtensa.h</code>	3,519	Special-register accessors with their required synchronisation	7
<code>linker.ld</code>	7,095	IRAM / DRAM / DROM, vector placement, <code>.dram.rodata</code> , heap bounds	4

Scheduling and time

File	Bytes	Contents	Chapter
<code>task.c</code>	26,656	Task table, fabricated frames, the selection loop, ageing, CPU accounting, <code>task_sleep</code>	8, 9
<code>task.h</code>	10,185	Frame layout, priorities, ageing constants, the blocking protocol	8, 9
<code>timer.c</code>	5,602	CCOMPARE1 tick, three defects' worth of comments	7
<code>timer.h</code>	763		7
<code>critical.h</code>	1,341	<code>crit_enter/crit_exit</code> , and a capitalised warning about misuse	11
<code>mutex.c</code>	3,785	Direct handoff, the granted bitmask, non-owner refusal	11
<code>mutex.h</code>	2,864		11

Memory

File	Bytes	Contents	Chapter
<code>heap.c</code>	8,582	Address-ordered list, split/coalesce, ten-code <code>heap_check()</code>	10
<code>heap.h</code>	2,292		10
<code>arena.c</code>	3,154	Arena lifecycle, zeroing, <code>arena_contains()</code>	10
<code>arena.h</code>	2,040	The isolation statement	1, 10

File	Bytes	Contents	Chapter
kstring.c	1,996	memcpy, memset, memmove, memcmp	5, 14

The virtual machine

File	Bytes	Contents	Chapter
vm.h	11,572	The ISA, the syscall table, the fault codes, vm_t	13, 14
vm.c	23,405	Dispatch, checked accessors, vp_fill/vp_text, twelve syscalls	14, 17
app.c	6,830	Application table, retire(), app_tick()	16
app.h	4,126	Strip geometry and the close-button argument	16, 24
ipc.c	2,631	Mailboxes, copy-in/copy-out	16
ipc.h	2,291	The "copied, never shared" rationale	16

Drivers

File	Bytes	Contents	Chapter
display.c	32,900	Bit-banged and SPI2 backends, DMA, spans, font, panic mode, display_resync	18
display.h	6,850	Pin map, lock timing names, panic-mode contract	18
touch.c	21,247	XPT2046 burst, PD handling, pressure gate, calibration table	19
touch.h	5,011		19
calib.c	10,348	Four inset targets, the pair check, the fit	19
calib.h	1,407		19
gpio.h	8,237	Two-bank accessors, IO_MUX table, pin interrupts, the measured bit 15	19, 22
flash.c	11,450	SPI1 user mode, register save/restore, the explicit clock divider	20
flash.h	2,697		20
store.c	3,902	The 13-word record, checksum, version, one-writer discipline	20
store.h	3,148		20
sd.c	9,550	Bit-banged SPI card init and block read	21
sd.h	3,950	Seven per-stage error codes, the SPI-mode argument	21
intr.c	8,853	Routing, dispatch, the hang defence, intr_selftest	22
intr.h	4,313	Sources vs lines	22
adc.c	17,585	SAR ADC1 and four probes	22
adc.h	3,597		22
i2c.c	9,242	Bit-banged master, clock stretching, the two-direction self-test	22

File	Bytes	Contents	Chapter
i2c.h	3,615		22
audio.c	12,615	LEDC PWM, the DAC post-mortem, probes with their limits printed	23
audio.h	3,519		23
uart.c	4,387	TX FIFO, the AHB receive alias, the output tee	12, 26
uart.h	635		12
console.c	433	Message-granularity console lock	11
console.h	1,335		11
efuse.c	2,910	Factory MAC, verified against the on-chip CRC8	27
efuse.h	848		27
phyinit.c	5,098	The radio blob's clock, init data, calibration buffer	27
phyinit.h	264		27
wifimac.c	44,812	MAC init, liveness by motion, TSF, rx/tx chains, beacon decode	27
wifimac.h	6,579	The behavioural TSF identification	27
wifi_osi_impl.c	14,972	Static pools, tagged handles, the call0 side of the OSI table	27
wifi_osi_impl.h	2,228		27

Interface

File	Bytes	Contents	Chapter
desktop.c	24,845	Icon grid, cursor, press latching, chrome, the overlay, six assertions	24
desktop.h	5,981	The launcher-not-window-manager argument, the cursor argument	24
raycast.c	22,003	Fixed-point marcher, face shading, wall seams, navigation, framebuffer	25
raycast.h	3,024		25
notes.c	15,089	Multi-tap keypad, compose and inbox views	26
notes.h	2,101	The native-code limitation	26
messages.c	3,193	Flash-backed message store, separate sector	26
messages.h	1,929		26
term.c	15,002	On-panel shell: layout, ring scrollbar, keypad, three assertions	26
term.h	1,676		26
shell.c	53,226	~70 commands, execute(), shell_run_line()	12, 16, 26
shell.h	1,921	The program table type	16

Failure handling

File	Bytes	Contents	Chapter
panic.c	8,141	Two entry points, record-before-report, the panel screen, halt_forever	12
panic.h	790		12
watchdog.c	5,108	Disable-all, TIMG0 arm/feed/disarm, liveness on distinct switches	12
watchdog.h	2,284		12

Kernel entry

File	Bytes	Contents	Chapter
kmain.c	74,035	Nine tasks, six self-tests, the boot report, the colour strip, the reporter	6, 8, 9, 16

Generated

kernel/generated/ holds build products, one header per tools/*.vasm plus the sine table. Never edited; regenerated on every build.

```
app_a.h app_b.h app_blit.h app_draw.h app_gfx_rogue.h app_paint.h
app_ping.h app_pong.h app_rogue.h demo.h sintab.h spin.h
```

A.2 Tools

File	Lines	Contents	Chapter
vasm.py	381	Two-pass bytecode assembler; emits a C header with label offsets	15
gen_sintab.py	33	Fixed-point sine table generator	25
demo.vasm	49	sum(1..10) through memory, a call, four syscalls	15
spin.vasm	23	The kernel's hosted program: a bounds-checked store per iteration	15
app_a.vasm	13	Counter, 3 instructions per iteration	15, 16
app_b.vasm	17	Squares, 4 instructions per iteration	15, 16
app_rogue.vasm	31	Walks a store off the end of its arena	15, 16
app_gfx_rogue.vasm	38	Asks to fill the whole panel, forever	15, 17
app_paint.vasm	40	The first interactive program	15, 17
app_blit.vasm	46	Builds an image and blits it	15, 17
app_draw.vasm	49	Draws text and rectangles	15
app_ping.vasm	29	Sends a counter to application 1	15, 16
app_pong.vasm	47	Receives and replies	15, 16

A.3 Vendor

Path	Bytes	Contents	Chapter
vendor/bootloader.bin	17,536	ESP-IDF second-stage bootloader, Apache 2.0	3
vendor/partitions.bin	3,072	Partition table, Apache 2.0	3
vendor/README.md	1,915	What they are, where they came from, how to rebuild them	3
vendor/windowed/	—	-mabi=windowed C objects: the OSI table, the ROM-symbol answers	2, 27
vendor/phy/	—	libphy_natos.a, libpp_natos.a — patched Espressif archives	5, 27

A.4 Build and docs

Path	Contents	Chapter
build.ps1	Assemble bytecode → compile call0 → compile windowed → link → elf2image → flash	5
README.md	Project overview, the two shaping decisions, the shell table	—
LICENSE	MIT	—
docs/README.md	Report index, status table, twelve standing rules	29
docs/UM-NATOS-0NN-*.md	Twenty-eight engineering reports	Appendix E
docs/pdf/	Rendered PDFs of every report	—
docs/style/	build_pdfs.py, figures.py, report.css	—
docs/book/	This book	—

A.5 The ten largest files, and what that says

Rank	File	Bytes
1	kmain.c	74,035
2	shell.c	53,226
3	wifimac.c	44,812
4	display.c	32,900
5	task.c	26,656
6	desktop.c	24,845
7	vm.c	23,405
8	raycast.c	22,003
9	touch.c	21,247

Rank	File	Bytes
10	window.S	18,744

Two observations.

kmain.c and **shell.c** are the two largest files and neither is a subsystem. They are, respectively, the boot report plus six self-tests, and roughly seventy diagnostic commands (UM-NATOS-026 counted 25 when it was written; the growth since is almost entirely the bring-up probes of Chapters 22, 23 and 27). About a fifth of the kernel's source is instrumentation — which, given Chapter 28, is the correct proportion.

task.c at 26,656 bytes implements a scheduler in 689 lines, of which a substantial fraction is comment. The kernel proper — `start.S`, `vectors.S`, `task.c`, `timer.c`, `heap.c`, `arena.c`, `mutex.c`, `critical.h` — is under 55 KB of source and compiles to a few thousand bytes of text.

B. Bytecode ISA Reference

Derived from `kernel/vm.h` (authoritative) and `tools/vasm.py` (the producer).

B.1 Machine model

Property	Value
Registers	16, <code>r0</code> – <code>r15</code> , 32-bit, no special roles
Instruction width	4 bytes, fixed
PC	Byte offset into the arena; must be 4-byte aligned
Call stack	32 entries, in kernel memory , unaddressable by the program
Address space	The arena only. Offset 0 is the start of the loaded image
Condition codes	None. Comparisons write 0 or 1 to a register
Endianness of immediates	Little — byte 2 is the low half

B.2 Encoding

byte 0	opcode	
byte 1	a	destination register, or the sole operand
byte 2	b	source register, or immediate low byte
byte 3	c	source register / offset, or immediate high byte

Formats, as `vasm.py` names them:

Format	Operands	a	b	c
none	—	0	0	0
ab	<code>rA</code> , <code>rB</code>	reg	reg	0
abc	<code>rA</code> , <code>rB</code> , <code>rC</code>	reg	reg	reg
ai	<code>rA</code> , <code>imm16</code>	reg	imm low	imm high
i	<code>label</code>	0	disp low	disp high
air	<code>rA</code> , <code>label</code>	reg	disp low	disp high
abo	<code>rA</code> , <code>rB</code> , <code>off</code>	reg	reg	0–255
sys	<code>name</code> <code>n</code>	0	num low	num high

Register validation. Every operand that is an index is checked against 16 and faults with `VM_FAULT_REG` otherwise — including operands an opcode does not use, because "operands that an opcode does not use are still indices in a well-formed program". `HALT`, `NOP` and `RET` are exempt entirely; immediate and offset forms exempt `b` and `c` respectively.

Branch displacements count INSTRUCTIONS, signed 16-bit, relative to the instruction *after* the branch:

```
target_pc = next_pc + simm * 4
```

The assembler refuses a misaligned target and a displacement outside -32768...32767.

B.3 Opcodes

Control — 0x00–0x04

Op	Mnemonic	Format	Semantics
0x00	halt	none	Stop; vm_run returns VM_RUN_HALTED
0x01	nop	none	—
0x02	mov rA, rB	ab	$a \leftarrow b$
0x03	ldi rA, imm	ai	$a \leftarrow \text{imm16 (zero-extended)}$
0x04	ldih rA, imm	ai	$a \leftarrow (a \ \& \ 0xFFFF) \ \ (\text{imm16} \ll 16)$

LDI then LDIH is how a 32-bit constant is built. `app_blit.vasm` uses the pair to make `0x001FFFE0` — two RGB565 pixels in one word.

Arithmetic and logic — 0x10–0x1D

Op	Mnemonic	Format	Semantics
0x10	add rA, rB, rC	abc	$a \leftarrow b + c$
0x11	sub rA, rB, rC	abc	$a \leftarrow b - c$
0x12	mul rA, rB, rC	abc	$a \leftarrow b \times c$
0x13	div rA, rB, rC	abc	$a \leftarrow b / c$; $c == 0$ faults
0x14	mod rA, rB, rC	abc	$a \leftarrow b \% c$; $c == 0$ faults
0x15	and rA, rB, rC	abc	
0x16	or rA, rB, rC	abc	
0x17	xor rA, rB, rC	abc	
0x18	shl rA, rB, rC	abc	$a \leftarrow b \ll (c \ \& \ 31)$
0x19	shr rA, rB, rC	abc	logical, $a \leftarrow b \gg (c \ \& \ 31)$
0x1A	sar rA, rB, rC	abc	arithmetic, $a \leftarrow (\text{int32})b \gg (c \ \& \ 31)$
0x1B	not rA, rB	ab	$a \leftarrow \sim b$
0x1C	neg rA, rB	ab	$a \leftarrow -b$
0x1D	addi rA, imm	ai	$a \leftarrow a + \text{sign_extend}(\text{imm16})$

Shift counts are **masked to 5 bits**, not faulted:

A shift of 32 or more is undefined in C and would let a program's behaviour depend on the host compiler, which is precisely what a VM exists to prevent.

Arithmetic wraps; there is no overflow trap.

Comparison — 0x20–0x25

All write 0 or 1 into rA.

Op	Mnemonic	Semantics
0x20	seq rA, rB, rC	b == c
0x21	sne rA, rB, rC	b != c
0x22	slt rA, rB, rC	(int32)b < (int32)c
0x23	sltu rA, rB, rC	b < c unsigned
0x24	sle rA, rB, rC	(int32)b <= (int32)c
0x25	sleu rA, rB, rC	b <= c unsigned

Comparisons produce 0 or 1 into a register rather than setting flags, so branches need only test a register for zero. That removes a whole class of state — condition codes with their own save/restore obligations across a context switch.

Control flow — 0x30–0x34

Op	Mnemonic	Format	Semantics
0x30	jmp label	i	unconditional
0x31	brz rA, label	air	branch if a == 0
0x32	brnz rA, label	air	branch if a != 0
0x33	call label	i	push next_pc; branch. Depth 32; overflow faults
0x34	ret	none	pop; empty stack faults

Memory — 0x40–0x43

Address is rB + c, where c is an unsigned byte offset 0–255. **Every access is bounds-checked against the arena.**

Op	Mnemonic	Semantics	Extra check
0x40	ldw rA, rB, off	a ← u32 at [b+off]	4-byte aligned
0x41	ldb rA, rB, off	a ← u8 at [b+off]	—
0x42	stw rA, rB, off	u32 at [b+off] ← a	4-byte aligned
0x43	stb rA, rB, off	u8 at [b+off] ← a	—

Services — 0x50

Op	Mnemonic	Format
0x50	sys name	sys

B.4 Syscalls

Arguments in `r0 – r5`; results in `r0 – r2`.

#	Name	Arguments	Result	Ends slice?
0	exit	<code>r0 = status</code>	— (halts)	—
1	putc	<code>r0 = character</code>	—	no
2	puts	<code>r0 = arena offset of a NUL-terminated string</code>	—	no
3	putd	<code>r0 = value</code>	—	no
4	ticks	—	<code>r0 ← timer_ticks()</code>	no
5	fill	<code>r0=x r1=y r2=w r3=h r4=colour</code>	—	yes
6	text	<code>r0=str r1=x r2=y r3=fg r4=bg r5=scale</code>	—	yes
7	dims	—	<code>r0 ← (w << 16) ∨ h</code>	no
8	touch	—	<code>r0←touched r1←x r2←y</code>	no
9	blit	<code>r0=offset r1=x r2=y r3=w r4=h</code>	<code>r0 ← 1 drawn, 0 not</code>	yes
10	send	<code>r0=dest id r1=offset r2=len</code>	<code>r0 ← 1 delivered, 0 refused</code>	no
11	recv	<code>r0=offset r1=max</code>	<code>r0 ← bytes, r1 ← sender</code>	no

The slice rule: a syscall whose cost is measured in milliseconds ends the caller's slice regardless of remaining quantum; one that reads a published snapshot does not.

Confinement: - Display coordinates are viewport-relative; a program is never told where its viewport sits. - A touch outside the viewport is reported as **no touch at all**. - `send` names a destination **application**, never an address. - `blit` bounds `w` and `h` against the panel *before* multiplying, so `w × h × 2` provably cannot wrap. - `text` and `puts` walk the string one bounds-checked byte at a time; an unterminated string faults rather than reading past the arena.

Limits: message body ≤ 64 B (`IPC_MSG_MAX`); string buffer 48 B (`VM_STR_MAX`), truncated not faulted; text scale clamped to 1–4.

B.5 Faults

Code	Name	Raised by
0	NONE	—
1	OPCODE	Unknown opcode byte
2	REG	Register index ≥ 16
3	BOUNDS	Access outside the arena
4	ALIGN	Misaligned word access, or odd blit offset

Code	Name	Raised by
5	DIV0	div/mod by zero
6	PC	PC outside the arena, or misaligned
7	CALL_DEPTH	33rd nested call
8	RET	ret with an empty call stack
9	SYSCALL	Unknown service number
10	STRING	puts string unterminated inside the arena

Six of the ten have been exercised on hardware. PC, CALL_DEPTH, SYSCALL and STRING are implemented and untested.

Every fault records the code, the PC of the faulting instruction, and a detail value (the offending offset, opcode, register index or syscall number).

B.6 vm_run() return values

Value	Meaning
VM_RUN_HALTED	HALT or SYS_EXIT. The program finished
VM_RUN_FAULTED	See vm_fault(). Terminal
VM_RUN_QUANTUM	Budget spent, or an expensive syscall ended the slice. Call again

State lives entirely in vm_t, so a resumption continues at the exact instruction boundary where it stopped.

B.7 Assembler directives

Directive	Effect
label:	Records the current byte offset
.string "..."	Bytes plus a NUL. Escapes: \n \r \t \0 \\ \" \'
.byte a, b, ...	Raw bytes
.word v, ...	32-bit little-endian; pads to 4-byte alignment first
.align n	Pad to a multiple of n
.space n	n zero bytes
; ... or # ...	Comment (string-aware)

Values accept decimal, 0x hex, 'c' character literals, a bare label name, and @label for a label's byte offset.

Instructions are padded to 4-byte alignment automatically, because "the VM faults otherwise, so pad rather than emit something that cannot execute".

B.8 A worked decode

tools/spin.vasm, assembled to 28 bytes:

Offset	Bytes	Decode
0	03 01 18 00	ldi r1, 0x0018 — @counter
4	03 02 00 00	ldi r2, 0
8	03 09 01 00	ldi r9, 1
12	10 02 02 09	add r2, r2, r9
16	42 02 01 00	stw r2, r1, 0
20	30 00 fd ff	jmp -3 → offset 12
24	00 00 00 00	counter: .word 0

Displacement check: the `jmp` is at 20, so `next` is 24; $24 + (-3 \times 4) = 12$, which is `loop:`.

B.9 Producing a program

```
python tools/vasm.py tools/myprog.vasm -o kernel/generated/myprog.h --name
vm_myprog
```

Emits `vm_myprog[]`, `VM_MYPROG_LEN`, and `VM_MYPROG_AT_<LABEL>` for every label. `build.ps1` does this automatically for every `tools/*.vasm`.

To make it launchable, add a row to `PROGRAMS[]` in `kmain.c`:

```
{ "myprog", vm_myprog, VM_MYPROG_LEN, 512u, VM_MYPROG_AT_COUNTER },
```

— name, image, length, arena bytes, and the offset of the progress word the kernel reads.

C. Shell Command Reference

The shell is a native task polling UART0, and — since Chapter 26 — the same `execute()` reached from an on-panel keypad. **One command set, not two.**

Over serial at 115200. On the panel, via the `shell` icon.

UM-NATOS-026 records "all 25 commands reachable without a host" as the metric at the time it was written. The table has since grown to roughly seventy, most of them the bring-up probes of §C.8 — the claim that matters is unchanged, because there is still exactly one command set.

C.1 Everyday commands

Command	Effect
<code>help</code>	The command list
<code>ps</code>	Applications: id, name, state, arena size, instructions retired, published value, fault diagnosis
<code>progs</code>	Loadable programs with image and arena sizes
<code>run <name></code>	Start a program by name , never by index
<code>kill <id></code>	Stop an application and release its arena
<code>mem</code>	<code>free</code> , <code>largest</code> , <code>blocks</code> , <code>high_water</code> , <code>check</code>
<code>stacks</code>	Per-task stack headroom, in bytes free of 2,048
<code>3d [off]</code>	Hand the region to the raycaster, or back to the launcher
<code>fb</code> <code>[on\ off]</code>	Framebuffer for the 3D view

`mem`'s three numbers are chosen so no one of them can hide a problem the others would show: `largest` beside `free` is the fragmentation indicator, and `check` beside both is the structural one.

C.2 Storage

Command	Effect
<code>sd</code>	Probe the microSD card; reports the failing stage by name if it fails
<code>sdread <lba></code>	Read and dump one 512 B block, decoding an MBR or FAT signature

`sdread 0` proves the bus. `sdread 240` proves the addressing mode — see Chapter 21 §21.5 for why those are different claims.

C.3 Touch

Command	Effect
cal	Run the four-target calibration on the panel
calshow	The last run's outcome, all four readings, and the calibration in use
taps	Dump the touch press log — raw and mapped coordinates, pressure
tapsclear	Empty it
touchcfg <prio> <sleep>	Retune touch scheduling without a reflash

`calshow` exists because the calibration's result line is printed the instant the fourth target is tapped — "exactly when nobody has a capture attached".

`touchcfg` is the runtime-tunable knob that turned four hypotheses into one flash (Chapter 27 §27.11).

C.4 Sensors and buses

Command	Effect
adc	Read every ADC1 channel
ldr	Watch the light sensor only
ldrscan	Watch all eight channels at once — the four touch pins are the control group
i2c	Self-test both lines in both directions, then scan 0x08–0x77
intr	Interrupt matrix counters: per-line service counts, spurious, disabled mask
tone <hz>	Tone on GPIO26. tone 0 stops. Try 3000, not 440
beep	A short 3 kHz beep

`i2c` refuses to let a scan be believed unless the bus test passed:

bus looks sane; a scan can be believed

or

bus is NOT sane; ignore any scan result

C.5 Deliberate failure

Command	Induces	Expected
hang	Masks interrupts, spins	Watchdog resets the board — rst:0x7
fault	Executes ill	Panic, halt, evidence retained on serial, flash and panel
smash	Clobbers the running task's guard word	Panic at the next switch, naming the task

The last three exist on purpose. A recovery path that has never been observed to fire is confidence without evidence.

C.6 Mixed-ABI and vendor code

Command	Effect
wintest [n]	Call windowed-ABI code at recursion depth n; the return value is a checksum over every handler invocation
vendorcall [n]	Call a -mabi=windowed object built by the vendor compiler
romcall	Call <code>crc32_1e</code> at <code>0x4005CFEC</code> in the ESP32 ROM
bridgetest	A windowed function calling <i>back</i> into <code>call0</code> code every iteration

Four escalating proofs, each establishing something the previous one did not (Chapter 2 §2.7).

C.7 WiFi

Command	Effect
phyver	PHY version, from the blob's own <code>printf</code> shim
phyinit	Ungate the radio clock and call <code>register_chipv7_phy</code>
macinit	Bring up the MAC (needs <code>phyinit</code> first)
maclive	Which MAC registers move on their own — liveness by motion
mactsf	TSF rate, counted against the CPU's own cycle counter
macaddr	Factory MAC from eFuse, verified against its stored CRC8
macirq	Route the MAC interrupt onto a CPU line
macrx	Arm the receiver, promiscuous
chan <1..13>	Tune a channel
scan	Decoded beacons: BSSID, SSID, count
macstat	Frame counters, recycled descriptors
beacon / beaconoff	Transmit a discoverable SSID
probe	Send probe requests and count frames addressed to us
txstat	Frames handed to hardware / completions reaped / forced
txpwr	Read and set <code>most_tpw</code>
hwinit	The MAC hardware init chain
ositest	Drive the OSI table through its function pointers, windowed→ <code>call0</code> and back

`probe` is the unforgeable transmit test: a probe *response* addressed to this station cannot be produced by any local fault.

C.8 Bring-up probes

Grouped separately in `help`, with a warning:

bring-up probes (they poke at live hardware):

Command	Effect
irqtest	Inject an edge per candidate INT_ENA bit; report which reaches this CPU
irqpoke	Inject one GPIO edge without touching the pin's configuration
adcprobe	Sweep the RTC sensor-pad mux
adconv	Prove a conversion actually runs — DONE low, then high 13 spins later
adcdri ve	Sweep against GPIO32, a pin this kernel drives . Also drives touch MOSI
findspk	Square wave on each candidate speaker pin
spktest	Prove the square-wave generator works — and prints its own blind spot

`irqtest` re-derives the PRO CPU enable bit on any board rather than trusting a comment. `adcdrive` commands the input voltage rather than observing it.

C.9 Display and measurement

Command	Effect
spicl k 0\ 1\ 2	Panel clock: 40 / 20 / 10 MHz. Returns the register read back
dfreeze	Stop every drawer in the display task, leaving the buffer static
fbsum	Sample the framebuffer — used with <code>dfreeze</code> and a control reading
blittest	Static colour-bar test pattern through the raycaster's exact blit call
resync	Re-issue the panel window and end the transaction, drawing nothing
fifopoke	A 16-byte FIFO transfer, bypassing DMA
sleep test	Measure what <code>task_sleep</code> actually costs
tickrate	Ticks against wall clock
efusedump	eFuse block 0
audio	LEDC register read-back, including the clock gate

Four of these — `spiclk`, `dfreeze`, `blittest`, `resync` — were built during a single investigation and are the reason it produced a positive result. Chapter 27 §27.13 names the method:

Make the variable runtime-tunable before forming a theory about it.

C.10 Behaviour

Every command is parsed once. `shell_run_line()` copies a caller's string into the shell's own buffer and calls `execute()` — the same path a serial line takes. A line

longer than the buffer is **refused, not truncated**, because "a truncated command is a different command, and `kill 12` truncated to `kill 1` kills something".

One command, one uninterrupted response. `execute()` holds the console lock for its whole run, so a telemetry line cannot land in the middle of a table.

Unknown commands are rejected, not ignored.

`shell_poll()` **never blocks**, so a user holding a key cannot starve the system.

Receive is polled, not interrupt-driven — "a console that drops a keystroke under load is a better outcome than a second interrupt source competing with the scheduler tick".

Enter works. It did not, for the shell's entire existence before Chapter 12 §12.5, because the RX FIFO was read through the APB address rather than its AHB alias and the path ran exactly one byte behind.

D. Address and Register Map

Every hardware address this kernel touches, with the file that owns it and the chapter that explains it.

Evidence grading (Chapter 0b): **M** = measured on hardware, **T** = transcribed from a vendor header or manual, **R** = reverse-engineered or identified behaviourally.

D.1 Memory regions

Region	Range	Grade	Chapter
SRAM0 (IRAM view)	0x40070000–0x400A0000	T	4
SRAM1 (DRAM view)	0x3FFE0000–0x40000000	T	4
SRAM1 (IRAM view)	0x400A0000–0x400C0000	T	4
SRAM2 (DRAM view)	0x3FFAE000–0x3FFE0000	T	4
Flash instruction (XIP)	0x400D0000+	T	4
Flash read-only data	0x3F400000+	T	4

What the linker script claims

Region	Origin	Length	Grade
iram	0x40080000	0x20000 (128 KB)	M — confirmed by the 24 KB span experiment
dram	0x3FFB0000	0x2C200 (~176 KB)	M — stack top read back as 0x3FFDC200
drom	0x3F400020	0x400000 - 0x20	M — every printed string is .rodata

The 0x20 offset makes the virtual address congruent with the flash offset across a 64 KB MMU page: 24-byte image header plus 8-byte segment header.

Symbols the linker defines

Symbol	Meaning
_vecbase	Start of .vectors; 1024-byte aligned
_bss_start, _bss_end	Zeroed by start.S
_rodata_start, _rodata_end	Flash-mapped read-only data
_stack_top	ORIGIN(dram) + LENGTH(dram) = 0x3FFDC200
_boot_stack_size	4 KB, permanently reserved
_heap_start, _heap_end	Everything between .bss and the boot stack

D.2 Vector offsets from `VECBASE`

Offset	Slot	Populated
0x000	Window overflow 4	yes
0x040	Window underflow 4	yes
0x080	Window overflow 8	yes
0x0C0	Window underflow 8	yes
0x100	Window overflow 12	yes
0x140	Window underflow 12	yes
0x180	Level 2	—
0x1C0	Level 3	yes → <code>_handler_level3</code>
0x200	Level 4	—
0x240	Level 5	—
0x280	Debug	—
0x2C0	NMI	—
0x300	Kernel exception	yes → <code>_handler_panic</code>
0x340	User exception	yes → <code>_handler_panic</code>
0x3C0	Double exception	yes → <code>_handler_panic</code>

Each slot is 64 bytes and holds a single `j`. Verified in the linked ELF with `nm before flashing` (Chapter 4 §4.3).

D.3 Flash layout

Offset	Contents	Size
0x1000	Second-stage bootloader (borrowed)	17,536 B
0x8000	Partition table (borrowed)	3,072 B
0x10000	nat-os image	37,248 B
0x200000	Persistent record sector	4,096 B
0x201000	Message store sector	4,096 B

D.4 Xtensa special registers

Accessed through `xtensa.h`, which bakes in the required synchronisation.

Register	Access	Sync	Use
CCOUNT	read	—	Free-running cycle counter
CCOMPARE1	read/write	ESYNC	The tick; writing also acknowledges
INTENABLE	read/write	ESYNC	Per-line enable mask

Register	Access	Sync	Use
INTERRUPT	read	—	Lines asserting now, regardless of enable
PS	read/write	RSYNC	INTLEVEL in bits 3:0, EXCM bit 4
VECBASE	read/write	ISYNC	Vector table base
EPC1 / EPC3	read	—	Interrupted PC (exception / level 3)
EPS3	read	—	Interrupted PS
EXCCAUSE	read	—	Exception cause code
SAR	read/write	—	Shift amount; saved in the frame
LBEG/LEND/LCOUNT	read/write	—	Zero-overhead loop; saved in the frame
WINDOWBASE/WINDOWSTART	read/write	RSYNC	Windowed excursions only

Interrupt sources and lines:

Constant	Value	Meaning
XT_TIMER1_INTERRUPT	15	CCOMPARE1, internal, level 3
INTR_LINE_GPIO	23	Level 3, level-triggered
INTR_LINE_WIFI_MAC	27	Level 3, level-triggered
INTR_SRC_GPIO_PRO	22	The GPIO peripheral source
INTR_SRC_WIFI_MAC	0	First entry in the silicon's table

D.5 DPORT

Address	Register	Use	Chapter
0x3FF00104 + 4×src	PRO CPU interrupt map	One word per source; holds a line number	22
0x3FF000C0	PERIP_CLK_EN	SPI2 bit 6, LEDC bit 11, SPI DMA bit 22	18, 23
0x3FF000C4	PERIP_RST_EN	Same bit assignments	18, 23
0x3FF000CC	WIFI_CLK_EN	WiFi/BT common clock, mask 0x3C9	27
0x3FF005A8	SPI_DMA_CHAN_SEL	Bits 2:3 select SPI2's DMA channel	18

Bit 1 in the same register clocks the flash controller this code executes from, so the write is read-modify-write and never a plain store.

D.6 GPIO

Address	Register	Bank
0x3FF44008 / 0x3FF4400C	OUT_W1TS / OUT_W1TC	0–31
0x3FF44014 / 0x3FF44018	OUT1_W1TS / OUT1_W1TC	32–39
0x3FF44024 / 0x3FF44028	ENABLE_W1TS / _W1TC	0–31
0x3FF44030 / 0x3FF44034	ENABLE1_W1TS / _W1TC	32–39

Address	Register	Bank
0x3FF4403C / 0x3FF44040	IN / IN1	
0x3FF44044 / 0x3FF4404C	STATUS / STATUS_W1TC	0–31
0x3FF44050 / 0x3FF44058	STATUS1 / STATUS1_W1TC	32–39
0x3FF44054	STATUS1_W1TS	Edge injection — Chapter 22 §22.5
0x3FF44088 + 4×n	GPIO_PIN<n>	INT_TYPE 9:7, INT_ENA 17:13
0x3FF44530 + 4×n	FUNC<n>_OUT_SEL_CFG	256 = SIG_GPIO_OUT_IDX

GPIO_PIN_INT_ENA_PRO is bit 15 — measured, not read. Bit 13 (the field's low bit, matching the vendor header) delivers to the APP CPU, which this kernel never starts.

IO_MUX — not in pin order

Pin	Address	Pin	Address
GPIO2	0x3FF49040	GPIO25	0x3FF49024
GPIO12	0x3FF49034	GPIO26	0x3FF49028
GPIO13	0x3FF49038	GPIO27	0x3FF4902C
GPIO14	0x3FF49030	GPIO32	0x3FF4901C
GPIO15	0x3FF4903C	GPIO33	0x3FF49020
GPIO21	0x3FF4907C	GPIO36	0x3FF49004 (SENSOR_VP)
GPIO22	0x3FF49080	GPIO39	0x3FF49010 (SENSOR_VN)

The trap: 0x84 is U0RXD and 0x88 is U0TXD, between GPIO22 (0x80) and GPIO23 (0x8C). Counting up from GPIO21 puts GPIO23 on the console's receive pad (Chapter 21 §21.4).

Fields: MCU_SEL 14:12 (2 = GPIO), FUN_DRV 11:10, FUN_IE bit 9, FUN_PU bit 8.

D.7 Peripherals

UART0

Address	Register
0x3FF40000	FIFO — write to transmit
0x3FF4001C	STATUS — TX count 23:16, RX count 7:0
0x60000000	AHB alias — read to receive

The APB address for receive runs exactly one byte behind. Behavioural finding; no erratum cited (Chapter 12 §12.5).

SPI1 — flash

0x3FF42000 base. CMD +0x00, ADDR +0x04, CTRL +0x08, CTRL2 +0x14, CLOCK +0x18, USER +0x1C, USER1 +0x20, USER2 +0x24, MOSI_DLEN +0x28, MISO_DLEN +0x2C, w(n) +0x80.

Six registers saved and restored per transaction, **including on the failure path**. Clock set explicitly to ~8 MHz; the inherited cache-read divider samples MISO one clock early.

SPI2 — display

0x3FF64000 base, same layout. Plus DMA: DMA_CONF +0x100, DMA_OUT_LINK +0x104, DMA_INT_CLR +0x11C.

SPI2_CLKDIV = 0x00001001 — sysclk/2, 40 MHz. The ceiling on this board, established by trying 80 MHz and looking at the panel.

Watchdogs

Address	Register
0x3FF4808C / 0x3FF480A4	RTC WDTCONFIG0 / WDTWPROTECT
0x3FF5F048 / 0x3FF5F064	TIMG0 WDTCONFIG0 / WDTWPROTECT
0x3FF5F04C / 0x3FF5F050 / 0x3FF5F060	TIMG0 prescaler / stage 0 / feed
0x3FF60048 / 0x3FF60064	TIMG1 WDTCONFIG0 / WDTWPROTECT

Write-protect key 0x50D83AA1, shared by all three. Re-locked after every change.

SAR ADC1

0x3FF48800 (SENS) base. READ_CTRL +0x0000, MEAS_WAIT2 +0x000C, START_FORCE +0x002C, ATTEN1 +0x0034, MEAS_START1 +0x0054.

SAR1_EN_PAD_FORCE is bit 31 — written as bit 27 from memory, which is *inside* the twelve-bit SAR1_EN_PAD field at shift 19 (Chapter 22 §22.10).

RTC IO base 0x3FF48400; PAD_DAC1 +0x84, PAD_DAC2 +0x88.

LEDc

0x3FF59000 base. HSCH0_CONF0 +0x0000, HPOINT +0x0004, DUTY +0x0008, CONF1 +0x000C, HSTIMER0_CONF +0x0140, CONF +0x0190.

GPIO matrix signal 71 = LEDC_HS_SIG_OUT0. **Clock-gated out of reset** — DPORT bit 11.

eFuse

0x3FF5A004 = BLK0_RDATA1 (MAC[31:0]), 0x3FF5A008 = BLK0_RDATA2 (MAC[47:32] low half, CRC8 bits 23:16). **Documented**, unlike the rest of the WiFi path.

WiFi MAC

0x3FF73000 base. CTRL 0x3FF73CB8. TSF 0x3FF73C00 — identified behaviourally as the only word advancing at exactly 1000 kHz across repeated samples, confirmed against CCOUNT to 0.1% over half a second.

Reverse-engineered from open-mac except where noted.

D.8 Board pin assignments

Function	Pins
Display	SCLK 14, MOSI 13, MISO 12, CS 15, DC 2, BL 21
Touch	CLK 25, MOSI 32, MISO 39, CS 33, IRQ 36
microSD	CS 5, SCK 18, MISO 19, MOSI 23
Light sensor	34 (ADC1 channel 6)
Speaker	26
I ² C	SDA 22, SCL 27
RGB LED	4, 16, 17
Console	U0TXD 1, U0RXD 3

GPIO 34–39 are **input only**, which is why touch MISO and IRQ sit on 39 and 36 — "they can never be driven by mistake".

Every other pin is spoken for. SDA 22 and SCL 27 are the only two brought out to a header and unclaimed.

D.9 Magic constants

Constant	Value	Purpose
STACK_GUARD	0x57ACC0DE	Task stack base word
STACK_FILL	0xEEEEEEEE	Untouched-stack pattern
BLK_FREE	0xF2EEB10C	Free heap block
BLK_USED	0x05EDB10C	Live heap block
MUTEX_FREE	-2	Unheld — not -1 , which is the boot context
STORE_MAGIC	0x59444F53	"SODY"
WDT_WKEY	0x50D83AA1	Watchdog write-protect key
H_TAG	0x05100000	WiFi OSI handle tag
M0 .data canary	0xC0DEFACE	Proves .data was copied
smash value	0xDEADBEEF	Deliberate guard corruption

Every one is chosen to be implausible as data, so its corruption announces itself rather than requiring interpretation.

E. Report Index and Cross-Reference

The twenty-eight engineering reports under `docs/`, their revisions, and the chapters of this book that draw on each.

E.1 The set

ID	Title	Rev	Status	Chapters
001	System Architecture and Scope	1.0	current	1, 13
002	Boot Chain and Image Format	1.0	measured on hardware	3
003	Xtensa ABI Selection	1.0	verified by codegen inspection	2
004	Memory Map and Allocation Policy	1.0	current	4
005	Build and Flash Pipeline	1.0	current	5
006	Milestone 0 Verification	1.0	PASS	6
007	Development Roadmap M1–M5	1.1	all milestones complete	6, 31
008	Milestone 1 Verification	1.1	PASS	7
009	Milestone 2 Verification	1.3	PASS	8, 9
010	Milestone 3 Verification	1.0	PASS	10
011	Flash Cache and Read-Only Data Placement	1.0	complete	4
012	Milestone 4 Verification	1.1	PASS	13, 14, 15, 17
013	Milestone 5 Verification	1.1	PASS	16
014	Locking Primitives	1.2	complete	11
015	Display Driver	1.4	complete	18, 25
016	Display Syscalls, and a Total System Freeze	1.0	complete	17
017	Touchscreen, and a Verification Method That Failed Three Times	1.2	complete	19, 28
018	Persistence, and a Read Defect That Looked Like the Wrong Thing	1.0	complete	20
019	Failure Handling, and Three Mechanisms That Had Never Fired	1.2	complete	12
020	microSD over SPI, and a Pad Table That Is Not in Pin Order	1.0	complete	21
021	The Launcher, and Four Defects It Found by Existing	1.3	complete	24
022	The Note Pad, and a Workaround Wearing a Costume	1.1	complete	26

ID	Title	Rev	Status	Chapters
023	The Interrupt Matrix, and Four Ways to Deliver an Edge to Nobody	1.0	infrastructure only	22
024	The ADC, and a Wrong Bit in a Right Register	1.0	verified	22
025	I ² C, and Why a Preempted Master Is Legal	1.0	bus only	22
026	The Shell on the Panel, and Not a Menu of It	1.1	complete	26
027	Audio, and Three Ways to Be Silent	1.0	working	23
028	WiFi, Touch, and the Column That Ate the 3D View	1.1	rx working, tx not	27, 9, 28

E.2 Reading orders

New to the project (from `docs/README.md`): 001 → 002 → 004 → 003.

Picking up implementation work: 006 (what is known-good) then 007 (what is next and what it will break).

Reproducing a build: 005 alone is sufficient.

Debugging anything: 028 §8, 017 §6, 018 §5.1 — the three catalogues of instruments that lied. This book's Chapter 28 consolidates them.

E.3 Reports that were revised, and what changed

Revision is unusually informative here, because the project revises *in place* with the superseded text left standing.

Report	Rev	What the revision added
007	1.1	§2.1 — eleven reports of unplanned work against five of planned
008	1.1	§8 — the second writer to CCOMPARE1, and 183 ms of stopped clock
009	1.1–1.3	§11 — blocking, sleeping, priorities; §11.4 — ageing
012	1.1	§10 — seven syscalls added after M4, and the check each needed
013	1.1	§8 — messaging, copied never shared
014	1.1–1.2	§9 — priority inheritance; §10 — the same defect from the other side
015	1.1–1.4	SPI2, DMA, the renderer's two defects, §5.8 overturning §5.7
017	1.1–1.2	§8 — input confinement; §4.1, §7.1–7.4 — the inverted axis and the corner calibration
019	1.1–1.2	§6 — the fault persisted; §7 — the panic screen
021	1.1–1.3	§6 — icons and the close button; §6.7 correcting §6.6

Report	Rev	What the revision added
022	1.1	The calibration fault corrected; §3.2's characterisation retracted
026	1.1	§4.1 — scrollbar
028	1.1	§10.1 — the frozen-renderer measurement

E.4 Conclusions published and later retracted

Recorded here in one place because the project's own README lists honesty about these as a rule.

Claim	Where published	Where retracted
"Watchdogs are not armed at this point in boot"	006 §6, 008 §6	009 §8 — measured, <code>rst:0x10</code> on every boot
"The IRAM origin conflicts with the bootloader"	004 §5 (original)	004 §5.1–5.3 — disproved by a 24 KB span experiment
"Pressure never exceeds 1 under a firm press"	017 (original)	017 §5.3 — actual 1123
"PENIRQ never asserts across two full drags"	017 (original)	017 §5.3 — actual 17
"Raw X increases left to right"	017 §7	017 §7.1 — it decreases
"The touch mapping reads 24 px low on X"	022 §3.2	022 §3.2 note — it is a magnification, not an offset
"SPI_CTRL2 MISO delay is disproven"	018 (during)	018 §5 — untested; the run never flashed
"The framebuffer makes no measurable difference"	015 §5.7	015 §5.8 — both figures taken through two faults
"The layout broke the 3D view"	021 §6.6	021 §6.7 — geometry alone breaks nothing; it was the camera
"A rising completion count means the frame went out"	028 §3 (initial)	028 §3 — a phone caught it
"Referencing libphy costs 48 KB of IRAM"	MAC-NEXT.md	028 §3 — 2,459 bytes
"The chrome column paints over the 3D view" (post-relayout)	desktop.c guard	desktop.c — geometry says otherwise; narrowed to an assertion

Twelve retractions across twenty-eight reports. Every one is still in the record next to its correction.

E.5 Predictions that came true

Predicted in	Prediction	Realised in
011 §4	A flash write will collide with the data cache	018 §3 — "It broke exactly as predicted, on the first version of the driver"
010 §8	arena_contains() exists and nothing calls it	012 — closed by the interpreter
010 §7	A full framebuffer and concurrent applications cannot coexist	015 — the driver has none
002 §6	Flash cache is configured but unused; will become load-bearing	011
007 §4	An incorrect register save will manifest later, in unrelated code	009 §6 — exactly that shape
007 §7	The isolation claim will be tested by a program written to escape	013 §5.2

And one that has not:

| 009 §9 | Nothing has audited the other consequences of PS.EXCM | **Still open** |

E.6 Where each standing rule came from

Rule	Report §
Clear PS.EXCM before calling C	009 §6
One writer, or every writer maintains the shadow	008 §8
A yield must never defer the clock it depends on	016 §3
Contention cost is a count, not a duration	014 §10.5
An instrument reporting itself invalid outranks the number beside it	017 §6
A negative result needs a demonstrably-run experiment	018 §5
A startup artefact is not evidence	019 §4.1
Never infer direction from a gesture's endpoint	017 §7.1
A cross-check only tests what its samples distinguish	020 §4.2
A correct diagnosis does not license arbitrary scope	021 §6.6
Tolerating a defect is not fixing it	022 §3.3
Reverting two changes destroys which one mattered	021 §6.7

E.7 The status table

`docs/README.md` maintains a live status table. Its current state, condensed:

Complete and verified on hardware: M0–M5, isolation, the full stack, locking, task blocking, scheduling with ageing, the 3D renderer, the display, application

graphics, touch, the ADC, the on-screen shell, audio, application input, application messaging, application bitmaps, the flash cache, persistence, failure handling, fault reporting, the launcher, notes, stack margins.

Infrastructure only: the interrupt matrix — "peripheral interrupts routable for the first time; PENIRQ proven by injection but never observed to fire from a finger".

Bus verified, nothing attached: I²C — "no byte has yet been transferred".

Reading only: microSD.

Measured then, unmeasured now: the DRAM budget.

Closed: the bootloader IRAM overlap, the panic handler, the watchdogs.

Ordered, not in hand: the JTAG debug probe.

E.8 The report format

Every report follows the same shape, and it is worth naming because the shape does work:

1. **Abstract** — what this establishes, in two paragraphs, including what went wrong.
2. **Design decisions** — usually a table of alternatives with the selected one marked, and the deciding argument in one sentence.
3. **Implementation** — file by file.
4. **Verification method** — how the claim was turned into a number.
5. **Results** — raw captured output, then interpretation.
6. **The defect** — where there was one, with wrong hypotheses recorded.
7. **Metrics** — a table, including process metrics like "build cycles spent" and "wrong hypotheses recorded".
8. **What this does not establish** — the gaps.
9. **References** — other reports and the source files.

Section 8 is the one the README calls "the most useful part", and Chapter 30 is all twenty-eight of them merged.

F. Every Measured Number

Every quantitative claim in this book, in one place, with its grade (**M**asured / **D**erived / **T**ranscribed) and its chapter.

F.1 The machine

Quantity	Value	Grade	Ch
CPU clock	80 MHz — not the 240 MHz maximum	D	7
Tick interval	800,000 cycles $\approx$ 10 ms	M	7, 8
Interrupt handler prologue overhead	30 cycles , constant	M	7
Internal SRAM	520 KB	T	1
DRAM claimed by the linker	180,736 B	M	4
PSRAM	none — ESP32-D0WD-V3 rev 3.1, coding scheme none	M	1
Flash	4 MB, chip ID 0x684016	M	20

F.2 Image sizes, by milestone

Milestone	.text	.data	.bss	Image
M0	1,124 B	4 B	4 B	1,216 B
M1	3,340 B	4 B	544 B	3,424 B
M2	—	—	—	5,312 B
M3	—	—	—	6,720 B
Flash cache	—	—	—	6,736 B
M4	—	—	—	10,560 B
M5	—	—	—	14,464 B
Locking	—	—	—	16,256 B
Display	—	—	—	18,896 B
Display syscalls	—	—	—	19,856 B
Touch	—	—	—	21,888 B
Current	119,151 B of 131,072 IRAM	—	—	37,248 B

F.3 Memory

Quantity	Value	Grade	Ch
Heap at M3	166,432 B	M	10
Heap after flash-mapped .rodata	167,680 B — grew by exactly the 1,248 B moved	M	4
Heap at M5 (TASK_MAX 4→8)	158,048 B	M	16
Heap now	lower, unremeasured	—	30
.rodata relocated to flash	1,248 B (0.7% of heap)	M	4
Heap header overhead	16 B per block	—	10
Boot stack reservation	4,096 B (2% of DRAM)	—	4
Full 240×320 framebuffer would cost	153,600 B = 92.3% of the heap	D	1
Span buffer instead	480 B	—	18
3D view framebuffer	80,640 B	—	25
Font	475 B, in flash, 0 DRAM	M	18
Task stack	2 KB × 12 slots	—	8
Stack headroom, worst	1,604 B free of 2,048 (app-host)	M	12
Stack headroom, best	1,844 B free of 2,048 (report)	M	12
Minimum margin across all tasks	78%	M	12
Context frame	96 B / 21 words	—	8
Mutex	24 B	—	11
Persistent record	52 B, version 3	—	20
Message store	~1.4 KB in one 4 KB sector	—	26
Terminal scrollbar ring	48 lines, under 2 KB .bss	—	26
PHY calibration buffer	1,904 B .bss	—	27
PHY private stack	6 KB, 272 B used	M	27

F.4 Scheduling

Quantity	Value	Grade	Ch
M2 ticks observed	3,418	M	8
M2 switches	1,140 / 1,139 / 1,139	M	8
M2 corruption events	0	M	8
Register checks under interruption (M1)	1,556,680, 0 corrupt	M	7
Priority levels	3, plus ageing credit up to 3	—	9
Ageing threshold	30 ticks per level	—	9
Worst-case wait for a ready task	~600 ms, bounded	D	9

Quantity	Value	Grade	Ch
Longest wait observed under stress	35 ticks (350 ms)	M	9
Ageing rescues at shipped settings	0 — inert, as intended	M	9
Reporter lines in 20 s, starved / with ageing	0 → 8	M	9
task_sleep(50) before / after	107 cycles / 639 ms (64 ticks)	M	9
timer_ticks() rate before the fix	217/s where 100 is correct	M	7
Comparator overrun before the fix	14,630,119 cycles = 18 tick periods = 183 ms	M	7
Milestones the comparator defect survived	3	—	7
Self-tests that noticed it	1 of 11	—	7

F.5 Locking

Quantity	Value	Grade	Ch
Acquisitions measured	3,133	M	11
Contentions	200	M	11
Non-owner unlocks / lost updates	0 / 0	M	11
Throughput, pathological (lock every iteration)	~2,060	M	11
Throughput, 1-in-32	~50,143	M	11
Starvation defect: worker A / worker B	238,542 / 0	M	11
Draw lock held, applications running	979 ms = 12% of wall clock	M	11
Aggregate blocked	13,051 ms in 8.08 s of wall clock	M	11
Blocked per contention vs hold	63 ms against a 24 ms hold	M	11
Effect of narrowing the hold 25%	13,051 → 13,085 ms — none	M	11
Blocked, display / application host	3,880 ms (65%) / 5,894 ms (98%)	M	11
Frames/s, blocking → best-effort	3.0 → 9.9	M	11
Frames/s with all applications killed	11.1	M	11
Primitives skipped under best-effort	45 of 1,325 = 3.4%	M	11
Raycaster: lock per column → per frame	25,075,460 μs → 129,000 μs = 194×	M	18

F.6 The heap

Quantity	Value	Grade	Ch
Alloc/free cycles tested	10,000	M	10
Blocks after test	1 — every split undone by a coalesce	M	10

Quantity	Value	Grade	Ch
High-water mark	5,120 B	M	10
Allocation failures	1 (the deliberate one)	M	10
Refused frees	2 (both deliberate)	M	10
heap_check() failures	0	M	10
Arena bounds cases, M3 / M4 cross-check	10 / 35	—	10, 14

F.7 The virtual machine

Quantity	Value	Grade	Ch
Opcodes	35 (roadmap ceiling ~40)	—	14
Registers / call depth	16 / 32	—	14
Syscalls	12, on one opcode	—	17
sum(1..10)	70 instructions	M	14
Demo program	120 B, 6 labels	M	14
Quantum resumptions during the demo	4	M	14
Fault classes exercised	6 of 10	M	14
Bytecode instructions under the scheduler	3,291,313	M	14
Preemptions survived	450	M	14
Accounting: 1,097,103 × 3 + 3 vs observed	3,291,312 vs 3,291,313 — drift 1	D	14
Dispatch cost	~ 109 cycles /instruction (ceiling)	D	14
M5 instruction budget per app	60,000 each	M	16
App A: 19,999 × 3 + 3	= 60,000 exactly	D	16
App B: 14,999 ²	= 224,970,001 , the published value	D	16
Rogue fault offset	256, of a 256 B arena	M	16
Heap after release	158,048 B — exactly baseline	M	16
Runaway program	exactly 500 instructions under a 500 quantum	M	14

F.8 Confinement

Quantity	Value	Grade	Ch
Fills audited	136	M	17
Viewport escapes	0	M	17
Lowest row painted	250 = slot 2's exact bottom edge, of 320	M	17
Touches delivered / withheld	81 / 109,211	M	17
Confinement failures	0	M	17

Quantity	Value	Grade	Ch
Messages sent / delivered / refused	278 / 277 / 157,957	M	16
Bad buffers offered	0	M	16
String buffer	48 B, per-byte bounds-checked	—	17

F.9 The display

Quantity	Value	Grade	Ch
Full-screen fill, bit-banged	387 ms (~397 kB/s, ~3.2 MHz effective)	M	18
— original estimate	~150 ms — wrong by 2.6×	—	18
Full-screen fill, SPI2 FIFO	78 ms	M	18
— predicted improvement	~12×; actual 5×	—	18
Full-screen fill, SPI2 + DMA	43 ms , dma=320/0	M	18
Theoretical floor at 40 MHz	~31 ms	D	18
Transfers per screen: DMA / FIFO	320 / 2,560	D	18
Bytes for init + clear	153,634 = 240×320×2 + 34 command bytes	M	18
SPI clock	40 MHz (sysclk/2)	M	18
At 80 MHz	works electrically, visibly noisy	M	18
Blit after the DMA stall	55.9 ms instead of a possible ~22 ms	M	18
Good transfers before the spurious stall	63,910	M	18
Waits per second the raycaster issues	~2,900	D	18
Panic screen bytes drawn	~176,000 (fault 175,955, smash 176,704)	M	12

F.10 The renderer

Quantity	Value	Grade	Ch
Rays per frame	240, one per column	—	25
Frame: march / compose / blit	2.6 / 13.9 / 41.9 ms	M	25
Frame rate	~9.1 fps	M	25
Share of frame on the bus	72%	D	25
RAY_COLW 2→1: predicted / actual	0.2 ms / ~9 ms — wrong by 40×	—	25
Face shading + wall seams cost	unmeasurable — 9.1 fps before and after	M	25
Distinct cells visited in 20 s, before / after	4 / 12	M	25
Framebuffer, first measurement	fb off 126,650 μs / fb on 131,411 μs — no difference	M	18
Framebuffer, after two fixes	worth having; default on	M	18

Quantity	Value	Grade	Ch
Renderer at HIGH, alone / busy NORMAL band	2 → 8.5 fps / ~3 fps	M	9
Frame rate after the comparator fix	12.4 → 16.0 fps	M	7
Overlay cost per frame	324 pixels	—	24

F.11 Touch

Quantity	Value	Grade	Ch
Poll rate	100 Hz, HIGH priority, real 10 ms sleep	—	19
Conversions per read	11, two discarded	—	19
Peak pressure	2,065 (threshold 300)	M	19
z1 under touch / idle	1,123 / 0	M	19
z2 under touch / idle	2,992 / ~4,090	M	19
Pressure: real contact / approach-release	~2,000 / ~10 — a factor of 100	M	19
Raw span, horizontal / vertical drag	3,050 / 2,772 counts	M	19
raw_x left / right edge	~3,300 / ~480 — decreases	M	19
Corner readings TL/TR/BL/BR (raw_x)	3360 / 591 / 3258 / 376	M	19
Calibration error, corner-derived	23% on X, 11% on Y	M	19
Mapping error at x = 30 / 120 / 210	-29 / -8 / +12 px — changes sign	M	19
Months the X axis was inverted	~3	—	19
Rail samples per tap before the gate	~2 of 3	M	19
Samples per reporter interval, before / after	~15 / ~150	M	9
Attempts before the driver worked	4	—	19
Self-inflicted interrupt edges before masking	3,917	M	22
Finger-driven wakes ever observed	0	M	22

F.12 Storage

Quantity	Value	Grade	Ch
Flash user clock	~8 MHz (80 MHz / 1 / 10)	—	20
Chip ID read / expected before the fix	0x34200B / 0x684016 — exactly >> 1	M	20
Divider × edge combinations tested in one boot	16	M	20
Registers saved/restored per transaction	6	—	20
Boots survived in test	16	M	20
Frames across resets	256 → 512 → 512	M	20

Quantity	Value	Grade	Ch
Save cadence	every 256 frames (~60 s at 4.4 fps; now less)	M	20
Renderer rate when the cadence was set	4.4 fps — 81 frames in 18.3 s	M	20
SD identification clock	~250 kHz (spec ≤400 kHz)	—	21
SD error codes	7, one per stage	—	21
Card under test	~250 MB, SDSC, FAT16	M	21
Partition	type 0x06, LBA 240, 490,000 sectors	M	21
IO_MUX entries cross-checked / that could have caught the fault	4 / 0	—	21

F.13 Sensors and sound

Quantity	Value	Grade	Ch
ADC channels / resolution / attenuation	8 / 12-bit / 11 dB	—	22
Light sensor swing (hand over board)	265 counts	M	22
Control-group maximum	47 counts	M	22
Sensor usable range	273–538 of 0–4095	M	22
Wrong bits in the ADC driver	1	—	22
I ² C pull-ups	internal, ~45 kΩ (vs 2.2–10 kΩ wanted)	T	22
Devices found on a bare bus	0	M	22
Bytes ever transferred over I²C	0	—	22
Speaker: 440 Hz / 3 kHz	inaudible / clear	M	23
Usable from	~1 kHz (estimate, not measurement)	—	23
Click	3 kHz, 2 ticks (~20 ms)	—	23
CPU while sounding	none	—	23
Pins swept before the fault was found	15	—	23
Faults stacked	3	—	23

F.14 WiFi

Quantity	Value	Grade	Ch
OSI table entries declared / implemented	116 / 39	—	27
MAC moving words: cold / after phy / after mac	0 / 6 / 13–15	M	27
TSF rate, two measurements	1000 kHz / 1001 kHz	M	27
— over	622 ms and 508 ms of CPU time	M	27
MAC address	5c:01:3b:50:3f:64, CRC8 0x08	M	27
— reversed order gives	0x8f	M	27

Quantity	Value	Grade	Ch
PHY stack used	272 B of 6,144	M	27
Frames received / descriptors recycled	372 / 374 across 4 buffers	M	27
Networks decoded	2	M	27
Beacon rate before / after the HIGH change	3 Hz → 9.4 Hz (theoretical)	M	27
Frames transmitted / completed / answered	178 / 178 / 0	M	27
Probe requests / responses	20 / 0	M	27
Transmit power (most_tpw)	0x28 = 40 quarter-dBm = 10 dBm	M	27
IRAM cost of the MAC init chain	2,459 B — not the 48 KB believed	M	27
update_rx_chain worst-case wait	1 iteration	M	27

F.15 Process

Quantity	Value	Ch
Build cycles spent on the M2 defect	12	8
Wrong hypotheses recorded (M2)	3	8
Instructions in the M2 fix	5	8
Commits the display freeze survived	3	17
Hypotheses tested against stale firmware	2	20
Tools written to avoid a capture failure that reproduced it	2 of 2	19
Instruments caught lying, one session	7	27
Display theories eliminated by measurement	8	27
Conclusions published and retracted	12	E
Scripted edits that failed silently	4	12
Reverted attempt	~200 lines, four files	24
Layout assertions	6, spanning four files	24
Screen rows allocated	320 of 320	24
Commits	138	G
Reports	28	E
Kernel source	671,884 B across 79 files	A
Image	37,248 B	—

F.16 The four numbers worth remembering

37,248 bytes. The whole operating system.

0 escapes across 136 fills, and 0 confinement failures across 109,292 touch queries. The isolation claim, as numbers.

63 ms blocked against a 24 ms hold. Why contention cost is a count and not a duration.

3,291,313 instructions, 450 preemptions, drift of 1. The context switch, proved by arithmetic rather than by absence of complaint.

G. Timeline from the Commit History

138 commits across three days. The repository was initialised on 2026-08-14 — the same day the roadmap flagged its absence as an outstanding risk, "because the sibling project The Word Device was lost and recovered only because an editor's local history happened to retain it".

Commit subjects are quoted verbatim. They are unusually informative: several state a *retraction* rather than an addition.

G.1 Day one — 2026-08-14: M0 through M5, in one day

```
Initial commit – cyd-os through Milestone 1
WIP: M2 task switching (not working) + PDF report pipeline
Disable watchdogs; M2 narrowed to boot-context restore
M2 still failing: timer interrupt stops firing with all-fabricated tasks
M2 localized: fabricated-frame entry works, saved-frame resume does not
M2 works: preemptive task switching, 1200+ switches, zero corruption
Docs: UM-CYDOS-009 (M2 verification), two new figures, watchdog corrections
Root cause: PS.EXCM disables the zero-overhead loop; clear it before calling C
```

Eight commits, five of them about one defect. The sequence is the twelve build cycles of Chapter 8 §8.7 compressed: *not working* → *narrowed* → *still failing* → *localized* → *works* → and then, after the milestone had already passed, the **root cause**.

That ordering is worth noting. M2 was made to work by the `volatile` workaround before the cause was understood; the cause came afterwards and the workaround was removed.

```
M3: kernel heap and VM arena model, all exit criteria verified
UM-CYDOS-010 §7: rule out flash-as-framebuffer, and challenge the premise
Map .rodata from flash through the data cache
M4: register-based bytecode VM, assembler, and verified containment
Host the VM in a native task: both preemption mechanisms, composed
M5: multiple applications, isolated, with a shell. Roadmap M0-M5 complete.
Locking: critical sections and a blocking mutex, with a starvation defect found
```

"*challenge the premise*" is the commit that produced Chapter 1 §1.4's argument that the framebuffer should not exist. The flash-cache work follows immediately, because it was the prerequisite for M4 (Chapter 4 §4.4).

```
Display: ILI9341 driver, bit-banged SPI, no framebuffer
Display confirmed on hardware; measure the fill and correct a wrong estimate
Confirm display colour order; MADCTL BGR bit verified
WIP: display syscalls with viewport confinement (STALLS - do not flash)
Fix total system freeze: task_yield() must only ever move the tick EARLIER
Merge branch 'wip/display-syscall'
Merge display syscalls onto the freeze fix; restore the graphical demo
```

Three display commits, then a `WIP ... do not flash`, then the freeze fix. Chapter 17 §17.9 records that the freeze **predated** the syscall work and had survived three commits — visible here as the three display commits above it.

G.2 Day two — 2026-08-15: from kernel to device

Correct the previous commit: the shell was never broken
Prove viewport containment numerically, not visually
UM-CYDOS-016: display syscalls, and the freeze they were blamed for

The first commit of the day is a **retraction**. The second replaces a judgement ("I think it looks good") with a counter.

Touch: XPT2046 driver, and a GPIO bank bug it exposed
Fix build: hoist the touch latch above its user
Touch: PENIRQ detection, and a capture method that was destroying its own data
Touch works: PENIRQ detection confirmed, axes corrected by measurement
Touch calibration complete: both axes verified by measurement
UM-CYDOS-017: touchscreen, and the verification method that kept failing
Touch syscall: applications can read the pointer, confined to their viewport

Four attempts, visible as four commits. *"a capture method that was destroying its own data"* is Chapter 19 §19.4.

Hardware SPI2 for the display: 387 ms -> 78 ms full-screen fill
SYS BLIT: applications draw images held in their own arena
IPC: applications exchange messages without sharing memory
SPI2 DMA: 78 ms -> 43 ms full-screen fill
Animate the spectrum strip: crossfade between bars and a scrolling hue sweep
Raycaster: a rainbow dungeon rendered as vertical columns
Framebuffer for the 3D view, switchable - and the measurement says leave it off
Scheduler priorities, with sleep and priority inheritance

The framebuffer commit is the one Chapter 18 §18.7 overturns. *"switchable"* is what made the overturning possible.

Close two standing risks: arm the hang detector, pin the panic path to DRAM
Persistence: a record that survives a power cycle
Make the safety net loud: enforce stack guards, keep panics readable, fix UART rx
UM-CYDOS-019: failure handling, and three mechanisms that had never fired
Persist the panic reason, so a fault survives the power cycle that hides it
Panic to the panel, so a standalone board says why it stopped

"fix UART rx" is buried in the middle of a commit about something else — which is how Chapter 12 §12.5's finding was made: while building a harness to send `fault` over the serial line.

There was no idle task: nine `task_create` calls against a `TASK_MAX` of 8
A touch launcher, and a microSD driver
The touch X axis has been backwards since it was written
Revise 017: the direction test was decided by its own worst sample

Four commits, in order: the launcher exposes a missing idle task, the launcher lands, the launcher exposes the inverted axis, and the report explaining why the calibration missed it. Chapter 24 §24.1's *"none was found by inspection; all four were found by building something that depended on them"*.

Measure the renderer: it is busy 10% of the time, and the workers were not the problem
Scheduler fairness: age ready tasks so no priority can starve one forever
Instrument the display lock: contention with application drawing costs 3.7x the frame rate
Best-effort application drawing: 3.0 -> 9.9 fps
Shorten the display sleep now that it is the limiter; find the SPI ceiling by eye
Rename lock timing to say what it measures; revise 014 with the contention finding

"Rename lock timing to say what it measures" — Chapter 11 §11.9's blocked versus wait. A commit whose entire content is a naming decision, made because the wrong name would send the next reader in the wrong direction.

Rename the project from cyd-os to nat-os
The tick deadline raced into the future: every yield pushed it one interval
Document the comparator defect in 008

The 183 ms stall, found because shortening the display task's sleep removed the margin that had been concealing it.

Icons, and a close button an application cannot reach
Give the screen back to the launcher; the colour artwork becomes something you open
Chrome gets rows no view draws into; sweep the docs the relay layout invalidated
Revert the screen relay layout; keep the camera fix and add the layout assertions
Stamp the 3D view's close button into its framebuffer instead of over it
Revise 021: the overlay fix, and a correct diagnosis fixed at the wrong scope

Six commits containing the whole of Chapter 24 §24.8: the relay layout, the revert, the twenty-times-smaller fix, and the report about scope. Note that the revert commit **bundles two changes** — "keep the camera fix" — which is exactly what Chapter 24 §24.9 identifies as having destroyed the information.

3D view: one ray per column, and navigation decided per cell instead of per tick
Face shading, and 015 revision 1.4: a framebuffer claim that did not survive
Wall texture: panel seams in world space
Notes app with a multi-tap keypad, replacing the blit icon
Notes: save to flash, an inbox to read them back, and a close button somebody draws
Full screen for the launcher, and the postmortem that made it safe
Merge the full-screen layout and its postmortem
Revise 021 §6.7: the cause was found, and it was not the layout
calib: validate target pairs before fitting, and keep the outcome
docs: record what the calibration fault actually was (017 §7.4, 022 rev 1.1)

"a close button somebody draws" is Chapter 26 §26.8 — the button that was present, hit-testable, and drawn by nobody.

"the cause was found, and it was not the layout" is the acquittal.

G.3 Day three — 2026-08-16: peripherals, then the radio

```
intr: route peripheral interrupts, and switch the first consumer back off
adc: SAR ADC1, and the first thing this OS has measured outside itself
i2c: bit-banged two-wire master, and docs for it and the ADC
docs: consistency audit across all 26 reports against the source
```

"switch the first consumer back off" is in the commit subject — the interrupt matrix shipped with its only consumer disabled, and said so.

```
term: the shell on the panel, replacing the counter icon
audio: tones on the DAC, and a speaker that has not been found
docs: record the shell's scrollbar, and correct the record on where it landed
audio: LEDC tones on gpio26, and a click on every keypress
```

Two audio commits: the DAC attempt that produced silence, then LEDC. Chapter 23's three stacked faults sit between them.

```
display: overlap pixel conversion with DMA, and find out the timings lie
task: account CPU time per task, and time work with it instead of wall-clock
display: a single DMA timeout was silently halving throughput for the whole run
display: isolate the DMA timeout to the 3D view, and clear today's changes of it
display: the DMA timeout was measuring preemption, not a stalled engine
Revert today's display changes: the DMA timeouts were real, not preemption
docs: record the DMA stall evidence, and correct the previous commit
```

Seven commits, and the last two contradict the two before them. This is Chapter 18 §18.9 as it happened: a theory, a fix, a revert of the fix, and a report correcting the revert's commit message. The open status is the honest outcome.

```
window: implement the register-window vectors; windowed-ABI code now runs
window: link and call a real -mabi=windowed object from the call0 kernel
window: run Espressif ROM code -- crc32_le returns the textbook CRC-32
shell: list the three windowed-ABI commands in help
```

The ladder of Chapter 2 §2.7: hand-written windowed assembly, a vendor-compiled object, then real ROM code with a known answer.

```
vendor/phy: measure what Espressif's radio blob needs, and link it
vendor/phy: first attempt to link the blob into the kernel, and why it failed
vendor/phy: Espressif's radio blob runs inside nat-os
phy: the radio initialises -- register_chipv7_phy returns 0
wifi: the OSI table -- all 116 entries, generated, linking against libpp.a
wifi: the full MAC stack links -- zero unresolved symbols
wifi: implement the OSI table bodies over the call0 kernel
wifi: bring up the MAC and identify the TSF timer
```

"and why it failed" is a commit that exists to record a failure. *"all 116 entries ... containing nothing whatsoever"* — Chapter 27 §27.2's "it linked beautifully and would have collapsed the instant anything called it".

```
kernel: fix task_sleep, which was not sleeping
kernel: restore flat-out polling in the display and touch tasks
```

```
wifi: read the factory MAC address, verified against its own CRC
wifi: route the MAC interrupt onto a CPU line
wifi: receive chain from open-mac's source; channel tuning still faults
wifi: nat-os receives 802.11 frames
wifi: recycle rx descriptors so reception is continuous
```

"restore flat-out polling" is the commit Chapter 27 §27.10 calls a "restoration" that was not — it substituted `task_yield()` for `task_sleep(1)` and broke touch responsiveness, which was only diagnosable once `task_sleep` had been fixed the commit before.

```
wifi: transmit, and beacon a discoverable SSID
wifi: prove transmit does NOT reach the air, and build the test that shows it
wifi: rule out transmit power; the MAC hardware init is what is missing
wifi: link libpp and run the MAC hardware init chain
wifi: drop the rx task back to NORMAL; it was starving touch
```

"prove transmit does NOT reach the air, and build the test that shows it" — the probe-request experiment of Chapter 27 §27.8, committed as a retraction of the commit two above it.

```
touch: real 10 ms poll at HIGH priority, replacing a yield-per-sample loop
touch: make polling policy runtime-tunable; revert to the last known-good config
display: runtime-selectable panel clock, and touch polling policy
desktop: stop the app chrome painting over the 3D view
touch: make the responsive polling default
docs: UM-NATOS-028, the WiFi/touch/3D-view session
```

"make polling policy runtime-tunable" is the commit that turned four hypotheses into one flash, and the method note in Chapter 27 §27.13 says it "should have happened three rounds earlier".

```
desktop: narrow the chrome guard to a compile-time geometry assertion
calibration: give it the panel, and add raycast_open()
shell: add fbsum, and record what the 3D startup fault is NOT
shell: add the arena/framebuffer overlap check, and close out the evidence
shell: add dfreeze, and prove the fault is downstream of the framebuffer
docs: render UM-NATOS-028 to PDF, and let the builder use an installed Chrome
docs: UM-NATOS-028 rev 1.1 -- the frozen-renderer result, and rebuild the PDF
```

The last five working commits are all *instruments*, not fixes: `fbsum`, an overlap check, `dfreeze`. The project ends by building measurement rather than by guessing, which is the arc of the whole book.

"record what the 3D startup fault is NOT" — a commit whose deliverable is an enumeration of eliminated theories.

G.4 What the commit messages show

Retractions are commits. Nine commit subjects announce a correction:

Correct the previous commit: the shell was never broken
Revise 017: the direction test was decided by its own worst sample
Revert the screen relay; keep the camera fix and add the layout assertions
Revise 021: the overlay fix, and a correct diagnosis fixed at the wrong scope
Revise 021 §6.7: the cause was found, and it was not the layout
docs: correct the record on where it landed
Revert today's display changes: the DMA timeouts were real, not preemption
docs: record the DMA stall evidence, and correct the previous commit
wifi: prove transmit does NOT reach the air, and build the test that shows it

Failures are commits. *"and why it failed", "still failing", "a speaker that has not been found", "channel tuning still faults", "switch the first consumer back off".*

Documentation is a commit, and often a separate one. Fourteen commits are purely docs: or Revise NNN, including a *"consistency audit across all 26 reports against the source"* — a commit that checked the documentation against the code rather than the reverse.

Measurements are the deliverable. *"measure what the radio blob needs", "find out the timings lie", "prove the fault is downstream of the framebuffer", "record what the 3D startup fault is NOT".*

G.5 By the numbers

Commits	138
Days	3 (2026-08-14 to 2026-08-16)
Commits that announce a retraction	9
Commits that are documentation only	14
Commits about one defect (M2)	5
Commits about one defect (DMA stall)	7
Reverts	3
Milestones reached on day one	M0 through M5